



Papers fundacionales de la IA

Artificial Intelligence Evolution Program · eje transversal

52 hitos · de Rosenblatt (1958) a los sistemas agentic · actualizado 2026-08-16

Este documento enlaza a las fuentes primarias; no las reproduce. Los notebooks ejecutables viven en el repositorio.

1. P01 — El perceptrón: un modelo probabilístico de almacenamiento y organización de información en el cerebro (1958)
2. P02 — Aprender representaciones retropropagando errores (1986)
3. P03 — Memoria larga de corto plazo (1997)
4. P04 — Clasificación de ImageNet con redes neuronales convolucionales profundas (2012)
5. P05 — Estimación eficiente de representaciones de palabras en un espacio vectorial (2013)
6. P06 — Aprendizaje de secuencia a secuencia con redes neuronales (2014)
7. P07 — Traducción automática neuronal aprendiendo conjuntamente a alinear y traducir (2014)
8. P08 — La atención es todo lo que necesitas (2017)
9. P09 — BERT: preentrenamiento de Transformers bidireccionales profundos para comprensión del lenguaje (2018)
10. P10 — Los modelos de lenguaje son aprendices con pocos ejemplos (2020)
11. P11 — Generación aumentada por recuperación para tareas de PLN intensivas en conocimiento (2020)
12. P12 — Entrenar modelos de lenguaje para seguir instrucciones con retroalimentación humana (2022)
13. P13 — ReAct: sinergia entre razonar y actuar en modelos de lenguaje (2022)
14. P14 — Toolformer: los modelos de lenguaje pueden enseñarse a sí mismos a usar herramientas (2023)
15. P15 — Optimización directa de preferencias: tu modelo de lenguaje ya es un modelo de recompensa (2023)

16. P16 — Sistemas agentic contemporáneos: memoria, reflexión, multiagente e interoperabilidad (2023)
17. P17 — Modelos probabilísticos de difusión con eliminación de ruido (2020)
18. P18 — Aprender modelos visuales transferibles con supervisión de lenguaje natural (2021)
19. P19 — Entrenar modelos de lenguaje grandes con cómputo óptimo (2022)
20. P20 — Mamba: modelado de secuencias en tiempo lineal con espacios de estados selectivos (2023)
21. P21 — Mixtral: mezcla dispersa de expertos (2024)
22. P22 — DeepSeek-R1: incentivar la capacidad de razonamiento mediante aprendizaje por refuerzo (2025)
23. P23 — GloVe: vectores globales para representación de palabras (2014)
24. P24 — Representaciones profundas de palabras dependientes del contexto (2018)
25. P25 — Explorar los límites del aprendizaje por transferencia con un Transformer unificado texto a texto (2019)
26. P26 — Control a nivel humano mediante aprendizaje por refuerzo profundo (2015)
27. P27 — Dominar el go con redes neuronales profundas y búsqueda en árbol (2016)
28. P28 — El prompting de cadena de pensamiento provoca razonamiento en modelos de lenguaje grandes (2022)
29. P29 — Árbol de pensamientos: resolución deliberada de problemas con modelos de lenguaje grandes (2023)
30. P30 — Reflexion: agentes de lenguaje con refuerzo verbal (2023)
31. P31 — Agentes generativos: simulacros interactivos de comportamiento humano (2023)
32. P32 — Voyager: un agente encarnado de final abierto con modelos de lenguaje grandes (2023)
33. P33 — AutoGen: aplicaciones de nueva generación mediante conversación multiagente (2023)
34. P34 — RoFormer: Transformer mejorado con codificación posicional rotatoria (2021)
35. P35 — FlashAttention: atención exacta, rápida y eficiente en memoria, consciente de la E/S (2022)
36. P36 — Perdidos en el medio: cómo usan los modelos de lenguaje los contextos largos (2023)
37. P37 — MemGPT: modelos de lenguaje como sistemas operativos (2023)
38. P38 — Bayes variacional con autocodificación (2013)
39. P39 — Redes generativas adversarias (2014)
40. P40 — Dropout: una forma simple de evitar el sobreajuste en redes neuronales (2014)
41. P41 — Adam: un método de optimización estocástica (2014)
42. P42 — Explicar y aprovechar los ejemplos adversarios (2014)
43. P43 — Normalización por lotes: acelerar el entrenamiento profundo (2015)
44. P44 — Aprendizaje residual profundo para reconocimiento de imágenes (2015)
45. P45 — Destilar el conocimiento de una red neuronal (2015)
46. P46 — Una imagen vale 16x16 palabras: Transformers para reconocimiento de imágenes a escala (2020)
47. P47 — Predicción de estructura de proteínas de alta precisión con AlphaFold (2021)
48. P48 — LoRA: adaptación de rango bajo de modelos de lenguaje grandes (2021)
49. P49 — QLoRA: ajuste fino eficiente de modelos cuantizados (2023)
50. P50 — IA constitucional: inocuidad a partir de retroalimentación de IA (2022)
51. P51 — SWE-bench: ¿pueden los modelos resolver incidencias reales de GitHub? (2023)
52. P52 — Hacia la monosemanticidad: descomponer modelos de lenguaje con aprendizaje de diccionario (2023)

💡 La idea

Un paper leído es una anécdota. Un paper **ejecutado, roto e interpretado** es conocimiento.

Este eje convierte cada hito en una experiencia con la misma secuencia siempre:

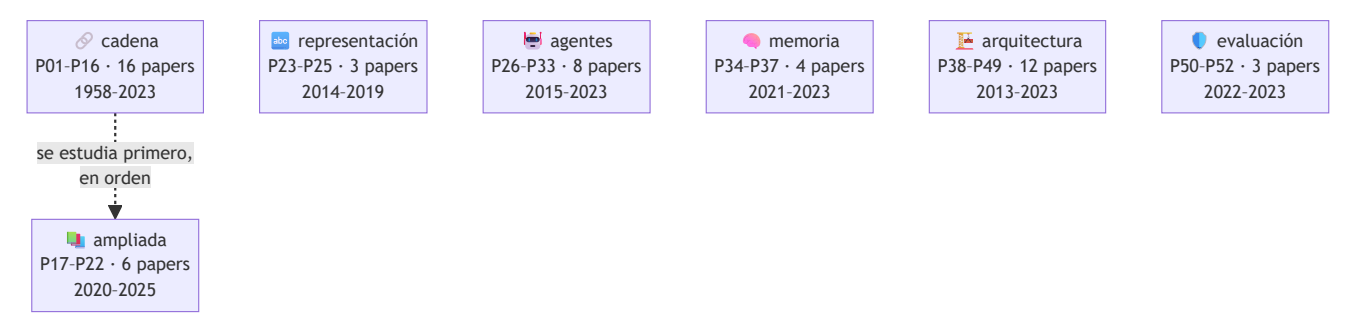
problema histórico → propuesta → intuición → matemática mínima → implementación → experimento → interpretación → limitaciones → siguiente hito

La última flecha es la importante. **Cada paper existe porque el anterior dejó algo sin resolver.** El perceptrón no separa XOR → backpropagation entrena capas ocultas → el gradiente se desvanece en secuencias largas → la LSTM lo controla → un vector fijo no sostiene frases largas → la atención lo elimina → si la atención basta, sobra la recurrencia → Transformer → ...y la atención cuesta $O(n^2)$, que es de donde arranca Mamba doce años después.

[!IMPORTANT] Este eje **no redistribuye papers**. Enlaza a la fuente primaria de cada uno, con autoría, año, venue, URL y fecha de consulta. Los notebooks implementan **miniaturas** del mecanismo en Python estándar: no reproducen los experimentos originales y lo declaran en cada salida.

🌀 Siete rutas · 52 papers

El eje tiene 7 bloques con propósitos distintos. **No se estudian igual.** Dentro de cada uno, los papers van **en orden cronológico**; entre bloques no hay orden, porque responden a preguntas diferentes.



🔗 Ruta mínima — la cadena canónica

P01–P16: la cadena canónica donde cada paper resuelve lo que el anterior dejó abierto. Se estudia en orden.

#	Paper	Año	Nivel	Lo que aportó
P01	El perceptrón	1958	L1	Primera máquina que aprende sus propios pesos a partir de ejemplos en lugar de ejecutar reglas escritas por una persona.
P02	Backpropagation	1986	L2	Un procedimiento práctico para entrenar capas ocultas: la red descubre representaciones intermedias que nadie diseñó.
P03	LSTM	1997	L2	Primera arquitectura recurrente capaz de mantener información a través de cientos de pasos sin que el gradiente se desvanezca.
P04	AlexNet	2012	L3	El resultado que convirtió el deep learning en la corriente principal: margen amplio sobre los métodos de visión hechos a mano.
P05	Word2Vec	2013	L2	El significado distribucional se vuelve barato: vectores densos entrenables sobre miles de millones de palabras.
P06	Seq2Seq	2014	L3	Una única red aprende a mapear secuencias de longitud variable a secuencias de longitud variable, de extremo a extremo.
P07	Atención (Bahdanau)	2014	L3	Nace la atención: el decodificador deja de depender de un único vector y consulta toda la entrada en cada paso.
P08	Transformer · <i>Attention Is All You Need</i>	2017	L4	Elimina la recurrencia y la convolución del modelado de secuencias: todo el cómputo de una capa se paraleliza.
P09	BERT	2018	L3	Consolida el patrón preentrenar-y-ajustar: un mismo modelo base sirve para muchas tareas con un ajuste pequeño.
P10	GPT-3 y el aprendizaje en contexto	2020	L3	El aprendizaje en contexto: la tarea se especifica en el prompt y el modelo se adapta sin actualizar ningún peso.
P11	RAG	2020	L3	Separa el conocimiento (índice consultable y actualizable) del razonamiento (parámetros del modelo).
P12	InstructGPT y RLHF	2022	L3	El salto de «modelo que completa texto» a «asistente que sigue instrucciones»: alineación con preferencias humanas.
P13	ReAct	2022	L2	El modelo deja de ser solo un generador de texto y pasa a ser el controlador de un bucle que observa y actúa.
P14	Toolformer	2023	L3	El uso de herramientas se aprende de forma autosupervisada: el criterio de utilidad es la propia pérdida del modelo.
P15	DPO	2023	L4	Alinear un modelo con preferencias humanas sin modelo de recompensa explícito ni bucle de aprendizaje por refuerzo.
P16	Sistemas agentic contemporáneos	2023	L5	El agente deja de ser un bucle y pasa a ser un sistema: memoria, reflexión, planificación, presupuesto, múltiples agentes y protocolos de interoperabilidad.

Ruta ampliada — lo que la cadena mínima no cubre

P17–P22: cobertura que la cadena mínima no da (generativa, multimodal, escalado) y continuación hasta 2025. Ordenada por año.

#	Paper	Año	Nivel	Lo que aportó
P17	Difusión (DDPM)	2020	L3	La generación deja de ser un salto en la oscuridad: se aprende a deshacer, paso a paso, un proceso de ruido conocido.
P18	CLIP	2021	L3	El texto se convierte en la etiqueta: un solo modelo clasifica categorías que nadie anotó, describiéndolas con palabras.
P19	Leyes de escalado con cómputo óptimo (Chinchilla)	2022	L4	Corrige la carrera por el tamaño: a cómputo fijo, los modelos de la época estaban infraentrenados en datos.
P20	Mamba	2023	L4	El primer competidor serio del Transformer en lenguaje: tiempo lineal y estado de tamaño fijo, sin atención.
P21	Mixtral (mezcla dispersa de expertos)	2024	L3	Desacopla capacidad de cómputo: 47 000 millones de parámetros totales, 13 000 millones activos por token.
P22	DeepSeek-R1	2025	L5	El razonamiento se incentiva con refuerzo puro, sin trazas humanas anotadas; y es el primer LLM de pesos abiertos publicado tras revisión por pares.

 Ruta de representación — cómo el lenguaje llegó a un formato único

P23–P25: cómo el lenguaje pasó de vectores estáticos a representaciones contextuales y de ahí a un formato único texto → texto. Ordenada por año.

#	Paper	Año	Nivel	Lo que aportó
P23	GloVe	2014	L2	Unifica las dos familias de embeddings: factorizar estadísticas globales de co-ocurrencia con la ventaja de los métodos predictivos.
P24	ELMo	2018	L3	Un vector por APARICIÓN y no por palabra: la polisemia deja de colapsar en un único punto del espacio.
P25	T5	2019	L3	Todo problema de texto se reescribe como texto → texto: un solo modelo, una sola pérdida, cero cabezas específicas.

 Ruta de agentes — decisión secuencial, razonamiento y multiagente

P26–P33: decisión secuencial y agentes. Empieza en el refuerzo profundo y la búsqueda guiada —de donde viene la idea de agente— y llega al razonamiento deliberado, la memoria, las habilidades reutilizables y el multiagente. Ordenada por año.

#	Paper	Año	Nivel	Lo que aportó
P26	DQN	2015	L3	El primer agente que aprende a actuar directamente desde píxeles, con la misma arquitectura y los mismos hiperparámetros en decenas de juegos.
P27	AlphaGo	2016	L4	Une las dos tradiciones de la IA: la búsqueda simbólica de la parte 01 y el aprendizaje profundo de la parte 04, en un solo sistema.
P28	Chain-of-Thought	2022	L2	Descomponer en pasos intermedios desbloquea tareas que el mismo modelo fallaba respondiendo de una vez.
P29	Tree of Thoughts	2023	L3	Devuelve la búsqueda clásica al razonamiento: explorar varias ramas, evaluarlas y poder retroceder.
P30	Reflexion	2023	L2	El agente aprende entre intentos sin tocar un solo peso: el refuerzo ocurre en el contexto, en lenguaje natural.
P31	Generative Agents	2023	L3	Resuelve la memoria de un agente que vive mucho tiempo: qué recordar, cuándo y por qué, cuando el contexto no da para todo.
P32	Voyager	2023	L3	El agente acumula habilidades reutilizables en vez de contexto: memoria procedimental que no se borra al terminar la tarea.
P33	AutoGen	2023	L4	El multiagente deja de ser una metáfora y pasa a ser un patrón de programación: agentes con rol que conversan hasta converger.

 Ruta de memoria y contexto — qué recuerda el modelo y cómo

P34–P37: cómo se codifica la posición, por qué el contexto largo es viable, por qué tenerlo no basta y cómo se gestiona como memoria jerárquica.

#	Paper	Año	Nivel	Lo que aportó
P34	RoPE	2021	L3	La posición se codifica rotando, y la atención pasa a depender solo de la distancia relativa.
P35	FlashAttention	2022	L4	El cuello de botella de la atención no eran los FLOPs sino las lecturas y escrituras a memoria.
P36	Lost in the Middle	2023	L3	Tener contexto largo no es usarlo: el rendimiento cae en forma de U cuando el dato relevante está en el medio.
P37	MemGPT	2023	L3	Aplica al contexto la idea de memoria virtual: una jerarquía que da la ilusión de memoria grande sobre una pequeña y rápida.

 Ruta de arquitectura y entrenamiento — el andamiaje de todo lo demás

P38–P49: el andamiaje que hace entrenable todo lo demás — generativa clásica, regularización, optimización, robustez, normalización, profundidad, compresión, visión con Transformer, ciencia aplicada y adaptación eficiente.

#	Paper	Año	Nivel	Lo que aportó
P38	VAE	2013	L3	Hace entrenable un modelo generativo latente: el truco de reparametrización deja pasar el gradiente a través del muestreo.
P39	GAN	2014	L3	Convierte la generación en un juego: dos redes compiten y ninguna necesita una verosimilitud explícita.
P40	Dropout	2014	L2	Apagar unidades al azar durante el entrenamiento equivale a entrenar un ensamblado exponencial de subredes que comparten pesos.
P41	Adam	2014	L2	Un paso de aprendizaje por dimensión, adaptado a la escala de su propio gradiente.
P42	Ejemplos adversarios	2014	L3	Una perturbación imperceptible cambia la predicción.
P43	Batch Normalization	2015	L2	Normalizar las activaciones dentro de la red permite tasas de aprendizaje mucho mayores y hace el entrenamiento profundo mucho menos frágil.
P44	ResNet	2015	L3	El atajo identidad hace apilables cientos de capas.
P45	Destilación de conocimiento	2015	L2	Las probabilidades del maestro contienen más información que la etiqueta correcta: el modelo pequeño aprende de esa estructura.
P46	Vision Transformer	2020	L3	Trata la imagen como una secuencia de parches y aplica un Transformer puro: la convolución deja de ser imprescindible en visión.
P47	AlphaFold 2	2021	L4	Resuelve en la práctica un problema abierto de cincuenta años en biología, y demuestra que la IA puede producir conocimiento científico, no solo productos.
P48	LoRA	2021	L3	Ajustar un modelo enorme entrenando una fracción diminuta de parámetros, sin coste añadido en inferencia.
P49	QLoRA y cuantización	2023	L3	Pone el ajuste fino de un modelo muy grande al alcance de una sola GPU de consumo.

Ruta de evaluación y seguridad — cómo se decide que un modelo sirve

P50–P52: cómo se decide que un modelo es aceptable — principios explícitos, criterios de evaluación verificables e interpretabilidad de lo que hay dentro.

#	Paper	Año	Nivel	Lo que aportó
P50	IA constitucional	2022	L4	Sustituye parte del juicio humano por un conjunto de principios explícitos y auditables, y por la autocritica del modelo.
P51	SWE-bench	2023	L3	Cambia el criterio de evaluación: no si el código parece bien, sino si los tests del repositorio real pasan.
P52	Superposición y autoencoders dispersos	2023	L5	Explica por qué una neurona no significa una cosa, y propone una forma de descomponer las activaciones en características interpretables.

¿Por qué el eje no llega a 2026?

Porque el propio eje se lo prohíbe. El criterio de ascenso exige, entre otras cosas, **12 meses desde la publicación** y evidencia de que otros trabajos han construido encima. Con fecha de revisión **2026-08-16**,

eso admite hasta mediados de 2025 — y ahí termina P22.

Lo posterior no se omite: vive en `frontier/current-topics.yaml` con fecha y fuente, y asciende a `foundational/` cuando cumple los criterios. Un paper de hace tres meses puede ser excelente y aun así no tener todavía lo que hace falta para enseñarlo como hito: réplicas, consecuencias y errores comunes documentados.

Tratamiento especial: *Attention Is All You Need*

El paper que sostiene casi todo lo que vino después se desmonta pieza por pieza en ocho miniaturas independientes, además de su ficha completa:

Miniatura	Qué aísla
To1	Por qué había que quitar la recurrencia
To2	Q, K, V y el producto escalar escalado
To3	Softmax, escala $\sqrt{d_k}$ y saturación
To4	Self-attention y máscara causal
To5	Multi-head attention
To6	Codificación posicional
To7	Residual, layer norm y feed-forward
To8	Encoder–decoder, complejidad y qué no dice el título

Y la ecuación 1 está desarrollada **con números, paso a paso**, en el anexo A04.

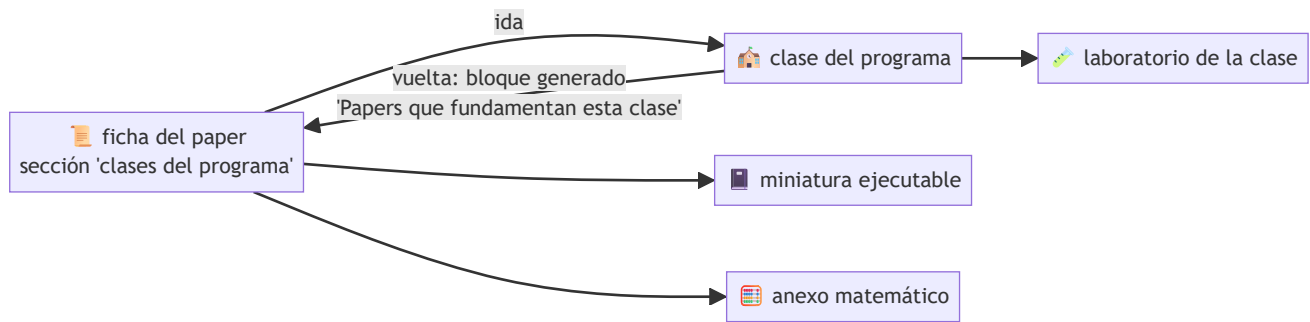
Anexos matemáticos

La sección 5 de cada ficha es deliberadamente corta: solo la matemática de **ese** paper. Las herramientas que reaparecen en todos se explican una vez, en un sitio, con ejemplo resuelto a mano y su error común:

Anexo	Cubre
A01 · Álgebra y geometría	Producto escalar, norma, coseno, hiperplanos, matrices
A02 · Probabilidad y verosimilitud	Softmax, entropía, KL, Bradley-Terry, gaussianas
A03 · Cálculo y gradientes	Regla de la cadena, retropropagación, gradiente de política
A04 · La atención, paso a paso	La ecuación 1 con números, máscara, multi-cabeza
A05 · Complejidad, coste y escalado	$O()$, memoria, FLOPs, 6ND, coste de inferencia

Ida y vuelta con las clases

El circuito está cerrado en los dos sentidos:



Las **50 clases** enlazadas llevan un bloque generado por `scripts/link_papers_to_classes.py` que lista sus papers, el año, qué desbloqueó cada uno y su notebook. Se regenera desde `papers.json`, así que no puede desincronizarse: `--check` lo verifica en CI.



Cómo se usa

```

pip install -e .

ai-evolution papers           # catálogo del eje
ai-evolution paper P08        # ficha resumida de un paper
ai-evolution paper-lab P08 --seed 7 # ejecuta la miniatura
python scripts/generate_papers.py # regenera índice, notebooks y manifiesto
  
```

Con los notebooks:

```
jupyter lab notebooks/papers/
```

Sin código: el eje también se lee en el [sitio del programa](#) o en el PDF imprimible (`python scripts/generate_pdfs.py --papers`).

Empieza por `P01_perceptron.ipynb` y sigue el orden de la ruta mínima. **Antes de ejecutar cada celda, escribe tu predicción** en la sección 7 — ese paso no es decorativo: es lo que se evalúa.



Estructura del eje

```
papers/
├─ README.md           ← este archivo
├─ ROADMAP.md          ← niveles L0-L5 y plan de estudio
├─ manifest.json       ← inventario con hash por artefacto (generado)
├─ guides/             ← cómo leer, 5 pasadas, plantilla, glosario, fuentes
├─ annexes/           ← 5 anexos matemáticos con ejemplos resueltos
├─ catalog/
│   └─ papers.json     ← fuente de verdad estructurada
│   └─ sources.yaml    ← venues y repositorios primarios
│   └─ PAPERS_INDEX.md ← índice legible (generado)
└─ foundational/PXX_slug/ ← una ficha de 18 secciones por paper

notebooks/papers/      ← 38 + 8 notebooks (generados)
instructor/papers/     ← plan de sesión por paper (generado)
student/papers/        ← ficha de estudio y bitácora (generado)
assessments/papers/    ← evaluación con rúbrica por paper (generado)
prompts/               ← prompts reutilizables del eje
```

Reglas del eje (verificadas automáticamente)

1. **Contexto antes que tecnicismo.** Nadie ve una ecuación antes de saber qué problema resuelve.
2. **Progresión antes que saturación.** Un hito por vez, en orden.
3. **Interpretación antes que ejecución mecánica.** Predecir, ejecutar, explicar.
4. **Implementación pequeña y explicable** antes que frameworks.
5. **Nunca atribuir al paper ideas posteriores.** Sección 11 de cada ficha.
6. **Nunca inventar** autores, fechas, datasets, métricas ni citas.
7. **Siempre distinguir** hecho documentado, simplificación didáctica, inferencia y práctica moderna.
8. **Siempre registrar** URL, autoría, año, venue y fecha de consulta.
9. **Sin APIs pagadas** como requisito de aprendizaje.
10. **Sin redistribuir** material con copyright.

Verificación

```
python scripts/generate_papers.py --check
python scripts/link_papers_to_classes.py --check
python -m unittest tests.test_papers -v
python scripts/validate_repository.py --strict
```

Se comprueba: JSON y YAML válidos, las 18 secciones de cada ficha en orden, los 17 momentos de cada notebook, `nbformat` correcto, que cada motor exista y sea determinista, que las clases enlazadas existan y **enlacen de vuelta**, ausencia de rutas absolutas y coherencia de los hashes del manifiesto en cualquier sistema operativo.

Ruta del eje de papers

Cómo recorrer los 22 hitos: qué nivel exige cada uno, en qué orden, con qué dedicación y qué evidencia deja cada tramo.

Los seis niveles

El eje tiene **dos rutas**: la mínima (P01–P16), que se estudia en orden porque es una cadena de dependencias, y la ampliada (P17–P22), ordenada por año, que aporta la cobertura que la cadena no da y continúa la historia hasta 2025. Ver README del eje.

Nivel	Nombre	Qué se hace	Evidencia que produce
L0	Orientación	Entender qué es un paper, cómo se publica y cómo se cita	Una cita completa y bien formada
L1	Fundamentos	Leer la idea y su contexto histórico, sin ejecutar nada	Ficha de lectura de pasada 1–2
L2	Implementación	Escribir o ejecutar la miniatura del mecanismo	Notebook ejecutado con predicción previa
L3	Análisis	Comparar variantes, medir e interpretar	Experimento controlado con tres semillas
L4	Reproducción parcial	Replicar una figura, tabla o ablación a escala reducida	Figura reproducida + límites declarados
L5	Investigación	Leer la frontera con fecha y fuente, proponer preguntas abiertas	Nota de linaje fechada

Lo no está asignado a ningún paper: es la puerta de entrada. Se cubre con las tres guías (cómo leer, 5 pasadas, fuentes) y con la clase 010 del programa.

Plan de cinco fases

Fase 1 — Cómo aprende una máquina (P01–P04)

Duración sugerida: 2 semanas · **Niveles:** L1–L3

De la neurona que corrige su error a la red profunda que gana un certamen. Al terminar sabes por qué el conexionismo tuvo un invierno y qué lo sacó de él.

Paper	Nivel	Pregunta que responde
P01 Perceptrón	L1	¿Puede una máquina ajustar sus propios parámetros?
P02 Backpropagation	L2	¿Cómo se entrena lo que no se observa directamente?
P03 LSTM	L2	¿Cómo se recuerda a través del tiempo?
P04 AlexNet	L3	¿Qué hizo falta para que la profundidad escalara?

Entregable de fase: un informe de 2 páginas explicando el invierno de las redes neuronales usando **solo** evidencia de P01 y P02, sin narrativa retrospectiva.

● Fase 2 — Del vector a la secuencia (P05–P07)

Duración sugerida: 2 semanas · **Niveles:** L2–L3

Las palabras adquieren geometría, las secuencias se transforman en secuencias, y el cuello de botella del vector fijo aparece y se elimina.

Paper	Nivel	Pregunta que responde
P05 Word2Vec	L2	¿Cómo se representa el significado sin definirlo?
P06 Seq2Seq	L3	¿Cómo se mapea longitud variable a longitud variable?
P07 Attention	L3	¿Cómo se deja de comprimir la entrada?

Entregable de fase: medir experimentalmente el cuello de botella (P06) y demostrar que la atención lo elimina (P07), con tres semillas y una conclusión que no exceda los datos.

● Fase 3 — El bloque que lo cambió todo (P08–P11)

Duración sugerida: 3 semanas · **Niveles:** L3–L4

El Transformer y sus dos descendencias, más la separación entre conocimiento y razonamiento. Es la fase más densa y la más rentable.

Paper	Nivel	Pregunta que responde
P08 Transformer + T01–T08	L4	¿Y si quitamos la recurrencia por completo?
P09 BERT	L3	¿Por qué mirar a ambos lados cambia la comprensión?
P10 GPT-3	L3	¿Qué aparece al escalar el decoder?
P11 RAG	L3	¿Cómo se cita lo que un modelo afirma?

Entregable de fase: las ocho miniaturas T01–T08 ejecutadas, más un texto de una página titulado «*qué NO dice el título del paper*».

● Fase 4 — Del modelo al agente (P12–P16)

Duración sugerida: 3 semanas · **Niveles:** L3–L5

Alineación, herramientas y sistemas. Aquí el eje entra en territorio con menos consenso, y eso se declara.

Paper	Nivel	Pregunta que responde
P12 InstructGPT	L3	¿Cómo se alinea un modelo con una intención?
P13 ReAct	L2	¿Cómo se ancla el razonamiento en observaciones reales?
P14 Toolformer	L3	¿Cómo se aprende <i>cuándo</i> usar una herramienta?
P15 DPO	L4	¿Hace falta RL para alinear?
P16 Sistemas agentic	L5	¿Qué convierte un bucle en un sistema?

Entregable de fase: una nota de linaje fechada sobre un tema de `frontier/current-topics.yaml`, con fuentes primarias y una declaración explícita de qué no está consolidado.

 Fase 5 — Ruta ampliada (P17–P22)

Duración sugerida: 3 semanas · **Niveles:** L3–L5

Lo que la cadena canónica no cubre —generación, multimodalidad y economía del cómputo— y la continuación hasta 2025.

Paper	Nivel	Pregunta que responde
P17 Difusión	L3	¿Cómo se genera sin adversario y sin borrosidad?
P18 CLIP	L3	¿Cómo se clasifica lo que nadie etiquetó?
P19 Leyes de escalado	L4	A cómputo fijo, ¿parámetros o datos?
P20 Mamba	L4	¿Se puede modelar secuencias sin pagar $O(n^2)$?
P21 Mixtral	L3	¿Se puede tener capacidad sin pagarla en cada token?
P22 DeepSeek-R1	L5	¿Se puede aprender a razonar sin trazas humanas?

Entregable de fase: una cuenta de servilleta propia usando el anexo A05: elige un modelo, estima su coste de entrenamiento y de inferencia, y decide si conviene denso o disperso para tu volumen.

 Dedicación estimada

Perfil	Ritmo	Alcance
Curioso	2 h/semana	Pasada 1–2 de todos los papers; miniaturas de Po1, Po7, Po8
Estudiante	6 h/semana	Ruta completa a nivel L2–L3, todas las miniaturas ejecutadas
Profesional	10 h/semana	L4 en Po8 y P15; reproducción parcial de dos figuras
Investigador	continuo	L5 permanente: pasada 5 con fecha en su subárea

 Definición de terminado

Un estudiante ha terminado el eje cuando, **sin abrir los papers**, puede:

- [] ubicar cada hito en la evolución de la IA y decir qué lo precede y qué lo sigue;
- [] explicar qué problema resolvió cada uno y por qué el anterior no bastaba;

- [] ejecutar la miniatura del mecanismo e interpretar su salida;
- [] señalar al menos un límite del paper que sus autores **no** declararon;
- [] diferenciar el paper original de las prácticas modernas que se le atribuyen;
- [] reconocer un claim mal formulado y pedir los cinco datos que le faltan (tarea, dataset, métrica, línea base, condiciones);
- [] citar cualquiera de los 22 con autoría, año, venue, URL y fecha de consulta;
- [] explicar por qué el eje termina en 2025 y qué criterio decide lo que entra.

Trabajo pendiente del eje

Lo que aún no está y se declara como tal, en vez de fingir completitud:

Pendiente	Estado
Páginas HTML del eje dentro del sitio PWA	✅ 66 páginas en <code>site/papers/</code> , con buscador en la portada
PDF imprimible del eje completo	✅ <code>docs/pdf/papers-fundacionales.pdf</code> , 434 páginas
Anexos matemáticos con ejemplos resueltos	✅ 5 anexos en <code>annexes/</code>
Enlaces de vuelta clase → paper	✅ 50 clases, generados y verificados en CI
Ampliación a generativa, multimodal y escalado (P17–P19)	✅ difusión, CLIP y leyes de escalado
Continuación hasta 2025 (P20–P22)	✅ Mamba, Mixtral y DeepSeek-R1
Fichas de segunda línea (GloVe, ELMo, T5)	❌ no iniciado
Reproducción parcial guiada de una figura real de Po8	❌ no iniciado
Traducción de las fichas a inglés	❌ no iniciado

Los papers de frontera **no** se añaden a `foundational/` : se registran en `frontier/current-topics.yaml` hasta que se consoliden.



Cómo leer un paper de IA

Leer un paper no es leer un texto de principio a fin. Es un **interrogatorio**: entras con preguntas, buscas dónde se responden y decides si la respuesta te convence.

Esta guía enseña qué preguntarle a un paper. El método en 5 pasadas enseña en qué **orden** hacerlo.



Anatomía de un paper y qué esconde cada parte

Sección	Qué dice	Qué NO dice	Trampa habitual
Título	La consigna del trabajo	No es un teorema	<i>Attention Is All You Need</i> no significa que baste la atención
Abstract	La versión más optimista	Las condiciones	Los resultados aparecen sin su línea base
Introducción	El problema y la promesa	Los fracasos previos de los autores	Presenta como nuevo lo que ya existía
Trabajo relacionado	Genealogía según los autores	Lo que ignoraron	Se citan a sí mismos con generosidad
Método	Lo que hay que reproducir	Los trucos no documentados	Un hiperparámetro decisivo en una nota al pie
Experimentos	Evidencia	Los experimentos que no salieron	Comparar contra una línea base débil
Ablaciones	Qué componente aporta qué	—	La sección más honesta; se salta casi siempre
Limitaciones	Los límites que los autores admiten	Los que no vieron	Puede faltar por completo
Apéndice	Los detalles que importan	—	Aquí vive lo que hace falta para reproducir

[!TIP] Si tienes 10 minutos, léete el **abstract**, mira las **figuras** y ve directo a las **ablaciones**. Ese recorrido te dice más que leer las 4 primeras páginas seguidas.



Las siete preguntas

Ninguna es sobre «qué dice el paper». Todas son sobre qué demuestra.

1. **¿Qué se hacía antes y por qué no bastaba?** Si no puedes responderla, no entiendes el paper: entiendes su solución fuera de contexto.
2. **¿Cuál es la afirmación central, en una frase?** Escríbela sin usar el vocabulario del paper. Si no puedes, aún no la entendiste.

3. **¿Qué evidencia la sostiene?** Tarea, dataset, métrica, línea base, número de ejecuciones, semillas, cómputo. Faltar cualquiera de estos seis es una debilidad, no un descuido de formato.
4. **¿Contra qué se compara?** Una mejora sobre una línea base mal ajustada no es una mejora. Comprueba si la línea base recibió el mismo esfuerzo de ajuste que el método propuesto.
5. **¿Qué se rompe si quito una pieza?** Es la pregunta de las ablaciones. Si no hay ablaciones, no sabes qué componente explica el resultado — y probablemente los autores tampoco.
6. **¿Qué NO demuestra?** Lo más valioso que puedes escribir sobre un paper. Casi siempre queda fuera del abstract.
7. **¿Qué haría falta para refutarlo?** Si no existe ningún resultado imaginable que lo contradiga, no es una afirmación empírica.

🚩 Señales de alarma

- **Métricas sin línea base.** «Alcanzamos 92 % de exactitud» no significa nada solo.
- **Una sola ejecución.** Sin varianza ni semillas, la diferencia puede ser ruido.
- **Benchmark contaminado.** Si el test pudo estar en el corpus de preentrenamiento, la métrica mide memorización, no capacidad.
- **Comparación entre desiguales.** Distinto cómputo, distintos datos, distinto ajuste.
- **Cherry-picking cualitativo.** Cinco ejemplos preciosos elegidos entre mil.
- **Ausencia de sección de limitaciones.** No significa que no las haya.
- **El código no existe, o existe y no reproduce las tablas.**
- **Un número que solo aparece en el abstract** y no se puede rastrear a ninguna tabla.

🕒 El error específico de leer papers históricos

Es el error que más penaliza este eje: **atribuir a un paper ideas que llegaron después.**

Se dice...	Pero en realidad...
«La LSTM tiene puerta de olvido desde 1997»	La añadieron Gers, Schmidhuber y Cummins en 1999/2000
«Rumelhart inventó la retropropagación»	La popularizó para redes; la diferenciación en modo reverso es de Linnainmaa (1970) y Werbos (1974)
«AlexNet inventó las CNN»	LeNet es de 1998; AlexNet demostró que escalaban
«El Transformer eliminó todo menos la atención»	También usa FFN, residuales, layer norm y codificación posicional
«GPT-3 aprende de los ejemplos del prompt»	Se condiciona ; no hay actualización de pesos

Un paper se lee **con la información que existía cuando se escribió**. La narrativa retrospectiva —«ya se veía venir»— es cómoda y casi siempre falsa.

Tres categorías que hay que separar siempre

Al escribir sobre un paper, cada frase pertenece a una de estas cajas. Etiquétalas:

Categoría	Ejemplo
Hecho documentado	«El paper reporta la configuración base con $N=6$ capas y $h=8$ cabezas (sección 3).»
Simplificación didáctica	«Q es lo que preguntas, K la etiqueta y V el contenido.»
Inferencia propia	«Probablemente la escala $\sqrt{d_k}$ importa más al crecer d_k .» ← verificable, pero mía
Práctica moderna posterior	«Hoy se usa RMSNorm y pre-norm en vez de post-norm.» ← no está en el paper

Mezclarlas es lo que convierte un resumen en desinformación con buena intención.

Cómo leer la matemática sin bloquearse

1. **Identifica los símbolos antes que las operaciones.** ¿Qué es un vector, qué una matriz, qué un escalar, qué un índice?
2. **Comprueba las dimensiones.** Si las formas no cuadran, has entendido mal algo. Es el depurador más rápido que existe.
3. **Evalúa el caso trivial.** ¿Qué pasa con $n=1$? ¿Con $d=1$? ¿Si el peso es 0? ¿Si es 1?
4. **Busca el caso límite.** ¿Qué ocurre cuando la secuencia es muy larga, la dimensión muy grande o la probabilidad se acerca a 0?
5. **Implementa la versión de 10 líneas.** La ecuación se entiende cuando corre.

Ese paso 5 es exactamente lo que hacen los notebooks de este eje.

Producto mínimo de una lectura

No has leído un paper hasta que puedes producir esto, sin volver a abrirlo:

- [] El problema anterior, en dos frases.
- [] La propuesta, en una.
- [] La ecuación o el diagrama central, dibujado de memoria.
- [] Una evidencia concreta con su tabla o figura de origen.
- [] Una limitación que el paper admite.
- [] Una limitación que el paper **no** admite.
- [] Una idea que hoy se le atribuye y llegó después.
- [] El hito siguiente que este paper hizo posible.

Esa lista es, literalmente, el esqueleto de la plantilla de ficha.

Referencia externa recomendada: S. Keshav, *How to Read a Paper* (ACM SIGCOMM CCR, 2007), origen del método de tres pasadas que este eje extiende a cinco. DOI



Dónde vive la investigación de IA

Regla del eje: una afirmación sobre un paper se sostiene con la **fuentes primaria**. Un blog, un hilo o un resumen generado por IA son pistas para encontrar el paper, nunca sustituto de haberlo abierto.

El error más frecuente al empezar es confundir **dónde se publica** con **dónde se busca**. Google Scholar no publica nada: indexa. arXiv no revisa nada: aloja. Saber qué hace cada sitio cambia cómo se cita y cuánta confianza merece lo que encuentras.



El mapa en una tabla

Sitio	Qué es realmente	Revisión por pares	Cuándo usarlo
arXiv	Repositorio de preprints	✗ No	Ver la investigación antes de que se publique formalmente
NeurIPS	Conferencia	✓ Sí	ML e IA en sentido amplio
ICML	Conferencia	✓ Sí	ML avanzado, teoría, optimización
ICLR	Conferencia	✓ Sí	Deep learning, representaciones, arquitecturas
ACL Anthology	Archivo abierto de ACL/NAACL/EMNLP	✓ Sí	PLN, LLM, traducción, evaluación lingüística
OpenReview	Plataforma de revisión abierta	✓ Sí (visible)	Leer las objeciones de los revisores y las réplicas
Semantic Scholar	Índice y grafo de citas	—	Rastrear qué papers citan a este
Google Scholar	Índice de citas	—	Encontrar versiones alternativas
Papers with Code	Enlace paper ↔ implementación	—	Buscar código, con reservas



arXiv — el primer lugar donde mirar

Es donde aparece primero casi toda la investigación relevante de IA, a menudo meses antes de la conferencia. Las categorías que importan para este programa:

Categoría	Contenido	Enlace
cs.AI	Inteligencia artificial en general	listado
cs.LG	Machine learning (la más activa)	listado
cs.CL	Lenguaje computacional: NLP, LLM, traducción	listado
cs.CV	Visión por computador	listado
stat.ML	Machine learning desde la estadística	listado

[!WARNING] Un **preprint no ha pasado revisión por pares**. Puede contener errores, ser retirado o cambiar entre versiones (v1 , v2 , v3). Cita siempre la versión que leíste. Un paper muy citado en arXiv que nunca fue aceptado en ningún venue es una señal a investigar, no a ignorar — pero es una señal.

Conferencias — el filtro de la comunidad

En IA, la publicación de referencia es la **conferencia**, no la revista. Es una diferencia cultural con otras disciplinas y explica los ciclos rápidos del campo.

- **NeurIPS** — la más grande y transversal. Publicó AlexNet (2012), el Transformer (2017), GPT-3, RAG, InstructGPT, Toolformer y DPO.
- **ICML** — machine learning avanzado; más peso en teoría y optimización.
- **ICLR** — nació en torno al deep learning y las representaciones. Usa OpenReview, así que todo su proceso editorial es público.
- **ACL / NAACL / EMNLP** — el mundo del lenguaje. Su archivo, **ACL Anthology**, es de acceso abierto y permanente: si un paper de PLN está ahí, esa es la cita canónica.

OpenReview — el sitio infravalorado

Es el único lugar donde se ve el paper y **su discusión**: qué objetaron los revisores, qué respondieron los autores, qué cambió entre versiones y por qué se aceptó o rechazó.

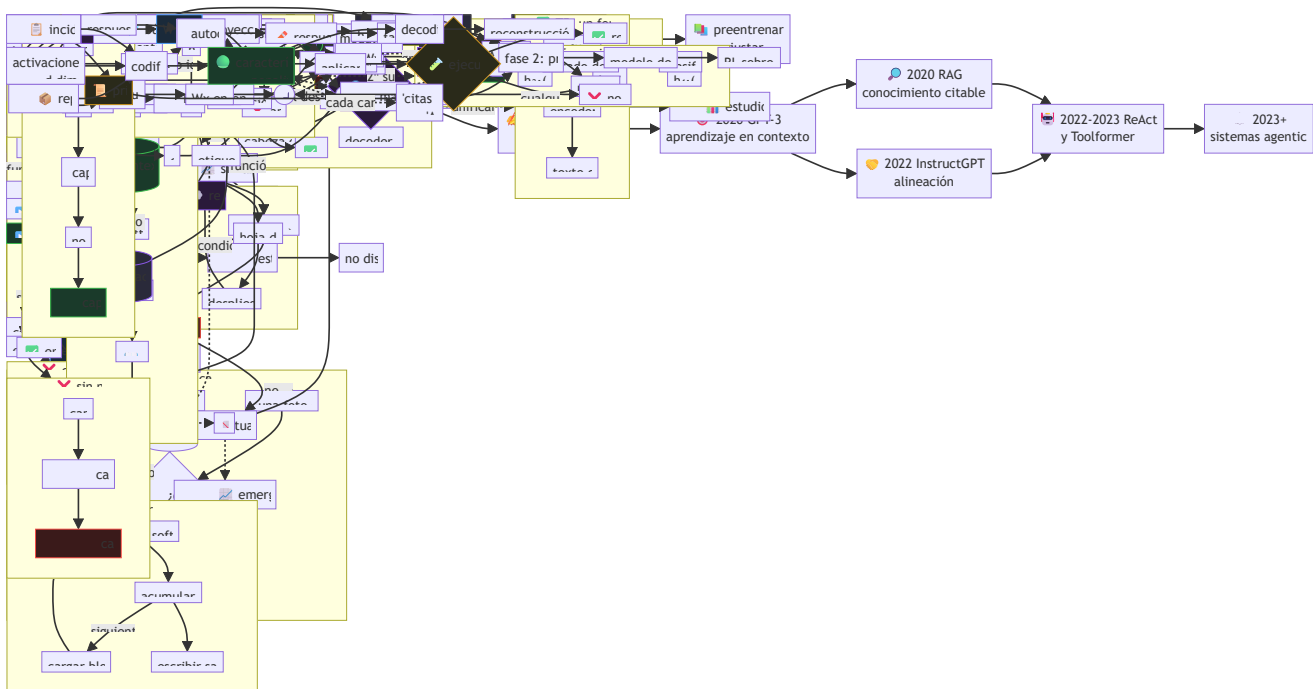
Para aprender a leer críticamente vale tanto como el paper. Ejercicio de nivel L3 de este eje: **buscar una revisión negativa de un paper aceptado y decidir si la objeción quedó realmente resuelta o solo respondida con elegancia**.

Buscadores — para rastrear, no para citar

Semantic Scholar y **Google Scholar** sirven para reconstruir linajes: quién citó a quién, qué vino antes y qué vino después. Semantic Scholar además expone una **API pública** que permite automatizar la vigilancia.

[!CAUTION] El número de citas mide **atención**, no calidad ni corrección. Papers muy citados han sido posteriormente matizados o refutados; papers poco citados han resultado fundamentales años después.

Así viajó la idea que sostiene casi todo lo que usas hoy:



Cada flecha del diagrama es un paper concreto con su ficha en este eje: índice completo.

1. Registra **autores, año, título, venue, URL y fecha de consulta**.
2. Cita la **versión** que leíste (arXiv:1706.03762v5 , no solo arXiv:1706.03762).
3. Prefiere el venue revisado si existe; usa arXiv cuando no exista o para la versión extendida.
4. **No cites un paper que no abriste**. Se detecta preguntando por el número de una figura.
5. No conviertas un número de un resumen ajeno en un dato tuyo sin verlo en la tabla original.
6. Distingue siempre: **hecho documentado** / **simplificación didáctica** / **inferencia propia**.

Este repositorio **enlaza** a las fuentes; no aloja PDFs de terceros con licencia restrictiva.

Los tres papers más antiguos del eje (PO1 Rosenblatt, PO2 Rumelhart y PO3 Hochreiter) están en revistas de suscripción. Vías legítimas: la versión de acceso abierto del autor, el repositorio institucional de su universidad, o una biblioteca. La versión en [ACL Anthology](#) siempre es abierta cuando existe.

Los datos de esta guía están también en formato legible por máquina en `catalog/sources.yaml`, con fecha de revisión.

Glosario del eje de papers

Términos que aparecen **leyendo papers**: vocabulario de publicación, de evaluación y de los mecanismos de la ruta mínima. Para el vocabulario general del programa (agentes, harness, context engineering, MLOps...) usa el glosario principal.

Cada entrada marca su tipo:  concepto técnico ·  proceso editorial ·  evaluación ·  trampa.

A

Ablación  — Quitar un componente y medir cuánto empeora el resultado. Es la única forma de saber qué pieza explica la mejora. Un paper sin ablaciones afirma que su método funciona, pero no por qué.

Abstract  — Resumen de 150–250 palabras. Contiene la versión más favorable de los resultados: casi nunca menciona la línea base ni las condiciones.

Alineación (alignment)  — Hacer que el comportamiento del modelo coincida con la intención humana. Ver P12 y P15.

Anacronismo de atribución  — Adjudicar a un paper una idea que apareció después. El error más penalizado en este eje. Ejemplo: la puerta de olvido de la LSTM.

arXiv  — Repositorio de preprints. **Sin revisión por pares.** Ver fuentes y venues.

Atención  — Mecanismo que calcula pesos que suman 1 sobre un conjunto de elementos y devuelve su combinación ponderada. Introducida para traducción en PO7.

Aprendizaje en contexto (in-context learning)  — Adaptación al condicionar con ejemplos en el prompt, **sin actualizar pesos.** Ver P10.

Autorregresivo  — Modelo que genera el elemento t condicionado a los anteriores: $p(x_t \mid x_{<t})$. La familia GPT.

B

Benchmark  — Conjunto estandarizado de tareas y métricas. Útil para comparar; peligroso cuando se optimiza directamente contra él (ley de Goodhart).

BLEU  — Métrica de traducción basada en solapamiento de n-gramas con referencias humanas. Correlaciona con calidad de forma imperfecta y no es comparable entre papers que usan distinta tokenización.

Bradley-Terry  — Modelo estadístico de comparaciones por pares: $p(a > b) = \sigma(r(a) - r(b))$. Base del modelo de recompensa en RLHF.

C

Camino máximo  — Número de pasos que debe recorrer una señal entre dos posiciones de una secuencia. $O(n)$ en un RNN, $O(1)$ en self-attention. Explica por qué el Transformer entrena mejor en dependencias largas.

Carrusel de error constante (CEC)  — Ruta aditiva de la celda LSTM por la que el gradiente viaja sin multiplicarse por activaciones. Ver P03.

Contaminación de benchmark  — Que los datos de test hayan aparecido en el corpus de entrenamiento. Convierte la métrica en una medida de memorización.

Cross-attention  — Atención donde las consultas vienen de una secuencia y las claves y valores de otra. Es lo que conecta decoder con encoder.

D

DPO (Direct Preference Optimization)  — Alinear con preferencias sin modelo de recompensa ni RL. Ver P15.

DOI  — Identificador persistente de una publicación. Preferible a una URL cuando existe.

E

Embedding  — Representación vectorial densa. Ver P05.

Entropía de la atención  — Cuánto se reparten los pesos α . Baja = atención concentrada; alta = repartida. Un softmax saturado tiene entropía casi nula y gradientes casi nulos.

Escalado (scaling laws)  — Relación empírica entre cómputo, datos, parámetros y pérdida. Kaplan et al. (2020) y Hoffmann et al. (2022).

F

Few-shot / one-shot / zero-shot  — Protocolos de evaluación según cuántos ejemplos se incluyen en el prompt. **No** son formas de entrenamiento.

Fine-tuning  — Ajustar los pesos de un modelo preentrenado sobre una tarea concreta. Distinto de condicionar con un prompt.

FFN por posición  — Red densa aplicada de forma independiente a cada posición dentro de un bloque Transformer. Concentra la mayoría de los parámetros del bloque.

G

Gradiente desvaneciente  — El gradiente se hace exponencialmente pequeño al propagarse por muchas capas o pasos temporales, y las primeras capas dejan de aprender.

GLUE  — Colección de tareas de comprensión del lenguaje usada para evaluar BERT y sucesores.

H

Hipótesis distribucional 🧱 — «Una palabra se caracteriza por la compañía que mantiene» (Firth, 1957; Harris, 1954). Fundamento de los embeddings.

Hoja de ruta del hito 🧱 — En este eje, la cadena problema → propuesta → límite → siguiente paper. Es lo que evita estudiar los papers como una lista inconexa.

K

KL (divergencia) 🧱 — Medida de cuánto se aleja una distribución de otra. En RLHF penaliza que la política se separe del modelo base; en DPO aparece implícita en el log-ratio.

L

Layer normalization 🧱 — Normaliza cada vector de activación a media 0 y varianza 1. Sin su epsilon, un vector constante produce división por cero.

Línea base (baseline) 🧱 — El método contra el que se compara. **Un resultado sin línea base no es un resultado.** Comprueba si recibió el mismo esfuerzo de ajuste.

M

Máscara causal 🧱 — Impide que la posición i atienda a posiciones futuras. Se aplica **antes** del softmax, poniendo los scores a $-\infty$.

Memoria paramétrica / no paramétrica 🧱 — Lo que el modelo sabe en sus pesos frente a lo que consulta en un índice externo. La separación que propone RAG.

MLM (Masked Language Modeling) 🧱 — Predecir tokens enmascarados usando contexto bidireccional. Objetivo de preentrenamiento de BERT.

O

OpenReview 🏛️ — Plataforma donde el proceso de revisión es público: revisiones, réplicas y decisión. La mejor escuela de lectura crítica disponible gratis.

Overclaiming ⚠️ — Afirmar más de lo que la evidencia sostiene. Forma más común: convertir una mejora en un benchmark en una afirmación sobre «comprensión» o «razonamiento».

P

Peer review (revisión por pares) 🏛️ — Evaluación por investigadores del área antes de publicar. arXiv no la tiene; NeurIPS, ICML, ICLR y ACL sí.

Preprint 🏛️ — Versión previa a la revisión formal. Citable, pero declarando que lo es.

Producto escalar escalado  — $QK^T / \sqrt{d_k}$. La división evita que el softmax se sature al crecer la dimensión.

Positional encoding  — Información de orden que se **suma** al embedding, porque la atención por sí sola es indiferente a las permutaciones.

R

RAG  — Recuperar documentos y generar condicionando en ellos. Ver P11.

Reproducibilidad  — Que otra persona, con el paper y el código, obtenga el mismo resultado. Requiere semillas, versiones, datos y cómputo declarados.

Residual (conexión)  — $x + \text{Sublayer}(x)$. Camino identidad que permite apilar decenas o cientos de capas sin que la señal se apague.

Reward hacking  — La política maximiza el modelo de recompensa explotando un atajo (por ejemplo, ser más larga) sin mejorar la calidad real.

RLHF  — Ajuste con retroalimentación humana en tres etapas: SFT, modelo de recompensa y RL con penalización KL. Ver P12.

S

Self-attention  — Atención de una secuencia sobre sí misma.

Semilla (seed)  — Valor que fija la aleatoriedad. Reportar resultados sin semilla ni número de ejecuciones impide distinguir mejora de ruido.

Softmax  — Convierte un vector de scores en una distribución de probabilidad. Se implementa restando el máximo para evitar desbordamiento.

SOTA (state of the art)  — «Estado del arte». Caduca. Sin fecha y benchmark concretos, la palabra no informa de nada.

T

Tool use  — Que el modelo invoque funciones externas. Aprendido de forma autosupervisada en P14.

Transformer  — Arquitectura basada solo en atención, FFN, residuales, layer norm y codificación posicional. Ver Po8.

V

Venue  — Dónde se publicó: conferencia, revista o repositorio. Parte obligatoria de una cita.

Varianza entre ejecuciones  — Diferencia de resultados al cambiar solo la semilla. Si la mejora reportada es menor que la varianza, no hay mejora demostrada.

Método de lectura en 5 pasadas

Origen y honestidad de la fuente: el método de **tres pasadas** es de S. Keshav, *How to Read a Paper* (ACM SIGCOMM CCR, 2007) — DOI. Las **pasadas 4 y 5** son una extensión pedagógica de este programa, no del paper de Keshav. Se marcan como tales para no atribuirle algo que no escribió.

La idea central: **nunca leas un paper una sola vez de principio a fin**. Haz varias pasadas, cada una con un objetivo distinto, y decide después de cada una si merece la siguiente. La mayoría de los papers no pasan de la pasada 1, y eso está bien.

El método de un vistazo

Pasada	Tiempo	Objetivo	Salida	¿Sigo?
1	10 min	¿De qué va y me sirve?	5 frases	Si no me sirve, paro
2	1 h	¿Qué propone y con qué evidencia?	Ficha secciones 1–9	Si no lo voy a usar ni evaluar, paro
3	4–5 h	¿Podría reimplementarlo?	Miniatura ejecutable	Si no necesito el detalle, paro
4	1–2 días	¿Se sostiene al replicarlo?	Figura o ablación reproducida	Solo para papers que van a fundamentar trabajo propio
5	continuo	¿Dónde encaja hoy?	Nota de linaje con fecha	Solo para el núcleo de tu área

1 Pasada 1 — Reconocimiento (10 minutos)

Lee: título, abstract, introducción, encabezados de sección, figuras, conclusiones, referencias (solo por encima). **No leas:** método, ecuaciones, tablas de resultados.

Al terminar responde las **cinco C**:

1. **Categoría** — ¿es un método nuevo, un análisis, un dataset, una evaluación, una posición?
2. **Contexto** — ¿con qué trabajos dialoga?
3. **Corrección** — ¿los supuestos parecen razonables a primera vista?
4. **Contribución** — ¿qué aporta, en una frase?
5. **Claridad** — ¿está bien escrito? (mala escritura correlaciona con trabajo confuso)

Criterio de parada: si no responde a una pregunta que tienes, cierra. Guarda la referencia y sigue. Leer todo lo que llega no es rigor: es falta de criterio.

2 Pasada 2 — Comprensión (1 hora)

Lee: todo el cuerpo con atención a figuras y tablas. **Ignora** las demostraciones.

Objetivos concretos:

- Escribir la **afirmación central** sin usar el vocabulario del paper.
- Identificar **tarea, dataset, métrica y línea base** de cada resultado principal.
- Marcar las referencias que necesitarás leer.
- Anotar cada punto que no entendiste (no lo disimules: la lista es tu plan de estudio).

Al terminar deberías poder **describir el paper a otra persona** con sus evidencias principales. Con esto ya puedes rellenar las secciones 1 a 9 de la ficha.

[!WARNING] Si al final de esta pasada no puedes nombrar la línea base, no has entendido el resultado: has memorizado un número.

3 Pasada 3 — Reimplementación mental (4–5 horas)

El objetivo: poder reconstruir el trabajo con los mismos supuestos.

Método: haz de nuevo el paper. Asume lo que asumen los autores y recrea el trabajo. Comparar tu reconstrucción con la real revela no solo las innovaciones del paper, sino sus supuestos ocultos y sus fallos.

En este eje, la pasada 3 tiene una salida obligatoria y concreta: **la miniatura ejecutable**, que ya está construida en `notebooks/papers/`. Tu trabajo es predecir la salida antes de ejecutarla y explicar cualquier diferencia.

Preguntas de esta pasada:

- ¿Qué hiperparámetro decide el resultado, y está documentado?
- ¿Qué pasaría si cambio este componente por el más simple posible?
- ¿Puedo derivar la ecuación central en una servilleta?

4 Pasada 4 — Reproducción parcial (*extensión de este programa*)

No siempre es posible reproducir un paper: el cómputo original puede costar cientos de miles de dólares. Pero casi siempre es posible reproducir **algo**:

- una **figura** con datos sintéticos o un subconjunto público;
- una **ablación** a escala reducida;
- una **tendencia** (que la métrica suba con el tamaño, aunque no llegues a sus valores).

Regla de honestidad: una tendencia reproducida a escala 1/1000 es evidencia de que entendiste el mecanismo, **no** de que el resultado del paper sea correcto. Decláralo así.

Salida: un notebook con semilla fija, la figura reproducida y una frase sobre qué parte del resultado original queda fuera de tu alcance y por qué.

5 Pasada 5 — Linaje y estado del arte (*extensión de este programa*)

Un paper no es un punto: es un nodo. Esta pasada lo sitúa.

1. Busca en Semantic Scholar qué papers lo citan.
2. Identifica: quién lo **extendió**, quién lo **contradijo** y quién lo **reemplazó**.
3. Comprueba si su resultado principal **sigue vigente** o fue matizado.
4. Anota la **fecha de consulta**. Esta pasada caduca.

Salida: una nota de linaje de 5 líneas con fecha, que en este repositorio vive en la sección 13 de cada ficha y, para lo aún no consolidado, en `frontier/current-topics.yaml`.

Cuánto invertir, según para qué

Tu objetivo	Pasada suficiente
Saber si existe algo relevante	1
Citarlo en un informe	2
Explicarlo en clase	3
Construir sobre él	4
Investigar en su área	5

Autoevaluación del método

- ☐ He parado en la pasada 1 al menos una vez esta semana (señal de tener criterio).
- ☐ Puedo nombrar la línea base de los últimos tres papers que leí.
- ☐ Tengo una lista escrita de lo que no entendí, no una sensación difusa.
- ☐ Antes de ejecutar cualquier miniatura, escribí mi predicción.
- ☐ Sé distinguir, en mis propias notas, hecho documentado de inferencia mía.



Plantilla de ficha de paper

Contrato de 18 secciones. Es **obligatorio y verificado automáticamente**: cada ficha de papers/foundational/ debe tener estas 18 secciones, con estos títulos exactos y en este orden. Lo comprueba `ai_evolution.papers.validate_papers()`.

Copia el bloque de abajo para dar de alta un paper nuevo. Después registra su entrada en `catalog/papers.json` y ejecuta:

```
python scripts/generate_papers.py
```

Por qué 18 secciones

Cada una responde a una pregunta que un resumen normal se salta:

Sección	Pregunta que fuerza a responder
1–3	¿Qué había antes, qué propone y quién lo firma?
4–6	¿Lo entiendo sin fórmulas, con fórmulas y como flujo?
7–8	¿Dónde está la evidencia en el paper original ?
9–11	¿Qué cambió, qué no demuestra y en qué se equivoca la gente?
12–13	¿De qué depende y qué hizo posible?
14–17	¿Cómo lo ejecuto, lo practico y me autoevalúo?
18	¿De dónde salió cada afirmación?



Plantilla (copiar desde aquí)

PXX – Título en español

> Una frase que sitúe el hito en la evolución de la IA.

****Nivel:**** L? . ****Motor:**** `nombre` . ****Notebook:**** enlace relativo a `notebooks/papers/PXX\_slug.ipynb`

1. Identificación

Campo Valor
--- ---
Título original
Autoría
Año
Venue
Fuente primaria
Acceso abierto / restringido
Fecha de consulta AAAA-MM-DD

2. Problema anterior

Qué se hacía antes y por qué no bastaba. ****Sin mencionar la solución del paper.****

3. Propuesta

La contribución en 3-5 frases. Qué es nuevo y qué reutiliza de trabajos previos.

4. Intuición sin fórmulas

Una analogía honesta. Debe declarar dónde deja de funcionar.

5. Matemática mínima

Solo las ecuaciones imprescindibles, con todos los símbolos definidos.

6. Arquitectura o flujo

Diagrama (mermaid o bloque `text`) del mecanismo, entrada a salida.

7. Qué observar en el paper original

Secciones, figuras y tablas concretas que hay que mirar, con su número.

8. Evidencia y resultados

Qué demostró, con tarea, dataset, métrica y línea base. Si un número no se ha verificado en la fuente, se dice explícitamente en lugar de escribirlo.

9. Impacto

Qué cambió después. Distinguir impacto documentado de narrativa retrospectiva.

10. Limitaciones

Las que el paper admite ****y**** las que no admite. Mínimo tres.

11. Errores comunes

Malentendidos frecuentes, especialmente atribuciones anacrónicas.

12. Relación con trabajos anteriores

De qué depende. Con referencia y año.

13. Relación con trabajos posteriores

Qué hizo posible. Con referencia y año.

14. Notebook asociado

Qué implementa la miniatura, qué ****no**** implementa y cómo ejecutarla.

15. Actividades Bloom

Seis actividades: recordar, explicar, aplicar, analizar, evaluar y crear.

16. Autoevaluación

Entre 5 y 8 preguntas. Ninguna de definición pura.

17. Respuestas esperadas

Qué debe contener una buena respuesta. No una respuesta modelo para copiar.

18. Fuentes primarias

Lista con autores, año, venue, URL y fecha de consulta.

✓ Reglas de calidad no negociables

1. **No inventar** autores, fechas, datasets, métricas ni citas. Si no lo verificaste, escribe «verificar en la tabla N del paper» en lugar del número.
2. **No atribuir** al paper ideas posteriores (sección 11 existe para eso).
3. **Marcar** siempre qué es hecho documentado, simplificación didáctica, inferencia propia o práctica moderna.
4. **Fuente primaria obligatoria** con URL y fecha de consulta.
5. **No redistribuir** PDFs con restricciones de copyright: se enlaza.
6. **Sin rutas absolutas** en ningún ejemplo ni notebook.
7. **Limitaciones antes que entusiasmo**: la sección 10 no puede quedar en una línea.

🔍 Cómo se verifica

```
python -c "import sys; sys.path.insert(0,'src'); from ai_evolution.papers import validate_papers; print(validate_papers(strict=True))"
```

Comprueba: las 18 secciones y su orden, la existencia del notebook y sus 17 momentos, que el motor exista, que haya fuente primaria con URL, que las clases enlazadas existan y que los SHA-256 del manifiesto estén al día.



Anexos matemáticos del eje de papers

Toda la matemática que aparece en las 52 fichas, explicada una sola vez y en un solo sitio. Cada anexo dice **qué es**, **por qué aparece**, **dónde se usa** y trae un **ejemplo resuelto a mano** que puedes comprobar con lápiz antes de ejecutar nada.

Por qué existen

Las fichas tienen una sección 5 «Matemática mínima» deliberadamente corta: solo las ecuaciones imprescindibles de **ese** paper. Pero las mismas herramientas reaparecen una y otra vez —el producto escalar está en Word2Vec, en la atención y en CLIP; el softmax está en cinco papers; la regla de la cadena sostiene el entrenamiento de los veintidós—.

Repetir la explicación en cada ficha sería ruido. Omitirla dejaría fuera a quien la necesita. Los anexos resuelven esa tensión: la ficha enlaza, el anexo explica.

Los cinco anexos

Anexo	Cubre	Papers que lo usan
A01 · Álgebra y geometría	Vectores, producto escalar, norma, coseno, hiperplanos, matrices	P01, P05, P07, P08, P18
A02 · Probabilidad y verosimilitud	Softmax, entropía, KL, verosimilitud, Bradley-Terry, gaussianas	P08, P09, P10, P12, P15, P17, P22
A03 · Cálculo y gradientes	Derivadas, regla de la cadena, retropropagación, comprobación numérica	P02, P03, P05, P12, P15
A04 · La atención, paso a paso	La ecuación 1 desarrollada con números, máscara, multi-cabeza	P07, P08, T01–T08
A05 · Complejidad, coste y escalado	Notación $O()$, memoria, FLOPs, leyes de escalado, coste de inferencia	P06, P08, P19, P20, P21, P22

Cómo usarlos

1. **No los leas seguidos.** Son material de consulta, no un curso de matemáticas.
2. Cuando una ficha te frene en la sección 5, abre el anexo que enlaza, lee **solo** el apartado que necesitas y vuelve.
3. Cada apartado termina con un **error común**: si vas con prisa, lee al menos esos.
4. Los ejemplos numéricos están resueltos a mano a propósito. Haz la cuenta tú antes de mirar el resultado — es el mismo contrato que el de los notebooks: **predecir, luego comprobar**.

Nivel asumido

Se asume bachillerato: operaciones con fracciones, potencias, logaritmos y la idea de derivada. Todo lo demás se construye aquí. Si algo del anexo A03 no se entiende, la clase 005 del programa cubre los prerrequisitos con más calma.

A01 · Álgebra y geometría

Los vectores no son «listas de números»: son **flechas con dirección y longitud**. Casi todo lo que hace un modelo de IA es medir ángulos entre flechas y moverlas.

Usado en: P01 · P05 · P07 · P08 · P18

1. Producto escalar

$$a \cdot b = \sum_i a_i b_i = a_1 b_1 + a_2 b_2 + \dots + a_n b_n$$

Qué mide. Cuánto «van en la misma dirección» dos vectores, escalado por sus longitudes:

$$a \cdot b = \|a\| \cdot \|b\| \cdot \cos(\theta)$$

- $a \cdot b > 0 \rightarrow$ ángulo agudo, apuntan hacia el mismo lado.
- $a \cdot b = 0 \rightarrow$ **perpendiculares**, no comparten nada.
- $a \cdot b < 0 \rightarrow$ apuntan en sentidos opuestos.

Ejemplo resuelto. $a = (3, 4)$, $b = (4, 3)$:

$$\begin{aligned} a \cdot b &= 3 \cdot 4 + 4 \cdot 3 = 24 \\ \|a\| &= \sqrt{9+16} = 5 & \|b\| &= \sqrt{16+9} = 5 \\ \cos(\theta) &= 24 / (5 \cdot 5) = 0,96 & \rightarrow \theta &\approx 16,3^\circ \end{aligned}$$

Casi alineados, como cabía esperar de dos vectores tan parecidos.

Dónde aparece. Es la operación QK^T de la atención (P08), la puntuación $w \cdot x$ del perceptrón (P01) y la similitud de CLIP (P18).

⚠ **Error común:** creer que un producto escalar grande significa «muy parecidos». Puede ser grande simplemente porque los vectores son **largos**. Para comparar dirección hay que normalizar — y eso es exactamente el coseno.

2. Norma y coseno

$$\begin{aligned} \|a\| &= \sqrt{a \cdot a} && \text{longitud del vector} \\ \cos(a, b) &= (a \cdot b) / (\|a\| \|b\|) && \text{similitud independiente de la longitud, en } [-1, 1] \end{aligned}$$

Ejemplo resuelto. $a = (1, 0)$, $b = (100, 0)$: el producto escalar vale 100 —enorme— pero $\cos = 100 / (1 \cdot 100) = 1$. Son **la misma dirección** con longitudes muy distintas.

Dónde aparece. Toda medida de similitud semántica del eje: vecinos de Word2Vec, ranking de RAG, matriz de CLIP.

⚠ **Error común:** usar coseno cuando la magnitud sí importa. En recuperación, un documento largo y uno corto con la misma proporción de términos dan el mismo coseno; a veces eso es lo que quieres y a veces no.

3. Hiperplanos y separabilidad

$$w \cdot x + b = 0$$

Esa ecuación define una **recta** en 2D, un **plano** en 3D y un **hiperplano** en n dimensiones. Divide el espacio en dos mitades: donde $w \cdot x + b > 0$ y donde es < 0 .

- w es **perpendicular** al hiperplano: marca hacia dónde crece la puntuación.
- b lo **desplaza** del origen. Sin b , la frontera pasaría siempre por el punto $(0, \dots, 0)$.
- La distancia de un punto x al hiperplano es $|w \cdot x + b| / \|w\|$.

Ejemplo resuelto (el perceptrón de AND). Con $w = (2, 1)$ y $b = -3$:

x	$w \cdot x + b$	lado	clase
(0,0)	$0+0-3 = -3$	negativo	0
(0,1)	$0+1-3 = -2$	negativo	0
(1,0)	$2+0-3 = -1$	negativo	0
(1,1)	$2+1-3 = 0$	frontera (≥ 0)	1

La recta $2x_1 + x_2 = 3$ deja solo $(1,1)$ en el lado positivo. Compruébalo dibujándolo.

Por qué XOR no se puede. Con $y=1$ en $(0,1)$ y $(1,0)$, y $y=0$ en $(0,0)$ y $(1,1)$:

$(0,0) \rightarrow 0$: $b < 0$
 $(0,1) \rightarrow 1$: $w_2 + b \geq 0$
 $(1,0) \rightarrow 1$: $w_1 + b \geq 0$
 $(1,1) \rightarrow 0$: $w_1 + w_2 + b < 0$

Sumando las filas 2 y 3: $w_1 + w_2 + 2b \geq 0 \rightarrow w_1 + w_2 + b \geq -b > 0$
que contradice la fila 4. No existe (w_1, w_2, b) .

⚠ **Error común:** decir «XOR no converge porque faltan épocas». No es un problema de presupuesto: es que **no existe** ningún hiperplano que lo resuelva. Ver P01.

4. Matrices como transformaciones

Una matriz $W \in \mathbb{R}^{m \times n}$ convierte un vector de n dimensiones en uno de m :

$$y = W \cdot x \qquad y_i = \sum_j W_{ij} x_j$$

Cada **fila** de W es un vector con el que se hace producto escalar. Por eso una capa densa $y = Wx + b$ es literalmente « m puntuaciones, una por fila».

Comprobación de dimensiones — el mejor depurador que existe. Si multiplicas $(m \times n)$ por $(n \times k)$, obtienes $(m \times k)$: la dimensión interior debe coincidir y desaparece.

$$\begin{aligned} Q (n \times d_k) \cdot K^T (d_k \times n) &\rightarrow (n \times n) && \leftarrow \text{la matriz de atención: un peso por par de posiciones} \\ (n \times n) \cdot V (n \times d_v) &\rightarrow (n \times d_v) && \leftarrow \text{la salida, una fila por posición} \end{aligned}$$

Si las formas no cuadran, has entendido mal el mecanismo. Comprobar dimensiones antes de programar ahorra horas.

 **Error común:** confundir Wx con xW . El orden importa y determina qué dimensión se contrae. En los papers, mira siempre si los vectores son fila o columna.

5. Proyección y subespacios

Multi-cabeza (Po8) parte d_{model} en h trozos de d_{model}/h . Cada cabeza trabaja en un **subespacio** distinto: una proyección de menor dimensión donde puede especializarse.

$$\begin{aligned} d_{\text{model}} = 512, \quad h = 8 &\rightarrow d_k = 64 \text{ por cabeza} \\ \text{Parámetros de 8 cabezas de 64} &= 8 \cdot (512 \cdot 64) = 512 \cdot 512 \\ \text{Parámetros de 1 cabeza de 512} &= 512 \cdot 512 \\ &\quad \uparrow \text{el MISMO total} \end{aligned}$$

Por eso multi-cabeza no cuesta más parámetros: se reparte el mismo presupuesto en más subespacios más pequeños.

A02 · Probabilidad y verosimilitud

Casi toda la IA moderna optimiza lo mismo: **hacer más probable lo que ocurrió**. Cambia el objeto — el siguiente token, la respuesta preferida, el ruido añadido— pero no la maquinaria.

Usado en: P08 · P09 · P10 · P12 · P15 · P17 · P22

1. Softmax

$$\text{softmax}(z)_i = e^{z_i} / \sum_j e^{z_j}$$

Convierte cualquier vector de números reales en una **distribución de probabilidad**: todos los valores quedan en $(0,1)$ y suman 1.

Ejemplo resuelto. $z = (2, 1, 0)$:

$$e^2 = 7,389 \quad e^1 = 2,718 \quad e^0 = 1,000 \quad \text{suma} = 11,107$$
$$\text{softmax} = (0,665, 0,245, 0,090) \quad \text{suma} = 1,000 \checkmark$$

Dos propiedades que hay que conocer:

(a) Invariante ante desplazamientos. $\text{softmax}(z) = \text{softmax}(z - c)$ para cualquier c . Por eso se implementa restando el máximo: evita que e^{800} desborde sin cambiar el resultado.

(b) La escala lo cambia todo. Multiplicar z por un factor concentra o reparte la masa:

z escalado	resultado	interpretación
$(0,5 \cdot z)$	(0,506, 0,307, 0,186)	repartido
z	(0,665, 0,245, 0,090)	normal
$(4 \cdot z)$	(0,999, 0,0003, 0,0000)	saturado

Esta es exactamente la razón del `√d_k` del Transformer: sin él, los productos escalares crecen con la dimensión, el softmax se satura y **el gradiente de todo lo no seleccionado se anula**.

⚠ **Error común:** aplicar una máscara **después** del softmax poniendo pesos a cero. La distribución deja de sumar 1. La máscara va **antes**, poniendo los logits a $-\infty$.

2. Entropía

$$H(p) = - \sum_i p_i \cdot \log(p_i)$$

Mide **cuán repartida** está una distribución. Alta = indecisión; baja = concentración.

Ejemplo resuelto con 4 opciones:

$p = (0,25, 0,25, 0,25, 0,25) \rightarrow H = -4 \cdot (0,25 \cdot \log 0,25) = 1,386 \leftarrow \text{máxima (= log 4)}$
 $p = (0,97, 0,01, 0,01, 0,01) \rightarrow H \approx 0,169 \leftarrow \text{casi determinista}$

En el eje se usa para medir si la atención está **enfocada** o **repartida** (To3).

3. Verosimilitud y entropía cruzada

Entrenar un modelo de lenguaje es **maximizar la verosimilitud** de un texto que ya ocurrió:

$\text{maximizar } \prod_t p(x_t | x_{<t}) \Leftrightarrow \text{minimizar } - \sum_t \log p(x_t | x_{<t})$

Se usan logaritmos por dos razones prácticas: convierten productos en sumas (más estable numéricamente) y evitan que multiplicar miles de probabilidades pequeñas colapse a cero.

Ejemplo resuelto. El modelo asigna 0,7 al token correcto:

pérdida = $-\log(0,7) = 0,357$
Si asignara 0,99 $\rightarrow -\log(0,99) = 0,010$ (casi sin penalización)
Si asignara 0,01 $\rightarrow -\log(0,01) = 4,605$ (penalización enorme)

La forma de $-\log$ es lo que hace que el modelo **odie estar muy seguro y equivocado**.

⚠ **Error común:** comparar pérdidas entre modelos con tokenizadores distintos. La pérdida es por token; si un modelo parte las palabras en más trozos, sus números no son comparables.

4. Divergencia KL

$KL(p \parallel q) = \sum_i p_i \cdot \log(p_i / q_i)$

Mide **cuánto se aleja** una distribución q de otra p . Vale 0 solo si son idénticas, y es **asimétrica**: $KL(p \parallel q) \neq KL(q \parallel p)$.

Dónde aparece. En RLHF penaliza que la política se aleje del modelo base:

$\text{objetivo} = E[r(x, y)] - \beta \cdot KL(\pi \parallel \pi_{\text{ref}})$

Sin ese término, la política deriva hacia texto degenerado con recompensa alta. En DPO la misma restricción queda **absorbida** dentro del log-ratio $\beta \cdot \log(\pi / \pi_{\text{ref}})$.

⚠ **Error común:** tratar la KL como una distancia. No lo es: no es simétrica y no cumple la desigualdad triangular.

5. Bradley-Terry: aprender de comparaciones

$$p(a > b) = \sigma(r(a) - r(b)) \quad \text{con} \quad \sigma(z) = 1/(1+e^{-z})$$

Modela la probabilidad de que **a** se prefiera a **b** en función de la **diferencia** de sus puntuaciones. Solo importa la diferencia: la escala absoluta de **r** no está determinada.

Ejemplo resuelto. $r(a) = 2,0$, $r(b) = 0,5$:

$$p(a > b) = \sigma(1,5) = 1/(1+e^{-1,5}) = 0,818$$

Un 82 % de las veces se preferiría **a**. Si ambas puntuaran igual, saldría exactamente 0,5.

Por qué se piden comparaciones y no notas. Pedir «¿cuál prefieres?» es más consistente entre anotadores que pedir «puntuá del 1 al 10»: la escala numérica la interpreta cada persona a su manera y deriva con el tiempo.

6. Gaussianas y el proceso de difusión

$$N(x; \mu, \sigma^2) \quad \text{la campana: media } \mu, \text{ desviación } \sigma$$

Propiedad clave que usa P17: **la composición de gaussianas es gaussiana**. Por eso se puede saltar de x_0 a x_t en un paso sin simular los t intermedios:

$$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1-\bar{\alpha}_t)} \cdot \varepsilon, \quad \varepsilon \sim N(0, I)$$

Ejemplo resuelto. Con $\bar{\alpha}_t = 0,25$:

$$\begin{aligned} x_t &= 0,5 \cdot x_0 + 0,866 \cdot \varepsilon & (\sqrt{0,25} = 0,5; \quad \sqrt{0,75} = 0,866) \\ \text{SNR} &= \bar{\alpha}_t / (1-\bar{\alpha}_t) = 0,25/0,75 = 0,333 \end{aligned}$$

La señal ya vale un tercio del ruido. Y despejando la imagen original:

$$x_0 = (x_t - 0,866 \cdot \varepsilon) / 0,5 \quad \leftarrow \text{el factor } 1/0,5 = 2 \text{ AMPLIFICA cualquier error en } \varepsilon$$

Ese factor $1/\sqrt{\bar{\alpha}_t}$ crece según avanza t , y es la razón de que el muestreo vaya paso a paso.

A03 · Cálculo y gradientes

Un modelo aprende porque alguien sabe responder a esta pregunta: «**si muevo este peso un poquito, ¿cuánto cambia el error?**». Eso es una derivada, y nada más.

Usado en: P02 · P03 · P05 · P12 · P15

1. Derivada: la pregunta que resuelve

$$f'(x) = \lim_{h \rightarrow 0} (f(x+h) - f(x)) / h$$

En castellano: **cuánto cambia la salida por unidad de cambio en la entrada**. El signo indica la dirección; la magnitud, la sensibilidad.

Gradiente es lo mismo con varias variables: un vector con una derivada parcial por parámetro. Apunta en la dirección de **máximo crecimiento**, así que para minimizar se va en contra:

$$\theta \leftarrow \theta - \eta \cdot \partial L / \partial \theta \quad \eta = \text{tasa de aprendizaje}$$

Ejemplo resuelto. $L(x) = (x-3)^2$, empezando en $x = 8$ con $\eta = 0,1$:

$$\partial L / \partial x = 2(x-3)$$

$x=8 \rightarrow \text{grad} = 10 \rightarrow x \leftarrow 8 - 0,1 \cdot 10 = 7,0$	$L: 25 \rightarrow 16$
$x=7 \rightarrow \text{grad} = 8 \rightarrow x \leftarrow 7 - 0,1 \cdot 8 = 6,2$	$L: 16 \rightarrow 10,24$
$x=6,2 \rightarrow \text{grad} = 6,4 \rightarrow x \leftarrow 6,2 - 0,1 \cdot 6,4 = 5,56$	$L: 10,24 \rightarrow 6,55$

Converge hacia $x = 3$, donde el gradiente vale 0. Haz dos pasos más a mano.

 **Error común:** creer que gradiente 0 significa «óptimo global». Significa punto crítico: puede ser mínimo, máximo o punto de silla.

2. Regla de la cadena

Si $y = f(u)$ y $u = g(x)$, entonces:

$$dy/dx = (dy/du) \cdot (du/dx)$$

Las sensibilidades **se multiplican** a lo largo de la composición. Toda la retropropagación es esta regla aplicada con orden.

Ejemplo resuelto. $y = (3x + 1)^2$ en $x = 2$:

$$\begin{aligned}
 u &= 3x+1 = 7 & du/dx &= 3 \\
 y &= u^2 = 49 & dy/du &= 2u = 14 \\
 dy/dx &= 14 \cdot 3 = 42
 \end{aligned}$$

Comprobación numérica: $((3 \cdot 2,001+1)^2 - (3 \cdot 1,999+1)^2) / 0,002 = 42,0 \checkmark$

3. Retropropagación

Para la red $x \rightarrow h = \sigma(w_1x + b_1) \rightarrow o = \sigma(w_2h + b_2)$ con $L = (o - y)^2$:

$$\begin{aligned}
 \sigma(z) &= 1/(1+e^{-z}) & \sigma'(z) &= \sigma(z) \cdot (1 - \sigma(z)) \\
 \delta_o &= \partial L / \partial o_{in} = 2(o - y) \cdot \sigma'(o_{in}) \\
 \partial L / \partial w_2 &= \delta_o \cdot h \\
 \delta_h &= (w_2^T \delta_o) \odot \sigma'(h_{in}) & \leftarrow \text{el error VIAJA hacia atrás} \\
 \partial L / \partial w_1 &= \delta_h \cdot x
 \end{aligned}$$

Por qué es eficiente. Calcular cada derivada parcial por separado costaría una evaluación por parámetro. Al reutilizar los δ ya calculados, el coste total es del orden de una sola pasada hacia adelante — sea la red de 9 parámetros o de 9 000 millones.

4. Por qué el gradiente se desvanece

La sigmoide tiene un techo: $\sigma'(z) \leq 0,25$, y ese máximo solo se alcanza en $z = 0$.

Al encadenar capas, los factores se **multiplican**:

capas	factor acumulado (con $\sigma' \approx 0,25$)
1	0,25
5	0,00098
10	0,00000095
20	$\approx 9 \cdot 10^{-13}$

Las primeras capas dejan de recibir señal útil. Es exactamente el problema de Po3 trasladado al tiempo en vez de a la profundidad, y la razón de que existan la LSTM y las conexiones residuales.

La solución estructural en una línea:

Camino multiplicativo: $\partial h_T / \partial h_0 = \prod \sigma'(\cdot) \cdot W \rightarrow \text{colapsa o explota}$
Camino aditivo: $c_t = f \cdot c_{t-1} + \dots \rightarrow \partial c_t / \partial c_{t-1} = f \approx 1$

Sumar en vez de multiplicar. Esa es la idea que comparten la celda LSTM, las residuales de ResNet y cada bloque del Transformer.

5. Comprobación numérica del gradiente

La única forma de saber que tu derivación es correcta:

Diferencia centrada: $f'(x) \approx (f(x+\epsilon) - f(x-\epsilon)) / (2\epsilon)$

Ejemplo resuelto. $f(x) = (x-3)^2$ en $x = 2$, cuya derivada real es -2 :

ϵ	resultado	error
1e-1	-2,00000000	$\sim 1e-16$ (esta función es exacta con centrada)
1e-5	-2,00000000	$\sim 1e-11$
1e-12	-2,000178	$1,8e-4 \leftarrow$ peor : la precisión de coma flotante se come el resultado

Hay un punto óptimo: ϵ **demasiado grande** mide una secante en vez de la tangente; ϵ **demasiado pequeño** hace que $f(x+\epsilon) - f(x-\epsilon)$ sea ruido de redondeo.

Regla práctica: $\epsilon \approx 1e-5$, diferencia **centrada**, y se compara el error relativo:

```
error_relativo = |analítico - numérico| / max(|analítico|, |numérico|, 1e-8)
```

< 1e-7 $\rightarrow$ correcto
< 1e-4 $\rightarrow$ sospechoso, revisa
> 1e-2 $\rightarrow$ hay un error en la derivación

⚠ Error común: usar diferencia hacia adelante $(f(x+\epsilon) - f(x))/\epsilon$. Su error es $O(\epsilon)$ frente a $O(\epsilon^2)$ de la centrada: pierdes varios dígitos de precisión con el mismo número de evaluaciones.

6. Gradiente de política (REINFORCE)

Cuando la salida se **muestrea** en vez de calcularse —el caso de P12 y P22— no se puede derivar directamente a través del muestreo. Se usa:

$$\nabla_{\theta} E[r] = E[r \cdot \nabla_{\theta} \log \pi_{\theta}(a)]$$

En castellano: **sube la probabilidad de lo que salió bien y baja la de lo que salió mal**, proporcionalmente a la recompensa.

Línea base. Restar la recompensa media reduce muchísimo la varianza sin sesgar el estimador:

$$\nabla_{\theta} E[r] = E[(r - b) \cdot \nabla_{\theta} \log \pi_{\theta}(a)]$$

Sin línea base, si todas las recompensas son positivas, **todas** las acciones suben de probabilidad y solo el ruido decide cuál sube más. Con línea base, solo suben las que están por encima de la media.

A04 · La atención, paso a paso

La ecuación 1 del Transformer desarrollada **con números concretos**, para que deje de ser una fórmula y pase a ser un cálculo que puedes hacer a mano.

Usado en: Po7 · Po8 · To1–To8

La ecuación

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$$

Cuatro operaciones encadenadas. Vamos una por una con el mismo ejemplo.

El ejemplo que usaremos

Tres tokens, $d_k = d_v = 2$. Consulta = el primer token.

$$Q = \begin{bmatrix} 1,0 & 0,0 \end{bmatrix} \quad \leftarrow \text{una sola consulta}$$

$$K = \begin{bmatrix} 1,0 & 0,0 \\ 0,0 & 1,0 \\ 0,9 & 0,1 \end{bmatrix} \quad \begin{array}{l} \leftarrow \text{clave 0: idéntica a } Q \\ \leftarrow \text{clave 1: perpendicular a } Q \\ \leftarrow \text{clave 2: muy parecida a } Q \end{array}$$

$$V = \begin{bmatrix} 10,0 & 0,0 \\ 0,0 & 10,0 \\ 5,0 & 5,0 \end{bmatrix} \quad \begin{array}{l} \leftarrow \text{valor 0} \\ \leftarrow \text{valor 1} \\ \leftarrow \text{valor 2} \end{array}$$

Paso 1 — Compatibilidad: $Q \cdot K^T$

Producto escalar de la consulta con cada clave (ver A01):

$$\begin{aligned} Q \cdot K_0 &= 1,0 \cdot 1,0 + 0,0 \cdot 0,0 = 1,00 \\ Q \cdot K_1 &= 1,0 \cdot 0,0 + 0,0 \cdot 1,0 = 0,00 \\ Q \cdot K_2 &= 1,0 \cdot 0,9 + 0,0 \cdot 0,1 = 0,90 \end{aligned}$$

Los tres números $(1,00 \cdot 0,00 \cdot 0,90)$ dicen cuánto «le interesa» cada posición a la consulta. **Aún no son probabilidades:** pueden ser negativos y no suman nada en particular.

Paso 2 — Escala: $/ \sqrt{d_k}$

$$\sqrt{d_k} = \sqrt{2} = 1,414$$

$$\text{scores escalados} = (0,707, \quad 0,000, \quad 0,636)$$

¿Por qué dividir? Si q y k tienen componentes independientes de media 0 y varianza 1, entonces $q \cdot k$ tiene **varianza** d_k : su magnitud típica crece como $\sqrt{d_k}$. Dividir devuelve la varianza a 1.

Qué pasa si no se hace, con dimensiones realistas:

d_k	magnitud típica de $q \cdot k$	softmax resultante
2	$\sim 1,4$	repartido, sano
64	~ 8	ya concentrado
512	~ 23	saturado : un token se lleva $\sim 1,0$ y el resto ~ 0

Con el softmax saturado, el gradiente de todas las posiciones no seleccionadas es prácticamente cero: **la capa deja de aprender**.

Paso 3 — Normalizar: softmax

```
e^{0,707} = 2,028
e^{0,000} = 1,000
e^{0,636} = 1,889
                suma = 4,917

α = (0,412 , 0,203 , 0,384)          suma = 1,000 ✓
```

Tres propiedades que hay que verificar siempre:

- 1. Todos los pesos son **positivos**.
- 2. **Suman exactamente 1**.
- 3. El orden se conserva: la clave más compatible tiene el mayor peso.

 Si «normalizas» dividiendo por la suma de los scores en vez de con softmax, con scores (2, -1, 0,5) obtienes (1,33 , -0,67 , 0,33) : pesos **negativos** y mayores que 1. Deja de ser una distribución. Por eso es softmax y no una división.

Paso 4 — Mezclar: $\alpha \cdot v$

```
salida = 0,412 \cdot (10,0) + 0,203 \cdot (0,0) + 0,384 \cdot (5,0)
         0,412 \cdot (0,0) + 0,203 \cdot (10,0) + 0,384 \cdot (5,0)

= ( 4,12 + 0,00 + 1,92 , 0,00 + 2,03 + 1,92 )
= ( 6,04 , 3,95 )
```

Propiedad clave: como los pesos son positivos y suman 1, la salida es una **combinación convexa** de las filas de v . Nunca puede salirse de la envolvente de los valores.

La atención **mezcla**, no inventa. Si un dato no está en v , no puede aparecer en la salida.

La máscara causal

Para generar de izquierda a derecha, la posición i no puede mirar a $j > i$. Se aplica **antes** del softmax poniendo esos scores a $-\infty$:

```
scores = [ 0,71  -∞  -∞ ]      e^{-∞} = 0
          [ 0,32  0,67  -∞ ]
          [ 0,16  0,15  0,69 ]

α       = [ 1,000  0,000  0,000 ]  ← la posición 0 solo se ve a sí misma
          [ 0,327  0,673  0,000 ]
          [ 0,164  0,145  0,691 ]
          ↑ cada fila sigue sumando 1
```

Matriz **triangular inferior**, y la masa sobre el futuro es exactamente 0.

⚠ **Error común:** poner los pesos a cero *después* del softmax. La fila deja de sumar 1 y la mezcla queda mal escalada. Comprobación rápida: suma cada fila; si no da 1, la máscara está mal puesta.

Multi-cabeza

```
MultiHead(X) = Concat(head1, ..., headh) · WO
headi = Attention(X·WiQ, X·WiK, X·WiV),    dk = dmodel / h
```

Con $d_{\text{model}} = 512$ y $h = 8$, cada cabeza trabaja en 64 dimensiones. El presupuesto total de parámetros **no cambia** (ver A01 §5): lo que cambia es que hay ocho subespacios donde especializarse en vez de uno.

Autocomprobación

Repite el cálculo con $Q = [0, 0, 1, 0]$ (la consulta ahora apunta a la segunda dimensión) y verifica:

- [] Los tres productos escalares valen $(0,00 \cdot 1,00 \cdot 0,10)$.
- [] Tras escalar y aplicar softmax, los pesos suman 1.
- [] El peso mayor corresponde ahora a la **clave 1**.
- [] La salida está entre el mínimo y el máximo de cada columna de V .
- [] Con máscara causal en la primera fila, $\alpha = (1, 0, 0)$ sea cual sea Q .

Si las cinco te salen, entiendes la ecuación 1. Compruébalo ejecutando T02.

A05 · Complejidad, coste y escalado

La diferencia entre una idea que funciona y una que se despliega casi siempre es una cuenta de servilleta. Este anexo enseña a hacerla.

Usado en: P06 · P08 · P19 · P20 · P21 · P22

1. Notación O(): qué dice y qué no

$O(f(n))$ describe **cómo crece** el coste al crecer n , ignorando constantes.

$O(n)$

duplicar n duplica el coste

$O(n^2)$

duplicar n cuadruplica el coste

$O(n^3)$

duplicar n multiplica por 8

Lo que NO dice: cuál es más rápido *hoy, en tu máquina, con tu n* . Un algoritmo $O(n^2)$ con constante pequeña puede ganar a uno $O(n)$ con constante enorme durante todo el rango que te importa.

⚠

Error común: «es $O(n)$, luego es más rápido». Siempre hay que preguntar **a partir de qué n** .

2. El cruce: atención frente a recurrencia

Del Transformer, tabla 1:

Tipo de capa	Operaciones	Pasos secuenciales	Camino máximo
Self-attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrente	$O(n \cdot d^2)$	$O(n)$	$O(n)$

¿Cuándo cuesta más la atención?

$n^2 \cdot d > n \cdot d^2$

↔

$n > d$

Con $d = 512$, la atención es **más barata** en operaciones hasta secuencias de 512 tokens, y más cara por encima.

Pero las otras dos columnas no dependen de n : la atención siempre gana en paralelismo ($O(1)$ pasos secuenciales) y siempre gana en distancia entre posiciones ($O(1)$ frente a $O(n)$). Por eso se impuso aunque el cómputo crezca al cuadrado: **el hardware moderno paraleliza bien y espera mal**.

3. La cuenta que decide el hardware: memoria

Coste de la matriz de atención, en float32 (4 bytes), **por cabeza y por capa**:

memoria = n² · 4 bytes

n	matriz de atención	¿viable?
512	1 MB	trivial
2 048	16 MB	cómodo
8 192	268 MB	notable
32 768	4,3 GB	problemático
128 000	65,5 GB	inviable por cabeza y capa

Multiplica por el número de cabezas y de capas y verás por qué el contexto largo real **no** usa atención densa ingenua: usa variantes dispersas, con E/S optimizada o compresión.

Frente a eso, un SSM mantiene un estado de tamaño **d · N constante**: con d = 512 y N = 16 son 8 192 valores, valga la secuencia 1 000 o 1 000 000.

4. FLOPs de entrenamiento: la regla de 6ND

$C \approx 6 \cdot N \cdot D$

N = parámetros, D = tokens de entrenamiento

El 6 sale de contar, por parámetro y por token, las multiplicaciones-acumulaciones de la pasada hacia adelante (≈2) y hacia atrás (≈4).

Ejemplo resuelto. N = 7 · 10¹⁰, D = 1,4 · 10¹²:

$C \approx 6 \cdot 7 \cdot 10^{10} \cdot 1,4 \cdot 10^{12} \approx 5,9 \cdot 10^{23} \text{ FLOPs}$

Para dimensionar: una GPU de gama alta con ~10¹⁵ FLOP/s **efectivos** tardaría ~5,9 · 10⁸ s ≈ 19 años. Con 1 000 GPU al 40 % de utilización, unos 17 días. Ahí se ve por qué esto es una actividad industrial.

5. Escalado: dónde gastar el presupuesto

De Chinchilla:

$L(N, D) = E + A/N^\alpha + B/D^\beta$

sujeto a 6ND = C

- E es el **error irreducible**: no baja con cómputo.
- Los otros dos términos son lo que compras con parámetros y con datos.
- El óptimo iguala las contribuciones marginales de ambos → N y D deben crecer **a la vez**.

 **Error común:** citar «20 tokens por parámetro» como constante universal. Es una razón derivada de

exponentes ajustados en un rango concreto y una arquitectura concreta.

6. La cuenta que casi nadie hace: inferencia

Coste de entrenamiento (una vez): $C_{\text{train}} \approx 6 \cdot N \cdot D$
Coste de inferencia (cada petición): $C_{\text{inf}} \approx 2 \cdot N \cdot \text{tokens_generados}$

Ejemplo resuelto. $N = 7 \cdot 10^{10}$, $D = 1,4 \cdot 10^{12}$, respuestas de 500 tokens:

peticiones	coste de inferencia	¿domina?
10^6	$7,0 \cdot 10^{19}$	no (entrenamiento: $5,9 \cdot 10^{23}$)
10^9	$7,0 \cdot 10^{22}$	todavía no
10^{12}	$7,0 \cdot 10^{25}$	sí, 100× el entrenamiento

Consecuencia de diseño: si vas a servir a gran escala, conviene entrenar un modelo **más pequeño que el óptimo de Chinchilla** y darle aún más datos. El óptimo de entrenamiento y el de despliegue son problemas distintos.

7. Dispersión: cuando los parámetros dejan de ser uno

En una mezcla de expertos, N deja de ser un solo número:

$N_{\text{totales}} = 47 \cdot 10^9$ ← determina la MEMORIA necesaria
 $N_{\text{activos}} = 13 \cdot 10^9$ ← determina el CÓMPUTO por token

Memoria de pesos en fp16 = $N_{\text{totales}} \cdot 2 \text{ bytes} = 94 \text{ GB}$
Cómputo por token $\propto N_{\text{activos}} = 28 \%$ del denso equivalente

⚠ **El error más caro de esta arquitectura:** dimensionar la GPU por los parámetros **activos**. Hay que cargar todos los expertos, porque cualquier token puede activar cualquiera. MoE ahorra cómputo, **no** memoria.

8. Checklist antes de crearte una cifra de rendimiento

Cuando alguien diga «nuestro modelo es más eficiente», pregunta:

- [] ¿Eficiente en **qué**: FLOPs, memoria, latencia, coste en dinero, energía?
- [] ¿A qué n (longitud de secuencia)? ¿Dónde está el cruce?
- [] ¿En **entrenamiento** o en **inferencia**? No se optimizan igual.
- [] ¿Con qué **lote**? Muchas ventajas desaparecen con lote 1.
- [] ¿Se cuenta la **memoria de pesos** o solo la activa?
- [] ¿Con qué **hardware** y qué utilización efectiva?

- [] ¿Se compara contra una línea base **igual de optimizada**?

Las tres últimas son las que más veces faltan.

P01 — El perceptrón

El momento en que una máquina deja de ejecutar reglas escritas por una persona y empieza a ajustar sus propios parámetros a partir de ejemplos.

Nivel: L1 · Motor: perceptron · Notebook: P01_perceptron.ipynb

1. Identificación

Campo	Valor
Título original	<i>The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain</i>
Autoría	Frank Rosenblatt
Año	1958
Venue	<i>Psychological Review</i> , 65(6), 386–408
Fuente primaria	doi.org/10.1037/h0042519
Acceso	Restringido (revista de suscripción)
Fecha de consulta	2026-08-16

2. Problema anterior

En los años 50 un programa hacía exactamente lo que su autor había escrito. McCulloch y Pitts (1943) habían mostrado que una neurona de umbral podía computar funciones lógicas, pero sus pesos los fijaba el diseñador: la red no cambiaba con la experiencia. Hebb (1949) había propuesto que el aprendizaje biológico era plasticidad sináptica, pero como principio, no como algoritmo.

Faltaba lo del medio: **un procedimiento concreto y ejecutable para que un sistema modifique su comportamiento observando datos etiquetados**. Sin eso, «aprender» era una metáfora.

3. Propuesta

Rosenblatt propone una unidad que:

1. recibe entradas `x`, las pondera con `w` y suma un sesgo `b` ;
2. decide con una función escalón: positivo → clase 1, negativo → clase 0;
3. y —la parte nueva— **corrige `w` y `b` solo cuando se equivoca**, empujando el hiperplano hacia el ejemplo mal clasificado.

Lo enmarca como modelo probabilístico de almacenamiento de información en el cerebro, no como herramienta de ingeniería. El Mark I Perceptron lo implementó en hardware con conexiones ajustables físicamente.

4. Intuición sin fórmulas

Imagina puntos de dos colores sobre una hoja y una regla que intenta separarlos. Cada vez que un punto queda del lado equivocado, empujas la regla un poco hacia él. Si existe alguna posición de la regla que los separe, este procedimiento la encuentra.

Dónde deja de funcionar la analogía: si no existe ninguna recta que los separe, la regla no se «acerca poco a poco» a una solución aproximada — oscila indefinidamente. No hay degradación elegante.

5. Matemática mínima

$$z = w \cdot x + b = \sum_i w_i x_i + b$$

$$\hat{y} = 1 \quad \text{si } z \geq 0$$

$$\hat{y} = 0 \quad \text{si } z < 0$$

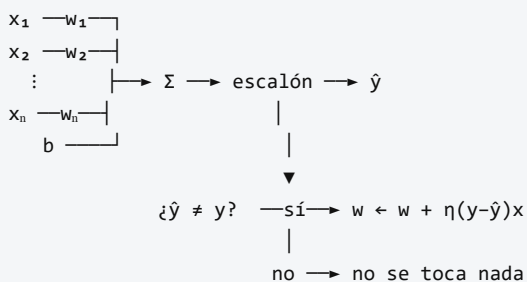
Regla de actualización (solo si $\hat{y} \neq y$):

$$w \leftarrow w + \eta(y - \hat{y})x$$

$$b \leftarrow b + \eta(y - \hat{y})$$

- $x \in \mathbb{R}^n$ entrada · $w \in \mathbb{R}^n$ pesos · $b \in \mathbb{R}$ sesgo · $\eta > 0$ tasa de aprendizaje.
- $w \cdot x + b = 0$ define un **hiperplano**: la frontera de decisión.
- **Teorema de convergencia** (Novikoff, 1962 — posterior al paper): si los datos son linealmente separables con margen $\gamma > 0$ y radio R , el algoritmo converge en a lo sumo $(R/\gamma)^2$ correcciones.

6. Arquitectura o flujo



La última línea es la esencia del paper: **sin error no hay aprendizaje**.

7. Qué observar en el paper original

- La **motivación biológica**: Rosenblatt escribe para psicólogos, no para ingenieros. El vocabulario es de sistemas nerviosos, no de optimización.
- El planteamiento **probabilístico** de la conectividad, hoy poco recordado frente a la versión algorítmica simplificada que circula en los manuales.
- La ausencia de: función de pérdida diferenciable, descenso de gradiente y capas ocultas entrenables. Ninguna de esas ideas está en este trabajo.

8. Evidencia y resultados

El artículo es teórico y conceptual, acompañado del programa experimental del Mark I. La demostración formal de convergencia para datos separables es **posterior** (Novikoff, 1962; Block, 1962).

Verificar en la fuente primaria antes de citar cualquier cifra de desempeño: este eje no reproduce números de los experimentos originales de 1958 porque no fueron formulados con el protocolo de tarea/dataset/métrica/línea base que hoy se exige.

La miniatura de este eje sí produce evidencia verificable, con otro alcance: AND converge en 6 épocas con 11 correcciones; XOR no converge en 200 épocas.

9. Impacto

- Inaugura el **conexionismo** como programa de investigación.
- Es el ancestro directo de toda neurona artificial posterior: la forma $\sigma(w \cdot x + b)$ es la misma en un MLP, en una CNN y en la FFN de un Transformer.
- Su límite provocó el primer invierno de las redes neuronales tras el análisis de Minsky y Papert (1969) — un impacto negativo que resultó igual de determinante que el positivo.

Cuidado con la narrativa retrospectiva: la historia habitual («Minsky mató las redes neuronales») es una simplificación. El libro de 1969 es un análisis matemático riguroso de qué puede y qué no puede computar un perceptrón; la interpretación institucional que se hizo de él es otra cosa.

10. Limitaciones

1. **Solo fronteras lineales.** Si las clases no son linealmente separables, no hay solución dentro de la clase de hipótesis. XOR es el contraejemplo mínimo.
2. **No degrada con elegancia.** Con datos no separables no converge a una «mejor aproximación»: oscila. No devuelve nada útil.
3. **Sin noción de margen.** Cualquier hiperplano que separe le vale; no busca el mejor. Eso lo resolverán las SVM tres décadas después.
4. **Sin probabilidades.** La salida es 0 o 1: no expresa confianza.
5. **Sensible a la escala** de las características y al orden de presentación de los ejemplos.
6. **Sin capas ocultas entrenables.** El paper no ofrece forma de entrenar representaciones intermedias.

11. Errores comunes

Error	Corrección
«El perceptrón usa descenso de gradiente»	Usa una regla de corrección de error . La función escalón no es diferenciable.
«Minsky y Papert demostraron que las redes neuronales no sirven»	Demostraron límites del perceptrón de una capa . El propio libro discute redes multicapa.
«El teorema de convergencia está en el paper de 1958»	Es de Novikoff y Block, 1962.
«El perceptrón fracasó»	Funcionó exactamente como su teoría predecía. Lo que falló fueron las expectativas construidas sobre él.
«No converge porque le faltan épocas»	Con XOR no existe solución que alcanzar. Es un límite de capacidad, no de presupuesto.

12. Relación con trabajos anteriores

- **McCulloch y Pitts (1943)** — neurona de umbral con pesos fijos. Rosenblatt añade el aprendizaje.
- **Hebb (1949)** — la plasticidad sináptica como mecanismo de aprendizaje. Rosenblatt la convierte en un algoritmo con señal de error supervisada.

13. Relación con trabajos posteriores

- **Minsky y Papert (1969)** — *Perceptrons*: formalizan qué no puede computar.
- **Novikoff (1962)** — teorema de convergencia con margen.
- **Po2 Backpropagation (1986)** — resuelve el límite entrenando capas ocultas.
- **Vapnik y colaboradores (1992–1995)** — SVM: separadores lineales con margen máximo y kernels, otra respuesta al mismo límite.

14. Notebook asociado

P01_perceptron.ipynb

Qué implementa: la regla de aprendizaje original en 20 líneas, sobre AND y XOR, con historial de errores por época.

Qué NO implementa: el modelo probabilístico de conectividad del paper, el hardware Mark I, ni ningún experimento de percepción visual.

```
ai-evolution paper-lab P01 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la regla de actualización y define cada símbolo sin mirar.
Explicar	Explica por qué la regla no hace nada cuando la predicción es correcta, y qué consecuencia tiene.
Aplicar	Ejecuta el perceptrón sobre OR y NAND. Anota épocas hasta converger.
Analizar	Demuestra algebraicamente que XOR no es linealmente separable (4 desigualdades).
Evaluar	Un compañero afirma: «con más épocas, XOR converge». Refuta con evidencia experimental y con el argumento algebraico.
Crear	Diseña un conjunto de datos 2D donde el perceptrón converja muy lentamente sin dejar de ser separable, y explica qué propiedad geométrica lo provoca.

16. Autoevaluación

1. ¿Por qué la función escalón impide usar descenso de gradiente?
2. Con $w = (0, 0)$, $b = 0$ y $\eta = 1$, ejecuta a mano dos épocas de AND. ¿Qué valores obtienes?
3. ¿Qué garantiza exactamente el teorema de convergencia, y bajo qué condición?
4. ¿Qué le ocurre a la cota $(R/\gamma)^2$ cuando el margen γ tiende a cero? ¿Qué significa eso en la práctica?
5. Si añades la característica $x_3 = x_1 \cdot x_2$ a XOR, ¿se vuelve separable? ¿Qué te dice eso sobre la relación entre representación y separabilidad?
6. Nombra dos límites del perceptrón que **no** sean «no resuelve XOR».
7. ¿Qué idea que hoy se asocia al perceptrón apareció en realidad después de 1958?

17. Respuestas esperadas

1. La derivada del escalón es 0 en todas partes salvo en el salto, donde no existe. Sin gradiente informativo, no hay dirección de descenso.
2. Debe llegar a valores intermedios y converger hacia $w = (2, 1)$, $b = -3$ alrededor de la sexta época. Lo evaluable es el **procedimiento** paso a paso, no el número final.
3. Que si existe un hiperplano separador con margen $\gamma > 0$, el algoritmo hace un número **finito** de correcciones, acotado por $(R/\gamma)^2$. No garantiza nada si no hay separabilidad.
4. La cota diverge. Datos casi colineales o casi solapados pueden necesitar un número enorme de correcciones aun siendo técnicamente separables: la garantía existe pero es inútil.
5. Sí. Muestra que la separabilidad no es propiedad de los datos sino del **espacio de representación**. Es la idea que fundamenta tanto los kernels como las capas ocultas.
6. Se aceptan: ausencia de margen máximo, salida no probabilística, no degradación elegante, sensibilidad a la escala, dependencia del orden de presentación.
7. El teorema de convergencia (1962), el descenso de gradiente sobre pérdidas diferenciables, la idea de margen máximo y cualquier mención a capas ocultas entrenables.

18. Fuentes primarias

- Rosenblatt, F. (1958). *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain*. **Psychological Review**, 65(6), 386–408. doi.org/10.1037/h0042519 · consultado 2026-08-16.

- Minsky, M. y Papert, S. (1969). *Perceptrons*. MIT Press. [ficha del editor](#) · consultado 2026-08-16.
- McCulloch, W. y Pitts, W. (1943). *A Logical Calculus of the Ideas Immanent in Nervous Activity*. **Bulletin of Mathematical Biophysics**, 5, 115–133. doi.org/10.1007/BF02478259 · consultado 2026-08-16.

Evaluación — P01

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *The Perceptron: A Probabilistic Model for Information Storage and Organization in the Brain* (1958, Psychological Review, 65(6), 386–408) · **Nivel:** L1

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P01_perceptron.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P02 — Backpropagation

El procedimiento que hizo entrenables las capas ocultas: la red deja de recibir las representaciones y empieza a descubrirlas.

Nivel: L2 · Motor: `backprop` · Notebook: `P02_backpropagation.ipynb`

1. Identificación

Campo	Valor
Título original	<i>Learning representations by back-propagating errors</i>
Autoría	David E. Rumelhart, Geoffrey E. Hinton, Ronald J. Williams
Año	1986
Venue	<i>Nature</i> , 323, 533–536
Fuente primaria	doi.org/10.1038/323533a0
Acceso	Restringido (revista de suscripción)
Fecha de consulta	2026-08-16

2. Problema anterior

El perceptrón solo traza fronteras lineales. La solución evidente —apilar capas— chocaba con un obstáculo práctico: **¿cuánta culpa del error final tiene un peso de la capa intermedia?** Ese peso no está conectado directamente con la salida; su efecto pasa por todas las unidades que vienen después.

Se le llamó el *credit assignment problem*. Sin resolverlo, las capas ocultas eran una idea sin algoritmo: se podían dibujar, no entrenar.

3. Propuesta

Aplicar la **regla de la cadena** recorriendo la red hacia atrás:

- propagación hacia adelante: se calcula la salida y la pérdida;
- propagación hacia atrás: el error de cada capa se expresa en función del error de la siguiente, multiplicando por los pesos y por la derivada de la activación;
- actualización: cada peso se mueve en la dirección opuesta a su gradiente.

La aportación decisiva del artículo no es la fórmula —ya existía en otras formulaciones— sino **mostrar que funciona en la práctica y que las unidades ocultas aprenden representaciones internas útiles y no diseñadas por nadie**. El título lo dice: *learning representations*.

4. Intuición sin fórmulas

Una cadena de montaje produce una pieza defectuosa. El responsable de calidad no reparte la culpa al azar: retrocede puesto por puesto preguntando «¿cuánto habría cambiado el defecto si tú hubieras hecho tu parte un poco distinta?». Cada puesto recibe una responsabilidad proporcional a su influencia real.

Dónde deja de funcionar la analogía: la cadena de montaje tiene un número fijo de puestos con influencia comparable. En una red profunda, la influencia se multiplica en cada paso, y ese producto puede colapsar a cero o dispararse — el problema que Po3 tendrá que atacar.

5. Matemática mínima

Red $x \rightarrow h = \sigma(w_1x + b_1) \rightarrow o = \sigma(w_2h + b_2)$, pérdida $L = (o - y)^2$, con $\sigma(z) = 1/(1+e^{-z})$ y $\sigma'(z) = \sigma(z)(1 - \sigma(z))$:

$$\delta_o = \partial L / \partial o_{in} = 2(o - y) \cdot \sigma'(o_{in})$$

$$\partial L / \partial w_2 = \delta_o \cdot h^T$$

$$\partial L / \partial b_2 = \delta_o$$

$$\delta_h = (w_2^T \delta_o) \odot \sigma'(h_{in}) \quad \leftarrow \text{el error "viaja" hacia atrás}$$

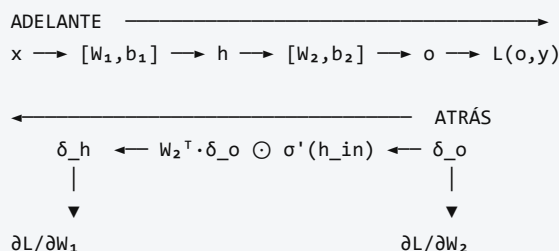
$$\partial L / \partial w_1 = \delta_h \cdot x^T$$

$$\partial L / \partial b_1 = \delta_h$$

$$\text{Actualización: } \theta \leftarrow \theta - \eta \cdot \partial L / \partial \theta$$

El único ingrediente nuevo respecto al cálculo elemental es el orden: computar δ de atrás hacia adelante evita recalcular derivadas repetidas. Eso es lo que lo hace viable.

6. Arquitectura o flujo



7. Qué observar en el paper original

- El artículo de *Nature* es **muy breve** (4 páginas). La formulación extendida está en el capítulo 8 de *Parallel Distributed Processing* (Rumelhart, Hinton y Williams, 1986).
- El experimento de las **representaciones internas**: la red descubre codificaciones en las unidades ocultas que corresponden a regularidades del dominio (por ejemplo, simetría o relaciones familiares) sin que se las especifique.
- Lo que **no** aparece: ReLU, dropout, normalización por lotes, Adam, inicialización cuidadosa, GPU. Todo eso es posterior y es lo que hizo escalar el método décadas después.

8. Evidencia y resultados

El paper demuestra, con problemas pequeños, que una red multicapa entrenada así:

- resuelve tareas no linealmente separables;
- **desarrolla representaciones internas interpretables** en las unidades ocultas.

Los problemas del artículo son de juguete para los estándares actuales; su valor es de **existencia** («esto se puede entrenar»), no de escala. Verificar las figuras concretas en la fuente primaria antes de citar cualquier resultado numérico.

La miniatura de este eje produce evidencia complementaria y verificable: XOR resuelto con 9 parámetros, y comprobación numérica del gradiente con error $\approx 1e-8$.

9. Impacto

- Desbloquea las redes multicapa y cierra el invierno abierto por el análisis de 1969.
- Es el algoritmo que **sigue entrenando todo hoy**: cada modelo del resto de esta ruta — LSTM, AlexNet, Transformer, GPT — se entrena con retropropagación.
- Establece la idea de *representación aprendida*, que es el concepto central del deep learning y da nombre a una conferencia entera (ICLR).

10. Limitaciones

1. **Gradiente desvaneciente o explosivo.** Al encadenar muchas capas, el producto de derivadas colapsa o diverge. El paper no lo aborda; será el tema de PO3.
2. **Óptimos locales y mesetas.** No hay garantía de convergencia al mínimo global.
3. **Coste de memoria.** Hay que guardar las activaciones de la pasada hacia adelante.
4. **Sensibilidad a la inicialización y a la tasa de aprendizaje.** Con sigmoides y pesos mal escalados, el entrenamiento se estanca en la zona de saturación.
5. **Poca plausibilidad biológica.** El cerebro no dispone de un canal simétrico que transporte el error hacia atrás con los mismos pesos (*weight transport problem*).
6. **Requiere activaciones diferenciables**, lo que excluye la función escalón de PO1.

11. Errores comunes

Error	Corrección
«Rumelhart, Hinton y Williams inventaron la retropropagación»	La popularizaron para redes neuronales . La diferenciación en modo reverso es anterior: Linnainmaa (1970), Werbos (1974). El propio artículo reconoce trabajo previo.
«Backpropagation es un algoritmo de optimización»	Es un algoritmo para calcular gradientes . Quien optimiza es SGD, Adam u otro.
«Backprop garantiza encontrar la mejor solución»	Encuentra un punto donde el gradiente se anula, no el óptimo global.
«El paper usa ReLU»	No. Usa sigmoides. ReLU se generaliza a partir de 2010–2012.
«Basta con más capas»	Sin las técnicas posteriores (inicialización, normalización, residuales), más capas empeoran el entrenamiento.

12. Relación con trabajos anteriores

- **P01 Perceptrón (1958)** — el límite lineal que motiva todo.
- **Linnainmaa (1970)** — diferenciación automática en modo reverso. doi.org/10.1007/BF01931367
- **Werbos (1974)** — tesis doctoral que aplica la idea a modelos de ciencias sociales.
- **Minsky y Papert (1969)** — el análisis que definió el problema a superar.

13. Relación con trabajos posteriores

- **P03 LSTM (1997)** — ataca el gradiente desvaneciente en secuencias.
- **P04 AlexNet (2012)** — demuestra que el método escala con GPU, ReLU, dropout y datos masivos.
- **Glorot y Bengio (2010), He et al. (2015)** — inicialización que evita la saturación.
- **Autograd moderno** (Theano, TensorFlow, PyTorch, JAX) — la generalización del método a grafos de cómputo arbitrarios.

14. Notebook asociado

P02_backpropagation.ipynb

Qué implementa: una red 2-2-1 con sigmoides entrenada sobre XOR, con gradientes derivados **a mano** (sin autograd) y verificación numérica del gradiente.

Qué NO implementa: el experimento de representaciones internas del paper, ni ninguna red de tamaño realista.

```
ai-evolution paper-lab P02 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la expresión de δ_h en función de δ_o y explica cada factor.
Explicar	Explica por qué se calcula δ de atrás hacia adelante y no al revés.
Aplicar	Cambia la sigmoide por <code>tanh</code> en el notebook y ajusta la derivada correspondiente.
Analizar	Calcula el producto de derivadas de sigmoide en 10 capas suponiendo $\sigma' \approx 0,25$. ¿Qué queda del gradiente?
Evaluar	Un modelo entrena y la pérdida baja, pero la verificación numérica del gradiente da <code>1e-2</code> de diferencia. ¿Confías en el entrenamiento? Justifica.
Crear	Implementa la comprobación de gradiente como una función reutilizable que reciba cualquier red y devuelva el error relativo máximo.

16. Autoevaluación

1. ¿Por qué la retropropagación es eficiente frente a calcular cada derivada parcial por separado?
2. ¿Qué papel juega $\sigma'(h_{in})$ en la propagación del error hacia atrás?
3. ¿Por qué $\sigma' \leq 0,25$ es una mala noticia en redes profundas?
4. ¿Qué diferencia hay entre backpropagation y descenso de gradiente estocástico?
5. ¿Por qué la verificación numérica del gradiente usa diferencia centrada y no hacia adelante?
6. ¿Qué ocurre si inicializas todos los pesos a cero? ¿Y todos al mismo valor no nulo?
7. ¿Qué idea del deep learning moderno **no** está en este paper y suele atribuírsele?

17. Respuestas esperadas

1. Porque reutiliza los δ ya calculados: el coste es proporcional al de una pasada hacia adelante, en lugar de una evaluación por parámetro.
2. Mide la sensibilidad local de la unidad. Si la neurona está saturada, $\sigma' \approx 0$ y el error deja de propagarse por esa ruta.
3. Porque los factores se multiplican: $0,25^{10} \approx 1e-6$. Las capas iniciales reciben una señal despreciable y dejan de aprender.
4. Backprop **calcula** el gradiente; SGD **usa** ese gradiente para actualizar. Son piezas distintas y se pueden sustituir por separado.
5. Porque su error es $O(\epsilon^2)$ frente a $O(\epsilon)$ de la diferencia hacia adelante: da varios dígitos más de precisión con el mismo número de evaluaciones.
6. Con todos a cero (o todos iguales), todas las unidades ocultas reciben el mismo gradiente y permanecen idénticas para siempre: la simetría no se rompe y la capa oculta es inútil.
7. ReLU, dropout, normalización por lotes, Adam, conexiones residuales, GPU. Todo posterior.

18. Fuentes primarias

- Rumelhart, D. E., Hinton, G. E. y Williams, R. J. (1986). *Learning representations by back-propagating errors*. **Nature**, 323, 533–536. doi.org/10.1038/323533a0 · consultado 2026-08-16.
- Linnainmaa, S. (1976). *Taylor expansion of the accumulated rounding error*. **BIT Numerical Mathematics**, 16, 146–160. doi.org/10.1007/BF01931367 · consultado 2026-08-16.

- Werbos, P. (1974). *Beyond Regression: New Tools for Prediction and Analysis in the Behavioral Sciences*. Tesis doctoral, Universidad de Harvard.

Evaluación — P02

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Learning representations by back-propagating errors* (1986, Nature, 323, 533–536) · **Nivel:** L2

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P02_backpropagation.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P03 — LSTM

La arquitectura que convirtió «recordar durante mucho tiempo» en un problema de ingeniería resoluble, sustituyendo una multiplicación encadenada por una suma con compuertas.

Nivel: L2 · Motor: lstm · Notebook: P03_lstm.ipynb

1. Identificación

Campo	Valor
Título original	Long Short-Term Memory
Autoría	Sepp Hochreiter, Jürgen Schmidhuber
Año	1997
Venue	Neural Computation, 9(8), 1735–1780
Fuente primaria	doi.org/10.1162/neco.1997.9.8.1735
Acceso	Restringido
Fecha de consulta	2026-08-16

2. Problema anterior

Una red recurrente procesa una secuencia reutilizando el mismo estado. Para entrenarla se despliega en el tiempo y se aplica retropropagación: el gradiente atraviesa un paso por cada instante.

Ahí está el problema. En cada paso el gradiente se multiplica por (aproximadamente) $w \cdot \sigma'$. Si ese factor es menor que 1, el producto colapsa exponencialmente; si es mayor, explota. Hochreiter lo había diagnosticado formalmente en su tesis de 1991. La consecuencia práctica: **una RNN clásica no aprende dependencias separadas por más de unas decenas de pasos**, y no por falta de datos ni de capacidad, sino por aritmética.

3. Propuesta

Introducir una **celda de memoria** cuyo estado se actualiza **sumando**, no aplicando una transformación no lineal encadenada. Esa ruta aditiva es el *constant error carousel* (CEC): por ella el gradiente viaja sin multiplicarse por derivadas de activación.

Alrededor del CEC se colocan **compuertas multiplicativas** que aprenden qué información entra (**i**) y qué información se expone al resto de la red (**o**). Las compuertas protegen la celda de escrituras y lecturas irrelevantes.

Precisión histórica obligatoria: la versión de 1997 tiene compuertas de **entrada** y **salida**. La compuerta de **olvido** —la que hoy aparece en todos los diagramas— la añadieron Gers, Schmidhuber y Cummins (1999/2000).

4. Intuición sin fórmulas

Una cinta transportadora que atraviesa toda la fábrica. Los operarios pueden depositar cosas encima (compuerta de entrada) y consultar lo que lleva (compuerta de salida), pero la cinta avanza sin que su contenido se transforme por el camino. Lo que subió al principio puede seguir intacto al final.

Dónde deja de funcionar la analogía: la cinta real no borra nada; la LSTM moderna sí, y lo hace con una compuerta aprendida. Y si esa compuerta aprende a cerrarse, el contenido se desvanece igual que en una RNN.

5. Matemática mínima

Formulación moderna (con compuerta de olvido, posterior al paper):

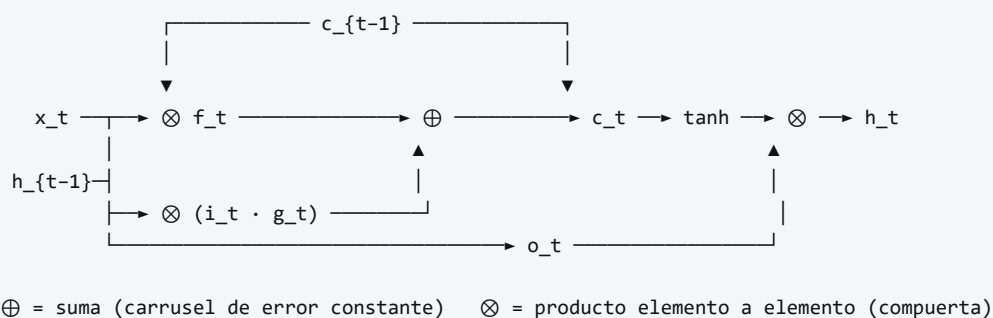
$f_t = \sigma(W_f \cdot [h_{t-1}, x_t] + b_f)$	olvido
$i_t = \sigma(W_i \cdot [h_{t-1}, x_t] + b_i)$	entrada
$o_t = \sigma(W_o \cdot [h_{t-1}, x_t] + b_o)$	salida
$g_t = \tanh(W_g \cdot [h_{t-1}, x_t] + b_g)$	candidato
$c_t = f_t \odot c_{t-1} + i_t \odot g_t$ ← SUMA: aquí está el CEC	
$h_t = o_t \odot \tanh(c_t)$	

La clave está en la penúltima línea:

$$\partial c_t / \partial c_{t-1} = f_t$$

Con $f_t \approx 1$ durante T pasos: $\partial c_T / \partial c_0 \approx 1$
Con una RNN tanh y factor 0,4: $0,4^{40} \approx 1,2 \cdot 10^{-16}$

6. Arquitectura o flujo



7. Qué observar en el paper original

- El artículo es **largo** (46 páginas) y buena parte es el **análisis del flujo de error**: ahí está el argumento, no en la arquitectura.
- La justificación de por qué las compuertas deben ser multiplicativas y el estado aditivo.
- Los experimentos con **problemas sintéticos de dependencia larga** (retardos de cientos de pasos) diseñados para que ninguna RNN previa pudiera resolverlos.
- Comprueba tú mismo: busca la palabra «forget gate» en el artículo de 1997. No está.

8. Evidencia y resultados

El paper compara la LSTM con RNN clásicas, BPTT truncado y otras alternativas de la época sobre tareas construidas para exigir memoria a largo plazo. La LSTM resuelve retardos con los que las alternativas fracasan.

Los detalles de tareas, retardos y tasas de éxito están en la sección de experimentos del artículo. Este eje no reproduce esas cifras: verificarlas en la fuente antes de citarlas.

La miniatura de este eje aporta la evidencia del mecanismo: tras 40 pasos, un factor de decaimiento $0,378$ deja el gradiente en $\approx 10^{-17}$, mientras que una compuerta de olvido en $0,98$ lo deja en $\approx 0,45$.

9. Impacto

- Durante casi dos décadas fue la arquitectura por defecto para secuencias: reconocimiento de voz, escritura manuscrita, traducción, series temporales.
- Hizo posible Seq2Seq y, con él, la traducción automática neuronal — el problema que llevó a la atención y al Transformer.
- Instaló una idea que sobrevive a la arquitectura: **las rutas aditivas preservan el gradiente**. Es exactamente el mismo principio que las conexiones residuales de ResNet y de cada bloque Transformer.

10. Limitaciones

1. **Sigue siendo secuencial**. El paso t necesita el $t-1$: no se puede paralelizar sobre la longitud. Este es el límite que motiva Po8.
2. **No elimina el gradiente desvaneciente; lo pone bajo control de una compuerta aprendida**. Si f aprende valores bajos, el desvanecimiento vuelve.
3. **Cuatro veces más parámetros** que una RNN simple del mismo tamaño de estado.
4. **Sigue comprimiendo** toda la historia en un estado de tamaño fijo.
5. **Difícil de interpretar**: qué guarda cada dimensión de la celda no es legible.
6. **La versión de 1997 carece de compuerta de olvido**, y sin ella el estado de celda puede crecer sin límite en secuencias largas — motivo por el que se añadió después.

11. Errores comunes

Error	Corrección
«La LSTM de 1997 tiene compuerta de olvido»	No. Gers, Schmidhuber y Cummins (1999/2000). Es el anacronismo más repetido del campo.
«La LSTM resuelve el gradiente desvaneciente»	Lo mitiga cuando la compuerta de olvido se mantiene cerca de 1 en el intervalo relevante.
«La GRU es una LSTM simplificada del mismo paper»	La GRU es de Cho et al. (2014), 17 años después.
«Las LSTM quedaron obsoletas»	Siguen siendo competitivas en series temporales, dispositivos con poca memoria y latencia baja. «Obsoleto» es una afirmación sobre un contexto, no sobre una arquitectura.
«El estado de celda y el estado oculto son lo mismo»	<code>c_t</code> es la memoria protegida; <code>h_t</code> es lo que la celda expone , filtrado por <code>o_t</code> .

12. Relación con trabajos anteriores

- **Po2 Backpropagation (1986)** — el método de entrenamiento cuyo problema en el tiempo se ataca aquí.
- **Elman (1990)** — redes recurrentes simples.
- **Hochreiter (1991)** — tesis que diagnostica formalmente el gradiente desvaneciente.
- **Werbos (1990)** — retropropagación en el tiempo (BPTT).

13. Relación con trabajos posteriores

- **Gers, Schmidhuber y Cummins (2000)** — compuerta de olvido.
doi.org/10.1162/089976600300015015
- **Cho et al. (2014)** — GRU, versión con menos compuertas. [arXiv:1406.1078](https://arxiv.org/abs/1406.1078)
- **Po6 Seq2Seq (2014)** — dos LSTM encadenadas para traducir.
- **He et al. (2015), ResNet** — el mismo principio aditivo aplicado a la profundidad.
- **Po8 Transformer (2017)** — elimina la recurrencia entera.

14. Notebook asociado

P03_lstm.ipynb

Qué implementa: una pasada completa de la celda con valores explícitos de las cuatro compuertas, y la comparación cuantitativa del decaimiento del gradiente entre una RNN tanh y la ruta aditiva de la celda.

Qué NO implementa: entrenamiento real de una LSTM, BPTT, ni las tareas sintéticas del paper.

```
ai-evolution paper-lab P03 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Enumera las cuatro compuertas de la LSTM moderna y di cuáles estaban en 1997.
Explicar	Explica por qué una suma preserva el gradiente y un producto encadenado no.
Aplicar	Calcula el estado de celda tras 3 pasos con $f = 0,9$, $i = 0,5$ y $g = 1$ constantes.
Analizar	¿Cuál es el valor mínimo de f que conserva el 1 % del gradiente tras 100 pasos? Resuélvelo analíticamente y compruébalo.
Evaluar	Un artículo divulgativo dice: «la LSTM recuerda porque tiene memoria». Reescríbelo con precisión técnica en dos frases.
Crear	Diseña una tarea sintética mínima donde una RNN simple falle y una LSTM acierte, y justifica por qué tu tarea discrimina entre ambas.

16. Autoevaluación

1. ¿Qué significa exactamente «carrusel de error constante»?
2. ¿Por qué las compuertas usan sigmoide y el candidato usa tanh?
3. ¿Cuál es la diferencia funcional entre c_t y h_t ?
4. Si $f_t = 1$ e $i_t = 0$ para siempre, ¿qué hace la celda?
5. ¿Qué límite de la LSTM **no** resuelve el mecanismo de compuertas?
6. ¿Por qué la LSTM no se puede paralelizar sobre la longitud de la secuencia?
7. ¿Qué componente de la LSTM actual no aparece en el paper de 1997?

17. Respuestas esperadas

1. La ruta por la que el estado de celda se actualiza sumando, de modo que la derivada $\partial c_t / \partial c_{t-1}$ vale f_t en lugar de un producto de derivadas de activación. Con $f \approx 1$, el error se conserva a través de muchos pasos.
2. La sigmoide devuelve $[0, 1]$: es una compuerta, un porcentaje de paso. La tanh devuelve $[-1, 1]$: es contenido con signo, que puede sumar o restar de la memoria.
3. c_t es la memoria interna protegida. $h_t = o_t \odot \tanh(c_t)$ es la parte que la celda expone al resto de la red. Se puede recordar algo sin exponerlo.
4. Conserva su contenido indefinidamente sin admitir nada nuevo: memoria congelada.
5. La secuencialidad. El cómputo sigue siendo $O(n)$ pasos no paralelizables, y el camino entre dos posiciones distantes sigue siendo largo.
6. Porque h_t depende de h_{t-1} : hay una dependencia de datos estricta entre pasos.
7. La compuerta de olvido (y también la conexión *peephole*, de Gers y Schmidhuber, 2000).

18. Fuentes primarias

- Hochreiter, S. y Schmidhuber, J. (1997). *Long Short-Term Memory*. **Neural Computation**, 9(8), 1735–1780. doi.org/10.1162/neco.1997.9.8.1735 · consultado 2026-08-16.
- Gers, F. A., Schmidhuber, J. y Cummins, F. (2000). *Learning to Forget: Continual Prediction with LSTM*. **Neural Computation**, 12(10), 2451–2471. doi.org/10.1162/089976600300015015 · consultado 2026-08-16.

- Cho, K. et al. (2014). *Learning Phrase Representations using RNN Encoder–Decoder*. [arXiv:1406.1078](#)
· consultado 2026-08-16.

Evaluación — P03

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Long Short-Term Memory* (1997, Neural Computation, 9(8), 1735–1780) · **Nivel:** L2

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P03_lstm.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P04 — AlexNet

El resultado que sacó al deep learning del laboratorio: no una idea nueva, sino la combinación que demostró que las ideas viejas escalaban.

Nivel: L3 · Motor: convnet · Notebook: P04_alexnet.ipynb

1. Identificación

Campo	Valor
Título original	ImageNet Classification with Deep Convolutional Neural Networks
Autoría	Alex Krizhevsky, Ilya Sutskever, Geoffrey E. Hinton
Año	2012
Venue	NeurIPS (entonces NIPS) 2012
Fuente primaria	papers.nips.cc — actas de 2012 · versión CACM 2017
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Hasta 2012, la visión por computador se hacía con **descriptores diseñados a mano** (SIFT, HOG) seguidos de un clasificador. Un humano decidía qué características eran relevantes; el aprendizaje solo ocurría en la última etapa.

Las CNN existían desde LeNet (LeCun et al., 1998) y funcionaban en dígitos manuscritos, pero la comunidad dudaba de que escalaran a imágenes naturales de mil categorías. Faltaban tres cosas a la vez: **datos** (ImageNet llegó en 2009), **cómputo** (las GPU programables) y **técnicas de entrenamiento** que evitaran que una red profunda se estancara o memorizara.

3. Propuesta

Una CNN profunda —5 capas convolucionales y 3 completamente conectadas— entrenada directamente sobre ImageNet, combinando:

- **ReLU** en lugar de sigmoide o tanh: no satura para valores positivos y acelera mucho el entrenamiento;
- **entrenamiento en dos GPU** con la red repartida entre ambas, por límite de memoria;
- **dropout** en las capas densas, para reducir el sobreajuste;
- **aumento de datos** (recortes, reflejos, alteración de color);
- **pooling solapado** y normalización de respuesta local.

Ninguna pieza es enteramente nueva. La contribución es la **combinación funcionando a escala** y el resultado que la respaldó.

4. Intuición sin fórmulas

Un detector de bordes no debería reaprenderse en cada esquina de la imagen. La convolución desliza el mismo detector por toda la imagen: muchos menos parámetros y, sobre todo, la posición del objeto deja de importar. Apilando capas, los detectores simples (bordes) se componen en detectores complejos (texturas, partes, objetos).

Dónde deja de funcionar la analogía: un kernel no «reconoce» nada por sí solo. Lo que existe es un patrón de activación distribuido en muchos filtros; la interpretación limpia de «esta neurona detecta caras» es casi siempre una simplificación.

5. Matemática mínima

Convolución (correlación cruzada):

$$(I * K)[r, c] = \sum_i \sum_j I[r+i, c+j] \cdot K[i, j]$$

Activación:

$$\text{ReLU}(z) = \max(0, z) \quad \text{ReLU}'(z) = 1 \text{ si } z > 0, 0 \text{ si } z < 0$$

Max-pooling:

$$P[r, c] = \max \text{ de la ventana correspondiente}$$

Dropout (entrenamiento):

$$\tilde{h} = h \odot m, \quad m \sim \text{Bernoulli}(p)$$

Recuento de parámetros. Una capa densa que conecte una entrada de $H \times W$ con una salida de $H' \times W'$ necesita $H \cdot W \cdot H' \cdot W'$ pesos. Un kernel $k \times k$ necesita k^2 , reutilizados en toda la imagen. Esa reducción es lo que hace viable la profundidad.

6. Arquitectura o flujo

```
imagen 224x224x3
|
├─ conv1 → ReLU → norm → maxpool
├─ conv2 → ReLU → norm → maxpool
├─ conv3 → ReLU
├─ conv4 → ReLU
├─ conv5 → ReLU → maxpool
|
├─ fc6 → ReLU → dropout
├─ fc7 → ReLU → dropout
└─ fc8 → softmax (1000 categorías)
```

Repartida entre 2 GPU, con comunicación solo en capas concretas.

7. Qué observar en el paper original

- La **figura de la arquitectura** con la red partida en dos flujos: es una restricción de hardware convertida en decisión de diseño.
- La sección de **ReLU**: la comparación de velocidad de convergencia frente a tanh es uno de los argumentos más citados del artículo.
- La **visualización de los filtros de la primera capa**: bordes orientados y manchas de color, sin que nadie los programara.
- La **sección de reducción de sobreajuste**: aumento de datos y dropout.
- La tabla de resultados de ILSVRC-2012 con el error top-1 y top-5, y el margen respecto al segundo clasificado.

8. Evidencia y resultados

El sistema ganó el certamen ILSVRC-2012 de clasificación con un **error top-5 en torno al 15 %, frente a alrededor del 26 % del siguiente participante** — un margen inusual en una competición donde las mejoras anuales eran de uno o dos puntos.

Los valores exactos de top-1 y top-5, para el modelo individual y para el conjunto, están en la tabla de resultados del artículo. Verificarlos ahí antes de citarlos: circulan versiones distintas según se refieran al modelo único, al *ensemble* o al conjunto de validación.

El paper incluye **ablaciones** que muestran cuánto pierde el modelo al quitar capas convolucionales, y experimentos de recuperación por vecinos en el espacio de la penúltima capa.

9. Impacto

- Cambió el método por defecto de la visión por computador en menos de dos años.
- Consolidó la **GPU** como plataforma estándar de entrenamiento.
- Popularizó ReLU y dropout, que pasaron a ser configuración por defecto.
- Estableció la práctica de **preentrenar en un dominio grande y transferir** a tareas con pocos datos.
- Fuera del campo, es el resultado que convirtió el deep learning en tema de portada y desencadenó el ciclo de inversión posterior.

10. Limitaciones

1. **Coste de cómputo y datos.** El resultado depende de ImageNet y de GPU: no es reproducible con un dataset pequeño.
2. **Sensible a la traslación, no a la rotación ni a la escala.** La equivarianza es solo traslacional; el resto se aproxima con aumento de datos.
3. **Frágil ante perturbaciones adversarias**, como mostró trabajo posterior (Szegedy et al., 2013; Goodfellow et al., 2014).
4. **Poco interpretable** más allá de la primera capa.
5. **Sesgo del dataset.** Lo que la red «sabe» está determinado por qué contiene ImageNet y cómo se etiquetó; esa discusión llegó años después.

6. **La normalización de respuesta local** que propone acabó abandonada: trabajos posteriores mostraron que aporta poco frente a la normalización por lotes.

11. Errores comunes

Error	Corrección
«AlexNet inventó las CNN»	LeNet es de 1998; la convolución con retropropagación es anterior. AlexNet demostró que escalaban .
«AlexNet inventó ReLU y dropout»	Ambas son anteriores o contemporáneas (Nair y Hinton, 2010; Hinton et al., 2012). AlexNet las combinó y las popularizó.
«Ganó porque era más profunda»	Ganó por la combinación de profundidad, datos, GPU, ReLU, dropout y aumento de datos. La ablación del propio paper lo respalda.
«Las CNN son mejores que los métodos clásicos»	Afirmación vacía sin tarea, dataset, métrica y línea base.
«El 15 % de error significa que entiende las imágenes»	Significa que acierta una métrica en un conjunto concreto. Nada más.

12. Relación con trabajos anteriores

- **Po2 Backpropagation (1986)** — el método de entrenamiento.
- **LeCun et al. (1998)** — LeNet: convolución, pooling y retropropagación.
- **Deng et al. (2009)** — ImageNet, el dataset sin el cual no hay resultado.
- **Nair y Hinton (2010)** — unidades rectificadas.
- **Hinton et al. (2012)** — dropout. [arXiv:1207.0580](#)

13. Relación con trabajos posteriores

- **VGG (2014), GoogLeNet (2014), ResNet (2015)** — la carrera de profundidad que AlexNet abrió. ResNet resuelve el problema de entrenar cientos de capas con conexiones residuales, el mismo principio aditivo de Po3.
- **Szegedy et al. (2013)** — ejemplos adversarios: el reverso del éxito.
- **Po8 Transformer (2017) y ViT (2020)** — el fin del monopolio convolucional en visión.
- **Russakovsky et al. (2015)** — análisis del certamen ILSVRC. [arXiv:1409.0575](#)

14. Notebook asociado

P04_alexnet.ipynb

Qué implementa: convolución 2D con kernels de borde escritos a mano, ReLU, max-pooling, demostración de equivarianza traslacional y comparación de recuento de parámetros frente a una capa densa equivalente.

Qué NO implementa: ningún entrenamiento, ninguna GPU, ningún dato de ImageNet. Los kernels están escritos, no aprendidos — que es justamente lo contrario de lo que hizo AlexNet.

15. Actividades Bloom

Nivel	Actividad
Recordar	Enumera las cinco técnicas que AlexNet combinó y di cuál es anterior al paper.
Explicar	Explica por qué ReLU acelera el entrenamiento frente a tanh, en términos de derivada.
Aplicar	Aplica un kernel horizontal a la imagen del notebook y explica por qué no responde.
Analizar	Calcula el número de parámetros de una convolución $3 \times 3 \times 64 \rightarrow 128$ y compáralo con la densa equivalente sobre un mapa 56×56 .
Evaluar	Un informe afirma «nuestra CNN alcanza 94 % de exactitud». Escribe las cinco preguntas que debes hacer antes de aceptarlo.
Crear	Diseña un experimento que separe la contribución de ReLU de la del aumento de datos, y di qué necesitarías para ejecutarlo.

16. Autoevaluación

1. ¿Qué significa que la convolución sea *equivariante* a la traslación, y en qué se diferencia de ser *invariante*?
2. ¿Por qué compartir pesos reduce el sobreajuste además del cómputo?
3. ¿Qué hace exactamente el dropout durante el entrenamiento, y qué se hace en inferencia?
4. ¿Por qué la red se partió entre dos GPU?
5. ¿Qué componente del paper original acabó siendo abandonado por la comunidad?
6. ¿Por qué un resultado en ILSVRC-2012 no autoriza a afirmar nada sobre «visión» en general?
7. Nombra dos ideas anteriores a 2012 sin las cuales AlexNet no existiría.

17. Respuestas esperadas

1. Equivariante: si la entrada se desplaza, el mapa de activación se desplaza igual. Invariante: la salida no cambia. El pooling añade invarianza **local**; la convolución sola es equivariante.
2. Porque impone una restricción estructural: el mismo detector se aplica en todas las posiciones. Menos grados de libertad con la misma capacidad de detección.
3. Desactiva unidades al azar con probabilidad p , forzando representaciones redundantes. En inferencia se usan todas, con la escala correspondiente para mantener la magnitud esperada.
4. Por límite de memoria de las GPU de la época (3 GB). Es una restricción de hardware, no una decisión conceptual.
5. La normalización de respuesta local (LRN).
6. Porque la métrica está definida sobre un dataset concreto, con sus categorías, su distribución y sus sesgos. Generalizar a «visión» es un salto sin evidencia.
7. Se aceptan: retropropagación (1986), LeNet (1998), ImageNet (2009), ReLU (2010), dropout (2012), CUDA/GPU programables.

18. Fuentes primarias

- Krizhevsky, A., Sutskever, I. y Hinton, G. E. (2012). *ImageNet Classification with Deep Convolutional Neural Networks*. **NeurIPS 2012**. [actas](#) · consultado 2026-08-16.
- Krizhevsky, A., Sutskever, I. y Hinton, G. E. (2017). Versión revisada en **Communications of the ACM**, 60(6), 84–90. doi.org/10.1145/3065386 · consultado 2026-08-16.
- Russakovsky, O. et al. (2015). *ImageNet Large Scale Visual Recognition Challenge*. [arXiv:1409.0575](#) · consultado 2026-08-16.

Evaluación — P04

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *ImageNet Classification with Deep Convolutional Neural Networks* (2012, NeurIPS (NIPS) 2012) ·

Nivel: L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P04_alexnet.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P05 — Word2Vec

El momento en que el significado de una palabra se vuelve una dirección en el espacio, y esa dirección se puede calcular a escala de miles de millones de palabras.

Nivel: L2 · Motor: word2vec · Notebook: P05_word2vec.ipynb

1. Identificación

Campo	Valor
Título original	<i>Efficient Estimation of Word Representations in Vector Space</i>
Autoría	Tomas Mikolov, Kai Chen, Greg Corrado, Jeffrey Dean
Año	2013
Venue	arXiv:1301.3781 · presentado en el taller de ICLR 2013
Fuente primaria	arXiv:1301.3781
Complemento imprescindible	arXiv:1310.4546 (muestreo negativo, NIPS 2013)
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Representar palabras como identificadores dispersos (*one-hot*) tiene una consecuencia devastadora: **todas las palabras están a la misma distancia entre sí**. «Perro» no está más cerca de «gato» que de «hipoteca». Cualquier modelo que parta de ahí tiene que aprender la similitud desde cero para cada tarea.

Existían representaciones distribuidas: el modelo neuronal de lenguaje de Bengio et al. (2003) ya producía vectores de palabras como efecto colateral. Pero incluía una capa oculta no lineal y una softmax sobre todo el vocabulario: entrenarlo sobre corpus grandes era prohibitivo.

3. Propuesta

Dos arquitecturas **log-lineales, sin capa oculta**, diseñadas para ser baratas:

- **CBOW**: predice la palabra central a partir de su contexto.
- **Skip-gram**: predice el contexto a partir de la palabra central.

El artículo complementario (Mikolov et al., 2013b) añade el ingrediente que las hace prácticas: **muestreo negativo**, que sustituye la softmax sobre todo el vocabulario por unas pocas clasificaciones binarias «esta pareja es real / esta pareja es inventada».

El resultado inesperado: el espacio resultante exhibe **estructura lineal**. Ciertas relaciones semánticas y sintácticas aparecen como desplazamientos aproximadamente constantes.

4. Intuición sin fórmulas

«Dime con quién apareces y te diré qué significas.» Si dos palabras aparecen rodeadas de las mismas palabras, acabarán con vectores parecidos — sin que nadie escriba jamás una definición.

Dónde deja de funcionar la analogía: el método captura **distribución**, no significado. Antónimos como «caliente» y «frío» aparecen en contextos casi idénticos, y por tanto quedan cerca. El espacio no distingue «similar» de «intercambiable».

5. Matemática mínima

Skip-gram con muestreo negativo. Para un par observado (centro c , contexto o) y k negativos $n_1 \dots n_k$ muestreados de una distribución de ruido:

```
L = - log σ(v_c · u_o) - Σ_i log σ(-v_c · u_{n_i})  
  
σ(z) = 1/(1+e-z)  
v_c : vector de entrada (el que se usa como embedding final)  
u_o : vector de salida (contexto)
```

Similitud y analogía:

```
similitud(a, b) = cos(v_a, v_b) = (v_a · v_b) / (||v_a|| ||v_b||)  
  
analogía a : b :: c : ? → argmax_d cos(v_b - v_a + v_c, v_d), d ∉ {a, b, c}
```

La exclusión de $\{a, b, c\}$ del ranking **es parte del protocolo**, no un detalle de implementación: sin ella el resultado cambia por completo.

6. Arquitectura o flujo

```
CBOW          SKIP-GRAM  
contexto → suma → proyección   centro → proyección → predice  
      |                               |  
      └─ predice el centro ─┘       └─ cada palabra  
                                   └─ del contexto  
  
Entrenamiento (skip-gram + negativos):  
par real      (rey, reino) → empujar v_rey · u_reino hacia arriba  
par inventado (rey, hornear) → empujar v_rey · u_hornear hacia abajo
```

7. Qué observar en el paper original

- La **tabla de coste computacional**: el argumento central del artículo es de eficiencia, no de calidad. Se trata de poder entrenar sobre miles de millones de palabras.

- El **conjunto de analogías** semánticas y sintácticas que los autores construyen, y la distinción entre ambos tipos.
- La comparación de calidad frente a dimensión del vector y tamaño del corpus.
- En el artículo complementario: **muestreo negativo**, submuestreo de palabras frecuentes y detección de frases.

8. Evidencia y resultados

Los autores evalúan sobre un conjunto de preguntas de analogía del tipo «*Atenas es a Grecia lo que Oslo es a ____*» (semánticas) y «*camina es a caminando lo que nada es a ____*» (sintácticas), midiendo exactitud de la respuesta exacta.

Las cifras de exactitud por tipo de analogía, dimensión y tamaño de corpus están en las tablas del artículo. Este eje no las reproduce: verificarlas allí antes de citarlas.

La miniatura de este eje produce evidencia a escala de juguete y honesta sobre su alcance: con un corpus de 8 frases, **rey - hombre + mujer** devuelve **reina** como vecino más cercano en las tres semillas probadas, con un coseno claramente separado del segundo candidato.

9. Impacto

- Los embeddings preentrenados se convierten en el punto de partida por defecto de cualquier sistema de PLN durante varios años.
- Fundan la **búsqueda vectorial** y, con ella, todo el ecosistema de bases de datos de vectores que sostiene hoy RAG.
- La estructura lineal del espacio abre una línea de investigación sobre **sesgo**: si el espacio codifica «rey - hombre + mujer = reina», también codifica asociaciones sociales problemáticas (Bolukbasi et al., 2016; Caliskan et al., 2017).
- Popularizan la idea de que **el preentrenamiento no supervisado produce representaciones reutilizables** — la premisa completa de BERT y GPT.

10. Limitaciones

1. **Un vector por palabra.** «Banco» tiene una única representación para el asiento, la entidad financiera y la orilla. Lo resolverán ELMo y BERT con embeddings contextuales.
2. **Sin composición.** No hay forma principada de obtener el vector de una frase.
3. **Vocabulario cerrado.** Una palabra no vista no tiene vector (lo abordará FastText con subpalabras).
4. **Captura distribución, no significado.** Antónimos quedan cerca.
5. **Hereda y amplifica sesgos** del corpus.
6. **La aritmética de analogías es más frágil de lo que sugiere su fama:** depende del protocolo de exclusión, de la normalización y del subconjunto evaluado.

11. Errores comunes

Error	Corrección
«Word2Vec inventó los embeddings»	Bengio et al. (2003) ya producía vectores de palabras. Word2Vec los hizo baratos a gran escala.
«rey - hombre + mujer = reina es una igualdad»	Es el vecino más cercano de un vector calculado, tras excluir las tres palabras de la consulta. No es una identidad algebraica.
«El muestreo negativo está en el paper de enero de 2013»	Está en el artículo complementario de octubre de 2013 (arXiv:1310.4546).
«Word2Vec entiende el significado»	Modela co-ocurrencia. Que la geometría resultante sea interpretable no implica comprensión.
«Los embeddings son neutrales porque los aprende una máquina»	Reflejan el corpus. La neutralidad no es un efecto secundario del automatismo.

12. Relación con trabajos anteriores

- **Harris (1954), Firth (1957)** — hipótesis distribucional.
- **Bengio et al. (2003)** — modelo neuronal de lenguaje con representaciones distribuidas.
- **LSA / análisis semántico latente (1990)** — factorización de matrices de co-ocurrencia.

13. Relación con trabajos posteriores

- **GloVe (Pennington et al., 2014)** — enfoque basado en factorizar co-ocurrencias globales. *ACL Anthology*
- **FastText (2016)** — subpalabras; resuelve el vocabulario cerrado.
- **ELMo (2018)** — embeddings **contextuales**: un vector distinto por aparición. *ACL Anthology*
- **P09 BERT (2018)** — representaciones contextuales profundas.
- **P11 RAG (2020)** — recuperación densa apoyada en embeddings.

14. Notebook asociado

P05_word2vec.ipynb

Qué implementa: skip-gram con muestreo negativo entrenado desde cero en Python puro sobre un corpus de 8 frases, con vecinos por coseno y evaluación de analogía con protocolo de exclusión explícito.

Qué NO implementa: CBOW, submuestreo de frecuentes, detección de frases, ni ninguna evaluación a escala. Con 21 palabras de vocabulario, todo resultado es ilustrativo.

```
ai-evolution paper-lab P05 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la pérdida de skip-gram con muestreo negativo y explica cada término.
Explicar	Explica por qué el muestreo negativo es más barato que la softmax completa.
Aplicar	Ejecuta el notebook con tres semillas y anota qué cambia y qué se mantiene.
Analizar	Quita la exclusión de {a, b, c} del ranking de analogías y explica el resultado.
Evaluar	Un artículo afirma que los embeddings «demuestran que el modelo entiende relaciones de género». Evalúa la afirmación.
Crear	Construye 10 analogías propias en español, ejecútalas y clasifica los fallos por tipo (frecuencia, polisemia, corpus).

16. Autoevaluación

1. ¿Qué diferencia hay entre CBOW y skip-gram, y cuándo conviene cada uno?
2. ¿Por qué la softmax completa es costosa y cómo lo evita el muestreo negativo?
3. ¿Por qué «caliente» y «frío» acaban cerca en el espacio?
4. ¿Qué papel juega la ventana de contexto en el tipo de similitud que se aprende?
5. ¿Por qué hay dos matrices de vectores (entrada y salida) y cuál se usa como embedding?
6. ¿Qué problema de la polisemia no puede resolver este método, y qué lo resolvió?
7. ¿Qué parte del método que hoy se llama «Word2Vec» no está en el paper de enero de 2013?

17. Respuestas esperadas

1. CBOW predice el centro desde el contexto (más rápido, mejor con palabras frecuentes); skip-gram predice el contexto desde el centro (mejor con palabras raras y corpus pequeños).
2. Porque normalizar exige sumar sobre todo el vocabulario en cada paso. El muestreo negativo sustituye eso por $k+1$ clasificaciones binarias, con k típicamente entre 5 y 20.
3. Porque aparecen en contextos casi idénticos. El método mide sustituibilidad distribucional, no relación semántica.
4. Ventanas pequeñas favorecen similitud sintáctica y funcional; ventanas grandes favorecen similitud temática.
5. Cada palabra juega dos papeles (centro y contexto) y necesita un vector para cada uno. Habitualmente se usa la matriz de entrada como embedding final.
6. Que una palabra tenga un único vector para todos sus sentidos. Lo resolvieron los embeddings contextuales: ELMo y después BERT.
7. El muestreo negativo, el submuestreo de palabras frecuentes y la detección de frases: todo ello está en arXiv:1310.4546.

18. Fuentes primarias

- Mikolov, T., Chen, K., Corrado, G. y Dean, J. (2013). *Efficient Estimation of Word Representations in Vector Space*. arXiv:1301.3781 · consultado 2026-08-16.
- Mikolov, T., Sutskever, I., Chen, K., Corrado, G. y Dean, J. (2013). *Distributed Representations of Words and Phrases and their Compositionality*. **NIPS 2013**. arXiv:1310.4546 · consultado 2026-08-

16.

- Pennington, J., Socher, R. y Manning, C. (2014). *GloVe: Global Vectors for Word Representation*. **EMNLP 2014**. [ACL Anthology](#) · consultado 2026-08-16.

Evaluación — P05

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Efficient Estimation of Word Representations in Vector Space* (2013, arXiv:1301.3781 · ICLR 2013 (workshop)) · **Nivel:** L2

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P05_word2vec.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P06 — Seq2Seq

Una sola red aprende a convertir una secuencia de longitud variable en otra de longitud distinta, de extremo a extremo. Y, al hacerlo, revela el cuello de botella que definirá los tres papers siguientes.

Nivel: L3 · Motor: seq2seq · Notebook: P06_seq2seq.ipynb

1. Identificación

Campo	Valor
Título original	Sequence to Sequence Learning with Neural Networks
Autoría	Ilya Sutskever, Oriol Vinyals, Quoc V. Le
Año	2014
Venue	arXiv:1409.3215 · NeurIPS (NIPS) 2014
Fuente primaria	arXiv:1409.3215
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Las redes profundas funcionaban con entradas y salidas de **dimensión fija**: una imagen de 224×224, un vector de características. Pero traducir «el gato negro duerme» a inglés produce una secuencia de longitud distinta a la de entrada, y esa longitud no se conoce de antemano.

La traducción automática del momento se hacía con **sistemas estadísticos por frases**: muchos componentes ajustados por separado (modelo de traducción, modelo de lenguaje, reordenamiento, ajuste de pesos). Funcionaban bien, pero cada pieza se optimizaba con un criterio propio y no con el objetivo final.

3. Propuesta

Dos LSTM encadenadas:

- el **codificador** lee la secuencia de entrada token a token y termina con un estado interno c , un vector de tamaño fijo que pretende resumirla entera;
- el **decodificador** parte de c y genera la salida token a token, alimentándose de lo que ya generó.

Todo se entrena con un único objetivo: maximizar la probabilidad de la traducción correcta.

El artículo aporta además un truco empírico decisivo: **invertir el orden de la secuencia fuente**. Con la entrada invertida, las primeras palabras de origen quedan temporalmente cerca de las primeras de destino, lo que acorta las dependencias que la LSTM debe mantener.

4. Intuición sin fórmulas

Leer una frase entera, cerrar los ojos y escribir la traducción solo de memoria. Con frases cortas funciona. Con un párrafo, cuando llegas al final ya no recuerdas con precisión cómo empezaba.

Dónde deja de funcionar la analogía: una persona puede releer. El decodificador de este modelo no puede: solo dispone de c .

5. Matemática mínima

Codificador: $c = h_n$, con $h_t = \text{LSTM}(h_{t-1}, x_t)$

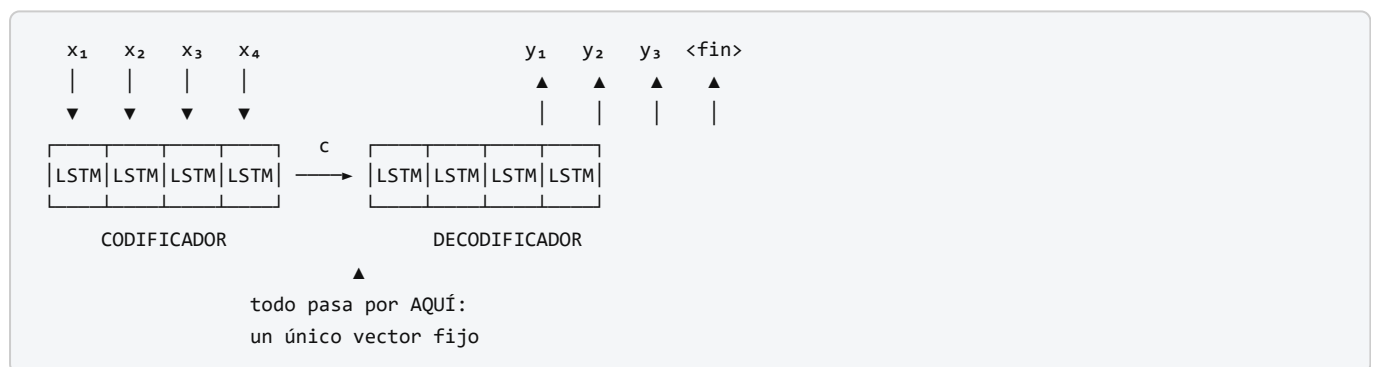
Decodificador: $p(y_1 \dots y_m \mid x_1 \dots x_n) = \prod_i p(y_i \mid y_{<i}, c)$

Entrenamiento: maximizar $\sum \log p(y \mid x)$ sobre el corpus paralelo

Inferencia: búsqueda en haz (beam search) sobre la secuencia de salida

El punto crítico: $c \in \mathbb{R}^d$ tiene dimensión **constante**, mientras que la información de $x_1 \dots x_n$ crece con n . Comprimir algo de tamaño creciente en algo de tamaño fijo tiene un coste, y ese coste crece con la longitud.

6. Arquitectura o flujo



7. Qué observar en el paper original

- La **discusión sobre invertir la secuencia fuente**: es una de las observaciones empíricas más comentadas del artículo, y los autores admiten no tener una explicación completa.
- El uso de **LSTM profundas** (varias capas apiladas) y su efecto en el resultado.
- La **búsqueda en haz** en inferencia y el efecto del tamaño del haz.
- La **visualización PCA** de los estados c : frases con significado parecido quedan cerca, y el espacio muestra sensibilidad al orden de las palabras y a la voz activa/pasiva.
- El experimento de **rescorear** las n -mejores hipótesis del sistema estadístico, en lugar de traducir directamente.

8. Evidencia y resultados

Evaluación sobre traducción **inglés** $\rightarrow$ **francés** de WMT'14, medida en BLEU, comparada contra un sistema estadístico por frases de referencia.

El artículo reporta dos regímenes: traducción directa con un conjunto de LSTM, y rescoring de las hipótesis del sistema estadístico —este último mejor que el primero—. También documenta que la calidad **cae con la longitud de la frase**, y que invertir la fuente mejora el resultado de forma consistente.

Los valores exactos de BLEU para cada configuración están en las tablas del artículo. Verificarlos allí: circulan cifras mezcladas entre el modelo único, el conjunto y el rescoring.

La miniatura de este eje mide el fenómeno estructural, no el BLEU: al pasar de longitud 2 a 32, la similitud del vector de contexto con el **primer** token se desploma mientras la del **último** se mantiene alta.

9. Impacto

- Convierte la traducción automática en un problema de aprendizaje de extremo a extremo, y en pocos años el paradigma estadístico por frases queda desplazado.
- Establece el patrón **codificador–decodificador** que sigue siendo la arquitectura de referencia para transformar una secuencia en otra.
- Populariza la **búsqueda en haz** como procedimiento estándar de decodificación.
- Y, sobre todo, **hace visible el cuello de botella**: al medir la caída de calidad con la longitud, define el problema que Bahdanau resolverá el mismo año.

10. Limitaciones

1. **Cuello de botella del vector fijo.** Es el límite estructural: capacidad constante frente a información creciente.
2. **La calidad cae con la longitud de la frase.** Documentado en el propio artículo.
3. **Significa secuencial:** no paraleliza sobre la longitud.
4. **Invertir la fuente es un parche**, no una solución. Reordena qué información se pierde.
5. **Coste de entrenamiento alto** para la época, y dependencia de grandes corpus paralelos.
6. **Sin alineación explícita:** no hay forma de saber qué parte de la entrada generó qué parte de la salida.

11. Errores comunes

Error	Corrección
«Seq2Seq usa atención»	No. La atención llega con P07, unas semanas después.
«El problema se arregla agrandando el estado»	Un estado mayor retrasa el problema; no cambia que la capacidad sea constante y la longitud variable.
«Invertir la entrada resuelve el cuello de botella»	Lo mitiga reordenando las dependencias. El cuello sigue ahí.
«Seq2Seq inventó el codificador–decodificador»	Cho et al. (2014) publicó una formulación muy próxima meses antes (arXiv:1406.1078). Ambos trabajos son contemporáneos e independientes.
«BLEU mide calidad de traducción»	Mide solapamiento de n-gramas con referencias. Correlaciona, imperfectamente, y no es comparable entre tokenizaciones distintas.

12. Relación con trabajos anteriores

- **P03 LSTM (1997)** — la celda que hace viable el codificador.
- **Cho et al. (2014)** — RNN encoder–decoder y GRU, contemporáneo. [arXiv:1406.1078](#)
- **Modelos estadísticos de traducción** (Brown et al., 1993; Koehn et al., 2003) — la línea base contra la que se compara.

13. Relación con trabajos posteriores

- **P07 Attention (2014)** — elimina el vector fijo.
- **P08 Transformer (2017)** — conserva el patrón codificador–decodificador y elimina la recurrencia.
- **BART, T5 (2019–2020)** — codificador–decodificador preentrenado.
- **Modelos de imagen a texto y voz a texto** — el mismo patrón aplicado a otras modalidades.

14. Notebook asociado

P06_seq2seq.ipynb

Qué implementa: una medición directa del cuello de botella. Un codificador de juguete comprime secuencias de longitud creciente en un vector fijo y se mide cuánta información del principio sobrevive.

Qué NO implementa: LSTM entrenadas, traducción real, búsqueda en haz ni BLEU. El codificador es una mezcla con decaimiento fijo: aísla el fenómeno, no lo mide en un modelo real.

```
ai-evolution paper-lab P06 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la factorización $p(y \setminus x)$ del decodificador y explica de qué depende cada término.
Explicar	Explica por qué invertir la secuencia fuente ayuda, en términos de distancia entre dependencias.
Aplicar	Ejecuta el notebook y anota a partir de qué longitud el primer token deja de ser recuperable.
Analizar	Argumenta por qué el problema es estructural y no de capacidad, con un contraejemplo numérico.
Evaluar	Alguien propone «duplicar la dimensión del estado» como solución. Evalúa la propuesta con evidencia.
Crear	Diseña una métrica que capture la pérdida de información del principio de la secuencia y justificala.

16. Autoevaluación

1. ¿Por qué las redes profundas previas no servían para traducir?
2. ¿Qué es exactamente el vector c y qué se le pide?
3. ¿Por qué la calidad cae con la longitud de la frase?
4. ¿Qué hace la búsqueda en haz y por qué no se usa simplemente el token más probable en cada paso?
5. ¿Invertir la fuente resuelve o mitiga el problema? Justifica.

6. ¿Qué información se pierde por completo en este modelo respecto a la alineación?
7. ¿Qué idea que hoy asociamos a Seq2Seq **no** está en este paper?

17. Respuestas esperadas

1. Porque exigían entrada y salida de dimensión fija, y la traducción tiene longitudes variables y desconocidas de antemano.
2. El estado final del codificador: un vector de dimensión fija que debe contener toda la información necesaria para reconstruir la salida.
3. Porque la información de entrada crece con n mientras la capacidad de c es constante. El estado acaba dominado por los últimos tokens leídos.
4. Mantiene las k hipótesis parciales más probables. La decodificación voraz puede quedar atrapada tras una primera elección localmente buena y globalmente mala.
5. Mitiga. Acorta la distancia entre las primeras palabras de origen y de destino, pero la compresión a tamaño fijo permanece intacta.
6. Qué parte de la entrada dio lugar a qué parte de la salida: no hay ningún mecanismo que lo exponga ni lo aprenda.
7. La atención. Es de Bahdanau, Cho y Bengio, y la publicación de arXiv es de septiembre de 2014, prácticamente simultánea.

18. Fuentes primarias

- Sutskever, I., Vinyals, O. y Le, Q. V. (2014). *Sequence to Sequence Learning with Neural Networks*. **NIPS 2014**. arXiv:1409.3215 · consultado 2026-08-16.
- Cho, K. et al. (2014). *Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation*. **EMNLP 2014**. arXiv:1406.1078 · consultado 2026-08-16.

Evaluación — P06

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Sequence to Sequence Learning with Neural Networks* (2014, arXiv:1409.3215 · NeurIPS (NIPS) 2014) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P06_seq2seq.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P07 — Atención (Bahdanau)

El decodificador deja de depender de un único vector y aprende a mirar la parte de la entrada que necesita en cada paso. Nace el mecanismo que tres años después dará nombre al paper más influyente del campo.

Nivel: L3 · Motor: bahdanau · Notebook: P07_attention_bahdanau.ipynb

1. Identificación

Campo	Valor
Título original	Neural Machine Translation by Jointly Learning to Align and Translate
Autoría	Dzmitry Bahdanau, Kyunghyun Cho, Yoshua Bengio
Año	2014 (arXiv) · 2015 (ICLR)
Venue	arXiv:1409.0473 · ICLR 2015
Fuente primaria	arXiv:1409.0473
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Seq2Seq comprime la frase de origen en un vector de tamaño fijo. Los propios autores de aquel trabajo documentaron que la calidad **cae con la longitud**: un vector de dimensión constante no puede sostener frases cada vez más largas.

El diagnóstico era claro y el parche disponible (invertir la entrada) no atacaba la causa. La pregunta abierta era: **¿y si el decodificador no tuviera que depender de un único vector?**

3. Propuesta

Que el decodificador construya **un vector de contexto distinto en cada paso de salida**, como suma ponderada de **todos** los estados del codificador. Los pesos los calcula una pequeña red que puntúa la compatibilidad entre el estado actual del decodificador y cada estado de entrada, y se normalizan con softmax.

Dos consecuencias, ambas en el título:

- **align** — los pesos son una alineación suave entre origen y destino, y se pueden inspeccionar;
- **translate** — todo se entrena junto, con la pérdida de traducción. **Nadie etiqueta la alineación**: emerge sola.

El codificador además pasa a ser **bidireccional**, para que cada estado h_j resuma el contexto a ambos lados de la posición j .

4. Intuición sin fórmulas

En lugar de memorizar la frase entera y cerrar los ojos, el traductor deja el texto original sobre la mesa y, para cada palabra que escribe, vuelve a mirar la parte que le hace falta.

Dónde deja de funcionar la analogía: un traductor humano sabe **por qué** mira donde mira. Los pesos α indican dónde se concentró la mezcla, no una razón. Confundir ambas cosas es el error que la sección 11 documenta.

5. Matemática mínima

Puntuación de compatibilidad (aditiva, con una capa oculta):

$$e_{ij} = v^T \cdot \tanh(W \cdot s_{i-1} + U \cdot h_j)$$

Normalización:

$$\alpha_{ij} = \exp(e_{ij}) / \sum_k \exp(e_{ik}) \quad \rightarrow \quad \sum_j \alpha_{ij} = 1$$

Vector de contexto del paso i :

$$c_i = \sum_j \alpha_{ij} \cdot h_j$$

Salida:

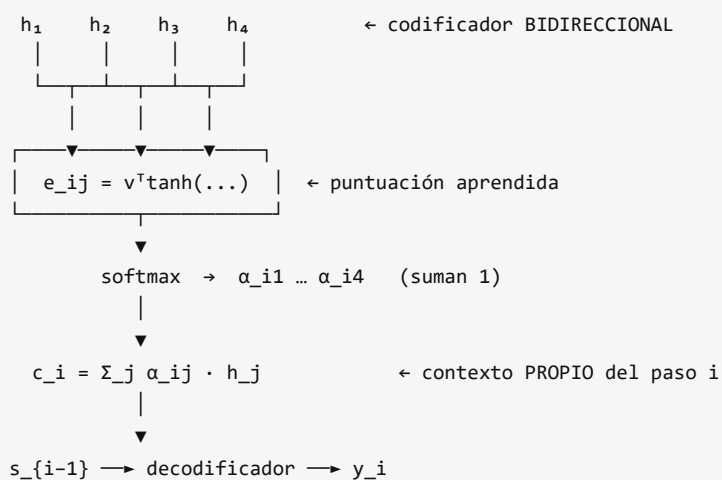
$$s_i = f(s_{i-1}, y_{i-1}, c_i)$$

$$p(y_i | y_{<i}, x) = g(y_{i-1}, s_i, c_i)$$

- h_j : estado del codificador bidireccional en la posición j de origen.
- s_{i-1} : estado del decodificador antes de emitir el token i .
- v, W, U : parámetros **aprendidos** junto con el resto del modelo.

Obsérvese que c_i depende de i : **hay un contexto por paso de salida**, no uno por frase.

6. Arquitectura o flujo



7. Qué observar en el paper original

- Las **matrices de alineación** visualizadas como mapas de calor entre frase de origen y traducción. Es la figura más reproducida del artículo y muestra que la alineación aprendida se corresponde con la intuición lingüística, incluida la reordenación entre idiomas.
- La **curva de BLEU frente a longitud de frase**: el modelo con atención no se degrada como el de vector fijo. Ese gráfico **es** el argumento del paper.
- La justificación del **codificador bidireccional**.
- El detalle de que la red de puntuación es pequeña y se entrena de forma conjunta.

8. Evidencia y resultados

Traducción inglés → francés sobre WMT'14, comparando el modelo con atención (llamado *RNNsearch*) contra el codificador–decodificador de vector fijo (*RNNenc*), con dos límites de longitud de entrenamiento.

El resultado central no es una cifra puntual sino una **forma de curva**: el modelo de vector fijo pierde calidad conforme crece la longitud de la frase, y el modelo con atención mantiene su rendimiento.

Los valores de BLEU por configuración están en las tablas y figuras del artículo. Verificarlos allí antes de citarlos.

La miniatura de este eje aporta evidencia del mecanismo: una atención aditiva con 18 parámetros aprende una alineación correcta 4/4 y produce distribuciones con entropía cercana a cero, con $\sum \alpha = 1$ en cada fila.

9. Impacto

- Fija la atención como componente estándar de la traducción automática neuronal.
- Introduce la idea de **acceso a contenido por compatibilidad aprendida**, que es exactamente lo que generaliza el Transformer.
- Abre la línea de **interpretabilidad por atención**, con su promesa y su posterior crítica.
- Reformula el problema: de «comprimir bien» a «recuperar lo relevante en cada paso» — un cambio de marco que reaparece en RAG.

10. Limitaciones

1. **Coste cuadrático** en el producto (longitud de entrada × longitud de salida): se calcula una puntuación por cada par de posiciones.
2. **Sigue siendo recurrente**. No paraleliza sobre la longitud; ese límite persiste hasta Po8.
3. **La atención no es una explicación**. Trabajo posterior mostró que existen distribuciones de atención muy distintas que producen la misma salida.
4. **Una sola «cabeza»**: un único tipo de relación por paso. El multi-head de Po8 lo generaliza.
5. **La alineación aprendida no siempre coincide** con la alineación lingüística, y cuando coincide no hay garantía de que sea la causa de la traducción.
6. **Depende de corpus paralelos grandes**.

11. Errores comunes

Error	Corrección
«La atención muestra en qué se fijó el modelo, luego explica su decisión»	Jain y Wallace (2019) mostraron que se pueden construir distribuciones de atención muy distintas con la misma salida. Es una pista, no una explicación causal.
«Este paper inventó el Transformer»	Inventó el mecanismo de atención para traducción. El Transformer (2017) lo generaliza y elimina la recurrencia.
«La atención de Bahdanau es producto escalar»	Es aditiva : una capa con tanh. La multiplicativa es de Luong et al. (2015).
«Los pesos α son parámetros del modelo»	Son calculados en cada paso a partir de la entrada. Los parámetros son v , W , U .
«La alineación se supervisa»	No. Emerge de entrenar únicamente la traducción. Esa es la parte notable.

12. Relación con trabajos anteriores

- **P06 Seq2Seq (2014)** — el cuello de botella que motiva el trabajo.
- **Cho et al. (2014)** — codificador–decodificador con GRU, del mismo grupo. [arXiv:1406.1078](#)
- **Graves (2013)** — mecanismos de atención sobre secuencias en generación de escritura.
- **Alineación en traducción estadística** (modelos IBM, 1993) — la noción clásica de alineación que aquí se vuelve suave y diferenciable.

13. Relación con trabajos posteriores

- **Luong et al. (2015)** — atención multiplicativa, global y local. [arXiv:1508.04025](#)
- **P08 Transformer (2017)** — self-attention multi-cabeza sin recurrencia.
- **Jain y Wallace (2019)** — *Attention is not Explanation*. [arXiv:1902.10186](#)
- **Wiegrefe y Pinter (2019)** — *Attention is not not Explanation*: la réplica. Leer ambas es un ejercicio de nivel L3.

14. Notebook asociado

P07_attention_bahdanau.ipynb

Qué implementa: atención aditiva con W y U diagonales, entrenada por descenso de gradiente hasta producir una matriz de alineación correcta; medición de entropía por fila y comprobación de que los pesos suman 1.

Qué NO implementa: traducción real, codificador bidireccional entrenado, ni alineación latente. **Aquí la alineación se supervisa**; en el paper emerge sola. La diferencia es importante y se declara en el propio notebook.

```
ai-evolution paper-lab P07 --seed 7
```


15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe las tres ecuaciones de la atención aditiva y nombra cada símbolo.
Explicar	Explica por qué c_i lleva subíndice i y qué habría cambiado si no lo llevara.
Aplicar	Ejecuta el notebook con tres semillas y compara entropías.
Analizar	Sustituye el softmax por una normalización dividiendo por la suma y explica qué se rompe.
Evaluar	Lee el resumen de Jain y Wallace (2019) y decide si invalida el uso de la atención para depurar modelos. Argumenta.
Crear	Diseña un experimento que distinga «la atención señala lo relevante» de «la atención causa la salida». Di qué necesitarías para ejecutarlo.

16. Autoevaluación

1. ¿Qué problema concreto de Seq2Seq resuelve este mecanismo, y cómo se mide que lo resuelve?
2. ¿Por qué los pesos α deben sumar 1?
3. ¿Qué se aprende exactamente: los pesos α o los parámetros v, W, U ?
4. ¿Por qué el codificador es bidireccional?
5. ¿Qué diferencia hay entre atención aditiva y multiplicativa?
6. ¿Por qué una matriz de atención interpretable no equivale a una explicación?
7. ¿Qué límite de este trabajo persiste y motiva el paper siguiente?

17. Respuestas esperadas

1. El cuello de botella del vector fijo. Se mide con la curva de calidad frente a longitud de frase: con atención deja de degradarse.
2. Porque c_i es una **combinación convexa** de los estados del codificador. Sumar 1 garantiza que el contexto viva en el casco convexo de h_j y que la mezcla sea comparable entre pasos.
3. Se aprenden v, W, U . Los α se **calculan** en cada paso a partir de la entrada concreta.
4. Para que h_j resuma el contexto a ambos lados de la posición j , y no solo lo anterior.
5. La aditiva usa una capa con tanh y un vector v ; la multiplicativa usa directamente un producto escalar (opcionalmente con una matriz). La segunda es más barata y es la que adopta el Transformer con la escala $\sqrt{d_k}$.
6. Porque correlación entre peso y salida no implica causalidad; existen distribuciones alternativas que producen la misma predicción.
7. La recurrencia: el cómputo sigue siendo secuencial en la longitud, y eso limita la escala del entrenamiento.

18. Fuentes primarias

- Bahdanau, D., Cho, K. y Bengio, Y. (2014). *Neural Machine Translation by Jointly Learning to Align and Translate*. **ICLR 2015**. arXiv:1409.0473 · consultado 2026-08-16.

- Luong, M.-T., Pham, H. y Manning, C. D. (2015). *Effective Approaches to Attention-based Neural Machine Translation*. **EMNLP 2015**. arXiv:1508.04025 · consultado 2026-08-16.
- Jain, S. y Wallace, B. C. (2019). *Attention is not Explanation*. **NAACL 2019**. arXiv:1902.10186 · consultado 2026-08-16.

Evaluación — P07

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Neural Machine Translation by Jointly Learning to Align and Translate* (2014, arXiv:1409.0473 · ICLR 2015) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P07_attention_bahdanau.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P08 — Transformer · *Attention Is All You Need*

El paper que quitó la recurrencia del modelado de secuencias y, con ella, el último obstáculo para entrenar a gran escala. Casi todo lo que vino después es una rama de este bloque.

Nivel: L4 · Motor: `transformer` · Notebook: `P08_transformer.ipynb` · Miniaturas: T01–T08

1. Identificación

Campo	Valor
Título original	<i>Attention Is All You Need</i>
Autoría	Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, Illia Polosukhin
Año	2017
Venue	arXiv:1706.03762 (junio) · NeurIPS 2017 (diciembre)
Fuente primaria	arXiv:1706.03762 · actas NeurIPS 2017
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Tras P07, la atención había resuelto el cuello de botella. Pero el modelo seguía siendo **recurrente**, y eso arrastraba dos costes:

- Cómputo secuencial.** El estado `ht` necesita `h{t-1}`. No se puede paralelizar sobre la longitud de la secuencia, por muchas GPU que haya. El entrenamiento sobre corpus grandes se vuelve el cuello de botella real.
- Camino largo entre posiciones.** La señal entre la posición 1 y la 100 atraviesa 99 pasos. Cada paso es una oportunidad para que el gradiente se degrade.

Las alternativas convolucionales (ByteNet, ConvS2S) reducían la secuencialidad pero el camino entre posiciones distantes seguía creciendo con la distancia: logarítmica o linealmente.

La pregunta del paper: **si la atención ya conecta cualquier par de posiciones directamente, ¿para qué sigue haciendo falta la recurrencia?**

3. Propuesta

Una arquitectura codificador–decodificador construida **solo** con:

- **self-attention multi-cabeza** (la única operación que mezcla información entre posiciones);
- **redes feed-forward por posición** (aplicadas de forma independiente a cada token);
- **conexiones residuales** y **layer normalization** alrededor de cada subcapa;
- **codificación posicional** sumada a los embeddings, porque la atención por sí sola es indiferente al orden.

Sin recurrencia y sin convolución. Todas las posiciones de una capa se computan a la vez, y el camino entre dos posiciones cualesquiera es de longitud constante.

4. Intuición sin fórmulas

Cada palabra pregunta al resto de la frase «¿quién de vosotros me importa?», recibe una respuesta ponderada y se actualiza con ella. Y todas las palabras lo hacen **a la vez**, no en fila. Un RNN es una fila de personas pasándose un mensaje al oído; el Transformer es una sala donde todos leen el mismo tablón simultáneamente.

Dónde deja de funcionar la analogía: en la sala, cada persona tiene que leer a todas las demás. Con n personas eso son n^2 lecturas. Por eso el contexto largo es caro, y por eso el Transformer no es «gratis»: cambió coste secuencial por coste cuadrático.

5. Matemática mínima

Atención por producto escalar escalado (ecuación 1)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{Q \cdot K^T}{\sqrt{d_k}}\right) \cdot V$$

- $Q \in \mathbb{R}^{n \times d_k}$ consultas · $K \in \mathbb{R}^{m \times d_k}$ claves · $V \in \mathbb{R}^{m \times d_v}$ valores.
- $Q \cdot K^T$ mide compatibilidad; **softmax** la convierte en pesos que suman 1; el producto por V mezcla la información.

Por qué $\sqrt{d_k}$: si las componentes de q y k son independientes con media 0 y varianza 1, entonces $q \cdot k$ tiene media 0 y **varianza d_k** . Sin normalizar, la magnitud de los scores crece con la dimensión, el softmax se satura y los gradientes se apagan.

Multi-cabeza

$$\begin{aligned} \text{MultiHead}(Q, K, V) &= \text{Concat}(\text{head}_1, \dots, \text{head}_h) \cdot W^O \\ \text{head}_i &= \text{Attention}(Q \cdot W_i^Q, K \cdot W_i^K, V \cdot W_i^V) \\ d_k &= d_v = d_{\text{model}} / h \end{aligned}$$

Partir d_{model} en h subespacios permite atender a varios tipos de relación a la vez **sin aumentar el número de parámetros**, porque cada cabeza trabaja en una dimensión h veces menor.

Máscara causal

$$\text{score}_{ij} \leftarrow -\infty \quad \text{para } j > i \quad (\text{antes del softmax})$$

Se aplica **antes** del softmax, no después: poner a cero los pesos tras normalizar rompería la distribución.

Codificación posicional

```
PE(pos, 2i)   = sin( pos / 10000^{2i/d_model} )
PE(pos, 2i+1) = cos( pos / 10000^{2i/d_model} )
```

Se **suma** al embedding (no se concatena: eso cambiaría `d_model` y rompería la compatibilidad con las residuales).

Subcapa completa

```
salida = LayerNorm( x + Sublayer(x) )

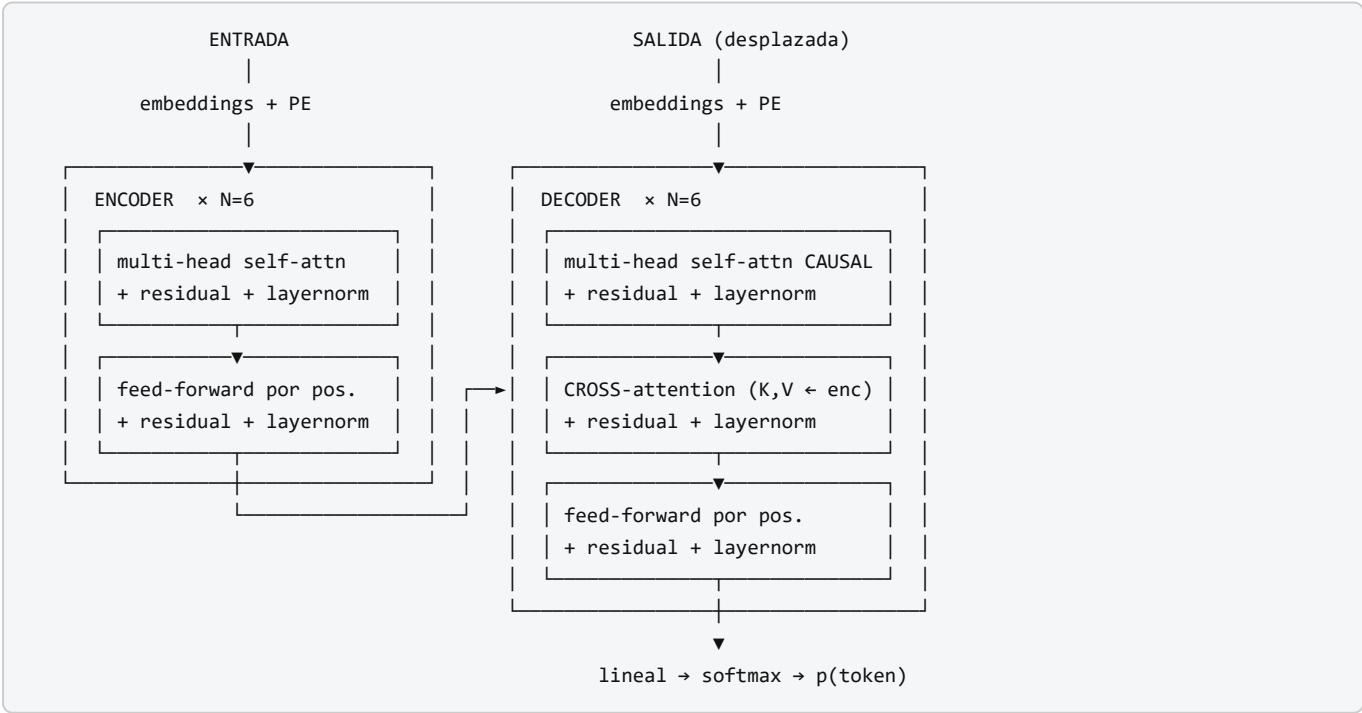
FFN(x) = max(0, x·W1 + b1) · W2 + b2      (misma FFN en todas las posiciones)
```

Complejidad (tabla 1 del paper)

Tipo de capa	Operaciones	Pasos secuenciales	Camino máximo
Self-attention	$O(n^2 \cdot d)$	$O(1)$	$O(1)$
Recurrente	$O(n \cdot d^2)$	$O(n)$	$O(n)$
Convolutacional	$O(k \cdot n \cdot d^2)$	$O(1)$	$O(\log_k n)$

La atención sale más barata en operaciones cuando `n < d`, y siempre gana en paralelismo y en longitud de camino.

6. Arquitectura o flujo



Configuración base del paper: `N=6`, `d_model=512`, `h=8`, `d_k=d_v=64`, `d_ff=2048`, dropout `0,1`, *label smoothing* `0,1`, optimizador Adam con calentamiento de la tasa de aprendizaje.

7. Qué observar en el paper original

Dónde	Qué buscar
Figura 1	El diagrama de la arquitectura. Cuenta cuántas subcapas hay en el decoder frente al encoder: son tres contra dos.
Sección 3.2.1	La ecuación 1 y la justificación explícita de $\sqrt{d_k}$. Está escrita, no es folclore.
Sección 3.2.2	Por qué multi-cabeza y por qué <code>d_k = d_model/h</code> .
Sección 3.3	La FFN por posición. Es donde vive la mayoría de los parámetros del bloque.
Sección 3.5	Codificación posicional, y la comparación con una versión aprendida (resultados casi idénticos; eligen la sinusoidal por extrapolación a longitudes mayores).
Tabla 1	Complejidad por tipo de capa. Es la tabla que justifica todo el diseño.
Tabla 2	Resultados de traducción y coste de entrenamiento en FLOPs . Compara calidad <i>y</i> cómputo.
Tabla 3	Las ablaciones . La sección más honesta: qué pasa al variar <code>h</code> , <code>d_k</code> , el dropout o la codificación posicional.
Sección 5.4	Regularización: dropout y label smoothing. Detalles sin los cuales no se reproduce.

8. Evidencia y resultados

Traducción sobre **WMT 2014**, en dos pares: inglés→alemán e inglés→francés, medida en BLEU.

- El modelo grande reporta **28,4 BLEU en EN→DE** y **41,8 BLEU en EN→FR**, superando a los mejores modelos y conjuntos publicados hasta entonces.
- El modelo base alcanza resultados competitivos con **un coste de entrenamiento sustancialmente menor** que las alternativas; el modelo grande se entrenó en el orden de días sobre 8 GPU P100.
- Generaliza a otra tarea (análisis sintáctico de constituyentes en inglés) con cambios mínimos.

Los valores exactos por configuración, junto con los FLOPs de entrenamiento, están en la tabla 2 del artículo, y las ablaciones en la tabla 3. **Verificarlos allí antes de citarlos:** es habitual encontrar cifras mezcladas entre modelo base, modelo grande y conjuntos.

La miniatura de este eje aporta evidencia del mecanismo, no del resultado: sin la escala $\sqrt{d_k}$ la entropía media de la atención cae de `1,09` a `0,49` con `d_model=8`, y la matriz causal es exactamente triangular inferior con filas que suman 1.

9. Impacto

- **Toda la familia de modelos de lenguaje actual** descende de este bloque: BERT usa el encoder, GPT usa el decoder, T5 y BART usan ambos.
- La paralelización hizo económicamente viable el entrenamiento a escala, lo que a su vez hizo observables las **leyes de escalado** (Kaplan et al., 2020; Hoffmann et al., 2022).
- Desplazó a las CNN en visión (ViT, 2020) y llegó a audio, código, proteínas y series temporales.
- Convirtió el bloque en una **pieza de infraestructura**: se optimiza a nivel de kernel de GPU, de compilador y de hardware.

🚫 Qué no significa el título

Attention Is All You Need es una consigna, no un teorema. El propio modelo del paper necesita, además de la atención:

Pieza	Sin ella...
Feed-forward por posición	El bloque pierde la mayor parte de su capacidad; la atención sola es una mezcla lineal ponderada
Conexiones residuales	El gradiente no atraviesa 6 bloques apilados
Layer normalization	El entrenamiento se desestabiliza
Codificación posicional	El modelo es ciego al orden : «el perro muerde» y «muerde el perro» son idénticos
Escala $\sqrt{d_k}$	El softmax se satura y los gradientes se apagan
Label smoothing y warmup	No se reproducen los resultados reportados

Lo que el título afirma con precisión es que **no hacen falta recurrencia ni convolución**. Eso es mucho, y es distinto de «basta la atención».

10. Limitaciones

1. **Coste y memoria** $O(n^2)$ en la longitud de secuencia. Con $n = 128\ 000$, la matriz de atención de una sola cabeza y capa ocupa decenas de gigabytes en `float32`.
2. **Extrapolación limitada** a longitudes mucho mayores que las de entrenamiento, pese a la codificación sinusoidal.
3. **Hambriento de datos**. Sin sesgo inductivo de localidad (a diferencia de una CNN), necesita más ejemplos para aprender estructura.
4. **La atención no es explicación**, igual que en P07.
5. **Sin memoria persistente** entre secuencias: todo lo que hay es la ventana de contexto.
6. **Coste ecológico y económico** del entrenamiento, no discutido en el artículo.
7. **El resultado se demuestra en traducción**, no en «lenguaje» en general. La generalización posterior es un hecho histórico, no una afirmación del paper.

11. Errores comunes

Error	Corrección
«El Transformer solo usa atención»	Ver la tabla de la sección 9. Usa FFN, residuales, layer norm y codificación posicional.
«El Transformer inventó la atención»	La inventó Bahdanau et al. (2014). Este paper la generaliza y elimina el resto.
«Es más eficiente que un RNN»	Es más paralelizable siempre; más eficiente en operaciones solo si $n < d$.
«GPT usa el Transformer completo»	GPT usa solo el decoder , sin cross-attention. BERT usa solo el encoder.
«La codificación posicional se concatena»	Se suma . Concatenar cambiaría <code>d_model</code> y rompería las residuales.
«La máscara causal se aplica al softmax»	Se aplica antes , poniendo los scores a <code>-∞</code> .
« $\sqrt{d_k}$ es una constante de ajuste empírica»	El paper la justifica por la varianza del producto escalar (sección 3.2.1).
«El Transformer entiende el lenguaje»	Ganó en BLEU sobre WMT 2014. Todo lo demás es interpretación.

12. Relación con trabajos anteriores

- **P07 Attention (2014)** — el mecanismo que se generaliza.
- **P06 Seq2Seq (2014)** — el patrón codificador–decodificador que se conserva.
- **P04 AlexNet (2012)** y **ResNet (2015)** — profundidad y conexiones residuales.
- **Ba, Kiros y Hinton (2016)** — layer normalization. [arXiv:1607.06450](https://arxiv.org/abs/1607.06450)
- **Lin et al. (2017)**, **Cheng et al. (2016)** — self-attention previa en otras formulaciones.
- **ByteNet y ConvS2S (2016–2017)** — alternativas convolucionales a la recurrencia.

13. Relación con trabajos posteriores

- **P09 BERT (2018)** — rama encoder.
- **P10 GPT-3 (2020)** — rama decoder llevada a escala.
- **T5, BART (2019–2020)** — rama codificador–decodificador preentrenada.
- **ViT (2020)** — el mismo bloque aplicado a parches de imagen.
- **Kaplan et al. (2020)** — leyes de escalado. [arXiv:2001.08361](https://arxiv.org/abs/2001.08361)
- **Hoffmann et al. (2022)** — Chinchilla: cómputo óptimo entre datos y parámetros. [arXiv:2203.15556](https://arxiv.org/abs/2203.15556)
- **Variantes eficientes** (atención dispersa, lineal, con E/S optimizada) — todas atacan el $O(n^2)$ que este paper introdujo.

14. Notebook asociado

Este hito tiene **tratamiento especial**: una miniatura general más ocho que desmontan el bloque pieza por pieza.

Notebook	Qué aísla
P08_transformer.ipynb	Vista integrada: escala, máscara, multi-cabeza, PE, residual, complejidad
T01	Por qué había que quitar la recurrencia
T02	Q, K, V y la ecuación 1
T03	Softmax, escala $\sqrt{d_k}$ y saturación
T04	Self-attention y máscara causal
T05	Multi-head attention
T06	Codificación posicional
T07	Residual, layer norm y feed-forward
T08	Encoder-decoder, complejidad y qué no dice el título

Qué NO implementan: proyecciones W^Q , W^K , W^V aprendidas, entrenamiento, tokenización BPE, traducción ni evaluación BLEU. Son miniaturas del mecanismo, en Python estándar.

```
ai-evolution paper-lab P08 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la ecuación 1 y define Q , K , V , d_k sin mirar.
Explicar	Explica por qué la codificación posicional es imprescindible, usando el experimento de permutación de T06.
Aplicar	Ejecuta T04 y verifica que la masa sobre el futuro es exactamente 0 y que cada fila suma 1.
Analizar	Con $d_{model}=512$ y $h=8$, calcula parámetros de la atención multi-cabeza y de la FFN de un bloque. ¿Cuál domina?
Evaluar	Un artículo divulgativo titula «la atención es todo lo que necesitas: el resto sobra». Escribe una refutación de 5 líneas apoyada en la tabla 3 del paper.
Crear	Diseña una ablación propia: elimina la codificación posicional de un modelo de juguete y define qué tarea revelaría el fallo.

16. Autoevaluación

- ¿Por qué se divide por $\sqrt{d_k}$ y qué ocurre exactamente si no se hace?
- ¿Cuál es la diferencia entre self-attention y cross-attention, y dónde aparece cada una?
- ¿Por qué la máscara causal se aplica antes del softmax?
- ¿Por qué h cabezas de dimensión d/h no cuestan más parámetros que una de dimensión d ?
- ¿Por qué el modelo sería ciego al orden sin codificación posicional? Da un ejemplo concreto.
- ¿En qué régimen de n y d la atención hace más operaciones que la recurrencia?
- ¿Qué gana y qué paga el Transformer respecto a un RNN? Una frase para cada cosa.
- ¿Qué afirma exactamente el título y qué no afirma?

17. Respuestas esperadas

1. Porque $q \cdot k$ tiene varianza d_k cuando las componentes son independientes con varianza 1. Sin escalar, los scores crecen con la dimensión, el softmax se satura hacia un vector casi one-hot y el gradiente de las posiciones no seleccionadas se anula.
2. En self-attention, Q , K y V proceden de la misma secuencia (encoder, y decoder con máscara). En cross-attention, Q viene del decoder y K , V del encoder: es el puente entre ambos.
3. Porque poner pesos a cero después de normalizar deja una distribución que ya no suma 1. Con $-\infty$ antes del softmax, el peso resultante es exactamente 0 y la fila sigue sumando 1.
4. Porque cada cabeza opera en d_{model}/h dimensiones. El total de las proyecciones concatenadas es el mismo que el de una única proyección de dimensión d_{model} .
5. Porque la atención es equivariante a permutaciones: reordenar la entrada solo reordena la salida. «El perro muerde al cartero» y «el cartero muerde al perro» producirían el mismo conjunto de representaciones.
6. Cuando $n > d$: $n^2 \cdot d > n \cdot d^2$ equivale a $n > d$. Con $d = 512$, a partir de secuencias de más de 512 tokens.
7. **Gana:** paralelismo total dentro de la capa y camino de longitud 1 entre cualquier par de posiciones.
Paga: coste y memoria cuadráticos en la longitud.
8. Afirma que **no hacen falta recurrencia ni convolución**. No afirma que baste la atención: el propio modelo lleva FFN, residuales, layer norm y codificación posicional.

18. Fuentes primarias

- Vaswani, A. et al. (2017). *Attention Is All You Need*. **NeurIPS 2017**. arXiv:1706.03762 · actas · consultado 2026-08-16.
- Ba, J. L., Kiros, J. R. y Hinton, G. E. (2016). *Layer Normalization*. arXiv:1607.06450 · consultado 2026-08-16.
- Kaplan, J. et al. (2020). *Scaling Laws for Neural Language Models*. arXiv:2001.08361 · consultado 2026-08-16.
- Hoffmann, J. et al. (2022). *Training Compute-Optimal Large Language Models*. arXiv:2203.15556 · consultado 2026-08-16.

Evaluación — P08

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Attention Is All You Need* (2017, arXiv:1706.03762 · NeurIPS (NIPS) 2017) · **Nivel:** L4

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P08_transformer.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P09 — BERT

El preentrenamiento deja de ser un truco y se convierte en la norma: un modelo base, muchas tareas, un ajuste pequeño para cada una.

Nivel: L3 · Motor: bert_mlm · Notebook: P09_bert.ipynb

1. Identificación

Campo	Valor
Título original	BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding
Autoría	Jacob Devlin, Ming-Wei Chang, Kenton Lee, Kristina Toutanova
Año	2018 (arXiv) · 2019 (NAACL)
Venue	arXiv:1810.04805 · NAACL-HLT 2019
Fuente primaria	arXiv:1810.04805 · ACL Anthology
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Con el Transformer disponible, la pregunta pasó a ser **cómo preentrenarlo**. Los modelos de lenguaje predicen el token siguiente y son, por construcción, **unidireccionales**: solo ven el pasado.

Para tareas de comprensión —responder preguntas, clasificar, extraer entidades— eso es un desperdicio: para interpretar una palabra hace falta lo que hay antes **y** después. ELMo (2018) lo había abordado concatenando dos LSTM entrenadas en direcciones opuestas, pero cada una seguía viendo un solo lado; la fusión era superficial.

El obstáculo técnico era real: **no se puede entrenar un modelo profundo y bidireccional con el objetivo de predecir el token siguiente**, porque a través de las capas cada token acabaría viéndose a sí mismo. La predicción sería trivial.

3. Propuesta

Cambiar el objetivo de preentrenamiento:

- 1. Masked Language Modeling (MLM).** Se enmascara un porcentaje de los tokens ($\approx 15\%$) y el modelo predice cada uno usando **todo** el contexto, a ambos lados. Enmascarar es lo que hace legítima la bidireccionalidad.
- 2. Next Sentence Prediction (NSP).** Dado un par de segmentos, predecir si el segundo sigue realmente al primero. Pensado para tareas que relacionan dos frases.

Y un procedimiento uniforme: **preentrenar una vez, ajustar todo el modelo** para cada tarea añadiendo una capa de salida mínima. La misma arquitectura sirve para clasificación, etiquetado y respuesta a preguntas.

Detalle importante del MLM: los tokens seleccionados no siempre se sustituyen por [MASK]. Se reparten entre [MASK], una palabra aleatoria y la palabra original, para que el modelo no aprenda a ignorar las posiciones sin máscara durante el ajuste fino, donde [MASK] no aparece.

4. Intuición sin fórmulas

Un examen de rellenar huecos. Para adivinar la palabra tapada lees toda la frase, no solo la mitad izquierda. Un modelo que renuncia al lado derecho está tirando la mitad de la evidencia.

Dónde deja de funcionar la analogía: rellenar huecos no es lo mismo que escribir un texto. BERT es excelente **comprendiendo** y no es un generador: su objetivo nunca fue producir texto token a token.

5. Matemática mínima

MLM:

$$L_{MLM} = - \sum_{t \in M} \log p(x_t \mid x_{\setminus M})$$

M = conjunto de posiciones enmascaradas

$x_{\setminus M}$ = la secuencia con esas posiciones ocultas (contexto completo a ambos lados)

NSP:

$$L_{NSP} = - \log p(\text{esSiguiente} \mid \text{segmentoA}, \text{segmentoB})$$

Objetivo total:

$$L = L_{MLM} + L_{NSP}$$

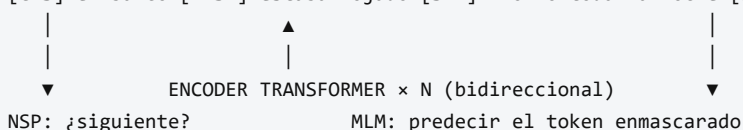
Entrada: [CLS] tokens_A [SEP] tokens_B [SEP], con tres embeddings sumados por posición (token + segmento + posición). El vector de [CLS] se usa como representación agregada de la secuencia para clasificación.

Escala del modelo: BERT-base con 12 capas, $d_{\text{model}}=768$, 12 cabezas, ≈ 110 M de parámetros; BERT-large con 24 capas, $d_{\text{model}}=1024$, 16 cabezas, ≈ 340 M.

6. Arquitectura o flujo

PREENTRENAMIENTO (no supervisado, corpus masivo)

[CLS] el banco [MASK] estaba mojado [SEP] llovió toda la noche [SEP]



AJUSTE FINO (supervisado, pocos datos por tarea)

mismo modelo + una capa de salida $\rightarrow$ clasificación / QA / etiquetado
se actualizan TODOS los parámetros

7. Qué observar en el paper original

- La **figura comparativa** entre BERT, GPT y ELMo: muestra visualmente qué significa «bidireccional profundo» frente a «unidireccional» y «bidireccional superficial».
- La **justificación de por qué no se puede usar el objetivo autorregresivo** para entrenar bidireccionalmente. Es el argumento central y suele resumirse mal.
- El detalle del **80/10/10** en el enmascarado y su motivación (la discrepancia entre preentrenamiento y ajuste fino).
- Los **estudios de ablación**: qué aporta NSP, qué aporta el número de pasos de preentrenamiento y qué pasa al usar el modelo como extractor de características congelado.
- La tabla de **GLUE** y las de **SQuAD**.

8. Evidencia y resultados

Evaluación sobre **GLUE** (once tareas de comprensión), **SQuAD v1.1 y v2.0** (respuesta a preguntas extractiva) y **SWAG** (inferencia de sentido común).

BERT establece nuevos máximos en las tres familias de tareas con el mismo procedimiento de ajuste, y el artículo reporta una mejora agregada en GLUE que llevó la puntuación por encima del 80 %. Las ablaciones muestran que **la bidireccionalidad es el factor dominante** y que la contribución de NSP es menor.

Las cifras exactas por tarea, para base y large, están en las tablas del artículo. Verificarlas allí antes de citarlas.

La miniatura de este eje aporta el argumento estructural, no las cifras: contando candidatos compatibles en un corpus de juguete, añadir el contexto derecho **nunca amplía** el conjunto de candidatos y en general lo reduce.

9. Impacto

- Convierte **preentrenar y ajustar** en el procedimiento estándar del PLN, desplazando a los modelos entrenados desde cero por tarea.
- Genera una familia enorme de descendientes: RoBERTa, ALBERT, DistilBERT, ELECTRA, DeBERTa y variantes por idioma y dominio.
- Sus embeddings contextuales son la base de los **recuperadores densos** que sostienen RAG.
- Instala la idea de **modelo fundacional**: un artefacto costoso de entrenar y barato de adaptar.

10. Limitaciones

1. **No genera texto.** El objetivo MLM no define una distribución autorregresiva utilizable para generación libre.
2. **Discrepancia preentrenamiento/ajuste.** [MASK] aparece en el preentrenamiento y nunca en el uso real; el reparto 80/10/10 mitiga el problema sin eliminarlo.

3. **Ineficiencia del MLM.** Solo se aprende de $\approx 15\%$ de los tokens por paso (ELECTRA atacó exactamente esto).
4. **NSP resultó de valor dudoso:** RoBERTa (2019) mostró que quitarlo no perjudica.
5. **Longitud de contexto limitada** (512 tokens en las configuraciones del paper).
6. **Coste de ajuste fino por tarea:** hay que guardar una copia completa del modelo por cada tarea, lo que motivó después los métodos de ajuste eficiente en parámetros.
7. **Sesgos del corpus** heredados y amplificados.

11. Errores comunes

Error	Corrección
«BERT es un LLM generativo»	Es un encoder para comprensión. La familia generativa viene de la rama decoder (P10).
«BERT fue el primer modelo bidireccional»	ELMo (2018) ya combinaba dos direcciones. BERT fue el primero profundamente bidireccional en todas las capas.
«MLM es lo mismo que predecir el token siguiente»	Son objetivos distintos y producen modelos con capacidades distintas.
«NSP es esencial en BERT»	Las ablaciones posteriores (RoBERTa) mostraron que se puede eliminar sin pérdida.
«BERT entiende el lenguaje»	Obtiene buenas puntuaciones en GLUE y SQuAD. Trabajos posteriores mostraron que parte de ese rendimiento se apoya en atajos estadísticos del dataset.
«Se enmascara siempre con [MASK]»	80 % [MASK] , 10 % palabra aleatoria, 10 % palabra original.

12. Relación con trabajos anteriores

- **P08 Transformer (2017)** — la arquitectura (solo el encoder).
- **ELMo (Peters et al., 2018)** — embeddings contextuales con LSTM bidireccionales. [ACL Anthology](#)
- **GPT-1 (Radford et al., 2018)** — preentrenar y ajustar con un decoder unidireccional; el trabajo con el que BERT se compara directamente.
- **ULMFit (Howard y Ruder, 2018)** — transferencia en PLN con LSTM. [arXiv:1801.06146](#)
- **P05 Word2Vec (2013)** — el antecesor estático de la idea de representación reutilizable.

13. Relación con trabajos posteriores

- **RoBERTa (2019)** — mismo modelo, mejor entrenado, sin NSP. [arXiv:1907.11692](#)
- **ELECTRA (2020)** — objetivo más eficiente que MLM. [arXiv:2003.10555](#)
- **Sentence-BERT (2019)** y los recuperadores densos → P11 RAG.
- **P10 GPT-3 (2020)** — la rama alternativa, que acabó dominando la conversación pública.

14. Notebook asociado

P09_bert.ipynb

Qué implementa: el argumento de la bidireccionalidad, mediante conteo de candidatos compatibles con contexto izquierdo frente a contexto completo sobre un corpus de juguete con polisemia («banco»).

Qué NO implementa: ningún Transformer, ningún preentrenamiento, ni NSP, ni el reparto 80/10/10, ni evaluación GLUE. Es un modelo de conteo que ilustra **por qué** el contexto derecho aporta información, no **cómo** lo aprovecha BERT.

```
ai-evolution paper-lab P09 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe los dos objetivos de preentrenamiento y qué predice cada uno.
Explicar	Explica por qué predecir el token siguiente impide entrenar bidireccionalmente en profundidad.
Aplicar	Añade frases al corpus del notebook y observa cómo cambia la ambigüedad del hueco.
Analizar	Compara las tres familias (encoder, decoder, encoder-decoder) por objetivo y tareas idóneas.
Evaluar	RoBERTa quitó NSP sin pérdida. ¿Qué dice eso sobre las ablaciones del paper original?
Crear	Diseña un objetivo de preentrenamiento alternativo y argumenta qué capacidad induciría.

16. Autoevaluación

1. ¿Por qué enmascarar hace legítima la bidireccionalidad?
2. ¿Qué problema resuelve el reparto 80/10/10?
3. ¿Por qué el MLM es menos eficiente por token que el objetivo autorregresivo?
4. ¿Para qué sirve el token [CLS] ?
5. ¿Por qué no conviene usar BERT para generar texto libre?
6. ¿Qué mostró RoBERTa sobre NSP, y qué implicación metodológica tiene?
7. ¿Qué significa exactamente una puntuación alta en GLUE, y qué no significa?

17. Respuestas esperadas

1. Porque el token objetivo se oculta de la entrada. Sin ocultarlo, la información fluiría por las capas y el modelo se vería a sí mismo: la predicción sería trivial.
2. La discrepancia entre preentrenamiento y uso: [MASK] nunca aparece en el ajuste fino ni en inferencia. Manteniendo a veces la palabra original o una aleatoria, el modelo debe construir una representación útil de **todas** las posiciones, no solo de las marcadas.
3. Porque solo aporta señal de aprendizaje el $\approx 15\%$ de posiciones enmascaradas, mientras que el objetivo autorregresivo produce una predicción por cada token de la secuencia.
4. Es una posición fija cuya representación final se usa como resumen de toda la secuencia para tareas de clasificación.
5. Porque su objetivo no define $p(x_t \mid x_{<t})$. Puede rellenar huecos, no continuar un texto de forma coherente token a token.

6. Que su aportación era prescindible. Implicación: una ablación del propio paper puede ser insuficiente si el resto de la configuración no está bien ajustada; la replicación independiente importa.
7. Que el modelo acierta un conjunto de tareas de comprensión con sus datasets y métricas. No significa comprensión general: parte del rendimiento puede provenir de correlaciones superficiales de esos datasets.

18. Fuentes primarias

- Devlin, J., Chang, M.-W., Lee, K. y Toutanova, K. (2019). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding*. **NAACL-HLT 2019**. arXiv:1810.04805 · ACL Anthology · consultado 2026-08-16.
- Peters, M. et al. (2018). *Deep Contextualized Word Representations (ELMo)*. **NAACL 2018**. ACL Anthology · consultado 2026-08-16.
- Liu, Y. et al. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach*. arXiv:1907.11692 · consultado 2026-08-16.

Evaluación — P09

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding* (2018, arXiv:1810.04805 · NAACL-HLT 2019) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P09_bert.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P10 — GPT-3 y el aprendizaje en contexto

La tarea deja de especificarse con un dataset etiquetado y pasa a especificarse **en el prompt**. Ningún peso cambia: cambia lo que hay escrito antes de la pregunta.

Nivel: L3 · Motor: `gpt3_icl` · Notebook: `P10_gpt3.ipynb`

1. Identificación

Campo	Valor
Título original	<i>Language Models are Few-Shot Learners</i>
Autoría	Tom B. Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah y otros (OpenAI)
Año	2020
Venue	arXiv:2005.14165 · NeurIPS 2020
Fuente primaria	arXiv:2005.14165
Acceso	Abierto (el modelo, no)
Fecha de consulta	2026-08-16

2. Problema anterior

El patrón que BERT consolidó exigía, para cada tarea nueva, un conjunto etiquetado y un ajuste fino que producía una copia entera del modelo. Eso tiene tres costes: **datos** anotados, **cómputo** de ajuste y **almacenamiento** por tarea.

Además, ese patrón no se parece a cómo una persona recibe una tarea: a un humano se le explica con una instrucción y dos ejemplos, no con diez mil casos etiquetados.

GPT-2 (2019) ya había mostrado indicios de resolver tareas sin ajuste fino. La pregunta abierta era si eso era una curiosidad o una **tendencia con el tamaño**.

3. Propuesta

Escalar la rama **decoder** del Transformer hasta 175 000 millones de parámetros, entrenarla con el objetivo autorregresivo de siempre sobre un corpus masivo, y evaluar sin ninguna actualización de gradiente en tres regímenes:

- **zero-shot** — solo la instrucción;
- **one-shot** — la instrucción y un ejemplo;
- **few-shot** — la instrucción y `k` ejemplos, con `k` limitado por la ventana de contexto.

El artículo mide sistemáticamente el rendimiento frente al **tamaño del modelo** en decenas de tareas, y documenta que la ventaja del régimen few-shot sobre el zero-shot **crece con la escala**.

4. Intuición sin fórmulas

En lugar de reentrenar a un empleado para cada tarea nueva, le dejas una nota con dos ejemplos resueltos y la tarea pendiente. Lo que hace es **seguir el patrón que ve en la nota**.

Dónde deja de funcionar la analogía: el empleado recuerda mañana lo que aprendió hoy. El modelo no. Cierras la sesión y no queda rastro: no hubo aprendizaje, hubo condicionamiento.

5. Matemática mínima

El objetivo de entrenamiento es exactamente el de siempre:

$$L = - \sum_t \log p_{\theta}(x_t \mid x_{<t})$$

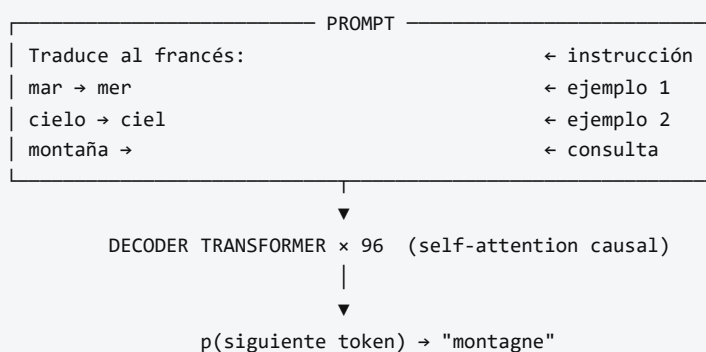
Lo nuevo es el **protocolo de evaluación**, no la matemática:

```
zero-shot : p(y | instrucción, x)
one-shot  : p(y | instrucción, (x1,y1), x)
few-shot  : p(y | instrucción, (x1,y1)...(xk,yk), x)
```

θ NO cambia en ninguno de los tres casos.

Escala del modelo mayor: 175 000 millones de parámetros, 96 capas, `d_model = 12 288`, 96 cabezas de atención, ventana de contexto de 2 048 tokens.

6. Arquitectura o flujo



Pesos actualizados: 0. Memoria entre llamadas: ninguna.

7. Qué observar en el paper original

- La **figura de las curvas de escala**: rendimiento frente a tamaño del modelo, con una curva por régimen (zero, one, few-shot). La separación entre curvas creciendo con el tamaño es el resultado

principal.

- La **tabla de composición del corpus** de entrenamiento (Common Crawl filtrado, WebText2, libros, Wikipedia) y sus pesos de muestreo.
- La **sección sobre contaminación de datos**: los autores analizan explícitamente el solapamiento entre los benchmarks y el corpus de entrenamiento, y reconocen limitaciones en ese análisis. Leerla es obligatorio antes de citar cualquier cifra.
- La **sección de impactos más amplios**: desinformación, sesgo y consumo energético. Es inusualmente extensa para la época.
- Las tareas donde el modelo **falla**: razonamiento aritmético de varios dígitos, inferencia en algunos conjuntos, comparaciones que requieren varios pasos.
- El artículo tiene decenas de páginas: casi todo es apéndice de evaluación.

8. Evidencia y resultados

Evaluación en más de veinte conjuntos de datos: modelado de lenguaje, respuesta a preguntas de libro cerrado, traducción, razonamiento de sentido común, comprensión lectora, SuperGLUE y tareas sintéticas (aritmética, uso de palabras nuevas, corrección gramatical).

Resultado central: **en varias tareas el régimen few-shot se aproxima a métodos ajustados específicamente**, y la brecha se estrecha conforme crece el modelo. En otras, sigue muy por detrás.

Las cifras por tarea y régimen están en las tablas del artículo. Verificarlas allí, y leer antes la sección de contaminación: parte del rendimiento en algunos conjuntos podría estar afectado por solapamiento con el corpus de entrenamiento.

La miniatura de este eje **no reproduce nada de esto**. Simula el fenómeno con inducción explícita de hipótesis para mostrar cómo cada ejemplo del prompt restringe el espacio de tareas compatibles.

9. Impacto

- Desplaza el centro de gravedad del PLN del **ajuste fino** al **diseño de prompts**, y crea de facto la práctica que hoy se llama ingeniería de prompts y de contexto.
- Hace del **tamaño** una variable de primer orden y motiva el estudio sistemático de las leyes de escalado.
- Populariza el acceso a modelos por **API** en lugar de por pesos, con las consecuencias de reproducibilidad que eso implica.
- Es el punto de partida de InstructGPT: un modelo potente que **no** hace lo que se le pide de forma fiable necesita alineación.

10. Limitaciones

1. **El contexto se paga en cada llamada**. Los ejemplos ocupan tokens, cuestan latencia y dinero, y compiten con el resto del contexto.
2. **Sensibilidad al prompt**. El orden de los ejemplos, el formato y hasta los separadores alteran el resultado. Eso convierte muchas comparaciones en poco fiables.

3. **Sin memoria entre llamadas.** No hay aprendizaje persistente de ningún tipo.
4. **Contaminación de benchmarks,** reconocida por los propios autores.
5. **Aritmética y razonamiento de varios pasos** siguen siendo débiles.
6. **Alucinación:** genera afirmaciones plausibles y falsas, sin forma de citar fuente. Es el problema que ataca RAG.
7. **Modelo cerrado:** sin pesos públicos, la replicación independiente es imposible.
8. **Coste energético y económico** del entrenamiento, no repetible por la mayoría de equipos.
9. **No sigue instrucciones de forma fiable.** Completa texto; que eso coincida con lo que quería el usuario es otra cosa.

11. Errores comunes

Error	Corrección
«Con los ejemplos del prompt, el modelo aprende»	Se condiciona . No hay gradiente, ni actualización, ni persistencia. Llamar «aprendizaje» a esto es la fuente de la mayoría de los malentendidos.
«GPT-3 inventó el prompting»	El artículo lo sistematiza y lo mide a escala. La idea de condicionar con instrucciones es anterior (GPT-2, 2019).
«GPT-3 es ChatGPT»	ChatGPT deriva de modelos alineados con instrucciones (P12). GPT-3 base no está ajustado para dialogar ni para obedecer.
«Few-shot supera al ajuste fino»	En algunas tareas se acerca. En muchas otras el ajuste fino sigue ganando claramente.
«El modelo razona»	Produce continuaciones estadísticamente plausibles. Que a veces coincidan con un razonamiento correcto no autoriza la afirmación.
«Los benchmarks del paper son limpios»	Los propios autores documentan solapamiento con el corpus y las limitaciones de su análisis.

12. Relación con trabajos anteriores

- **P08 Transformer (2017)** — la rama decoder.
- **GPT-1 (Radford et al., 2018)** — preentrenar y ajustar con decoder. Informe técnico de OpenAI.
- **GPT-2 (Radford et al., 2019)** — primeros indicios de tareas resueltas sin ajuste fino.
- **Kaplan et al. (2020)** — leyes de escalado, del mismo equipo y meses antes. [arXiv:2001.08361](#)
- **P09 BERT (2018)** — el paradigma que este trabajo cuestiona.

13. Relación con trabajos posteriores

- **P11 RAG (2020)** — ataca la alucinación y el conocimiento congelado.
- **P12 InstructGPT (2022)** — alineación con instrucciones.
- **Wei et al. (2022), Chain-of-Thought** — razonamiento paso a paso inducido en el prompt. [arXiv:2201.11903](#)
- **Hoffmann et al. (2022), Chinchilla** — corrige el reparto entre parámetros y datos. [arXiv:2203.15556](#)
- **P13 ReAct (2022)** — el modelo pasa a controlar un bucle.

14. Notebook asociado

P10_gpt3.ipynb

Qué implementa: una maqueta del aprendizaje en contexto mediante eliminación explícita de hipótesis: con o ejemplos hay cuatro tareas compatibles; con 2, queda una sola. La precisión en casos no vistos sube en consecuencia.

Qué NO implementa —y esto es lo importante—: nada de GPT-3. No hay modelo de lenguaje, ni 175 000 millones de parámetros, ni corpus. GPT-3 **no enumera hipótesis:** condiciona una distribución aprendida. La maqueta sirve para razonar sobre el mecanismo, no para afirmar nada sobre el modelo real.

```
ai-evolution paper-lab P10 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Define zero-shot, one-shot y few-shot con precisión.
Explicar	Explica por qué el aprendizaje en contexto no es aprendizaje, usando tres propiedades ausentes.
Aplicar	Cambia la tarea latente del notebook y mide cuántos ejemplos hacen falta para desambiguar.
Analizar	Diseña dos prompts con el mismo contenido y distinto orden de ejemplos. ¿Qué implicación tiene para comparar modelos?
Evaluar	Lee la sección de contaminación del paper y decide qué cifras aceptarías sin reservas.
Crear	Diseña un benchmark resistente a la contaminación y justifica cada decisión de diseño.

16. Autoevaluación

1. ¿Qué cambia exactamente en el modelo entre zero-shot y few-shot?
2. ¿Por qué el aprendizaje en contexto no es aprendizaje?
3. ¿Qué limita el número de ejemplos que puedes poner en el prompt?
4. ¿Qué es la contaminación de benchmark y por qué es crítica en este paper concreto?
5. ¿Por qué la sensibilidad al orden de los ejemplos es un problema metodológico serio?
6. ¿Qué diferencia hay entre GPT-3 y un asistente conversacional actual?
7. ¿Qué tres problemas de GPT-3 atacan directamente los tres papers siguientes de la ruta?

17. Respuestas esperadas

1. Nada en el modelo. Cambia únicamente el texto de entrada. Los pesos son idénticos.
2. Porque no hay actualización de parámetros, no hay persistencia entre llamadas y no hay generalización acumulativa: cada llamada parte de cero.
3. La ventana de contexto (2 048 tokens en el modelo del paper) y el coste por token.
4. Que ejemplos de test aparezcan en el corpus de preentrenamiento. Es crítico aquí porque el corpus es Common Crawl a gran escala, donde es probable que estén muchos benchmarks públicos; los autores lo

analizan y reconocen que su análisis es imperfecto.

5. Porque si reordenar los mismos ejemplos cambia el resultado, la métrica no mide solo la capacidad del modelo: mide también una elección arbitraria del evaluador. Sin reportar la varianza sobre órdenes, las comparaciones son frágiles.
6. El asistente está **alineado** con instrucciones mediante ajuste supervisado y aprendizaje con preferencias humanas. GPT-3 base solo continúa texto.
7. Conocimiento congelado y no citable → RAG. No seguir instrucciones → InstructGPT. No poder consultar el mundo ni actuar → ReAct.

18. Fuentes primarias

- Brown, T. B. et al. (2020). *Language Models are Few-Shot Learners*. **NeurIPS 2020**. arXiv:2005.14165 · consultado 2026-08-16.
- Kaplan, J. et al. (2020). *Scaling Laws for Neural Language Models*. arXiv:2001.08361 · consultado 2026-08-16.
- Bender, E. M., Gebru, T., McMillan-Major, A. y Shmitchell, S. (2021). *On the Dangers of Stochastic Parrots*. **FAccT 2021**. doi.org/10.1145/3442188.3445922 · consultado 2026-08-16.

Evaluación — P10

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Language Models are Few-Shot Learners* (2020, arXiv:2005.14165 · NeurIPS 2020) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P10_gpt3.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P11 — RAG

Separar lo que el sistema **sabe** de cómo **razona**: el conocimiento pasa a un índice consultable, actualizable y citable, en vez de quedar congelado en los pesos.

Nivel: L3 · Motor: rag · Notebook: P11_rag.ipynb

1. Identificación

Campo	Valor
Título original	Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks
Autoría	Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni y otros
Año	2020
Venue	arXiv:2005.11401 · NeurIPS 2020
Fuente primaria	arXiv:2005.11401
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Todo lo que un modelo preentrenado sabe está en sus pesos. Eso tiene cuatro consecuencias incómodas:

1. **No se puede actualizar** sin reentrenar o ajustar.
2. **No se puede auditar**: no hay forma de saber de dónde salió una afirmación.
3. **No se puede corregir** un dato erróneo de forma quirúrgica.
4. **El modelo alucina**: cuando no sabe, genera algo plausible con la misma confianza.

Trabajos previos habían mostrado que los modelos de lenguaje almacenan cantidades sorprendentes de conocimiento factual (Petroni et al., 2019), y a la vez que ese conocimiento es opaco y difícil de mantener.

3. Propuesta

Combinar **memoria paramétrica** (los pesos del modelo generativo) con **memoria no paramétrica** (un índice denso sobre un corpus de documentos):

1. Un **recuperador denso** (DPR) codifica la consulta y los documentos en el mismo espacio vectorial y devuelve los `k` pasajes más similares.
2. Un **generador seq2seq** (BART) produce la respuesta condicionada a la consulta y a los pasajes recuperados.
3. Ambos se entrenan de forma **conjunta**: la señal de la generación llega al codificador de la consulta (el índice de documentos se mantiene fijo, por coste).

El artículo propone dos variantes: **RAG-Sequence**, que usa el mismo documento para toda la respuesta, y **RAG-Token**, que puede cambiar de documento en cada token generado.

4. Intuición sin fórmulas

Un examen a libro cerrado frente a uno a libro abierto. A libro cerrado, si no lo recuerdas, lo inventas. A libro abierto, buscas la página, la citas y quien corrige puede verificarla.

Dónde deja de funcionar la analogía: en el examen a libro abierto, encontrar la página garantiza casi la respuesta. Aquí no: el generador puede recuperar el documento correcto y aun así contradecirlo. Recuperar y responder son dos fallos independientes.

5. Matemática mínima

Recuperación densa:

$$\text{sim}(x, z) = q(x)^T \cdot d(z) \rightarrow \text{top-k}(x) \text{ por producto escalar}$$

RAG-Sequence (un documento para toda la salida):

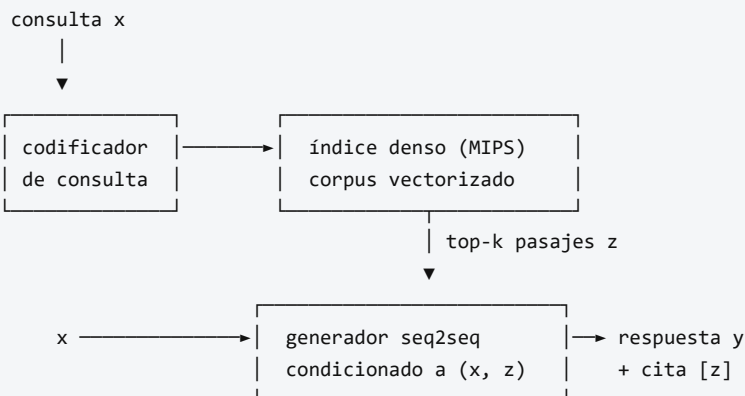
$$p(y \mid x) \approx \sum_{z \in \text{top-k}(x)} p_{\eta}(z \mid x) \cdot \prod_t p_{\theta}(y_t \mid x, z, y_{<t})$$

RAG-Token (documento distinto por token):

$$p(y \mid x) \approx \prod_t \sum_{z \in \text{top-k}(x)} p_{\eta}(z \mid x) \cdot p_{\theta}(y_t \mid x, z, y_{<t})$$

- p_{η} — recuperador: memoria **no paramétrica**, sustituible sin reentrenar.
- p_{θ} — generador: memoria **paramétrica**.
- El documento latente z se **marginaliza**: no se elige uno y se descarta el resto.

6. Arquitectura o flujo



Actualizar conocimiento = reindexar documentos. Sin tocar los pesos.

7. Qué observar en el paper original

- La **distinción** entre **RAG-Sequence** y **RAG-Token** y en qué tipo de pregunta gana cada una.

- La **marginalización sobre documentos**: es lo que diferencia RAG de «meter los documentos en el prompt». Los **k** documentos contribuyen a la vez, ponderados.
- El **experimento de actualización de conocimiento**: cambiar el índice cambia lo que el sistema responde, **sin reentrenar**. Es la demostración práctica de la tesis.
- La comparación contra modelos de **libro cerrado** (solo paramétricos) y contra sistemas extractivos.
- La discusión sobre **diversidad y especificidad** de las respuestas generadas frente a las extractivas.

8. Evidencia y resultados

Evaluación en tareas intensivas en conocimiento: **respuesta a preguntas de dominio abierto** (Natural Questions, TriviaQA, WebQuestions, CuratedTrec), **generación de preguntas de Jeopardy** y **verificación de hechos** (FEVER).

RAG establece nuevos máximos en varias de las tareas de respuesta a preguntas de libro abierto del momento, y genera respuestas más específicas y factuales que los modelos puramente paramétricos comparables. En verificación de hechos alcanza resultados cercanos a sistemas con supervisión de evidencia mucho más rica.

Las cifras por tarea (Exact Match, precisión) están en las tablas del artículo. Verificarlas allí antes de citarlas.

La miniatura de este eje aporta el mecanismo y un fallo real: con recuperación léxica, la consulta correcta sitúa el documento pertinente en primer lugar (score 0,78); una consulta sin respuesta en el corpus **también** devuelve un documento, con score bajo. De ahí la necesidad de un umbral de abstención.

9. Impacto

- Da nombre a un patrón que hoy es la arquitectura por defecto de la mayoría de aplicaciones empresariales de IA generativa.
- Convierte la **base de datos vectorial** en pieza de infraestructura estándar.
- Introduce la exigencia de **atribución**: una respuesta sin fuente pasa a considerarse incompleta en dominios regulados.
- Reformula el mantenimiento: actualizar conocimiento se convierte en un problema de **ingeniería de datos**, no de entrenamiento.

10. Limitaciones

1. **Tres fallos independientes.** (a) el documento no está en el índice; (b) está y no se recupera; (c) se recupera y el generador lo contradice. Evaluar RAG exige medir los tres por separado.
2. **Alucinación con cita.** El caso más peligroso: la cita es real y relevante, pero la afirmación no está en ella. Difícil de detectar a simple vista.
3. **Calidad limitada por la del corpus.** Un índice con documentos obsoletos o contradictorios produce respuestas obsoletas o contradictorias, con toda la apariencia de rigor.
4. **Coste de latencia** añadido por la recuperación.

5. **El índice de documentos permanece fijo** durante el entrenamiento conjunto del paper: no se reentrena el codificador de documentos.
6. **Fragmentación (chunking) y granularidad** son decisiones de ingeniería con enorme impacto y ninguna respuesta universal.
7. **No sabe abstenerse.** El paper no incorpora un mecanismo de «no lo sé»; hay que añadirlo.

11. Errores comunes

Error	Corrección
«RAG elimina las alucinaciones»	Las reduce y las hace auditables . Un modelo puede citar bien y afirmar mal.
«RAG es meter documentos en el prompt»	En el paper, el documento es una variable latente marginalizada y el sistema se entrena conjuntamente. Meter texto en el prompt es un caso particular, mucho más simple.
«Si la respuesta lleva cita, es correcta»	Hay que verificar que la afirmación esté en el documento citado.
«Recuperar bien basta»	Recuperación y generación fallan de forma independiente; hay que medir ambas.
«RAG usa búsqueda por palabras clave»	Usa recuperación densa (DPR). La léxica es una alternativa más simple, útil y a menudo complementaria.
«Con RAG el modelo ya no necesita saber nada»	La memoria paramétrica sigue haciendo el trabajo de comprender la consulta y redactar la respuesta.

12. Relación con trabajos anteriores

- **P05 Word2Vec (2013)** y **P09 BERT (2018)** — la representación densa que hace posible la recuperación semántica.
- **Karpukhin et al. (2020)**, **DPR** — el recuperador denso que RAG usa. [arXiv:2004.04906](#)
- **Lewis et al. (2019)**, **BART** — el generador seq2seq. [arXiv:1910.13461](#)
- **Petroni et al. (2019)** — cuánto conocimiento factual almacenan los modelos de lenguaje. [arXiv:1909.01066](#)
- **REALM (Gua et al., 2020)** — trabajo contemporáneo con preentrenamiento aumentado por recuperación. [arXiv:2002.08909](#)

13. Relación con trabajos posteriores

- **FiD (2021)** — fusión en el decodificador de muchos pasajes.
- **Self-RAG, RAG con reordenamiento y RAG con verificación (2023+)** — añaden crítica y abstención.
- **GraphRAG y RAG sobre grafos de conocimiento** — estructura explícita sobre el corpus.
- **P13 ReAct (2022)** — la recuperación deja de ser un paso fijo y se convierte en una **acción** que el modelo decide ejecutar.

14. Notebook asociado

Qué implementa: recuperación por similitud de coseno sobre frecuencias de términos, generación con citas explícitas, el contraste con la respuesta sin recuperación, un caso de **alucinación con cita** y un mecanismo de abstención por umbral.

Qué NO implementa: DPR, entrenamiento conjunto, marginalización sobre documentos ni BART. La recuperación es léxica, no densa: ilustra el patrón, no el método del paper.

```
ai-evolution paper-lab P11 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la fórmula de RAG-Sequence e identifica qué representa z .
Explicar	Explica la diferencia entre memoria paramétrica y no paramétrica con un ejemplo de mantenimiento.
Aplicar	Ejecuta el notebook y ajusta el umbral de abstención hasta que la consulta sin respuesta se rechace.
Analizar	Diseña tres consultas que provoquen cada uno de los tres modos de fallo.
Evaluar	Te presentan un sistema con «95 % de respuestas con cita». ¿Qué métricas pides antes de aceptarlo?
Crear	Diseña un protocolo de evaluación con tres métricas separadas (recuperación, fidelidad, abstención) y define cómo medir cada una.

16. Autoevaluación

1. ¿Qué significa marginalizar sobre el documento latente y por qué no es lo mismo que elegir el mejor?
2. ¿Cuál es la diferencia práctica entre RAG-Sequence y RAG-Token?
3. Nombra los tres modos de fallo de un sistema RAG y una métrica para cada uno.
4. ¿Por qué una alucinación con cita es más peligrosa que una sin cita?
5. ¿Qué se necesita para actualizar el conocimiento de un sistema RAG?
6. ¿Por qué hace falta un umbral de abstención y qué se rompe sin él?
7. ¿Qué parte del sistema sigue dependiendo de la memoria paramétrica del modelo?

17. Respuestas esperadas

1. Sumar sobre los k documentos ponderando por su probabilidad de recuperación, en vez de condicionar solo al mejor. Así la respuesta agrega evidencia de varios pasajes y es más robusta a un error del recuperador en la primera posición.
2. RAG-Sequence usa el mismo documento para toda la respuesta (mejor cuando la respuesta viene de una fuente única); RAG-Token puede cambiar de documento en cada token (mejor cuando la respuesta combina hechos de varias fuentes).
3. (a) cobertura del índice $\rightarrow$ recall del corpus; (b) recuperación $\rightarrow$ recall@k / MRR; (c) fidelidad $\rightarrow$ proporción de afirmaciones verificables en el contexto recuperado.

4. Porque la cita aporta una señal de credibilidad que el lector usa como atajo. Verificarla exige leer la fuente, y la mayoría no lo hace.
5. Reindexar los documentos. No hace falta tocar los pesos del modelo.
6. Porque el recuperador **siempre** devuelve algo: sin umbral, una consulta fuera del dominio recibe el documento «menos malo» y el generador construirá una respuesta sobre él.
7. Comprender la consulta, integrar los pasajes y redactar una respuesta coherente. RAG cambia de dónde vienen los hechos, no quién los redacta.

18. Fuentes primarias

- Lewis, P. et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. **NeurIPS 2020**. arXiv:2005.11401 · consultado 2026-08-16.
- Karpukhin, V. et al. (2020). *Dense Passage Retrieval for Open-Domain Question Answering*. **EMNLP 2020**. arXiv:2004.04906 · consultado 2026-08-16.
- Guu, K. et al. (2020). *REALM: Retrieval-Augmented Language Model Pre-Training*. arXiv:2002.08909 · consultado 2026-08-16.

Evaluación — P11

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks* (2020, arXiv:2005.11401 · NeurIPS 2020) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P11_rag.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P12 — InstructGPT y RLHF

El salto de «modelo que completa texto» a «asistente que hace lo que se le pide». Y la constatación incómoda de que «mejor» es una decisión de quién etiqueta, no una propiedad del modelo.

Nivel: L3 · Motor: rlhf · Notebook: P12_instructgpt_rlhf.ipynb

1. Identificación

Campo	Valor
Título original	Training language models to follow instructions with human feedback
Autoría	Long Ouyang, Jeff Wu, Xu Jiang, Diogo Almeida y otros (OpenAI)
Año	2022
Venue	arXiv:2203.02155 · NeurIPS 2022
Fuente primaria	arXiv:2203.02155
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

GPT-3 se entrena maximizando la verosimilitud del texto de internet. Ese objetivo **no es** «sé útil, honesto e inocuo». Es «predice qué viene después».

La consecuencia práctica: el modelo continúa el texto de la forma más plausible, que a menudo no es lo que el usuario quería. Ante «explícame X», puede generar más preguntas sobre X en lugar de explicarlo, porque en internet eso también es una continuación plausible.

A esto se le llama **desalineación de objetivo**: el objetivo de entrenamiento y la intención del usuario no coinciden. Y no se arregla con más escala, porque escalar mejora el objetivo equivocado.

3. Propuesta

Tres etapas encadenadas:

- SFT (ajuste supervisado).** Etiquetadores escriben respuestas de demostración a prompts reales; el modelo se ajusta sobre ellas.
- Modelo de recompensa (RM).** Para cada prompt se muestrean varias respuestas y los etiquetadores las **ordenan por preferencia**. Con esas comparaciones se entrena un modelo que asigna un escalar $r(x, y)$.
- Optimización por refuerzo.** La política se optimiza con PPO para maximizar r , con una **penalización KL** que la mantiene cerca del modelo SFT.

La decisión metodológica clave: **pedir comparaciones, no puntuaciones**. Ordenar dos respuestas es más fácil, más rápido y mucho más consistente entre anotadores que puntuar del 1 al 10.

4. Intuición sin fórmulas

Enseñar a alguien a cocinar sin darle recetas: le pones dos platos delante y le dices cuál está mejor. Con suficientes comparaciones, aprende qué buscas — aunque tú nunca hayas sabido formularlo con palabras.

Dónde deja de funcionar la analogía: el aprendiz humano entiende *por qué* un plato es mejor. El modelo de recompensa solo aprende **qué correlaciona** con la preferencia. Si tus platos preferidos resultan ser siempre los más grandes, aprenderá a servir raciones enormes.

5. Matemática mínima

Modelo de recompensa (Bradley-Terry)

$$p(y_w > y_l \mid x) = \sigma(r(x, y_w) - r(x, y_l))$$

$$L_{RM} = - E[\log \sigma(r(x, y_w) - r(x, y_l))]$$

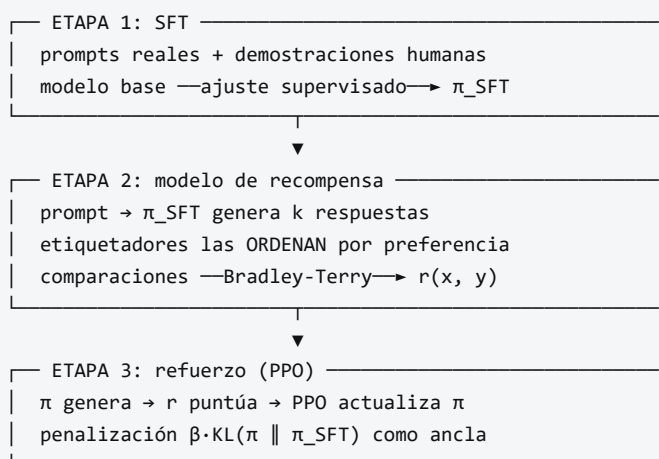
y_w = respuesta preferida, y_l = rechazada. Solo importa la **diferencia**: la escala absoluta de r no está determinada.

Optimización con restricción

$$\max_{\pi} E_{\{y \sim \pi(\cdot|x)\}} [r(x, y)] - \beta \cdot KL(\pi(\cdot|x) \parallel \pi_{SFT}(\cdot|x))$$

El término KL es imprescindible: sin él la política deriva hacia texto degenerado que engaña al modelo de recompensa. β fija el precio de alejarse del modelo base.

6. Arquitectura o flujo



7. Qué observar en el paper original

- El **resultado más citado**: los etiquetadores prefieren las salidas del modelo InstructGPT de **1 300 millones** de parámetros sobre las de GPT-3 de **175 000 millones**. Alinear resultó más rentable que escalar dos órdenes de magnitud.
- La sección sobre **quién etiquetó**: perfil demográfico de los anotadores, criterios de selección y guía de anotación. Es una de las secciones más honestas y menos leídas del artículo, y determina qué significa «mejor» en todo el trabajo.
- El **impuesto de alineación** (*alignment tax*): la caída de rendimiento en algunos benchmarks académicos, y la mezcla de gradientes de preentrenamiento que usan para mitigarla.
- Las secciones sobre **límites**: el modelo sigue alucinando, sigue siendo sensible al prompt y puede obedecer instrucciones dañinas si están bien formuladas.

8. Evidencia y resultados

Evaluación principal por **preferencia humana**: se comparan salidas de distintos modelos sobre la distribución de prompts real de la API y sobre conjuntos públicos.

- InstructGPT es preferido a GPT-3 por un margen amplio, **incluso con modelos mucho más pequeños**.
- Mejora en veracidad (TruthfulQA) y reducción de generación tóxica frente al modelo base.
- Generaliza a instrucciones y a idiomas poco representados en los datos de ajuste.
- Se documenta un retroceso en algunos benchmarks académicos, mitigado parcialmente.

Las proporciones exactas de preferencia y los resultados por benchmark están en las figuras y tablas del artículo. Verificarlos allí antes de citarlos.

La miniatura de este eje muestra el mecanismo del modelo de recompensa: cinco comparaciones bastan para que un `r` lineal ordene cuatro respuestas candidatas, situando arriba la útil y honesta y abajo la peligrosa.

9. Impacto

- Es la técnica que hizo posibles los asistentes conversacionales de uso masivo. **ChatGPT descende directamente de este trabajo**, no de GPT-3 base.
- Convierte la **alineación** en una disciplina de ingeniería con etapas, datos y métricas, no solo en una discusión conceptual.
- Instala la **preferencia humana como métrica**, con todo lo que eso implica: la evaluación deja de ser puramente automática.
- Abre la línea de trabajo sobre reward hacking, RLAIIF y alineación con principios explícitos (Constitutional AI, 2022).

10. Limitaciones

1. **Reward hacking.** Si una característica superficial correlaciona con la preferencia (por ejemplo, la longitud), la política aprende a explotarla.
2. **Los valores son los de los anotadores.** El modelo de recompensa no descubre qué es mejor: reproduce el criterio de un grupo concreto de personas con una guía concreta.
3. **Pipeline complejo y caro:** tres modelos, datos humanos y un bucle de RL delicado de estabilizar. Este es el problema que ataca DPO.
4. **Impuesto de alineación:** caída en algunos benchmarks académicos.
5. **Sigue alucinando.** La alineación cambia el estilo y la obediencia, no añade conocimiento ni verificación.
6. **Sicofancia:** si a los anotadores les gustan las respuestas que les dan la razón, el modelo aprende a darla.
7. **PPO es sensible** a hiperparámetros; reproducirlo requiere mucho oficio no documentado.

11. Errores comunes

Error	Corrección
«RLHF hace que el modelo diga la verdad»	Hace que produzca respuestas preferidas por los anotadores . La correlación con la verdad es parcial y no está garantizada.
«RLHF añade conocimiento al modelo»	No. Reorganiza el comportamiento sobre el conocimiento que ya tenía.
«El modelo de recompensa es objetivo por ser un modelo»	Es un resumen estadístico de juicios humanos concretos, con sus sesgos.
«InstructGPT es GPT-3 con más datos»	Son tres etapas y dos modelos adicionales. La diferencia es de procedimiento, no de escala.
«Este paper inventó RLHF»	Christiano et al. (2017) ya lo aplicaba a control con preferencias humanas. Este trabajo lo lleva a modelos de lenguaje a escala.
«El término KL es un detalle de regularización»	Sin él la política colapsa hacia texto degenerado con recompensa alta. Es estructural.

12. Relación con trabajos anteriores

- **P10 GPT-3 (2020)** — el modelo base y el problema.
- **Christiano et al. (2017)** — RL a partir de preferencias humanas. [arXiv:1706.03741](#)
- **Stiennon et al. (2020)** — RLHF aplicado a resumen; el precedente directo. [arXiv:2009.01325](#)
- **Schulman et al. (2017), PPO** — el algoritmo de optimización. [arXiv:1707.06347](#)
- **Bradley y Terry (1952)** — el modelo de comparaciones por pares.

13. Relación con trabajos posteriores

- **Bai et al. (2022), Constitutional AI** — reemplazar parte del juicio humano por principios explícitos y crítica del propio modelo. [arXiv:2212.08073](#)
- **P15 DPO (2023)** — el mismo objetivo sin modelo de recompensa ni RL.
- **RLAIF (2023)** — retroalimentación generada por IA.
- **Trabajo sobre sicofancia y reward hacking (2023+)** — el estudio sistemático de los fallos que este pipeline induce.

14. Notebook asociado

P12_instructgpt_rlhf.ipynb

Qué implementa: el modelo de recompensa Bradley-Terry entrenado por descenso de gradiente sobre comparaciones por pares, el ranking resultante, una selección best-of-n, y una demostración numérica de cómo el término KL penaliza alejarse del modelo base.

Qué NO implementa: SFT, PPO, generación de texto, ni etiquetadores humanos. Las «respuestas» son cuatro vectores de características escritos a mano.

```
ai-evolution paper-lab P12 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Enumera las tres etapas y qué produce cada una.
Explicar	Explica por qué se piden comparaciones en vez de puntuaciones absolutas.
Aplicar	Cambia una comparación en el notebook y observa cómo se reordena el ranking.
Analizar	Construye un conjunto de preferencias donde el modelo aprenda a premiar la longitud.
Evaluar	¿Un modelo alineado es más seguro o solo lo parece? Argumenta con dos límites del propio paper.
Crear	Diseña una guía de anotación de una página para un dominio que conozcas, y explica qué sesgo introduce cada regla.

16. Autoevaluación

1. ¿Qué significa exactamente «desalineación de objetivo» en este contexto?
2. ¿Por qué las comparaciones son mejores datos que las puntuaciones?
3. ¿Qué ocurre si se elimina el término KL del objetivo?
4. ¿Qué es el reward hacking y cómo lo detectarías en un sistema propio?
5. ¿Por qué un modelo de 1 300 millones alineado puede preferirse a uno de 175 000 millones sin alinear?
6. ¿Qué es el impuesto de alineación?
7. ¿Qué **no** arregla RLHF?

17. Respuestas esperadas

1. Que el objetivo optimizado (verosimilitud del texto) difiere de lo que el usuario quiere (una respuesta útil). Escalar mejora el primero sin acercar el segundo.
2. Porque son más consistentes entre anotadores: una escala numérica se interpreta de forma distinta por cada persona y deriva con el tiempo, mientras que «cuál prefieres» es estable.
3. La política deriva libremente hacia regiones donde el modelo de recompensa da valores altos pero el texto es degenerado. El KL ancla la política al modelo SFT.

4. Explotar una característica que correlaciona con la recompensa sin mejorar la calidad real. Se detecta evaluando con humanos independientes del modelo de recompensa y controlando por variables superficiales como la longitud.
5. Porque el modelo grande optimiza un objetivo distinto del que el usuario quiere. La alineación reorganiza el comportamiento, y eso vale más que capacidad bruta mal dirigida.
6. La pérdida de rendimiento en algunos benchmarks académicos que aparece al alinear, y que el paper mitiga mezclando gradientes de preentrenamiento.
7. La alucinación, la falta de conocimiento actualizado, la sensibilidad al prompt y el sesgo heredado del corpus. Cambia el comportamiento, no lo que el modelo sabe.

18. Fuentes primarias

- Ouyang, L. et al. (2022). *Training language models to follow instructions with human feedback*. **NeurIPS 2022**. [arXiv:2203.02155](https://arxiv.org/abs/2203.02155) · consultado 2026-08-16.
- Christiano, P. et al. (2017). *Deep Reinforcement Learning from Human Preferences*. [arXiv:1706.03741](https://arxiv.org/abs/1706.03741) · consultado 2026-08-16.
- Stiennon, N. et al. (2020). *Learning to summarize with human feedback*. [arXiv:2009.01325](https://arxiv.org/abs/2009.01325) · consultado 2026-08-16.
- Bai, Y. et al. (2022). *Constitutional AI: Harmlessness from AI Feedback*. [arXiv:2212.08073](https://arxiv.org/abs/2212.08073) · consultado 2026-08-16.

Evaluación — P12

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Training language models to follow instructions with human feedback* (2022, arXiv:2203.02155 · NeurIPS 2022) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P12_instructgpt_rlhf.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P13 — ReAct

El modelo deja de ser un generador de texto y pasa a ser el **controlador** de un bucle que observa el mundo y actúa sobre él. Es el paper donde empieza, en sentido técnico, el agente.

Nivel: L2 · Motor: `react` · Notebook: `P13_react.ipynb`

1. Identificación

Campo	Valor
Título original	<i>ReAct: Synergizing Reasoning and Acting in Language Models</i>
Autoría	Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, Yuan Cao
Año	2022 (arXiv) · 2023 (ICLR)
Venue	arXiv:2210.03629 · ICLR 2023
Fuente primaria	arXiv:2210.03629
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Existían dos líneas de trabajo que no se hablaban:

- **Razonar sin actuar.** El razonamiento en cadena (Wei et al., 2022) mejora tareas de varios pasos haciendo que el modelo escriba su razonamiento. Pero ese razonamiento ocurre **dentro** del modelo, sin contacto con el mundo: si un hecho intermedio es falso, nada lo corrige y el error se propaga con toda la apariencia de rigor.
- **Actuar sin razonar.** Los modelos que emiten acciones sobre un entorno (navegar, buscar, manipular) no descomponen la tarea ni replantean el plan cuando una acción no da lo esperado.

Faltaba el acoplamiento: que **lo que se observa condicione lo que se piensa a continuación**.

3. Propuesta

Intercalar dos tipos de salida en la misma secuencia generada:

Thought → Action → Observation → Thought → Action → Observation → ... → Finish

- **Thought** — texto libre: descomponer, planificar, replantear, decidir cuándo parar.
- **Action** — llamada a una herramienta del entorno (buscar, consultar, moverse).
- **Observation** — resultado real devuelto por el entorno; **no lo genera el modelo**.

Lo notable es la sinergia bidireccional: el razonamiento decide qué acción tomar, y la observación real ancla el siguiente razonamiento. El método se aplica mediante prompting con pocos ejemplos, sin entrenamiento adicional.

4. Intuición sin fórmulas

Un investigador que piensa en voz alta mientras consulta archivos. No decide toda la investigación de antemano ni abre carpetas al azar: piensa qué necesita, lo busca, lee lo que encontró y ese hallazgo cambia lo que busca después.

Dónde deja de funcionar la analogía: el investigador sabe cuándo su fuente es poco fiable. El bucle no: si la herramienta devuelve un dato erróneo, lo propaga sin dudar. Lo comprueba el notebook.

5. Matemática mínima

No hay ecuación nueva. Lo que cambia es el **espacio de acciones** del modelo:

Generación estándar:

$a_t \in \text{vocabulario de tokens}$

ReAct:

$a_t \in \text{vocabulario} \cup \text{espacio de acciones del entorno}$

$\text{contexto}_t = (\text{objetivo}, o_1, a_1, \dots, o_{\{t-1\}}, a_{\{t-1\}})$

$a_t \sim p_\theta(\cdot \mid \text{contexto}_t)$

Las observaciones **o** entran en el contexto pero **no** las genera el modelo: vienen del entorno. Ese es el anclaje.

6. Arquitectura o flujo

pregunta: «¿cuántos habitantes tiene la capital de Francia?»

Thought: no sé qué ciudad es; primero busco la capital	
Action: buscar("capital de Francia")	
Observation: "París"	← DEL ENTORNO
Thought: es París; ahora busco su población	
Action: buscar("población de París")	
Observation: "2 100 000"	← DEL ENTORNO
Thought: tengo el dato, puedo responder	
Action: finish("2 100 000")	

Sin el primer paso, la pregunta es irresoluble con una sola búsqueda.

7. Qué observar en el paper original

- La **comparación de cuatro modos**: solo razonar (CoT), solo actuar (Act), ReAct, y las combinaciones ReAct+CoT-SC. La tabla que las compara es el resultado central.
- Las **trazas de ejemplo** en los apéndices: contienen tanto casos de éxito como fallos, y los fallos son más instructivos.
- El análisis de **modos de error**: alucinación de razonamiento, bucles repetitivos, búsquedas poco informativas.
- Los **cuatro entornos**: HotpotQA y FEVER (razonamiento sobre conocimiento), ALFWorld y WebShop (toma de decisiones interactiva). La variedad importa: demuestra que el patrón no es específico de la búsqueda de texto.

8. Evidencia y resultados

- En **HotpotQA** y **FEVER**, ReAct reduce la alucinación frente a CoT puro, porque las observaciones de la Wikipedia anclan los hechos; en exactitud pura, CoT puede ganar en algunos casos, y el artículo lo reconoce en lugar de ocultarlo.
- En **ALFWorld** y **WebShop** —tareas interactivas— ReAct supera de forma clara a los métodos de solo actuar (imitación y refuerzo), con muy pocos ejemplos en el prompt.
- La combinación de ReAct con autoconsistencia obtiene los mejores resultados globales.

Las cifras por entorno y método están en las tablas del artículo. Verificarlas allí: la comparación es matizada y resumirla como «ReAct gana siempre» es incorrecto.

La miniatura de este eje aporta la evidencia estructural: una pregunta de dos saltos es irresoluble con una sola búsqueda, y la traza muestra cómo la primera observación **construye** la segunda consulta.

9. Impacto

- Es el patrón de referencia de casi todos los frameworks de agentes que vinieron después.
- Introduce la **traza auditable** como artefacto de primera clase: se puede revisar qué hizo el sistema y por qué, sin abrir los pesos.
- Convierte «llamar a una herramienta» en parte del bucle de razonamiento, no en un postprocesado.
- Prepara el terreno para Toolformer (aprender **cuándo** llamar) y para los sistemas agentic (memoria, reflexión, presupuesto).

10. Limitaciones

1. **Sin criterio de parada, el bucle no termina.** Es el fallo operativo número uno: la herramienta falla, el agente reintenta, el coste crece sin límite.
2. **La calidad está acotada por la de las herramientas.** Si la fuente miente, el agente miente con seguridad y con traza.
3. **La traza no es fiel.** El texto del «pensamiento» es una generación más y puede no describir el proceso real del modelo. Sirve para depurar, no como prueba.
4. **Coste:** cada paso es una llamada al modelo más una a la herramienta. La latencia se multiplica.

5. **Superficie de ataque:** una observación puede contener texto que intente redirigir al agente (inyección de prompt indirecta).
6. **Depende de un modelo grande:** con modelos pequeños, la calidad de los pensamientos se degrada rápidamente.
7. **Sin memoria entre episodios:** cada tarea empieza de cero.

11. Errores comunes

Error	Corrección
«La traza explica cómo razonó el modelo»	Es texto generado. Útil para auditar decisiones, no una prueba del proceso interno.
«ReAct siempre supera a CoT»	Depende de la tarea. El propio paper reporta casos donde CoT es mejor.
«ReAct es un framework»	Es un patrón de prompting . Los frameworks lo implementan; no son el paper.
«Un agente sin límite de pasos es más capaz»	Es más caro y más frágil. El criterio de parada es una función de seguridad, no una limitación.
«Si el agente cita una observación, el dato es correcto»	El dato es tan fiable como la herramienta que lo produjo.
«ReAct requiere entrenamiento especial»	Se aplica con pocos ejemplos en el prompt, sin entrenar.

12. Relación con trabajos anteriores

- **P10 GPT-3 (2020)** — el aprendizaje en contexto que hace viable aplicar el patrón con pocos ejemplos.
- **Wei et al. (2022), Chain-of-Thought** — razonar explícitamente en el prompt. [arXiv:2201.11903](#)
- **Wang et al. (2022), autoconsistencia** — muestrear varios razonamientos y votar. [arXiv:2203.11171](#)
- **P11 RAG (2020)** — recuperar como paso fijo; aquí pasa a ser una acción decidida por el modelo.

13. Relación con trabajos posteriores

- **P14 Toolformer (2023)** — aprender cuándo llamar a la herramienta.
- **Reflexion (Shinn et al., 2023)** — añadir autocritica y memoria entre intentos. [arXiv:2303.11366](#)
- **Generative Agents (Park et al., 2023)** — memoria episódica y recuperación por relevancia. [arXiv:2304.03442](#)
- **P16 Sistemas agentic** — el bucle convertido en sistema.
- **Model Context Protocol** — estandarización posterior del acceso a herramientas. [modelcontextprotocol.io](#)

14. Notebook asociado

`P13_react.ipynb`

Qué implementa: la comparación entre una estrategia de solo actuar y el bucle completo sobre una pregunta de dos saltos; un bucle sin criterio de parada como anti-patrón; una versión con límite de pasos,

detección de repetición y escalamiento; y un experimento donde la base de conocimiento devuelve un dato corrupto.

Qué NO implementa: ningún modelo de lenguaje. Los «pensamientos» están escritos a mano. En el paper los genera el modelo, y esa es precisamente la parte que puede fallar.

```
ai-evolution paper-lab P13 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe los tres elementos del bucle y di cuál no genera el modelo.
Explicar	Explica por qué una pregunta de dos saltos falla con una sola búsqueda.
Aplicar	Añade una tercera consulta encadenada al notebook y comprueba que la traza sigue siendo legible.
Analizar	Corrompe la base de conocimiento y analiza cómo se propaga el error por la traza.
Evaluar	Un agente resuelve una tarea en 20 pasos y otro en 4. ¿Cuál es mejor? Di qué necesitas saber antes de responder.
Crear	Diseña un verificador que contraste cada observación con una segunda fuente, y estima su coste en llamadas.

16. Autoevaluación

1. ¿Qué aporta la observación que no puede aportar el razonamiento interno?
2. ¿Por qué el razonamiento en cadena puro alucina hechos?
3. ¿Qué tres mecanismos evitan que un bucle se vuelva infinito?
4. ¿Por qué una traza legible no es una explicación del modelo?
5. ¿Qué riesgo de seguridad introduce leer observaciones de fuentes externas?
6. ¿En qué se diferencia ReAct de RAG?
7. ¿Qué límite de ReAct ataca directamente el paper siguiente?

17. Respuestas esperadas

1. Información del mundo que el modelo no tiene o tiene desactualizada, y una señal de corrección: si la acción no da lo esperado, el siguiente pensamiento puede replantear.
2. Porque genera los hechos intermedios desde sus parámetros, sin contrastarlos. Un eslabón falso se integra en la cadena y arrastra al resto con apariencia de solidez.
3. Límite de pasos, detección de estados o consultas repetidas, y escalamiento a un humano ante fallo persistente de la herramienta. Se aceptan también: presupuesto de tokens y de coste.
4. Porque es texto generado por el mismo proceso que produce la respuesta. Puede ser una racionalización posterior y no la causa de la acción.
5. Inyección de prompt indirecta: la observación puede contener instrucciones dirigidas al agente. Todo contenido observado debe tratarse como dato, nunca como instrucción.
6. En RAG la recuperación es un paso fijo del pipeline. En ReAct es una **acción** que el modelo decide ejecutar, cuántas veces y con qué consulta.

7. Que las herramientas y cuándo usarlas se especifican a mano en el prompt. Toolformer aprende ese «cuándo» de forma autosupervisada.

18. Fuentes primarias

- Yao, S. et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. **ICLR 2023**. arXiv:2210.03629 · consultado 2026-08-16.
- Wei, J. et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. **NeurIPS 2022**. arXiv:2201.11903 · consultado 2026-08-16.
- Shinn, N. et al. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. arXiv:2303.11366 · consultado 2026-08-16.

Evaluación — P13

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *ReAct: Synergizing Reasoning and Acting in Language Models* (2022, arXiv:2210.03629 · ICLR 2023) · **Nivel:** L2

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P13_react.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P14 — Toolformer

El modelo decide por sí solo cuándo le conviene llamar a una herramienta, usando como criterio su propia pérdida. Nadie etiqueta nada.

Nivel: L3 · Motor: `toolformer` · Notebook: `P14_toolformer.ipynb`

1. Identificación

Campo	Valor
Título original	<i>Toolformer: Language Models Can Teach Themselves to Use Tools</i>
Autoría	Timo Schick, Jane Dwivedi-Yu, Roberto Dessì, Roberta Raileanu y otros
Año	2023
Venue	arXiv:2302.04761 · NeurIPS 2023
Fuente primaria	arXiv:2302.04761
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

ReAct demostró que un modelo puede usar herramientas dentro de un bucle. Pero **qué herramientas existen y cuándo conviene llamarlas** venía escrito a mano en el prompt, o requería datos anotados por humanos.

Eso tiene dos problemas: anotar es caro, y lo anotado limita el modelo a las herramientas y situaciones que alguien previó. Además, los modelos de lenguaje fallan de forma sistemática y predecible en cosas que un programa de tres líneas resuelve: aritmética exacta, fechas, datos actualizados, traducción de términos raros.

La pregunta: **¿puede el modelo descubrir solo dónde le conviene pedir ayuda?**

3. Propuesta

Un procedimiento autosupervisado en cuatro pasos:

1. **Muestrear** posiciones candidatas del texto donde podría insertarse una llamada, y generar llamadas plausibles usando unos pocos ejemplos en el prompt.
2. **Ejecutar** cada llamada y obtener su resultado real.
3. **Filtrar**: conservar la llamada solo si el resultado **reduce la pérdida** de predecir el texto que viene después, por encima de un umbral τ .
4. **Reentrenar** el modelo sobre el corpus enriquecido con las llamadas que sobrevivieron.

La idea central es el criterio del paso 3: **la utilidad de una herramienta se mide por cuánto ayuda a predecir el texto siguiente**. Es una señal automática, disponible en cualquier corpus, sin ningún anotador.

Herramientas del artículo: calculadora, sistema de preguntas y respuestas, buscador, traductor y calendario.

4. Intuición sin fórmulas

Un estudiante con una calculadora sobre la mesa. Prueba a usarla en muchos sitios del examen y se queda con la costumbre solo donde le ayudó de verdad. Donde no, deja de usarla. Nadie le dice cuándo: lo deduce del resultado.

Dónde deja de funcionar la analogía: el estudiante sabe si la calculadora dio un número correcto. El modelo solo sabe si el resultado le **ayudó a predecir el texto siguiente**. Son cosas distintas: una herramienta que devuelve siempre lo mismo puede resultar «útil» por razones espurias.

5. Matemática mínima

Para una posición i , con llamada c y respuesta r :

$$\begin{aligned} L^+ &= L(x_{i:} \mid \text{prefijo} + [c \rightarrow r]) && \text{con el resultado de la herramienta} \\ L^- &= \min(L(x_{i:} \mid \text{prefijo}), && \text{sin llamada} \\ &\quad L(x_{i:} \mid \text{prefijo} + [c \rightarrow \emptyset])) && \text{con la llamada pero sin resultado} \end{aligned}$$

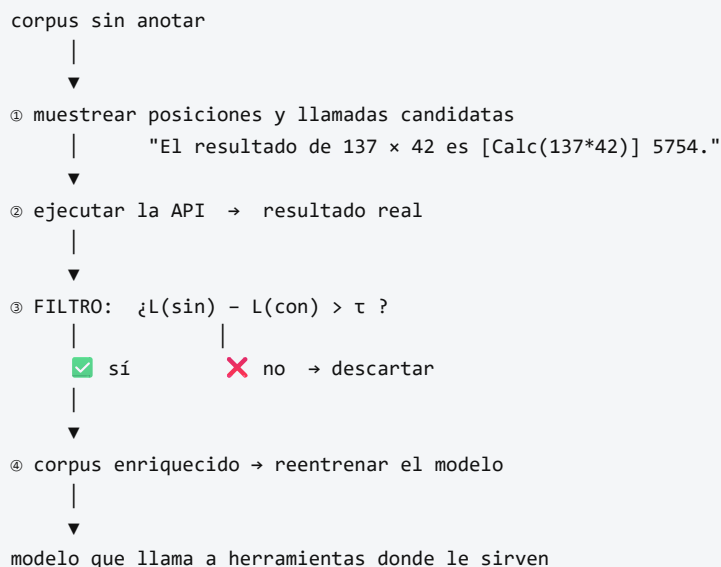
Se conserva la llamada si:

$$L^- - L^+ > \tau$$

L es la pérdida ponderada de predecir los tokens siguientes. τ es el umbral de filtrado.

El segundo término de L^- importa: compara contra **hacer la llamada sin recibir respuesta**, lo que aísla el valor del **resultado** frente al valor de la mera presencia del texto de la llamada.

6. Arquitectura o flujo



7. Qué observar en el paper original

- El **criterio de filtrado** y su justificación: es el corazón del método y ocupa poco espacio en el artículo.
- La **comparación de L** contra dos alternativas (sin llamada y con llamada sin resultado). Ese detalle evita conservar llamadas que solo ayudan por el texto que introducen.
- La **tabla de rendimiento por herramienta**: cuánto aporta cada una en qué benchmark. No todas aportan igual.
- El **análisis de escala**: el método necesita un modelo suficientemente capaz para generar llamadas plausibles; por debajo de cierto tamaño no funciona.
- La comprobación de que **no se degrada** la capacidad de modelado de lenguaje del modelo base.

8. Evidencia y resultados

El modelo base es un GPT-J de 6 700 millones de parámetros, enriquecido con el procedimiento y reentrenado.

Resultados evaluados en modo zero-shot sobre tareas donde cada herramienta debería ayudar: aritmética (calculadora), preguntas de conocimiento (buscador y sistema de QA), tareas multilingües (traductor) y preguntas sensibles a la fecha (calendario).

Toolformer mejora sustancialmente sobre el modelo base del mismo tamaño y, en varias de estas tareas, alcanza o supera a modelos mucho mayores **sin** herramientas, manteniendo su capacidad de modelado de lenguaje.

Las cifras por benchmark y herramienta están en las tablas del artículo. Verificarlas allí antes de citarlas.

La miniatura de este eje aísla el criterio de filtrado: de tres candidatas, sobrevive la que reduce la pérdida por encima de τ ; la llamada absurda, que la **aumenta**, se descarta sola.

9. Impacto

- Establece que el uso de herramientas puede **aprenderse**, no solo prompt-earse. Es un cambio conceptual respecto a ReAct.
- Introduce un criterio automático y barato de utilidad, reutilizable en otros contextos.
- Anticipa la normalización del *tool calling* como capacidad nativa de los modelos, en lugar de como capa externa.
- Su lógica sobrevive a su implementación: hoy el acceso a herramientas se estandariza con protocolos, pero la pregunta «¿esta llamada aporta valor medible?» sigue siendo la correcta.

10. Limitaciones

1. **Enseña cuándo llamar, no garantiza que la herramienta acierte.** Un Toolformer conectado a una API que devuelve basura aprenderá a llamarla con confianza.

2. **τ no tiene valor universal.** Bajo, conserva llamadas inútiles (coste y latencia); alto, descarta llamadas útiles.
3. **Coste de construcción del corpus:** hay que muestrear muchas candidatas y ejecutarlas todas.
4. **Cada llamada es independiente.** No hay composición: no aprende a encadenar dos herramientas para un resultado intermedio.
5. **Depende de la escala del modelo base:** por debajo de cierto tamaño, las llamadas candidatas no son suficientemente buenas.
6. **La reducción de pérdida es una aproximación de la utilidad**, no la utilidad misma. Puede premiar correlaciones espurias.
7. **Sin gestión de errores:** no aborda qué hacer cuando la API falla o tarda.

11. Errores comunes

Error	Corrección
«Toolformer garantiza respuestas correctas»	Garantiza que la llamada ayudó a predecir el texto . La corrección del resultado depende de la herramienta.
«Más llamadas a herramientas = mejor agente»	Cada llamada cuesta latencia y dinero. La métrica útil es la utilidad marginal por llamada, no el conteo.
«Es lo mismo que ReAct»	ReAct usa herramientas en el prompt; Toolformer aprende dónde insertarlas y reentrena con ello.
«Necesita anotadores humanos»	No. El criterio es la propia pérdida del modelo: por eso el título dice <i>teach themselves</i> .
«El filtro compara solo con no llamar»	Compara también contra llamar sin recibir respuesta, para aislar el valor del resultado.

12. Relación con trabajos anteriores

- **P13 ReAct (2022)** — usar herramientas dentro del bucle.
- **P10 GPT-3 (2020)** — aprendizaje en contexto, que permite generar las llamadas candidatas con pocos ejemplos.
- **WebGPT (Nakano et al., 2021)** — navegación con supervisión humana; el contraste directo con la autosupervisión de Toolformer. [arXiv:2112.09332](https://arxiv.org/abs/2112.09332)
- **PAL / Program-Aided Language Models (2022)** — delegar el cálculo a un intérprete. [arXiv:2211.10435](https://arxiv.org/abs/2211.10435)

13. Relación con trabajos posteriores

- **Tool calling nativo** en las APIs de modelos comerciales: la funcionalidad se vuelve producto.
- **Model Context Protocol** — estandarización del descubrimiento y la invocación de herramientas. modelcontextprotocol.io
- **P16 Sistemas agentic** — herramientas tipadas, permisos, presupuesto y seguridad de la cadena de suministro.
- **Trabajo sobre composición de herramientas (2023+)** — encadenar llamadas, que este paper no aborda.

14. Notebook asociado

P14_toolformer.ipynb

Qué implementa: el criterio de filtrado sobre tres candidatas (útil, marginal y absurda), un barrido de τ que muestra el compromiso entre conservar de más y de menos, y métricas que sustituyen al conteo bruto de llamadas.

Qué NO implementa: modelo de lenguaje, muestreo de candidatas, ejecución real de APIs ni reentrenamiento. Las pérdidas están fijadas para exhibir el criterio, y así se declara en la salida.

```
ai-evolution paper-lab P14 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe el criterio de filtrado y explica qué compara.
Explicar	Explica por qué comparar contra «llamada sin resultado» es necesario.
Aplicar	Barre τ en el notebook y anota cuántas llamadas sobreviven en cada caso.
Analizar	Diseña un caso donde la reducción de pérdida premie una llamada inútil.
Evaluar	Un equipo mide el éxito de su agente por «llamadas a herramientas por respuesta». Evalúa la métrica y propón tres alternativas.
Crear	Define una herramienta nueva para un dominio que conozcas y describe qué texto del corpus revelaría su utilidad.

16. Autoevaluación

1. ¿Qué señal sustituye a la anotación humana y por qué está disponible gratis?
2. ¿Qué pasa con τ muy bajo? ¿Y muy alto?
3. ¿Por qué el método necesita un modelo base suficientemente grande?
4. ¿Qué **no** garantiza este método sobre las respuestas de las herramientas?
5. ¿Por qué el conteo de llamadas es una mala métrica de calidad?
6. ¿En qué se diferencia de ReAct, en una frase?
7. ¿Qué capacidad relacionada con herramientas queda fuera del alcance de este paper?

17. Respuestas esperadas

1. La reducción de la pérdida de predecir el texto siguiente. Está disponible en cualquier corpus sin etiquetar, porque el «objetivo» es el propio texto que ya venía después.
2. Con τ bajo se conservan llamadas inútiles: el modelo aprende a invocar herramientas constantemente, con su coste y su latencia. Con τ alto se descartan llamadas útiles y el modelo vuelve a inventar los datos.

3. Porque las llamadas candidatas se generan con pocos ejemplos en el prompt. Si el modelo no produce llamadas plausibles, el filtro no tiene nada bueno que conservar.
4. Que sean correctas. El criterio mide utilidad predictiva, no veracidad.
5. Porque no distingue competencia de bucle. Siete llamadas pueden ser una descomposición excelente o un reintento caro que no converge.
6. ReAct **usa** herramientas mediante prompting; Toolformer **aprende** dónde llamarlas y reentrena el modelo con ese conocimiento.
7. La composición: encadenar la salida de una herramienta como entrada de otra. Cada llamada se evalúa de forma independiente.

18. Fuentes primarias

- Schick, T. et al. (2023). *Toolformer: Language Models Can Teach Themselves to Use Tools*. **NeurIPS 2023**. arXiv:2302.04761 · consultado 2026-08-16.
- Nakano, R. et al. (2021). *WebGPT: Browser-assisted question-answering with human feedback*. arXiv:2112.09332 · consultado 2026-08-16.
- Gao, L. et al. (2022). *PAL: Program-aided Language Models*. arXiv:2211.10435 · consultado 2026-08-16.

Evaluación — P14

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Toolformer: Language Models Can Teach Themselves to Use Tools* (2023, arXiv:2302.04761 · NeurIPS 2023) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P14_toolformer.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P15 — DPO

El resultado que eliminó el andamiaje: si la política óptima del objetivo RLHF tiene forma cerrada, la recompensa se puede despejar y el modelo se ajusta directamente con las preferencias. Sin modelo de recompensa. Sin aprendizaje por refuerzo.

Nivel: L4 · Motor: dpo · Notebook: P15_dpo.ipynb

1. Identificación

Campo	Valor
Título original	Direct Preference Optimization: Your Language Model is Secretly a Reward Model
Autoría	Rafael Rafailov, Archit Sharma, Eric Mitchell, Stefano Ermon, Christopher D. Manning, Chelsea Finn
Año	2023
Venue	arXiv:2305.18290 · NeurIPS 2023
Fuente primaria	arXiv:2305.18290
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

El pipeline de InstructGPT funciona, pero es caro y frágil:

- entrena **un modelo adicional** (el de recompensa) que hay que mantener y validar;
- requiere **muestreo on-policy** durante el entrenamiento por refuerzo;
- **PPO es sensible** a hiperparámetros y difícil de estabilizar;
- hay que mantener **varios modelos en memoria** a la vez (política, referencia, recompensa, crítico).

La pregunta: ¿todo ese aparato es necesario, o es un rodeo?

3. Propuesta

Un resultado teórico con una consecuencia práctica inmediata.

El resultado. La solución óptima del objetivo RLHF con restricción KL tiene forma cerrada:

$$\pi^*(y \mid x) = (1 / Z(x)) \cdot \pi_{\text{ref}}(y \mid x) \cdot \exp(r(x, y) / \beta)$$

La consecuencia. De ahí se despeja la recompensa:

$$r(x, y) = \beta \cdot \log[\pi^*(y|x) / \pi_{\text{ref}}(y|x)] + \beta \cdot \log Z(x)$$

Al sustituir esto en el objetivo Bradley-Terry, **el término $Z(x)$ se cancela** —porque aparece en ambas ramas de la comparación— y queda una pérdida de clasificación binaria sobre pares de preferencias, expresada únicamente en términos de la política.

En una frase: **la política ya es el modelo de recompensa**. No hace falta entrenar uno aparte para luego optimizar contra él; se optimiza la política directamente.

4. Intuición sin fórmulas

RLHF entrena un juez y luego entrena al alumno a gustarle al juez. DPO demuestra que el alumno **ya contiene** al juez: su preferencia se lee comparando lo que dice ahora con lo que decía antes de entrenarse. Basta con ajustarlo para que suba lo preferido y baje lo rechazado, sin alejarse demasiado de donde estaba.

Dónde deja de funcionar la analogía: el juez de RLHF puede evaluar respuestas nuevas, generadas durante el entrenamiento. DPO solo aprende de los pares que ya tiene: no explora.

5. Matemática mínima

Pérdida DPO:

$$L = - E_{\{(x, y_w, y_l)\}} [\log \sigma(\beta \cdot [\log(\pi(y_w|x)/\pi_{\text{ref}}(y_w|x)) - \log(\pi(y_l|x)/\pi_{\text{ref}}(y_l|x))])]$$

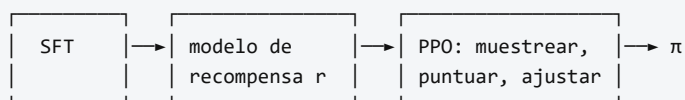
Recompensa implícita:

$$\hat{r}(x, y) = \beta \cdot \log[\pi(y|x) / \pi_{\text{ref}}(y|x)]$$

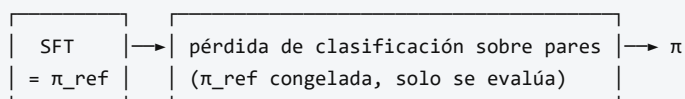
- y_w preferida, y_l rechazada, π_{ref} la política de referencia (habitualmente el modelo SFT), β el coeficiente que controla la desviación permitida.
- **π_{ref} es imprescindible.** Sin ella el objetivo premia subir $p(y_w)$ sin límite y la política colapsa a una distribución degenerada. El log-ratio es lo que convierte «subir» en «subir *relativamente a la referencia*», y β pone el precio.
- **La restricción KL no desapareció:** quedó absorbida en la forma del log-ratio.

6. Arquitectura o flujo

RLHF (tres etapas, dos modelos extra)



DPO (una etapa, ningún modelo extra)



7. Qué observar en el paper original

- La **derivación completa** de la forma cerrada y la cancelación de $Z(x)$. Es corta y vale la pena seguirla símbolo a símbolo: es el núcleo de todo el trabajo.
- El **análisis del gradiente**: el peso de cada par es mayor cuanto más se equivoca el modelo de recompensa implícito, lo que da una lectura intuitiva del comportamiento.
- La **comparación de la frontera recompensa–KL** frente a PPO. No basta con «gana en la métrica»: hay que ver a qué distancia de la referencia.
- Los experimentos de **generación controlada de sentimiento, resumen y diálogo**.
- La discusión sobre **generalización fuera de distribución**, donde los autores son cautos.

8. Evidencia y resultados

Tres escenarios:

- **generación controlada de sentimiento**, donde se puede computar la frontera óptima recompensa–KL y comparar objetivamente;
- **resumen** (Reddit TL;DR), evaluado con preferencia de un modelo juez;
- **diálogo de un turno** (Anthropic HH).

DPO iguala o supera a PPO en calidad de preferencia, con una implementación mucho más simple, más estable y sin ajustar un modelo de recompensa separado. En generación de sentimiento, DPO alcanza una frontera recompensa–KL mejor que PPO.

Las tasas de victoria y las curvas por escenario están en las figuras del artículo. Verificarlas allí antes de citarlas.

La miniatura de este eje muestra el mecanismo en su forma más desnuda: sobre una política de tres opciones, optimizar la pérdida DPO desplaza la masa hacia la respuesta preferida y produce recompensas implícitas positivas para ella y negativas para las rechazadas.

9. Impacto

- **Simplificó radicalmente la alineación** y la puso al alcance de equipos sin infraestructura de RL. Es hoy la vía por defecto para ajustar modelos abiertos con preferencias.
- Abrió una familia de métodos derivados: IPO, KTO, ORPO, SimPO y otros, cada uno con una variante de la pérdida.
- Reforzó una lección metodológica: **antes de construir maquinaria, comprobar si el problema tiene solución analítica**. El resultado estaba implícito en la formulación de RLHF desde el principio.

10. Limitaciones

1. **No explora**. Solo aprende de los pares disponibles; RLHF puede muestrear respuestas nuevas y evaluarlas. Si los pares no cubren una región, DPO no aprende nada de ella.

2. **Depende críticamente de la calidad y cobertura de las preferencias.** Basura entra, basura sale — igual que RLHF, pero sin un modelo de recompensa que se pueda inspeccionar por separado.
3. **Sin recompensa explícita que auditar.** En RLHF se puede estudiar r de forma aislada: qué premia, dónde falla. En DPO esa información está distribuida en la política.
4. **Sensible a β :** demasiado bajo, colapso; demasiado alto, apenas se mueve.
5. **Tendencia a reducir la probabilidad de ambas respuestas** del par en algunos regímenes, fenómeno documentado en trabajo posterior.
6. **Requiere π_{ref} en memoria** durante todo el entrenamiento.
7. **La equivalencia teórica con RLHF supone** un modelo de preferencias Bradley-Terry y un óptimo alcanzable; en la práctica ninguna de las dos cosas se cumple exactamente.

11. Errores comunes

Error	Corrección
«DPO es siempre mejor que RLHF»	Es más simple y a menudo igual de bueno. RLHF conserva la ventaja de explorar respuestas nuevas on-policy.
«DPO no tiene restricción KL»	La tiene, absorbida en el log-ratio contra π_{ref} y escalada por β .
«Se puede quitar π_{ref} y simplificar más»	Sin π_{ref} no hay ancla y la política colapsa. Es el anti-patrón del notebook.
«DPO no necesita datos de preferencia»	Los necesita exactamente igual. Lo que elimina es el modelo de recompensa, no los datos .
«La recompensa implícita es una metáfora»	Es una identidad: $\hat{r} = \beta \cdot \log(\pi/\pi_{\text{ref}})$ se deriva del óptimo del objetivo RLHF.
«DPO evita el reward hacking»	Reduce una superficie (el modelo de recompensa explotable), no el problema de fondo: sigue optimizando un proxy de la preferencia.

12. Relación con trabajos anteriores

- **P12 InstructGPT (2022)** — el pipeline que se simplifica.
- **Bradley y Terry (1952)** — el modelo de preferencias por pares. doi.org/10.2307/2334029
- **Schulman et al. (2017), PPO** — el algoritmo que DPO evita. [arXiv:1707.06347](https://arxiv.org/abs/1707.06347)
- **RL con regularización KL** — la formulación de la que se deriva la forma cerrada.

13. Relación con trabajos posteriores

- **IPO (2023)** — corrige supuestos del modelo de preferencias. [arXiv:2310.12036](https://arxiv.org/abs/2310.12036)
- **KTO (2024)** — aprende de señales binarias sin necesidad de pares. [arXiv:2402.01306](https://arxiv.org/abs/2402.01306)
- **ORPO, SimPO (2024)** — variantes que eliminan π_{ref} o fusionan etapas.
- **P16 Sistemas agentic** — la alineación como componente de un sistema, no como paso final.

14. Notebook asociado

P15_dpo.ipynb

Qué implementa: la pérdida DPO optimizada sobre una política categórica de tres opciones, la comparación entre π_{ref} y π_{DPO} , el cálculo de la recompensa implícita para varios β , y una demostración de por qué eliminar π_{ref} provoca colapso.

Qué NO implementa: modelo de lenguaje, generación, tokenización ni longitud variable. Tres opciones discretas no son una política sobre secuencias; el mecanismo es el mismo, la escala no.

```
ai-evolution paper-lab P15 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la pérdida DPO y la expresión de la recompensa implícita.
Explicar	Explica por qué $Z(x)$ se cancela y por qué eso es lo que hace viable el método.
Aplicar	Ejecuta el notebook con tres valores de β y compara las recompensas implícitas.
Analizar	Compara DPO y RLHF en cinco dimensiones: coste, estabilidad, exploración, auditabilidad y datos requeridos.
Evaluar	¿Cuándo elegirías RLHF pese a su complejidad? Da dos escenarios concretos.
Crear	Deriva la pérdida DPO desde el objetivo RLHF con restricción KL, paso a paso, en una página.

16. Autoevaluación

1. ¿Qué significa «tu modelo de lenguaje ya es un modelo de recompensa»?
2. ¿Por qué se cancela $Z(x)$ y qué pasaría si no se cancelara?
3. ¿Qué papel juega π_{ref} y qué ocurre exactamente si se elimina?
4. ¿Qué controla β ?
5. ¿Qué capacidad de RLHF se pierde con DPO?
6. ¿DPO necesita menos datos de preferencia que RLHF?
7. ¿Por qué la simplicidad de un método es un argumento técnico y no solo estético?

17. Respuestas esperadas

1. Que la recompensa implícita se puede leer como $\beta \cdot \log(\pi/\pi_{\text{ref}})$. La política contiene la información que el modelo de recompensa explícito codificaría, expresada como desviación respecto a la referencia.
2. Porque $Z(x)$ depende solo de x y aparece idéntico en las dos ramas de la comparación $r(x, y_w) - r(x, y_l)$. Si no se cancelara, habría que calcular una normalización sobre todas las salidas posibles, que es intratable.
3. Es el ancla. Sin ella, la pérdida premia aumentar $p(y_w)$ indefinidamente y la política colapsa a una distribución degenerada que siempre dice lo mismo.
4. Cuánto se permite que la política se aleje de la referencia: es el equivalente al coeficiente de la penalización KL en RLHF.
5. La exploración on-policy. RLHF genera respuestas nuevas durante el entrenamiento y las evalúa; DPO solo ve los pares del conjunto de datos.
6. No. Necesita los mismos datos. Elimina el **modelo** de recompensa, no la anotación.

7. Porque menos piezas significan menos hiperparámetros, menos modos de fallo, menos cómputo y mayor reproducibilidad. Todo eso es medible.

18. Fuentes primarias

- Rafailov, R. et al. (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model*. **NeurIPS 2023**. arXiv:2305.18290 · consultado 2026-08-16.
- Bradley, R. A. y Terry, M. E. (1952). *Rank Analysis of Incomplete Block Designs*. **Biometrika**, 39(3/4), 324–345. doi.org/10.2307/2334029 · consultado 2026-08-16.
- Azar, M. G. et al. (2023). *A General Theoretical Paradigm to Understand Learning from Human Preferences (IPO)*. arXiv:2310.12036 · consultado 2026-08-16.

Evaluación — P15

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Direct Preference Optimization: Your Language Model is Secretly a Reward Model* (2023, arXiv:2305.18290 · NeurIPS 2023) · **Nivel:** L4

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P15_dpo.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P16 — Sistemas agentic contemporáneos

Nodo de frontera, revisable. No es un paper: es un conjunto de trabajos posteriores a ReAct que convierten el bucle en un sistema. Se lee con fecha de consulta y se relee.

Nivel: L5 · **Motor:** `agentic` · **Notebook:** `P16_agentic_systems.ipynb`

[!WARNING] Esta ficha es la más volátil del eje y se aparta deliberadamente del formato de las quince anteriores en un punto: **agrupa varios trabajos** en lugar de analizar uno. Lo hace explícito para no fingir un consenso que no existe. Lo estable aquí son las **preguntas**; lo inestable son los nombres de framework de cada temporada. Última revisión: **2026-08-16**.

1. Identificación

Campo	Valor
Título original	— (nodo compuesto, no es un paper único)
Autoría	Varios equipos independientes
Año	2023 en adelante
Venue	arXiv, NeurIPS, ICLR, UIST y especificaciones abiertas
Fuentes primarias	ver sección 18
Acceso	Abierto
Fecha de consulta	2026-08-16

Trabajos que componen el nodo

Trabajo	Año	Qué añade al bucle
Reflexion (Shinn et al.)	2023	Autocrítica verbal y memoria entre intentos
Generative Agents (Park et al.)	2023	Memoria episódica con recuperación por relevancia, importancia y recencia
Voyager (Wang et al.)	2023	Currículo autónomo y biblioteca de habilidades reutilizables
AutoGen (Wu et al.)	2023	Orquestación conversacional entre varios agentes
Model Context Protocol	2024	Estandarización del acceso a herramientas, recursos y prompts

2. Problema anterior

ReAct demostró el bucle y Toolformer demostró que el uso de herramientas se aprende. Pero un bucle desnudo no sobrevive al contacto con una tarea real:

- **no recuerda** nada de un episodio al siguiente;

- **no se corrige**: si el plan es malo, lo ejecuta hasta el final;
- **no tiene presupuesto**: puede consumir indefinidamente;
- **no sabe parar**: ante un fallo de herramienta, reintenta;
- **no escala a varios agentes** sin un protocolo de coordinación;
- **no tiene una forma estándar** de descubrir e invocar herramientas.

Cada uno de estos huecos generó una línea de trabajo.

3. Propuesta

No hay una propuesta única. Hay una **descomposición en componentes** que la práctica ha ido estabilizando, aunque los nombres varíen:

Componente	Función	Origen representativo
Plan	Descomponer el objetivo en pasos verificables	ReAct, Voyager
Herramientas tipadas	Acciones con contrato de entrada, salida y efectos	Toolformer, MCP
Memoria	Episódica, semántica y de trabajo, con recuperación	Generative Agents
Reflexión	Criticar el propio resultado y reintentar mejor	Reflexion
Presupuesto	Límite de pasos, tokens, coste y tiempo	práctica operativa
Criterio de parada	Cuándo terminar, abortar o escalar	práctica operativa
Orquestación	Varios agentes con roles y protocolo	AutoGen
Permisos y aislamiento	Qué puede tocar el agente y con qué autoridad	seguridad de sistemas

4. Intuición sin fórmulas

Un becario brillante sin instrucciones, sin presupuesto y sin nadie a quien preguntar cuando algo falla. El problema no es su capacidad: es que el **sistema** a su alrededor no existe. Un agente que funciona en la demo y falla en producción casi nunca falla por el modelo.

Dónde deja de funcionar la analogía: el becario sabe cuándo está fuera de su competencia y pregunta. El agente no tiene ese sentido; hay que construirlo explícitamente como criterio de parada y escalamiento.

5. Matemática mínima

No hay una ecuación central. Lo que hay es un **contrato operativo** que sí se puede formalizar:


```
estado_t = (objetivo, plan, memoria_t, presupuesto_restante_t)
```

```
a_t ~  $\pi(\cdot \mid \text{estado}_t)$ 
```

```
o_t = entorno(a_t) ← puede fallar
```

```
memoria_{t+1} = actualizar(memoria_t, a_t, o_t)
```

```
presupuesto_{t+1} = presupuesto_t - coste(a_t)
```

PARAR si: objetivo cumplido

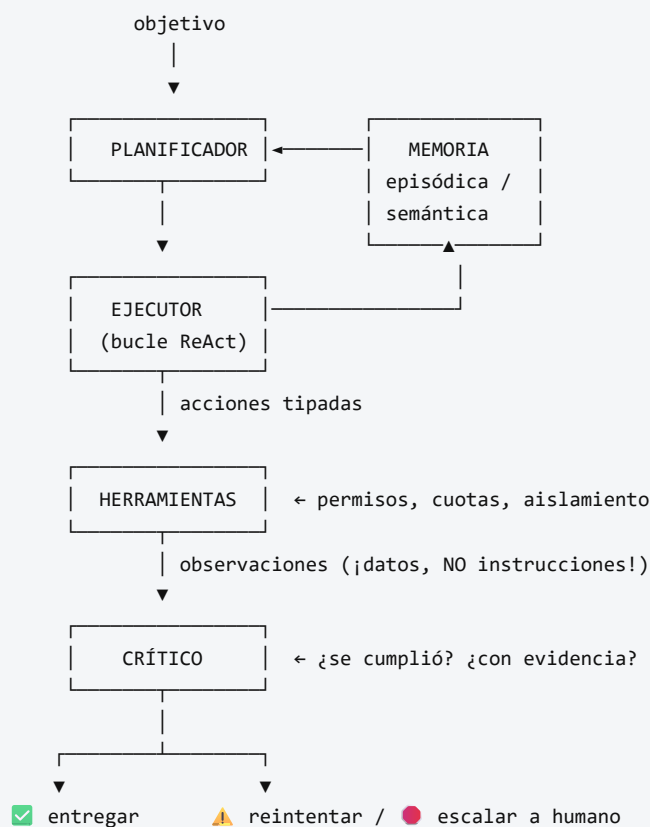
✓ presupuesto agotado

✓ fallo no recuperable → escalar a humano

✓ estado repetido → detectar bucle

La última línea es la que separa un prototipo de un sistema operable.

6. Arquitectura o flujo



Todo el bucle bajo PRESUPUESTO y con TRAZA persistida.

7. Qué observar en el paper original

Al ser un nodo compuesto, lo que hay que observar es **transversal**:

- En **Reflexion**: cómo se convierte un fallo en texto que mejora el siguiente intento, y cuál es el límite de esa mejora.
- En **Generative Agents**: la función de recuperación de memoria (relevancia, recencia, importancia) y la evaluación con humanos, no solo con benchmarks.

- En **Voyager**: la biblioteca de habilidades como forma de memoria **procedimental**, y la generación del currículo.
- En **AutoGen**: qué problemas mejoran con varios agentes y cuáles **no** — la sección más útil y la que menos se cita.
- En **MCP**: la separación entre *tools*, *resources* y *prompts*, y qué implica para permisos.
- **En todos**: cuántas ejecuciones se reportan y con qué varianza. Es la pregunta que más demos derriba.

8. Evidencia y resultados

Aquí hay que ser especialmente cuidadoso. El área tiene una brecha grande entre demostración y evidencia:

- muchos resultados se reportan sobre **pocas ejecuciones**, sin varianza;
- los **benchmarks de agentes** son jóvenes y algunos han mostrado problemas de contaminación o de especificación ambigua;
- la comparación entre frameworks rara vez controla el modelo base, el prompt y el presupuesto;
- el **coste** (llamadas, tokens, latencia) se omite con frecuencia, y es la mitad de la decisión.

Antes de citar cualquier cifra de esta área: comprobar número de ejecuciones, varianza, modelo base, presupuesto y fecha. Si falta alguno, la cifra no es comparable.

La miniatura de este eje no mide capacidad: muestra un agente con presupuesto explícito que se topa con un fallo de herramienta y **escala en lugar de responder igualmente**. Parar es un resultado correcto.

9. Impacto

- Traslada el foco del **prompt** al **sistema**: memoria, presupuesto, permisos, trazas y criterios de parada.
- Convierte la operación de agentes en una disciplina con su propio nombre y sus propias métricas (tasa de éxito, de escalamiento correcto, de respuesta inventada, coste por tarea).
- Hace de la **seguridad** un requisito de diseño: la inyección de prompt indirecta a través de observaciones es un vector real, no teórico.
- Empuja hacia la **estandarización** del acceso a herramientas, con las implicaciones de cadena de suministro que eso conlleva.

10. Limitaciones

1. **Evidencia débil y heterogénea.** Es la limitación principal de todo el nodo.
2. **Sin definición consensuada de «agente».** Cada trabajo usa la palabra para algo distinto.
3. **Fiabilidad compuesta.** Con 90 % de éxito por paso, diez pasos dan ≈ 35 %. La aritmética es implacable y se ignora demasiado a menudo.
4. **Coste y latencia** frecuentemente ausentes del reporte.
5. **Superficie de ataque amplia:** inyección indirecta, herramientas maliciosas, exfiltración de datos a través de acciones legítimas.
6. **Atribución de responsabilidad sin resolver:** quién responde cuando el agente actúa mal.

7. **Los nombres cambian más rápido que las ideas.** Buena parte de la literatura de una temporada queda obsoleta en la siguiente sin haber sido refutada: simplemente se abandona.

11. Errores comunes

Error	Corrección
«Funcionó una vez, está listo»	<code>n=1</code> no es evidencia. Sin distribución, varianza y casos adversarios, es una anécdota.
«Más agentes = mejor»	Añadir agentes multiplica coste, latencia y modos de fallo. Hay que demostrar que aporta.
«El agente es autónomo»	Opera bajo un objetivo, un presupuesto y unos permisos que alguien definió. La autonomía es un rango, no un interruptor.
«La traza demuestra cómo razonó»	Igual que en ReAct: es texto generado, útil para auditar decisiones, no para probar procesos.
«Si el agente responde, hizo su trabajo»	Un sistema que siempre responde está ocultando sus fallos. La abstención es una capacidad.
«El contenido que lee el agente son instrucciones»	Toda observación es dato . Tratarla como instrucción es la vulnerabilidad de inyección indirecta.
«Este es un campo maduro»	Es un campo activo con evidencia joven. Tratarlo como maduro es el error más caro.

12. Relación con trabajos anteriores

- **P13 ReAct (2022)** — el bucle base.
- **P14 Toolformer (2023)** — herramientas aprendidas.
- **P15 DPO (2023)** — alineación accesible del componente de decisión.
- **P11 RAG (2020)** — memoria consultable, antecedente directo de la memoria semántica de un agente.
- **Agentes racionales clásicos** (Russell y Norvig) — la definición de agente como percepción → decisión → acción es muy anterior a los LLM. Ver la clase 004 del programa.

13. Relación con trabajos posteriores

Por definición, esta sección **caduca**. Lo posterior a este nodo no se añade aquí: se registra con fecha y fuente en `frontier/current-topics.yaml`, y solo asciende a `papers/foundational/` cuando se consolida.

Preguntas abiertas que conviene seguir:

- evaluación de agentes con protocolos comparables y coste incluido;
- fiabilidad compuesta: cómo llevar tareas de muchos pasos a tasas de éxito operables;
- defensa contra inyección indirecta con garantías, no con heurísticas;
- memoria a largo plazo que no degrade con el volumen;
- atribución de responsabilidad y trazas verificables por terceros.

14. Notebook asociado

Qué implementa: un agente con plan, herramientas, memoria, presupuesto y criterio de parada que se topa con un fallo de verificación y escala en lugar de reintentar; el mapa componente → riesgo si falta; y el contraste entre un reporte anecdótico ($n=1$) y uno con distribución.

Qué NO implementa: ningún modelo de lenguaje. El agente **ejecuta** un plan, no lo planifica. Un agente real planifica con un LLM y puede equivocarse justo ahí.

```
ai-evolution paper-lab P16 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Enumera los seis componentes del contrato operativo y qué falla si quitas cada uno.
Explicar	Explica la fiabilidad compuesta con un ejemplo numérico de 10 pasos.
Aplicar	Reduce el presupuesto en el notebook y comprueba que el agente aborta limpiamente.
Analizar	Toma un trabajo de la tabla del nodo y analiza qué evidencia aporta y qué deja sin demostrar.
Evaluar	Te presentan un agente con «92 % de éxito». Escribe las siete preguntas que harías antes de aceptarlo.
Crear	Diseña el protocolo de evaluación de un agente de tu dominio: métricas, casos adversarios, presupuesto y criterio de aceptación.

16. Autoevaluación

1. ¿Qué distingue un bucle de un sistema agentic?
2. Con 95 % de éxito por paso, ¿cuál es la tasa esperada en una tarea de 15 pasos?
3. ¿Por qué la abstención es una capacidad y no un fallo?
4. ¿Qué es la inyección de prompt indirecta y qué regla la previene?
5. ¿Por qué añadir agentes puede empeorar un sistema?
6. ¿Qué debe contener el reporte de evaluación de un agente para ser aceptable?
7. ¿Por qué esta ficha lleva fecha de consulta destacada y las anteriores no tanto?

17. Respuestas esperadas

1. Los componentes que rodean al bucle: memoria, presupuesto, criterio de parada, permisos, trazas y escalamiento. El bucle es el motor; el sistema es el vehículo.
2. $0,95^{15} \approx 0,46$. Menos de la mitad. Es el argumento para acortar cadenas, verificar pasos intermedios y diseñar puntos de control.
3. Porque un sistema que siempre responde convierte sus fallos en respuestas plausibles. Poder decir «no lo sé» o «necesito ayuda» es lo que hace operable un sistema en producción.
4. Que contenido leído por el agente (una página, un documento, un correo) contenga texto dirigido a él. La regla: **toda observación es dato, nunca instrucción**; las instrucciones solo vienen del usuario por su canal.

5. Porque cada agente añade coste, latencia, puntos de fallo y ambigüedad de coordinación. El beneficio debe demostrarse contra una línea base de un solo agente bien construido.
6. Número de ejecuciones, varianza, tasa de éxito, de escalamiento correcto y de respuesta inventada, coste medio y percentil alto, modelo base, presupuesto, casos adversarios probados y fecha.
7. Porque agrupa trabajos recientes cuya evidencia aún se está consolidando. Las quince fichas anteriores describen resultados asentados y replicados; esta describe un frente activo.

18. Fuentes primarias

- Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K. y Yao, S. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. **NeurIPS 2023**. [arXiv:2303.11366](https://arxiv.org/abs/2303.11366) · consultado 2026-08-16.
- Park, J. S. et al. (2023). *Generative Agents: Interactive Simulacra of Human Behavior*. **UIST 2023**. [arXiv:2304.03442](https://arxiv.org/abs/2304.03442) · consultado 2026-08-16.
- Wang, G. et al. (2023). *Voyager: An Open-Ended Embodied Agent with Large Language Models*. [arXiv:2305.16291](https://arxiv.org/abs/2305.16291) · consultado 2026-08-16.
- Wu, Q. et al. (2023). *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. [arXiv:2308.08155](https://arxiv.org/abs/2308.08155) · consultado 2026-08-16.
- *Model Context Protocol* — especificación abierta. modelcontextprotocol.io · consultado 2026-08-16.

Evaluación — P16

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Sistemas agentic contemporáneos (nodo de frontera, revisable)* (2023, Conjunto de trabajos posteriores a ReAct — releer con fecha de consulta) · **Nivel:** L5

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P16_agentic_systems.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P17 — Difusión (DDPM)

Ruta ampliada · La generación deja de ser un salto en la oscuridad: se aprende a deshacer, paso a paso, un proceso de ruido que se conoce en forma cerrada.

Nivel: L3 · **Motor:** diffusion · **Notebook:** P17_diffusion.ipynb · **Anexo matemático:** probabilidad y verosimilitud

1. Identificación

Campo	Valor
Título original	<i>Denoising Diffusion Probabilistic Models</i>
Autoría	Jonathan Ho, Ajay Jain, Pieter Abbeel
Año	2020
Venue	arXiv:2006.11239 · NeurIPS 2020
Fuente primaria	arXiv:2006.11239
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Generar imágenes tenía dos familias con defectos opuestos. Las **GAN** producían muestras nítidas pero su entrenamiento adversario era inestable y colapsaba la diversidad: el generador encontraba unos pocos modos que engañaban al discriminador y se quedaba ahí. Los **VAE** eran estables y tenían verosimilitud tratable, pero sus muestras salían borrosas.

Faltaba un objetivo que fuera **estable como el de un VAE** y produjera **calidad de GAN**.

3. Propuesta

Definir un proceso **directo** que destruye la imagen añadiendo ruido gaussiano en **T** pasos —fijo, sin parámetros que aprender, y con forma cerrada para saltar a cualquier paso— y entrenar una red para invertirlo.

El giro decisivo es la parametrización: en vez de predecir la imagen limpia, la red predice **el ruido ϵ que se añadió**. Con esa elección, la cota variacional se simplifica a un error cuadrático medio entre el ruido real y el predicho.

El artículo establece además la conexión entre esta formulación y el *denoising score matching* con dinámica de Langevin, lo que conecta dos líneas de investigación que iban por separado.

4. Intuición sin fórmulas

Una foto que se va llenando de nieve de televisor hasta quedar en ruido puro. Como sabes exactamente cuánta nieve echaste en cada paso, puedes entrenar a alguien a estimarla y quitarla. Generar es empezar desde nieve pura y quitar nieve muchas veces.

Dónde deja de funcionar la analogía: al quitar nieve no recuperas *la* foto original, sino *una* foto plausible. El proceso inverso es estocástico: ahí está la generación, no en la reconstrucción.

5. Matemática mínima

Proceso directo (conocido, no se aprende):

$$q(x_t | x_{t-1}) = N(x_t; \sqrt{1-\beta_t} \cdot x_{t-1}, \beta_t \cdot I)$$

Salto directo a cualquier paso (la propiedad que lo hace entrenable):

$$x_t = \sqrt{\alpha_t} \cdot x_0 + \sqrt{1-\alpha_t} \cdot \epsilon, \quad \epsilon \sim N(0, I), \quad \alpha_t = \prod_{s \leq t} (1-\beta_s)$$

Despejando la imagen:

$$x_0 = (x_t - \sqrt{1-\alpha_t} \cdot \epsilon) / \sqrt{\alpha_t}$$

Pérdida simplificada:

$$L = E_{\{t, x_0, \epsilon\}} \| \epsilon - \epsilon_\theta(x_t, t) \|^2$$

La segunda línea es la que convierte un problema generativo en **regresión supervisada**: el par de entrenamiento (x_t, ϵ) se fabrica gratis desde cualquier imagen y cualquier t .

6. Arquitectura o flujo

7. Qué observar en el paper original

- La **derivación de la cota variacional** y cómo, al reparametrizar en términos de ϵ , colapsa en un error cuadrático simple. Es el corazón del artículo.
- La sección que conecta con **score matching y Langevin**: explica por qué esto no es un truco aislado sino un punto de encuentro entre dos formulaciones.
- El **planificador de β** elegido y su efecto en la calidad de las muestras.
- Los resultados en **CIFAR-10** y **LSUN**, y la discusión sobre verosimilitud frente a calidad perceptual: el modelo no gana en ambas a la vez, y los autores lo dicen.

8. Evidencia y resultados

Síntesis de imágenes de alta calidad en CIFAR-10 y LSUN, con resultados del estado del arte de la época en calidad de muestra.

Los valores exactos de FID e Inception Score por conjunto están en las tablas del artículo. Verificarlos allí antes de citarlos.

La miniatura de este eje aporta la mecánica: la SNR cae de $\sim 10^4$ a $\sim 4,5$ en 20 pasos, la reconstrucción con el ϵ correcto es exacta hasta precisión de máquina, y el factor de amplificación $\sqrt{(1-\bar{\alpha}_t)}/\sqrt{\bar{\alpha}_t}$ explica por qué el muestreo va paso a paso.

9. Impacto

- Desplazó a las GAN como método por defecto para generación de imágenes en pocos años.
- Es la base técnica de los sistemas de texto a imagen posteriores, que añaden **condicionamiento** (a menudo con un codificador de texto tipo CLIP) y difusión en espacio latente.
- Trasladó la idea a audio, vídeo, moléculas y política de robots: cualquier dominio donde «añadir ruido» esté bien definido.

10. Limitaciones

1. **Muestreo lento.** Generar exige cientos o miles de pasadas por la red, frente a una sola de una GAN. Toda la investigación posterior de destilación y solucionadores rápidos ataca esto.
2. **Coste de entrenamiento alto.**
3. **Verosimilitud frente a calidad perceptual:** optimizar la cota no maximiza la calidad percibida, y el artículo lo documenta.
4. **Sin condicionamiento** en esta versión: no hay forma de pedir *qué* generar.
5. **Sesgo y contenido del conjunto de entrenamiento** se reproducen en las muestras; el artículo no aborda esa dimensión.
6. **El proceso directo gaussiano** no encaja igual de bien en datos discretos (texto), donde hicieron falta formulaciones distintas.

11. Errores comunes

Error	Corrección
«El modelo predice la imagen limpia»	Predice el ruido . Es la parametrización que hace la pérdida simple y bien condicionada.
«La difusión la inventó este paper»	Sohl-Dickstein et al. (2015) la propuso antes; DDPM la hizo funcionar y la simplificó.
«Es determinista, luego no genera»	El proceso inverso es estocástico: de ahí sale la diversidad.
«Difusión = texto a imagen»	El condicionamiento por texto es posterior . Este paper genera sin condición.
«Más pasos siempre es mejor»	Hay rendimientos decrecientes, y el coste crece linealmente.

12. Relación con trabajos anteriores

- **Sohl-Dickstein et al. (2015)** — difusión no supervisada; el antecedente directo. [arXiv:1503.03585](#)
- **VAE (2013) y GAN (2014)** — las dos familias cuyas debilidades motivan el trabajo.
- **P02 Backpropagation** y **P04 AlexNet** — la red que predice ϵ es una CNN entrenada por retropropagación.

13. Relación con trabajos posteriores

- **P18 CLIP (2021)** — aporta el espacio compartido imagen-texto que permite condicionar la generación con palabras.
- **Difusión latente y modelos de texto a imagen (2022+)** — difusión en un espacio comprimido en vez de en píxeles.
- **Destilación de muestreo (2022+)** — reducir cientos de pasos a unos pocos.

14. Notebook asociado

P17_diffusion.ipynb

Qué implementa: el proceso directo con forma cerrada, la trayectoria de SNR, la reconstrucción desde ϵ y la amplificación del error por paso.

Qué NO implementa: la red ϵ_θ , el muestreo estocástico inverso, la U-Net ni ninguna imagen. Cuatro números no son una imagen, y aquí el ϵ se conoce en lugar de predecirse.

```
ai-evolution paper-lab P17 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la fórmula del salto directo a x_t y define $\bar{\alpha}_t$.
Explicar	Explica por qué predecir ϵ es mejor condicionado que predecir x_0 .
Aplicar	Calcula $\bar{\alpha}_t$ para un planificador lineal de 20 pasos y localiza dónde la SNR cruza 1.
Analizar	Deriva el factor de amplificación del error de ϵ sobre x_0 y explica su forma.
Evaluar	Un equipo reporta FID excelente y muestras poco diversas. ¿Qué métrica falta y por qué?
Crear	Diseña un proceso directo para datos discretos y argumenta qué se rompe respecto al gaussiano.

16. Autoevaluación

1. ¿Por qué el proceso directo no tiene parámetros que aprender?
2. ¿Qué propiedad permite entrenar sin simular los t pasos uno a uno?
3. ¿Por qué la pérdida acaba siendo un error cuadrático simple?
4. ¿De dónde sale la diversidad de las muestras?

5. ¿Por qué el muestreo es lento y qué se ha hecho después al respecto?
6. ¿Qué le falta a este paper para poder pedirle «un gato con sombrero»?
7. ¿Qué idea que hoy se asocia a «difusión» no está en este artículo?

17. Respuestas esperadas

1. Porque β_t es un planificador fijo elegido de antemano: añadir ruido gaussiano está completamente especificado, no hay nada que ajustar.
2. La composición de gaussianas es gaussiana, así que $q(x_t | x_0)$ tiene forma cerrada y se puede muestrear t al azar y saltar directamente.
3. Porque al reparametrizar la cota variacional en términos de ϵ , los términos se simplifican y queda $\|\epsilon - \epsilon_\theta\|^2$ con un peso que el paper decide ignorar.
4. Del proceso inverso, que es estocástico: se parte de ruido distinto y se añade ruido en cada paso de denoising.
5. Porque requiere una pasada por la red por cada paso. Después llegaron solucionadores de pocos pasos y destilación del muestreador.
6. Condicionamiento. Necesita un espacio donde el texto y la imagen sean comparables — el problema de P18.
7. El condicionamiento por texto, la difusión latente y la guía sin clasificador: todo posterior.

18. Fuentes primarias

- Ho, J., Jain, A. y Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models*. **NeurIPS 2020**. arXiv:2006.11239 · consultado 2026-08-16.
- Sohl-Dickstein, J. et al. (2015). *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*. arXiv:1503.03585 · consultado 2026-08-16.

Evaluación — P17

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Denoising Diffusion Probabilistic Models* (2020, arXiv:2006.11239 · NeurIPS 2020) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P17_diffusion.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P18 — CLIP

Ruta ampliada · El texto se convierte en la etiqueta: un solo modelo clasifica categorías que nadie anotó, descritas con palabras.

Nivel: L3 · Motor: clip · Notebook: P18_clip.ipynb · Anexo matemático: álgebra y geometría

1. Identificación

Campo	Valor
Título original	<i>Learning Transferable Visual Models From Natural Language Supervision</i>
Autoría	Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh y otros (OpenAI)
Año	2021
Venue	arXiv:2103.00020 · ICML 2021
Fuente primaria	arXiv:2103.00020
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Desde AlexNet, la visión funcionaba así: alguien define un conjunto cerrado de categorías, alguien etiqueta un millón de imágenes con ellas, y el modelo aprende a elegir entre esas categorías y ninguna más.

Eso tiene dos costes. El primero es económico: cada tarea nueva exige anotar de nuevo. El segundo es conceptual: el modelo aprende «clase 237», no *lo que la clase significa*. Un clasificador de ImageNet no sabe nada de una categoría que no estuviera en su lista.

3. Propuesta

Usar como supervisión lo que **ya viene con las imágenes de internet**: su texto asociado.

Se entrenan dos codificadores —uno de imagen, otro de texto— con un objetivo **contrastivo** sobre 400 millones de pares: acercar cada imagen a su texto y alejarla de los textos del resto del lote. En un lote de N , cada par correcto tiene $N-1$ negativos gratis.

Después, clasificar es comparar: se escriben las categorías como frases («una foto de un gato»), se codifican y se elige la más parecida a la imagen. **Sin clasificador entrenado y sin un solo ejemplo de la tarea**. El paper reporta que así iguala la exactitud de un ResNet-50 en ImageNet sin usar ninguno de sus 1,28 millones de ejemplos de entrenamiento.

4. Intuición sin fórmulas

Una clase con una pizarra de fotos y otra de pies de foto, desordenadas. La tarea es unir cada foto con su pie. Al hacerlo miles de millones de veces, aprendes qué aspecto tiene lo que las palabras describen — y luego puedes buscar «un perro con gorro» aunque nadie te enseñara nunca esa categoría.

Dónde deja de funcionar la analogía: el texto de internet no es una descripción cuidadosa; es lo que había alrededor de la imagen. Aprende correlaciones del pie de foto, no la definición del concepto.

5. Matemática mínima

Similitud (coseno, ambos vectores normalizados):

$$s_{ij} = (\mathbf{I}_i \cdot \mathbf{T}_j) / (\|\mathbf{I}_i\| \|\mathbf{T}_j\|)$$

Logits con temperatura aprendida τ :

$$\text{logits} = \mathbf{s} / \tau$$

Pérdida InfoNCE simétrica sobre un lote de N pares:

$$L = \frac{1}{N} \text{CrossEntropy}(\text{logits}, \text{etiquetas}=\text{diagonal}) \text{ por filas} \\ + \frac{1}{N} \text{CrossEntropy}(\text{logits}^T, \text{etiquetas}=\text{diagonal}) \text{ por columnas}$$

Clasificación zero-shot:

$$\hat{y} = \underset{c}{\text{argmax}} \cos(\mathbf{I}_{\text{imagen}}, \mathbf{T}_{\text{"una foto de un \{c\}"}})$$

La **diagonal** son los pares correctos. Todo lo demás del lote son negativos: por eso el tamaño de lote es un hiperparámetro de primer orden y no un detalle de implementación.

6. Arquitectura o flujo

7. Qué observar en el paper original

- El **tamaño del conjunto**: 400 millones de pares recogidos de internet, y cómo se construyó. La escala **es** la contribución tanto como el objetivo.
- La discusión sobre **eficiencia del objetivo**: por qué el contrastivo escala mejor que predecir el texto exacto de la imagen.
- La sección de **prompt engineering y ensamblado de plantillas**: la exactitud zero-shot varía varios puntos según cómo se escriba la clase. Los autores lo documentan abiertamente.
- El **análisis de limitaciones y sesgo**, inusualmente extenso: rendimiento pobre en tareas abstractas o de conteo, y asociaciones problemáticas heredadas del corpus.

- Los experimentos de **robustez ante desplazamiento de distribución**, donde CLIP aguanta mejor que modelos entrenados solo en ImageNet.

8. Evidencia y resultados

Evaluación en más de 30 conjuntos de visión, en régimen zero-shot y como extractor de características.

El resultado más citado: en ImageNet, la clasificación zero-shot iguala la exactitud de un ResNet-50 supervisado **sin usar sus 1,28 millones de ejemplos etiquetados**.

Las cifras por conjunto y por variante de codificador están en las tablas del artículo. Verificarlas allí antes de citarlas, y leer primero la sección de plantillas: el número depende de cómo se redacten las clases.

La miniatura de este eje muestra el mecanismo: la diagonal de la matriz de similitud sube y lo de fuera baja, y la clasificación zero-shot acierta 4/4 comparando la imagen con el **texto** de cada clase.

9. Impacto

- Convirtió el **texto en interfaz de la visión**: buscar, clasificar y filtrar imágenes describiéndolas.
- Su codificador de texto se volvió la pieza estándar para **condicionar generación**, lo que conecta directamente con P17.
- Popularizó el **preentrenamiento contrastivo multimodal** como receta general, extendida después a audio, vídeo y datos biomédicos.
- Reabrió la discusión sobre datos de internet: licencias, consentimiento y sesgo a escala.

10. Limitaciones

1. **Falla en tareas abstractas**: contar objetos, relaciones espaciales finas, texto dentro de la imagen.
2. **Sensible a la redacción de la clase**. «Zero-shot» esconde un ajuste de plantillas que, si se hace mirando el test, deja de ser zero-shot.
3. **Sesgo del corpus** heredado y amplificado; el propio paper lo mide y lo advierte.
4. **«Zero-shot» es relativo**: con 400 millones de pares de internet, pocas categorías comunes son realmente nuevas para el modelo.
5. **Coste de entrenamiento** fuera del alcance de casi cualquier equipo.
6. **Datos no públicos** en la versión original: la replicación independiente exigió construir corpus alternativos.
7. **Grano grueso**: distingue gato de perro mucho mejor que razas concretas.

11. Errores comunes

Error	Corrección
«Zero-shot significa que nunca vio gatos»	Significa que no vio este conjunto de etiquetas . Vio 400 millones de pares.
«CLIP es un clasificador»	Es un espacio compartido . La clasificación es una consulta de similitud sobre él.
«La plantilla del prompt es un detalle»	Cambia el resultado varios puntos. Es parte del protocolo y debe reportarse.
«CLIP genera imágenes»	No genera nada. Aporta el espacio que otros modelos usan para condicionar.
«Contrastivo = supervisado»	La supervisión es débil y gratuita : el emparejamiento que ya existía, no una anotación.

12. Relación con trabajos anteriores

- **P04 AlexNet (2012)** — el paradigma de categorías fijas que este trabajo rompe.
- **P05 Word2Vec (2013)** — la idea de que el significado vive en un espacio vectorial.
- **P08 Transformer (2017)** — el codificador de texto.
- **Russakovsky et al. (2015)** — ILSVRC y su marco de etiquetas cerradas. [arXiv:1409.0575](https://arxiv.org/abs/1409.0575)

13. Relación con trabajos posteriores

- **Generación condicionada por texto (2021+)** — usa el codificador de texto de CLIP o sucesores para guiar la difusión de P17.
- **Modelos de visión-lenguaje conversacionales (2023+)** — conectan un codificador visual a un LLM en vez de contrastar espacios.
- **Réplicas abiertas** con corpus públicos, que permitieron auditar el sesgo del método.

14. Notebook asociado

P18_clip.ipynb

Qué implementa: InfoNCE simétrico sobre cuatro pares, la matriz de similitud antes y después de entrenar, y clasificación zero-shot por comparación con el texto de la clase.

Qué NO implementa: codificadores de imagen o texto, los 400 millones de pares ni lotes grandes. Con lotes de 4, el contraste es trivial: la dificultad real del método está en la escala.

```
ai-evolution paper-lab P18 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la pérdida InfoNCE simétrica y di qué son los positivos y los negativos.
Explicar	Explica por qué el tamaño del lote es un hiperparámetro de primer orden.
Aplicar	Ejecuta el notebook con tres semillas y compara la separación diagonal/fuera.
Analizar	Diseña tres plantillas de texto para la misma clase y razona cuál debería funcionar mejor y por qué.
Evaluar	Alguien reporta 76 % zero-shot eligiendo la plantilla sobre el test. ¿Qué objetas?
Crear	Diseña un protocolo de evaluación zero-shot a prueba de ajuste encubierto de plantillas.

16. Autoevaluación

1. ¿Qué supervisión usa CLIP y por qué es barata?
2. ¿Por qué el objetivo contrastivo escala mejor que predecir el texto exacto?
3. ¿Qué papel juega la temperatura τ ?
4. ¿Cómo clasifica sin clasificador?
5. ¿En qué sentido «zero-shot» es una etiqueta discutible?
6. ¿Qué tipo de tareas se le dan mal y por qué?
7. ¿Qué aporta CLIP a un modelo de difusión?

17. Respuestas esperadas

1. El emparejamiento imagen-texto que ya existe en internet. Nadie anota categorías: la señal viene con los datos.
2. Porque predecir el texto exacto es una tarea mucho más difícil y con una salida enorme; distinguir cuál de N textos corresponde es una tarea más fácil que aún exige entender el contenido, y aprovecha $N-1$ negativos gratis por ejemplo.
3. Escala los logits antes del softmax: controla cuán concentrada queda la distribución. Es un parámetro aprendido, no fijo.
4. Codificando las categorías como frases y eligiendo la de mayor coseno con la imagen. El «clasificador» se construye al vuelo desde texto.
5. Porque el modelo vio 400 millones de pares de internet: la categoría es nueva para *el protocolo*, no necesariamente para *el modelo*.
6. Conteo, relaciones espaciales, distinciones de grano fino y tareas abstractas: la correlación pie-de-foto/imagen no las cubre bien.
7. Un espacio donde texto e imagen son comparables, que permite guiar la generación con palabras — lo que a P17 le faltaba.

18. Fuentes primarias

- Radford, A. et al. (2021). *Learning Transferable Visual Models From Natural Language Supervision*. **ICML 2021**. arXiv:2103.00020 · consultado 2026-08-16.
- Russakovsky, O. et al. (2015). *ImageNet Large Scale Visual Recognition Challenge*. arXiv:1409.0575 · consultado 2026-08-16.

Evaluación — P18

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Learning Transferable Visual Models From Natural Language Supervision* (2021, arXiv:2103.00020 · ICML 2021) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P18_clip.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P19 — Leyes de escalado con cómputo óptimo (Chinchilla)

Ruta ampliada · Corrige la carrera por el tamaño: a cómputo fijo, los modelos de la época estaban infraentrenados en datos.

Nivel: L4 · Motor: `scaling_laws` · Notebook: `P19_scaling_laws.ipynb` · Anexo matemático: complejidad, coste y escalado

1. Identificación

Campo	Valor
Título original	<i>Training Compute-Optimal Large Language Models</i>
Autoría	Jordan Hoffmann, Sebastian Borgeaud, Arthur Mensch y otros (DeepMind)
Año	2022
Venue	arXiv:2203.15556 · NeurIPS 2022
Fuente primaria	arXiv:2203.15556
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Tras GPT-3, la industria interpretó las leyes de escalado de Kaplan et al. (2020) como una consigna simple: **más parámetros**. Los modelos de los dos años siguientes crecieron un orden de magnitud en `N` mientras el número de tokens de entrenamiento `D` se mantenía prácticamente igual.

La pregunta que nadie había resuelto bien es distinta y más útil: **dado un presupuesto fijo de cómputo, ¿cuánto conviene gastar en parámetros y cuánto en datos?** Sin esa respuesta, cada laboratorio elegía por intuición y por marketing.

3. Propuesta

Medirlo. Los autores entrenan cientos de modelos variando `N` y `D` a lo largo de un rango amplio de presupuestos, ajustan una forma paramétrica a la pérdida y resuelven el problema de optimización con restricción de cómputo.

La conclusión: para minimizar la pérdida a cómputo fijo, **`N` y `D` deben escalarse aproximadamente en la misma proporción**. Los modelos grandes de la época habían gastado demasiado en parámetros y demasiado poco en tokens.

Y lo demuestran con la prueba más contundente posible: entrenan un modelo **considerablemente más pequeño** con muchos más tokens, al mismo cómputo, y supera a los grandes.

4. Intuición sin fórmulas

Un presupuesto fijo para montar una biblioteca: puedes comprar muchas estanterías y pocos libros, o pocas estanterías y muchos libros. Ninguno de los dos extremos es óptimo, y durante años el sector compró estanterías.

Dónde deja de funcionar la analogía: las estanterías vacías no perjudican; los parámetros infraentrenados sí consumen cómputo de entrenamiento y de inferencia para siempre.

5. Matemática mínima

Forma paramétrica ajustada empíricamente:

$$L(N, D) = E + A/N^\alpha + B/D^\beta$$

E = error irreducible (la entropía del propio lenguaje)

A/N^α = lo que se compra con parámetros

B/D^β = lo que se compra con datos

Restricción de presupuesto (aproximación estándar de FLOPs de entrenamiento):

$$C \approx 6 \cdot N \cdot D$$

Problema: minimizar $L(N, D)$ sujeto a $6ND = C$

Sustituyendo $D = C/(6N)$ y derivando respecto de N , la condición de óptimo iguala las contribuciones marginales de ambos términos. El resultado es una **razón óptima de tokens por parámetro** que depende de α y β , no del presupuesto.

Los valores ajustados de E , A , B , α y β están en el artículo. El motor de este eje usa constantes **didácticas** para que la curva tenga la forma correcta: la forma es lo transferible, los números concretos hay que leerlos en la fuente.

6. Arquitectura o flujo

7. Qué observar en el paper original

- Los **tres enfoques independientes** con los que estiman el óptimo. Que tres métodos distintos converjan es lo que hace creíble el resultado: no es un ajuste afortunado.
- El **experimento de validación**: entrenar el modelo predicho como óptimo y comprobar que supera a los mayores de la época al mismo cómputo.
- La discusión sobre **disponibilidad de datos**: si D debe crecer con N , el cuello de botella se desplaza al corpus. Ese punto envejeció muy bien.
- Las **limitaciones que los autores declaran**: rango de ajuste, arquitectura concreta, y que la ley describe pérdida de preentrenamiento, no capacidad en tareas.

8. Evidencia y resultados

Cientos de modelos entrenados en un rango amplio de presupuestos, tres estimaciones independientes del exponente óptimo, y una validación empírica entrenando el modelo predicho.

Los tamaños concretos, la razón óptima de tokens por parámetro y los resultados por benchmark están en las tablas del artículo. Verificarlos allí: es un paper donde los números concretos se citan mal con mucha frecuencia.

La miniatura de este eje reproduce el **razonamiento**, no los valores: a cómputo idéntico, la mejor y la peor asignación difieren claramente en pérdida, y al duplicar el presupuesto crecen N y D a la vez.

9. Impacto

- Reorientó la industria: los modelos posteriores se entrenan con muchos más tokens por parámetro que antes de 2022.
- Convirtió los **datos** en el recurso escaso y disparó el trabajo sobre curación, filtrado, repetición controlada de épocas y datos sintéticos.
- Introdujo en la práctica la distinción entre **óptimo de entrenamiento** y **óptimo de despliegue**: si un modelo se sirve a millones de peticiones, conviene entrenar uno más pequeño *por debajo* del óptimo de Chinchilla y darle aún más datos.
- Es un ejemplo de manual de cómo una medición cuidadosa refuta una intuición dominante.

10. Limitaciones

1. **Describe pérdida de preentrenamiento**, no capacidad, utilidad ni seguridad.
2. **Rango de ajuste acotado**: extrapolar varios órdenes de magnitud fuera de él no está justificado por el paper.
3. **Ignora el coste de inferencia**, que hoy suele dominar el coste total del ciclo de vida.
4. $C \approx 6ND$ **es una aproximación** que depende de la arquitectura y del régimen.
5. **Supone datos abundantes y de calidad uniforme**: repetir épocas o mezclar calidades cambia la ecuación.

6. **No cubre arquitecturas dispersas:** un modelo de mezcla de expertos tiene una relación distinta entre parámetros y cómputo.

11. Errores comunes

Error	Corrección
«Chinchilla dice que 20 tokens por parámetro es la regla»	Es una razón derivada de exponentes ajustados en un rango concreto y con una arquitectura concreta. Citarla como constante universal es sobregeneralizar.
«Menor pérdida = mejor modelo para mi tarea»	La ley predice pérdida de preentrenamiento. La utilidad en tu tarea es otra medición.
«El óptimo de entrenamiento es el modelo que hay que desplegar»	Si vas a servir mucho, conviene un modelo más pequeño y más entrenado.
«Kaplan se equivocó»	Kaplan et al. midieron algo real con un protocolo distinto. Chinchilla corrige el reparto óptimo, no invalida el marco.
«Ya no hace falta escalar parámetros»	Hace falta escalar ambos . El resultado es sobre el reparto, no sobre parar.

12. Relación con trabajos anteriores

- **P10 GPT-3 (2020)** — el modelo cuyo régimen de entrenamiento se cuestiona.
- **Kaplan et al. (2020)** — las leyes de escalado que este trabajo corrige. [arXiv:2001.08361](#)
- **Po8 Transformer (2017)** — la arquitectura sobre la que se mide.

13. Relación con trabajos posteriores

- **P21 Mixtral (2024)** — cambia la relación entre capacidad y cómputo, y por tanto la forma del problema.
- **P22 DeepSeek-R1 (2025)** — mueve el cómputo al momento de la inferencia, una variable que esta ley no modela.
- **Snell et al. (2024)** — escalar cómputo en inferencia puede rendir más que escalar parámetros. [arXiv:2408.03314](#)
- **Trabajo sobre calidad y repetición de datos (2023+)** — la consecuencia directa de volver escaso el corpus.

14. Notebook asociado

`P19_scaling_laws.ipynb`

Qué implementa: la forma paramétrica, la comparación de asignaciones a **cómputo idéntico**, el desplazamiento del óptimo al crecer el presupuesto y una comparación entre coste de entrenamiento y de inferencia.

Qué NO implementa: ningún entrenamiento. Y sus constantes son **didácticas**, no las ajustadas en el paper — el notebook lo declara en su salida.

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe $L(N, D)$ y explica qué representa cada término.
Explicar	Explica por qué E no se puede reducir con más cómputo.
Aplicar	Con la restricción $6ND = C$, calcula la pérdida de tres asignaciones y ordénalas.
Analizar	Deriva la condición de óptimo sustituyendo $D = C/(6N)$ y anulando la derivada.
Evaluar	Un equipo cita «20 tokens por parámetro» como ley universal. Formula dos objeciones.
Crear	Extiende el análisis añadiendo un término de coste de inferencia y resuelve el nuevo óptimo.

16. Autoevaluación

1. ¿Qué se mantiene constante al comparar asignaciones, y por qué es imprescindible?
2. ¿Qué significa E y por qué no baja al escalar?
3. ¿Por qué tres métodos independientes hacen más creíble el resultado?
4. ¿Qué recurso pasa a ser escaso como consecuencia de este paper?
5. ¿Por qué el óptimo de entrenamiento no es el óptimo de despliegue?
6. ¿Qué **no** predice esta ley?
7. ¿Por qué una arquitectura dispersa rompe la forma del problema?

17. Respuestas esperadas

1. El cómputo C . Sin fijarlo, comparar modelos es comparar presupuestos, no decisiones de diseño.
2. El error irreducible: la incertidumbre intrínseca del lenguaje. Ningún modelo puede predecir perfectamente el siguiente token, y E es esa cota.
3. Porque reduce la probabilidad de que el resultado sea un artefacto del método de ajuste. La convergencia de estimaciones independientes es evidencia; un solo ajuste es una hipótesis.
4. Los datos. Si D debe crecer con N , el corpus de calidad se vuelve el cuello de botella.
5. Porque el coste de inferencia es proporcional a N y se paga en **cada** petición, mientras que el de entrenamiento se paga una vez. Con volumen alto, conviene un modelo menor.
6. Capacidades concretas, utilidad, seguridad, comportamiento fuera de distribución, ni el resultado en tareas específicas.
7. Porque en un modelo disperso los parámetros totales y el cómputo por token dejan de ser proporcionales: $C \approx 6ND$ ya no se sostiene con la misma N .

18. Fuentes primarias

- Hoffmann, J. et al. (2022). *Training Compute-Optimal Large Language Models*. **NeurIPS 2022**. arXiv:2203.15556 · consultado 2026-08-16.

- Kaplan, J. et al. (2020). *Scaling Laws for Neural Language Models*. [arXiv:2001.08361](#) · consultado 2026-08-16.
- Snell, C., Lee, J., Xu, K. y Kumar, A. (2024). *Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters*. [arXiv:2408.03314](#) · consultado 2026-08-16.

Evaluación — P19

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Training Compute-Optimal Large Language Models* (2022, arXiv:2203.15556 · NeurIPS 2022) ·

Nivel: L4

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P19_scaling_laws.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P20 — Mamba

Ruta ampliada · El primer competidor serio del Transformer en lenguaje: tiempo lineal y estado de tamaño fijo, sin atención.

Nivel: L4 · Motor: `ssm` · Notebook: `P20_mamba.ipynb` · Anexo matemático: complejidad, coste y escalado

1. Identificación

Campo	Valor
Título original	<i>Mamba: Linear-Time Sequence Modeling with Selective State Spaces</i>
Autoría	Albert Gu, Tri Dao
Año	2023 (arXiv v1, diciembre)
Venue	arXiv:2312.00752 · COLM 2024 (Outstanding Paper Award)
Fuente primaria	arXiv:2312.00752
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

El Transformer compró paralelismo pagando $O(n^2)$ en cómputo y una caché que crece con la secuencia. Para contextos largos eso es prohibitivo, y no por falta de ingeniería: es la forma del mecanismo.

Existían alternativas subcuadráticas —atención lineal, convoluciones con puertas, modelos recurrentes, espacios de estados estructurados (S4)— y todas compartían el mismo destino: funcionaban bien en señales continuas (audio, series) y **perdían claramente frente a la atención en lenguaje**.

Los autores identifican por qué: esos modelos son **invariantes en el tiempo**. Sus parámetros no dependen de lo que están leyendo, así que no pueden decidir ignorar un token irrelevante ni retener uno importante. Les falta razonamiento sobre el **contenido**.

3. Propuesta

Hacer que los parámetros del espacio de estados sean **función de la entrada**. Esa es la selección: la puerta que decide cuánto se propaga y cuánto se olvida depende del token actual.

Tiene un coste inmediato: un SSM con parámetros variables ya no se puede expresar como una convolución, que era justo lo que hacía eficientes a los modelos previos. La segunda mitad de la contribución es resolver eso con un **algoritmo paralelo consciente del hardware** que opera en modo recurrente sin materializar el estado expandido en memoria lenta.

Con ambas piezas, integran el bloque en una arquitectura simplificada **sin atención y sin bloques MLP separados**, y reportan inferencia con 5× más rendimiento que Transformers comparables y escalado lineal en la longitud.

4. Intuición sin fórmulas

Un RNN es alguien tomando notas en una libreta de tamaño fijo: barato, pero acaba borrando. La atención es alguien que se guarda todos los papeles: no olvida nada, pero necesita releerlos todos cada vez. Mamba mantiene la libreta de tamaño fijo y añade criterio: **decide qué apuntar según lo que está leyendo**.

Dónde deja de funcionar la analogía: una libreta de tamaño fijo **es** compresión con pérdida. Por buena que sea la decisión, no puedes recuperar literalmente un token que decidiste no apuntar. La atención sí.

5. Matemática mínima

SSM invariante en el tiempo (S4 y anteriores):

$$h_t = A \cdot h_{t-1} + B \cdot x_t$$

$$y_t = C \cdot h_t$$
A, B, C FIJAS
→ equivalente a una convolución con un núcleo largo, y por tanto paralelizable

SSM selectivo (Mamba):

$$h_t = A(x_t) \cdot h_{t-1} + B(x_t) \cdot x_t$$

$$y_t = C(x_t) \cdot h_t$$
A, B, C DEPENDEN DE x_t
→ ya no es convolución; hace falta un escaneo paralelo explícito

Coste y memoria, para longitud n , dimensión d y estado N :

	Cómputo por capa	Memoria durante la generación	Camino entre posiciones
Atención	$O(n^2 \cdot d)$	$O(n \cdot d)$ — la caché KV crece	$O(1)$
SSM selectivo	$O(n \cdot d \cdot N)$	$O(d \cdot N)$ — constante	$O(n)$

Las dos últimas columnas son el compromiso completo: memoria constante a cambio de perder el acceso directo a cualquier posición.

6. Arquitectura o flujo

7. Qué observar en el paper original

- La **motivación por tareas sintéticas**: copia selectiva y cabezas de inducción. Están diseñadas para aislar exactamente la capacidad que falta a los SSM previos, y son la mejor parte pedagógica del artículo.
- La explicación de por qué la selección **rompe** la equivalencia con la convolución, y qué se hace al respecto. Es donde el paper es más honesto sobre su propio coste.
- El **algoritmo consciente del hardware**: qué se guarda en memoria rápida y qué se recomputa.
- Las **ablaciones** sobre qué parámetros conviene hacer selectivos.
- La comparación de **rendimiento de inferencia** y de escalado con la longitud, además de la calidad.

8. Evidencia y resultados

Evaluación en lenguaje, audio y genómica, con comparación frente a Transformers de tamaño comparable y frente a SSM previos.

El artículo reporta que **Mamba-3B supera a Transformers de su mismo tamaño e iguala a Transformers del doble de tamaño** en modelado de lenguaje, con **5× más rendimiento de inferencia** y escalado lineal, con mejoras que se mantienen en secuencias de hasta longitud de millones.

Las cifras por tarea, tamaño y línea base están en las tablas del artículo. Verificarlas allí: los resultados dependen mucho de la escala evaluada, y el rango del paper no llega al de los modelos frontera.

La miniatura de este eje aísla el mecanismo: en una tarea de copia selectiva, el SSM selectivo separa tokens marcados de relleno con el doble de margen que el invariante, y la tabla de complejidad muestra la memoria constante frente a la caché creciente.

9. Impacto

- Reabrió una pregunta que se daba por cerrada: **si la atención es realmente necesaria**.
- Impulsó una línea completa de arquitecturas **híbridas** que alternan bloques de atención y de SSM para quedarse con lo mejor de ambos (por ejemplo Jamba, 2024).
- Llevó el foco al **coste de inferencia y a la memoria de contexto largo**, que la comunidad trataba como problema de ingeniería y aquí se ataca desde la arquitectura.
- Ganó el **Outstanding Paper Award de COLM 2024**.

10. Limitaciones

1. **Un estado fijo es compresión con pérdida**. No hay recuperación exacta de un token concreto, cosa que la atención sí ofrece.
2. **Rendimiento peor en tareas de copia literal y recuperación exacta**, precisamente donde la atención brilla.
3. **La escala evaluada** en el artículo es menor que la de los modelos frontera; extrapolar paridad general no está respaldado.
4. **El algoritmo depende del hardware**: parte de la ventaja proviene de una implementación cuidadosa, no solo de la arquitectura.

- 5. **Ecosistema mucho más pequeño** que el del Transformer: menos herramientas, menos kernels optimizados, menos experiencia acumulada.
- 6. **La selección añade parámetros y complejidad** frente a un SSM invariante.

11. Errores comunes

Error	Corrección
«Mamba sustituye al Transformer»	Afirmación sin tarea, escala ni presupuesto. El paper reporta paridad en su rango, no en general.
«Los SSM son nuevos»	S4 y la línea de espacios de estados son anteriores. Lo nuevo es la selección .
«Es un RNN»	Comparte el estado recurrente, pero se entrena con un escaneo paralelo, no secuencialmente paso a paso.
«Tiempo lineal = siempre más rápido»	Para secuencias cortas, un Transformer bien optimizado puede ganar. La ventaja aparece con <code>n</code> grande.
«Memoria constante significa contexto infinito»	Significa que la memoria no crece. Lo que cabe en un estado fijo sigue siendo finito.

12. Relación con trabajos anteriores

- **P03 LSTM (1997)** — el estado recurrente con puertas; Mamba es su descendiente conceptual con entrenamiento paralelizable.
- **P08 Transformer (2017)** — el coste cuadrático que motiva todo el trabajo.
- **S4 y espacios de estados estructurados (2021-2022)** — la base directa, invariante en el tiempo.
- **P19 Leyes de escalado (2022)** — el marco económico donde se juzga si una arquitectura compensa.

13. Relación con trabajos posteriores

- **Jamba (AI21, 2024)** — híbrido Transformer-Mamba con mezcla de expertos, con contexto largo. [arXiv:2403.19887](#)
- **P21 Mixtral (2024)** — el otro eje de abaratamiento: los parámetros en vez de la secuencia.
- **Arquitecturas híbridas posteriores** — la conclusión práctica de la comunidad: alternar ambos bloques en vez de elegir uno.

14. Notebook asociado

P20_mamba.ipynb

Qué implementa: la tarea de copia selectiva comparando puertas fijas frente a puertas dependientes de la entrada, y la tabla de cómputo y memoria frente a la atención.

Qué NO implementa: el escaneo paralelo consciente del hardware —que es la mitad de la contribución—, parámetros aprendidos ni ningún entrenamiento. Las puertas están fijadas a mano para aislar el mecanismo.

```
ai-evolution paper-lab P20 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la recurrencia del SSM invariante y la del selectivo, y señala la diferencia.
Explicar	Explica por qué la selección impide expresar el modelo como convolución.
Aplicar	Ejecuta el notebook y compara la separación de ambos modelos.
Analizar	Calcula, para $n = 100\,000$ y $d = 512$, cuántas veces más memoria usa la caché KV que un estado de $N = 16$.
Evaluar	¿En qué tarea concreta apostarías por atención y no por un SSM? Justifica con el mecanismo.
Crear	Diseña una arquitectura híbrida y argumenta en qué capas pondrías atención y por qué.

16. Autoevaluación

1. ¿Qué significa que un SSM sea «invariante en el tiempo» y por qué es una limitación?
2. ¿Qué se gana y qué se pierde al hacer los parámetros dependientes de la entrada?
3. ¿Por qué la memoria del SSM no crece con la longitud?
4. ¿Cuál es el camino entre dos posiciones distantes en cada arquitectura?
5. ¿En qué tipo de tarea esperarías que la atención siga ganando?
6. ¿Por qué el algoritmo consciente del hardware es parte de la contribución y no un detalle?
7. ¿Qué afirmación sobre Mamba **no** está respaldada por el paper?

17. Respuestas esperadas

1. Que A , B y C no dependen del token que se está procesando. Aplica la misma transformación a todo, así que no puede decidir ignorar el relleno ni retener lo relevante.
2. Se gana razonamiento sobre el contenido; se pierde la equivalencia con la convolución y, con ella, la vía eficiente de entrenamiento que había que reconstruir.
3. Porque el estado tiene dimensión fija $d \cdot N$, decidida por la arquitectura y no por la entrada. Cada token actualiza ese estado en lugar de añadirse a una caché.
4. $O(1)$ en atención —cualquier posición mira a cualquier otra directamente— y $O(n)$ en el SSM, donde la información viaja a través del estado paso a paso.
5. En recuperación literal: copiar un identificador exacto visto miles de tokens atrás, búsqueda de un dato concreto en un contexto largo, tareas de *needle in a haystack*.
6. Porque sin él la selección sería inviable en la práctica: la ventaja de rendimiento que se reporta depende de esa implementación, no solo de la formulación.
7. Que sustituya al Transformer en general, o que alcance paridad a cualquier escala. El artículo evalúa un rango concreto y no afirma eso.

18. Fuentes primarias

- Gu, A. y Dao, T. (2023). *Mamba: Linear-Time Sequence Modeling with Selective State Spaces*. COLM 2024. arXiv:2312.00752 · consultado 2026-08-16.
- Lieber, O. et al. (2024). *Jamba: A Hybrid Transformer-Mamba Language Model*. arXiv:2403.19887 · consultado 2026-08-16.

Evaluación — P20

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Mamba: Linear-Time Sequence Modeling with Selective State Spaces* (2023, arXiv:2312.00752 · COLM 2024 (Outstanding Paper Award)) · **Nivel:** L4

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P20_mamba.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P21 — Mixtral (mezcla dispersa de expertos)

Ruta ampliada · Desacopla capacidad de cómputo: muchos parámetros disponibles, pocos activos por token.

Nivel: L3 · Motor: moe · Notebook: P21_moe.ipynb · Anexo matemático: complejidad, coste y escalado

1. Identificación

Campo	Valor
Título original	Mixtral of Experts
Autoría	Albert Q. Jiang y otros (Mistral AI)
Año	2024
Venue	arXiv:2401.04088
Fuente primaria	arXiv:2401.04088
Acceso	Abierto · pesos publicados con licencia Apache 2.0
Fecha de consulta	2026-08-16

2. Problema anterior

En un Transformer **denso**, cada token que entra activa **todos** los parámetros del modelo. Duplicar la capacidad duplica el coste de cada token, en entrenamiento y —lo que más duele— en cada petición de inferencia para siempre.

Chinchilla había dicho cuánto conviene gastar; no había dicho cómo obtener más capacidad **sin** pagarla en cada token. La capa dispersa de expertos existía desde Shazeer et al. (2017) y en trabajos posteriores, pero arrastraba fama de inestable y de difícil de reproducir, y los modelos que la usaban no eran abiertos.

3. Propuesta

Sustituir la capa feed-forward de cada bloque por **8 expertos** y un **router** que elige **2 por token**. La salida es la combinación ponderada de esos dos.

El modelo resultante tiene ≈47 000 millones de parámetros totales pero usa ≈13 000 millones por token. El artículo reporta que iguala o supera a Llama 2 70B y a GPT-3.5 en los benchmarks evaluados, con la variante ajustada a instrucciones superando también a varios modelos de chat de la época.

Y —la parte que más impacto tuvo— **publica los pesos bajo Apache 2.0**, lo que convirtió la arquitectura dispersa en algo que cualquiera podía inspeccionar, servir y ajustar.

4. Intuición sin fórmulas

Un hospital con ocho especialistas. Cada paciente no pasa por los ocho: recepción lo deriva a los dos que corresponden. El hospital tiene la capacidad de ocho; cada consulta cuesta dos.

Dónde deja de funcionar la analogía: los ocho especialistas tienen que estar **contratados y en el edificio** aunque solo trabajen dos. En un MoE hay que cargar todos los pesos en memoria aunque solo se computen dos expertos. Ahorra cómputo, no memoria.

5. Matemática mínima

Capa densa:

$$y = \text{FFN}(x)$$

$$\text{coste} \propto \text{ TODOS los parámetros}$$

Capa MoE dispersa con top-k:

$$\text{scores} = x \cdot W_{\text{router}}$$

(uno por experto)

$$\text{top} = \text{índices de los } k \text{ scores mayores}$$

$$g(x) = \text{softmax}(\text{scores}[\text{top}])$$

pesos normalizados sobre los k elegidos

$$y = \sum_{i \in \text{top}} g_i(x) \cdot E_i(x)$$

$$\text{coste} \propto k \text{ expertos}$$

Con $n_{\text{expertos}} = 8$ y $k = 2$, la **fracción activa** es $k/n = 25\%$.

Balanceo de carga. El router puede colapsar: unos pocos expertos reciben casi todos los tokens y el resto no recibe gradiente. Se mitiga con un término auxiliar que penaliza el desequilibrio, medible con el coeficiente de variación de la carga:

$$\text{CV} = \text{desviación_típica}(\text{carga}) / \text{media}(\text{carga}) \quad \rightarrow \quad \theta = \text{reparto perfecto}$$

6. Arquitectura o flujo

Los seis expertos en gris **no se computan** para este token — pero **sí están cargados en memoria**, porque el siguiente token puede necesitarlos.

7. Qué observar en el paper original

- La **tabla de parámetros totales frente a activos**, y cómo se traduce en coste de inferencia.
- El **análisis de especialización del router**: los autores estudian si los expertos se especializan por dominio o por sintaxis. El resultado es menos limpio de lo que la intuición sugiere, y merece leerse con cuidado.
- La comparación de **coste efectivo** frente a modelos densos de rendimiento similar.

- La sección de la variante **instruct** y su evaluación.
- Que es un **informe técnico de un modelo publicado**, no un estudio controlado de la arquitectura: hay que leerlo con esa expectativa.

8. Evidencia y resultados

Evaluación frente a Llama 2 70B y GPT-3.5 en un conjunto amplio de benchmarks: razonamiento, matemáticas, código y multilingüe.

El artículo reporta que Mixtral iguala o supera a ambos usando **muchos menos parámetros activos por token**, y que la variante ajustada a instrucciones supera en evaluaciones humanas a varios modelos de chat contemporáneos.

Las cifras por benchmark están en las tablas del artículo. Verificarlas allí, y tener presente que un informe técnico de un laboratorio sobre su propio modelo pide replicación independiente antes de darse por firme.

La miniatura de este eje muestra la mecánica y su fallo característico: con 8 expertos y top-2, la fracción activa es del 25 %, y el reparto de carga es desigual ($CV \approx 0,13$) hasta que se añade el término de balanceo ($CV \approx 0,04$).

9. Impacto

- Normalizó la **mezcla de expertos** como arquitectura de producción, no como curiosidad de investigación.
- Al publicar los pesos con licencia permisiva, permitió que la comunidad estudiara, sirviera y ajustara un modelo disperso real.
- Obligó a la industria a distinguir **parámetros totales**, **parámetros activos** y **memoria necesaria** al comparar modelos: antes se citaba un solo número.
- Empujó el trabajo sobre servido de modelos dispersos: enrutado eficiente, expertos repartidos entre dispositivos y descarga a memoria del sistema.

10. Limitaciones

1. **Ahorra cómputo, no memoria.** Hay que cargar los 47 000 M aunque solo se computen 13 000 M. Es el malentendido más caro de esta arquitectura.
2. **Colapso del router:** sin balanceo, unos pocos expertos acaparan y el modelo disperso se comporta como uno denso más caro.
3. **Complejidad de servido:** enrutar tokens a expertos repartidos entre GPU añade comunicación y hace la latencia menos predecible.
4. **La especialización es difícil de interpretar:** los expertos no se corresponden con conceptos limpios.
5. **Informe técnico, no estudio controlado:** no aísla la contribución de la arquitectura frente a los datos o al entrenamiento.
6. **Ajuste fino más delicado** que en un modelo denso equivalente.

11. Errores comunes

Error	Corrección
«13 000 M activos, luego cabe en una GPU de 13 000 M»	Falso y caro. Hay que cargar todos los expertos: dimensiona por los 47 000 M.
«Los expertos se especializan por tema»	El análisis del paper no muestra una especialización semántica limpia.
«MoE es siempre más barato»	Es más barato en FLOPs por token . En memoria y en complejidad de servido, no.
«Mixtral inventó la mezcla de expertos»	Shazeer et al. (2017) y trabajos posteriores son anteriores. Mixtral la hizo abierta y de producción.
«Más expertos siempre es mejor»	Más expertos con el mismo k agravan el desbalanceo y la complejidad de comunicación.

12. Relación con trabajos anteriores

- **Shazeer et al. (2017)** — la capa MoE dispersa con puertas, el antecedente directo. [arXiv:1701.06538](#)
- **Po8 Transformer (2017)** — la capa feed-forward que se sustituye, y donde vive la mayoría de los parámetros de cada bloque.
- **P19 Leyes de escalado (2022)** — el marco de coste que esta arquitectura reescribe: $C \approx 6ND$ deja de valer con la misma **N**.

13. Relación con trabajos posteriores

- **Jamba (2024)** — combina MoE con bloques de Mamba. [arXiv:2403.19887](#)
- **P22 DeepSeek-R1 (2025)** — la línea de modelos abiertos de gran escala donde lo disperso ya es la norma.
- **Trabajo de servido de modelos dispersos (2024+)** — enrutado, descarga y paralelismo de expertos.

14. Notebook asociado

P21_moe.ipynb

Qué implementa: un router top-2 sobre 8 expertos con 400 tokens, el conteo de parámetros totales frente a activos, la medición del desbalanceo con CV y el efecto del término de balanceo. Incluye el cálculo de presupuesto de memoria correcto frente al incorrecto.

Qué NO implementa: los expertos no computan nada —solo se cuenta a quién se enruta—, no hay entrenamiento conjunto, ni capacidad por experto, ni comunicación entre dispositivos, que es donde está la dificultad real.

```
ai-evolution paper-lab P21 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la salida de una capa MoE con top-k y define $g_i(x)$.
Explicar	Explica por qué MoE ahorra cómputo pero no memoria.
Aplicar	Calcula la fracción activa para (8, k=1), (8, k=2), (64, k=2).
Analizar	Ejecuta el notebook y explica qué mide el CV y por qué un CV alto es un problema de entrenamiento, no solo de eficiencia.
Evaluar	Un proveedor anuncia «modelo de 13 000 M» para un MoE 8×7B. ¿Qué le objetas?
Crear	Diseña un experimento que distinga si los expertos se especializan por dominio o por token frecuente.

16. Autoevaluación

1. ¿Qué se desacopla exactamente en una arquitectura dispersa?
2. ¿Por qué hace falta un término de balanceo de carga?
3. ¿Qué le pasa a un experto que no recibe tokens durante el entrenamiento?
4. ¿Por qué la memoria necesaria es la de **todos** los expertos?
5. ¿Cómo cambia $C \approx 6ND$ en un modelo disperso?
6. ¿Qué hace de este paper un informe técnico y no un estudio controlado?
7. ¿Qué ventaja no técnica tuvo publicar los pesos con Apache 2.0?

17. Respuestas esperadas

1. La **capacidad** (parámetros totales, que determinan cuánto puede saber el modelo) del **cómputo por token** (parámetros activos, que determinan cuánto cuesta cada token).
2. Porque el router tiende a concentrarse: si un experto empieza ligeramente mejor, recibe más tokens, mejora más y acapara. Es un bucle de realimentación positiva.
3. No recibe gradiente y deja de mejorar: su capacidad queda desperdiciada y el modelo disperso degenera hacia uno denso más pequeño y más caro de servir.
4. Porque el enrutado depende del token y cualquier token puede activar cualquier experto: no se puede saber de antemano cuáles harán falta.
5. Deja de valer con N = parámetros totales. El cómputo escala con los parámetros **activos**, así que hay que distinguir ambos en la ecuación.
6. Que describe un modelo concreto que se publica, sin ablaciones que aíslen la contribución de la arquitectura frente a los datos, el tamaño o la receta de entrenamiento.
7. Permitted replicación, auditoría y servido independientes, y convirtió lo disperso en algo estudiable por cualquiera en vez de en una afirmación de un laboratorio.

18. Fuentes primarias

- Jiang, A. Q. et al. (2024). *Mixtral of Experts*. arXiv:2401.04088 · consultado 2026-08-16.
- Shazeer, N. et al. (2017). *Outrageously Large Neural Networks: The Sparsely-Gated Mixture-of-Experts Layer*. arXiv:1701.06538 · consultado 2026-08-16.

Evaluación — P21

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Mixtral of Experts* (2024, arXiv:2401.04088) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P21_moe.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P22 — DeepSeek-R1

Ruta ampliada · El razonamiento se incentiva con refuerzo puro, sin trazas humanas anotadas. Y es el primer LLM de pesos abiertos publicado tras revisión por pares.

Nivel: L5 · **Motor:** r1_reasoning · **Notebook:** P22_deepseek_r1.ipynb · **Anexo matemático:** probabilidad y verosimilitud

1. Identificación

Campo	Valor
Título original	DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning
Autoría	DeepSeek-AI
Año	2025
Venue	arXiv:2501.12948 (enero 2025) · Nature 645, 633–638 (18 sept. 2025)
Fuente primaria	arXiv:2501.12948 · DOI Nature
Acceso	Abierto · pesos publicados
Fecha de consulta	2026-08-16

2. Problema anterior

El razonamiento en cadena de ReAct y del *chain-of-thought* funcionaba, pero dependía de **demostraciones humanas**: alguien tenía que escribir miles de razonamientos ejemplares para que el modelo los imitara. Eso es caro, no escala, y —más importante— **acota la capacidad del modelo a la de quien escribió los ejemplos**.

La alineación por preferencias de InstructGPT y DPO optimizaba lo que a un anotador *le parecía* mejor, una señal subjetiva y hackeable. Para razonar hay algo mejor disponible: en matemáticas y código, la respuesta **se puede comprobar**.

3. Propuesta

Entrenar con **refuerzo puro sobre una recompensa verificable**: no se premia el estilo del razonamiento ni se compara con una traza de referencia. Solo se comprueba si la respuesta final es correcta.

El resultado central es que el comportamiento de razonamiento **emerge** de ese incentivo. El artículo reporta la aparición de patrones como autorreflexión, verificación y adaptación dinámica de la estrategia, sin que nadie los demostrara. Y esos patrones, una vez presentes en un modelo grande, pueden **transferirse a modelos menores**.

El algoritmo de refuerzo concreto y las variantes del modelo se describen en el cuerpo del artículo. El resumen no los nombra: verificarlos allí antes de citarlos.

4. Intuición sin fórmulas

Enseñar a resolver problemas de matemáticas sin corregir el procedimiento: solo dices «bien» o «mal» al resultado. Con suficientes intentos, el alumno descubre por su cuenta que le renta comprobar antes de entregar — porque comprobar sube su tasa de aciertos, no porque se lo mandaran.

Dónde deja de funcionar la analogía: el alumno entiende *por qué* comprobar ayuda. El modelo solo ha encontrado una política con más recompensa esperada. Y esa distinción importa cuando el verificador no existe.

5. Matemática mínima

RLHF (P12): r = modelo de recompensa aprendido de preferencias humanas
→ subjetivo, aproximado, explotable (reward hacking)

Aquí : $r(x, y) = 1$ si `respuesta_final(y)` es correcta, 0 si no
→ objetivo, exacto, no explotable por estilo

Objetivo: maximizar $E_{y \sim \pi(\cdot|x)} [r(x, y)]$

La señal **no dice cómo razonar**. Solo dice si acertaste. Todo lo que aparezca entre el enunciado y la respuesta es instrumental: sobrevive si aumenta la probabilidad de acertar.

Consecuencia medible y no gratuita: las trayectorias que aciertan más son **más largas**, así que la política aprendida gasta más tokens por respuesta. El cómputo se desplaza del entrenamiento a la inferencia.

6. Arquitectura o flujo

Nadie etiqueta el contenido de las trazas. La única flecha con información humana es la que aporta **la solución conocida** del problema.

7. Qué observar en el paper original

- El **algoritmo de refuerzo** empleado y por qué se elige frente a PPO: está en el cuerpo, no en el resumen.
- El **diseño de la recompensa**: qué se verifica exactamente y cómo se evita que el modelo aprenda a producir el formato correcto sin resolver el problema.
- Las **curvas de longitud de respuesta** a lo largo del entrenamiento: el modelo aprende solo a escribir más. Es la evidencia más elocuente de que el comportamiento emerge.
- La sección de **destilación** a modelos pequeños y qué se conserva.
- Las **limitaciones declaradas** y los dominios donde el método no aplica.
- Que existen **dos versiones**: el preprint de enero de 2025 y la versión revisada por pares en *Nature* de septiembre de 2025. Cita la que leíste.

8. Evidencia y resultados

Evaluación en matemáticas, código y tareas STEM, comparando frente a líneas base entrenadas con demostraciones humanas.

El artículo reporta que el enfoque de refuerzo puro **supera a las líneas base supervisadas** y que los patrones de razonamiento emergentes se pueden transferir a modelos menores.

Las cifras por benchmark, los tamaños de modelo y el detalle de la destilación están en el artículo. Verificarlos allí — y preferir la versión de *Nature*, que pasó revisión.

La miniatura de este eje reproduce el mecanismo con tres estrategias: la política se desplaza hacia la que verifica, la exactitud esperada sube de $\sim 0,61$ a $\sim 0,81$ **sin una sola traza anotada**, y el coste en tokens casi se duplica en el mismo movimiento.

9. Impacto

- Consolidó el **razonamiento con recompensa verificable** como línea principal de trabajo, frente a la imitación de demostraciones.
- Trasladó el foco del **cómputo de entrenamiento** al **cómputo de inferencia**: una variable que las leyes de escalado de P19 no modelan.
- Como primer LLM de pesos abiertos publicado tras revisión por pares, marcó un precedente sobre qué nivel de escrutinio es exigible a un modelo, en un campo acostumbrado a informes técnicos autopublicados.
- Popularizó la **destilación de capacidad de razonamiento** a modelos pequeños y ejecutables localmente.

10. Limitaciones

1. **Solo funciona donde hay verificador.** Matemáticas y código lo tienen; redacción, diagnóstico o consejo legal, no. Es la limitación de fondo.
2. **Coste de inferencia mayor:** razonar más es gastar más tokens en cada respuesta.

- 3. **La traza no es una prueba.** Se optimizó la corrección de la respuesta final, no la validez de cada paso intermedio. Un razonamiento largo y seguro de sí mismo puede contener pasos inválidos.
- 4. **Riesgo de sobreajuste al verificador:** aprender a satisfacer la comprobación sin resolver el problema.
- 5. **Coste de entrenamiento** fuera del alcance de casi cualquier equipo.
- 6. **La destilación transfiere comportamiento, no garantías:** el modelo pequeño imita el patrón sin la misma base.
- 7. **Evaluar razonamiento es un problema abierto:** los benchmarks de matemáticas y código se saturan y se contaminan rápido.

11. Errores comunes

Error	Corrección
«El modelo aprendió a razonar»	Aprendió una política con mayor recompensa esperada, en la que aparecen comportamientos que parecen razonamiento y que funcionan. Cuál de las dos descripciones es correcta es una pregunta abierta, no un hecho establecido.
«La cadena de pensamiento explica la respuesta»	Es el mismo error que en ReAct, ahora con textos más largos y persuasivos.
«Esto sustituye a RLHF»	Resuelve el razonamiento en dominios verificables. La alineación con preferencias sigue siendo necesaria para lo demás.
«Más tokens de razonamiento = mejor respuesta»	Hay rendimientos decrecientes y un coste lineal. Conviene medir dónde deja de compensar.
«Refuerzo puro significa sin datos humanos»	Los problemas y sus soluciones conocidas son datos humanos. Lo que falta son las trazas de razonamiento .

12. Relación con trabajos anteriores

- **P12 InstructGPT (2022)** — el refuerzo con retroalimentación humana cuya señal subjetiva aquí se sustituye por una verificable.
- **P13 ReAct (2022)** y el *chain-of-thought* — el razonamiento explícito que dependía de demostraciones.
- **P15 DPO (2023)** — la línea de alineación simplificada.
- **P19 Leyes de escalado (2022)** — el marco de cómputo que este trabajo desplaza hacia la inferencia.

13. Relación con trabajos posteriores

- **Snell et al. (2024)** — escalar cómputo en inferencia puede rendir más que escalar parámetros; el marco teórico del mismo fenómeno. [arXiv:2408.03314](#)
- **Lo posterior a 2025-08 no está aquí:** por el criterio de ascenso del eje, vive en `frontier/current-topics.yaml` con fecha de revisión.

Preguntas abiertas que conviene seguir:

- cómo definir recompensas verificables fuera de matemáticas y código;
- cómo evaluar la **validez de los pasos**, no solo del resultado;
- cuánto cómputo de inferencia compensa y cómo decidirlo por consulta.

14. Notebook asociado

P22_deepseek_r1.ipynb

Qué implementa: una política sobre tres estrategias de resolución entrenada **solo** con la corrección del resultado, la curva de exactitud frente a coste en tokens, y el análisis de cuándo el presupuesto de inferencia hace inviable la estrategia que más acierta.

Qué NO implementa: ningún modelo de lenguaje, ninguna generación de trazas y ningún algoritmo de refuerzo del artículo. Las exactitudes de cada estrategia están fijadas por diseño; en el paper emergen del entrenamiento.

```
ai-evolution paper-lab P22 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Enuncia la diferencia entre la recompensa de RLHF y la de este trabajo.
Explicar	Explica por qué una recompensa verificable resiste el reward hacking clásico.
Aplicar	Ejecuta el notebook con tres semillas y compara exactitud y coste finales.
Analizar	Diseña una tarea donde el modelo pueda satisfacer al verificador sin resolver el problema.
Evaluar	Un sistema mejora 3 puntos gastando 5× tokens. ¿Compensa? Di qué necesitas saber para responder.
Crear	Propón un verificador para un dominio no verificable y argumenta honestamente sus fallos.

16. Autoevaluación

1. ¿Qué señal sustituye a las trazas humanas anotadas?
2. ¿Por qué esa señal es más robusta que un modelo de recompensa aprendido?
3. ¿Qué comportamiento emerge y por qué, si nadie lo demostró?
4. ¿Qué le pasa al coste de inferencia y por qué es inevitable?
5. ¿Por qué el método no se traslada directamente a redactar un informe médico?
6. ¿Qué significa que sea el primer LLM de pesos abiertos revisado por pares?
7. ¿Qué NO demuestra una traza de razonamiento larga y convincente?

17. Respuestas esperadas

1. La corrección **verificable** de la respuesta final, comparada con una solución conocida.
2. Porque no se puede engañar con estilo, longitud o confianza aparente: o el resultado coincide o no. Un modelo de recompensa aprendido sí puede explotarse por esas vías.

3. Verificación, reflexión y cambio de estrategia. Emergen porque **aumentan la probabilidad de acertar**, que es lo único que se premia.
4. Sube, porque las trayectorias que aciertan tienden a ser más largas. Es inevitable: el incentivo premia acertar, y razonar más ayuda a acertar más.
5. Porque no existe un verificador barato y objetivo de «buen informe médico». Sin verificador, la recompensa vuelve a ser un juicio subjetivo y reaparecen todos los problemas de RLHF.
6. Que un tercero independiente examinó el método y las afirmaciones antes de publicarse, en un campo donde la norma es el informe técnico autopublicado por el propio laboratorio.
7. Que los pasos intermedios sean válidos. Se optimizó la respuesta final; el texto intermedio es un medio, no un certificado auditado.

18. Fuentes primarias

- DeepSeek-AI (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning*. arXiv:2501.12948 · consultado 2026-08-16.
- DeepSeek-AI (2025). *DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning*. **Nature** 645, 633–638. doi.org/10.1038/s41586-025-09422-z · consultado 2026-08-16.
- Snell, C., Lee, J., Xu, K. y Kumar, A. (2024). *Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters*. arXiv:2408.03314 · consultado 2026-08-16.

Evaluación — P22

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning* (2025, arXiv:2501.12948 · Nature 645, 633–638 (2025)) · **Nivel:** L5

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P22_deepseek_r1.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P23 — GloVe

Ruta de representación · Unifica las dos familias de embeddings: aprovechar las estadísticas globales del corpus y conseguir la estructura lineal de los métodos predictivos.

Nivel: L2 · Motor: glove · Notebook: P23_glove.ipynb · Anexo: álgebra y geometría

1. Identificación

Campo	Valor
Título original	GloVe: Global Vectors for Word Representation
Autoría	Jeffrey Pennington, Richard Socher, Christopher D. Manning
Año	2014
Venue	EMNLP 2014 · ACL Anthology D14-1162
Fuente primaria	aclanthology.org/D14-1162 · DOI
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Había dos familias de métodos y cada una renunciaba a algo. Los métodos de **conteo** (LSA y derivados) construían la matriz de co-ocurrencia de todo el corpus —estadística global, entrenamiento rápido— pero producían espacios con peor estructura para analogías. Los métodos **predictivos** (Word2Vec) daban esa estructura, pero aprendían de ventanas locales, sin usar directamente las estadísticas agregadas del corpus.

La pregunta abierta: ¿qué propiedad de las estadísticas de co-ocurrencia es la que produce la estructura lineal, y se puede modelar directamente?

3. Propuesta

La respuesta de los autores: lo informativo **no es la co-ocurrencia, sino su razón**.

$P(\text{sólido} \mid \text{hielo}) / P(\text{sólido} \mid \text{vapor})$ es muy grande; para «gas» es muy pequeña; para «agua», que acompaña a ambos, es ≈ 1 . Esa razón discrimina lo que la co-ocurrencia bruta no.

Como las razones se convierten en diferencias al tomar logaritmos, proponen ajustar por mínimos cuadrados ponderados el producto de vectores al **logaritmo** de la co-ocurrencia. La función de peso evita que los pares muy frecuentes dominen y que los muy raros aporten ruido.

4. Intuición sin fórmulas

Word2Vec mira por una ventanita y aprende de lo que pasa cerca, millones de veces. GloVe cuenta primero todo el corpus en una tabla y luego busca vectores que expliquen esa tabla. Mismo destino, camino opuesto.

Dónde deja de funcionar la analogía: la tabla de conteos no es neutral. Cómo se define la ventana, si se pondera por distancia y qué se hace con las palabras muy frecuentes son decisiones que ya codifican una teoría del significado.

5. Matemática mínima

$$J = \sum_{ij} f(x_{ij}) \cdot (w_i \cdot \tilde{w}_j + b_i + \tilde{b}_j - \log x_{ij})^2$$

x_{ij} = veces que la palabra j aparece en el contexto de i

$w, \tilde{w}$ = vectores de palabra y de contexto (dos matrices, como en skip-gram)

$b, \tilde{b}$ = sesgos que absorben la frecuencia de cada palabra

$$f(x) = (x/x_{\max})^\alpha \quad \text{si } x < x_{\max}, \quad \text{si no } 1 \quad \text{con } \alpha = 3/4$$

Dos decisiones que conviene entender:

- El **logaritmo** convierte razones en diferencias, que es lo que la geometría vectorial sabe representar como desplazamientos.
- La **función de peso** es asimétrica a propósito: atenúa los pares raros (ruido estadístico) y satura en los muy frecuentes (que si no, dominarían la suma).

6. Arquitectura o flujo

Frente a Word2Vec, que recorre el corpus muchas veces por parejas, aquí el corpus se recorre **una vez** para construir X y después se entrena sobre los pares no nulos de esa matriz.

7. Qué observar en el paper original

- La **tabla de razones de co-ocurrencia** con hielo y vapor. Es el argumento entero del paper condensado en una tabla; si se entiende esa, se entiende todo lo demás.
- La **derivación** que va desde «queremos modelar razones» hasta la forma funcional concreta.
- La **función de peso** y por qué $\alpha = 3/4$.
- La comparación de **tiempo de entrenamiento** frente a los métodos predictivos: parte del argumento era práctico, no solo de calidad.

8. Evidencia y resultados

Evaluación en analogías de palabras, similitud y reconocimiento de entidades nombradas, frente a Word2Vec y a métodos de factorización, con distintos tamaños de corpus y de vector.

Las cifras por tarea, dimensión y tamaño de corpus están en las tablas del artículo. Verificarlas allí: las comparaciones de embeddings de esta época son muy sensibles al preprocesado y a los hiperparámetros, y ese es justamente el motivo de la sección 11.

La miniatura de este eje muestra el mecanismo: las razones separan limpiamente (≈ 25 para lo propio del hielo, $\approx 0,05$ para lo del vapor, ≈ 1 para lo compartido) y la pérdida baja mientras los vectores aprenden a reproducir el logaritmo de los conteos.

9. Impacto

- Los vectores GloVe preentrenados fueron durante años el punto de partida por defecto de cualquier sistema de PLN, junto con los de Word2Vec.
- Puso sobre la mesa una pregunta teórica —qué se está modelando exactamente— en un área que avanzaba de forma muy empírica.
- Consolidó la práctica de **publicar vectores preentrenados** como artefacto reutilizable, no solo el método.

10. Limitaciones

1. **Un vector por palabra:** la polisemia sigue colapsada. Lo resolverá ELMo.
2. **Memoria:** la matriz de co-ocurrencia de un corpus grande es enorme, aunque sea dispersa.
3. **Vocabulario cerrado:** sin vector para palabras no vistas.
4. **Sin composición:** no hay forma principitada de obtener el vector de una frase.
5. **Hereda los sesgos** del corpus, igual que cualquier método distribucional.
6. **La ventaja empírica sobre Word2Vec resultó frágil:** depende mucho de los hiperparámetros de ambos, como mostró trabajo posterior.

11. Errores comunes

Error	Corrección
«GloVe es contador y word2vec predictivo, son cosas distintas»	Levy y Goldberg (2015) mostraron que word2vec con muestreo negativo factoriza implícitamente una matriz relacionada. La dicotomía es menos profunda de lo que parecía.
«GloVe es mejor que word2vec»	Depende del corpus, la tarea y los hiperparámetros. Las comparaciones de la época estaban mal controladas.
«Modela la co-ocurrencia»	Modela el logaritmo de la co-ocurrencia, y lo hace porque el objeto de interés son las razones .
«La función de peso es un detalle»	Sin ella, los pares funcionales frecuentes dominan la pérdida y el espacio se degrada.

12. Relación con trabajos anteriores

- **P05 Word2Vec (2013)** — la familia predictiva que sirve de contraste.
- **LSA y factorización de matrices de co-ocurrencia (1990)** — la familia de conteo.
- **Harris (1954), Firth (1957)** — la hipótesis distribucional que ambas comparten.

13. Relación con trabajos posteriores

- **Levy y Goldberg (2015)** — el puente teórico entre ambas familias. ACL Anthology
- **FastText (2016)** — subpalabras, resuelve el vocabulario cerrado.
- **P24 ELMo (2018)** — el paso a representaciones contextuales.
- **P18 CLIP (2021)** — la misma idea de espacio compartido, con imágenes.

14. Notebook asociado

P23_glove.ipynb

Qué implementa: el ajuste por mínimos cuadrados ponderados sobre una matriz de co-ocurrencia de juguete con la estructura del ejemplo del paper, y la tabla de razones que justifica el objetivo.

Qué NO implementa: ningún corpus real. Con seis palabras no emerge semántica: se ve la mecánica del ajuste, no el resultado.

```
ai-evolution paper-lab P23 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la función objetivo y di qué representa cada término.
Explicar	Explica por qué se modela el logaritmo y no la co-ocurrencia directa.
Aplicar	Calcula las tres razones del ejemplo hielo/vapor a partir de la tabla del notebook.
Analizar	¿Qué pasa con la pérdida si $f(x) = 1$ para todo x ? Razónalo y compruébalo.
Evaluar	Lee el resumen de Levy y Goldberg (2015) y decide si la distinción contador/predictivo sigue siendo útil para enseñar.
Crear	Diseña una función de peso alternativa y argumenta qué propiedad conserva y cuál rompe.

16. Autoevaluación

1. ¿Por qué la razón de co-ocurrencias discrimina mejor que la co-ocurrencia?
2. ¿Qué papel juegan los sesgos b_i y b_j ?
3. ¿Por qué hay dos matrices de vectores?
4. ¿Qué problema resuelve la función de peso, por sus dos extremos?
5. ¿En qué se parece realmente a Word2Vec, según trabajo posterior?

6. ¿Qué limitación comparte con Word2Vec y no puede resolver?
7. ¿Qué ventaja práctica, no de calidad, defendía el paper?

17. Respuestas esperadas

1. Porque cancela la frecuencia general de la palabra de contexto. «Agua» es frecuente con todo; la razón entre dos palabras revela qué es específico de cada una.
2. Absorben la frecuencia global de cada palabra, para que el producto de vectores capture la parte informativa y no el simple hecho de que una palabra sea común.
3. Porque cada palabra juega dos papeles —centro y contexto— y necesita una representación para cada uno. Suelen sumarse o promediarse al final.
4. Por arriba, evita que los pares muy frecuentes dominen la suma; por abajo, atenúa los pares raros, cuyos conteos son ruido estadístico.
5. Levy y Goldberg mostraron que skip-gram con muestreo negativo factoriza implícitamente una matriz de información mutua puntual desplazada: ambos acaban factorizando estadísticas.
6. Un vector por palabra: la polisemia. Lo resuelven los embeddings contextuales.
7. El coste de entrenamiento: una sola pasada para construir la matriz y después entrenar solo sobre entradas no nulas.

18. Fuentes primarias

- Pennington, J., Socher, R. y Manning, C. D. (2014). *GloVe: Global Vectors for Word Representation*. **EMNLP 2014**. ACL Anthology D14-1162 · DOI · consultado 2026-08-16.
- Levy, O. y Goldberg, Y. (2015). *Improving Distributional Similarity with Lessons Learned from Word Embeddings*. **TACL**. ACL Anthology Q15-1016 · consultado 2026-08-16.

Evaluación — P23

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *GloVe: Global Vectors for Word Representation* (2014, EMNLP 2014 · ACL Anthology D14-1162) ·

Nivel: L2

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P23_glove.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P24 — ELMo

Ruta de representación · Un vector por **aparición** y no por palabra: la polisemia deja de colapsar en un único punto del espacio.

Nivel: L3 · Motor: `elmo` · Notebook: `P24_elmo.ipynb` · Anexo: álgebra y geometría

1. Identificación

Campo	Valor
Título original	<i>Deep Contextualized Word Representations</i>
Autoría	Matthew E. Peters, Mark Neumann, Mohit Iyyer, Matt Gardner, Christopher Clark, Kenton Lee, Luke Zettlemoyer
Año	2018
Venue	NAACL-HLT 2018 · ACL Anthology N18-1202
Fuente primaria	aclanthology.org/N18-1202
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Word2Vec y GloVe dan **un vector por palabra**. Eso significa que «banco del parque», «banco central» y «banco del río» comparten exactamente la misma representación: el sentido se pierde antes de que el modelo de la tarea empiece a trabajar.

Existían intentos de inducir sentidos y agruparlos, pero exigían decidir de antemano cuántos sentidos tiene cada palabra —una decisión artificial— y no aprovechaban la frase concreta.

3. Propuesta

Calcular la representación **en función de la frase entera**, usando los estados internos de un modelo de lenguaje bidireccional profundo entrenado sobre un corpus grande.

Dos decisiones importantes:

1. Las representaciones son **profundas**: no se usa solo la última capa, sino una combinación de todas.
2. Los pesos de esa combinación se **aprenden por tarea**, porque distintas capas capturan cosas distintas —las bajas, más sintaxis; las altas, más semántica—.

Se usaban como **características congeladas**: se calculaban los vectores y se alimentaban a un modelo específico de la tarea.

4. Intuición sin fórmulas

Un diccionario da una entrada por palabra. Un lector da un significado por aparición. ELMo deja de ser diccionario y pasa a leer: el vector de «banco» se calcula tras haber leído la frase.

Dónde deja de funcionar la analogía: un lector entiende; ELMo solo condiciona su representación en el contexto. Que dos apariciones queden lejos no implica que el modelo sepa que son sentidos distintos, solo que las representa distinto.

5. Matemática mínima

Modelo de lenguaje bidireccional: se entrenan dos direcciones y se suman sus verosimilitudes

$$\Sigma_k [\log p(t_k | t_1 \dots t_{k-1}) + \log p(t_k | t_{k+1} \dots t_N)]$$

Representación para la tarea:

$$\text{ELMo}_k = \gamma \cdot \sum_{j=0}^L s_j \cdot h_{k,j}$$

$h_{k,j}$ = estado de la capa j en la posición k

s_j = pesos por capa, normalizados con softmax y APRENDIDOS por tarea

γ = escala global, también aprendida

Lo esencial: ELMo_k depende de k y de toda la frase. Un embedding estático no tiene el subíndice k .

6. Arquitectura o flujo

7. Qué observar en el paper original

- El **análisis por capas**: la evidencia de que las capas bajas sirven mejor para tareas sintácticas y las altas para semánticas. Justifica que los pesos se aprendan por tarea.
- La comparación con usar **solo la capa superior**, que es lo que la intuición sugeriría.
- La sección de **desambiguación de sentidos**: es la comprobación directa de que la polisemia deja de colapsar.
- Que ELMo se **añade** a modelos existentes: el paper mejora seis tareas sin rediseñarlas.

8. Evidencia y resultados

Seis tareas de PLN —respuesta a preguntas, inferencia textual, análisis de sentimiento, etiquetado de roles semánticos, resolución de correferencia y reconocimiento de entidades— con mejoras al añadir ELMo a la arquitectura ya existente de cada una.

Las cifras por tarea y las ablaciones por capa están en las tablas del artículo. Verificarlas allí antes de citarlas.

La miniatura de este eje muestra el fenómeno central: con embedding estático las tres apariciones de «banco» tienen coseno 1,0 entre sí; con representación contextual bajan de forma desigual, y los sentidos más distintos quedan más separados.

9. Impacto

- Fue el paper que **normalizó las representaciones contextuales** en PLN, meses antes de BERT.
- Su análisis por capas abrió la línea de trabajo sobre **qué codifica cada capa** de un modelo de lenguaje, que sigue viva en interpretabilidad.
- Marcó la última etapa del paradigma «características congeladas»: BERT, poco después, ajustaría el modelo entero.

10. Limitaciones

1. **Bidireccionalidad superficial**: son dos modelos unidireccionales concatenados, no un modelo que atienda a ambos lados a la vez en cada capa. Ese es exactamente el argumento de BERT.
2. **Características congeladas**: no se ajusta el modelo base, así que se aprovecha menos.
3. **Basado en LSTM**: secuencial, no paraleliza sobre la longitud.
4. **Coste**: hay que ejecutar el modelo de lenguaje completo para obtener las representaciones.
5. **Los pesos por capa se aprenden por tarea**, lo que añade un ajuste extra.
6. **Sigue heredando** los sesgos del corpus de preentrenamiento.

11. Errores comunes

Error	Corrección
«ELMo es bidireccional como BERT»	Es una concatenación de dos direcciones independientes. BERT atiende a ambos lados en cada capa , y para eso necesita enmascarar.
«Basta con la última capa»	El paper muestra que distintas tareas prefieren distintas capas; por eso los pesos se aprenden.
«ELMo es un modelo de lenguaje»	Se entrena como tal, pero se usa como extractor de representaciones congeladas.
«Resolvió la polisemia»	Hace que las representaciones dependan del contexto. Que eso equivalga a distinguir sentidos es otra afirmación, y el paper la evalúa por separado.

12. Relación con trabajos anteriores

- **P05 Word2Vec** y **P23 GloVe** — los embeddings estáticos cuya limitación se ataca.

- **P03 LSTM (1997)** — la celda con la que se construye.
- **Howard y Ruder (2018), ULMFiT** — transferencia en PLN, contemporáneo. [arXiv:1801.06146](#)

13. Relación con trabajos posteriores

- **P09 BERT (2018)** — bidireccionalidad profunda real y ajuste fino del modelo completo; el paper con el que ELMo se compara explícitamente.
- **P25 T5 (2019)** — la unificación del formato de tarea.
- **Interpretabilidad por capas (2019+)** — la línea que abre su análisis.

14. Notebook asociado

P24_elmo.ipynb

Qué implementa: el vector de «banco» en tres frases con sentidos distintos, de forma estática y contextual, y la comparación de similitudes.

Qué NO implementa: ni LSTM ni modelo de lenguaje entrenado. La mezcla con decaimiento simula los estados internos, y los pesos por capa están fijados a mano en vez de aprendidos.

```
ai-evolution paper-lab P24 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la fórmula de <code>ELMo_k</code> y di qué es <code>s_j</code> .
Explicar	Explica por qué dos LSTM concatenadas no son lo mismo que atención bidireccional.
Aplicar	Añade una cuarta frase con «banco» como asiento y mira con cuál se agrupa.
Analizar	¿Qué pesos por capa esperarías para etiquetado gramatical frente a respuesta a preguntas?
Evaluar	¿Es «resolvió la polisemia» una afirmación defendible? Argumenta con lo que el paper mide.
Crear	Diseña una prueba que distinga «representación distinta» de «sentido distinguido».

16. Autoevaluación

1. ¿Qué significa que la representación sea contextual, en términos de la fórmula?
2. ¿Por qué se combinan varias capas en vez de usar la última?
3. ¿En qué sentido la bidireccionalidad de ELMo es superficial?
4. ¿Qué quiere decir que se use como «características congeladas»?
5. ¿Qué ventaja tiene eso, y qué se pierde?
6. ¿Qué problema de P23 resuelve y cuál no?
7. ¿Qué hizo BERT distinto pocos meses después?

17. Respuestas esperadas

1. Que lleva subíndice k y depende de los estados del modelo sobre **esa** frase: la misma palabra en otra frase da otro vector.
2. Porque las capas codifican información distinta y cada tarea necesita una mezcla distinta; el paper lo demuestra con ablaciones.
3. Porque son dos modelos unidireccionales entrenados por separado y concatenados. En ninguna capa hay una representación que haya atendido a ambos lados simultáneamente.
4. Que el modelo base no se actualiza: se calculan los vectores y se pasan a un modelo específico de la tarea, que es el único que se entrena.
5. Ventaja: barato, reutilizable, no requiere reentrenar el modelo grande. Pérdida: se aprovecha mucho menos el conocimiento del preentrenamiento.
6. Resuelve el vector único por palabra. No resuelve el vocabulario cerrado, la composición de frases ni el sesgo del corpus.
7. Bidireccionalidad profunda mediante enmascarado, y ajuste fino del modelo entero en vez de características congeladas.

18. Fuentes primarias

- Peters, M. E. et al. (2018). *Deep Contextualized Word Representations*. **NAACL-HLT 2018**. ACL Anthology N18-1202 · consultado 2026-08-16.
- Howard, J. y Ruder, S. (2018). *Universal Language Model Fine-tuning for Text Classification*. arXiv:1801.06146 · consultado 2026-08-16.

Evaluación — P24

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Deep Contextualized Word Representations* (2018, NAACL 2018 · ACL Anthology N18-1202) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P24_elmo.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P25 — T5

Ruta de representación · Todo problema de texto se reescribe como texto → texto: un solo modelo, una sola pérdida, cero cabezas específicas por tarea.

Nivel: L3 · Motor: t5 · Notebook: P25_t5.ipynb · Anexo: probabilidad y verosimilitud

1. Identificación

Campo	Valor
Título original	Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer
Autoría	Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, Peter J. Liu
Año	2019 (arXiv) · 2020 (JMLR)
Venue	arXiv:1910.10683 · JMLR 21(140)
Fuente primaria	arXiv:1910.10683 · JMLR
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Tras BERT, el campo se llenó de variantes: distintos objetivos de preentrenamiento, arquitecturas, corpus, tamaños y recetas de ajuste. Y **no se podían comparar**, porque cada trabajo cambiaba varias cosas a la vez y cada tarea usaba su propia cabeza, su propio formato y su propia métrica.

Además, ese zoo de cabezas —clasificación, regresión, extracción de índices, generación— era una fricción de ingeniería constante: nada se reutilizaba entre tareas.

3. Propuesta

Dos contribuciones que se apoyan la una en la otra.

El marco: reescribir toda tarea de texto como «texto de entrada → texto de salida», con un prefijo que indica cuál es. Clasificar es emitir la palabra `negative`; la regresión, emitir `4.2`; extraer, emitir el fragmento. Una sola pérdida: verosimilitud del texto de salida.

El estudio: una vez todo es comparable, hacer el barrido sistemático que faltaba —objetivos de preentrenamiento, arquitecturas, corpus, estrategias de ajuste, tamaños— midiendo cada decisión por separado. Y publicar el corpus resultante, **C4** (*Colossal Clean Crawled Corpus*).

4. Intuición sin fórmulas

Cinco máquinas distintas, cada una con su enchufe. T5 propone un enchufe único: si todo entra y sale como texto, sobra el adaptador.

Dónde deja de funcionar la analogía: el enchufe único tiene un coste real. Emitir un número como texto pierde precisión, y emitir una etiqueta como palabra permite que el modelo genere algo que no es ninguna de las etiquetas válidas.

5. Matemática mínima

Todas las tareas: maximizar $\log p_{\theta}(\text{texto_salida} \mid \text{texto_entrada})$

Lo único que cambia: el PREFIJO del texto de entrada

"translate English to German: That is good."	→ "Das ist gut."
"cola sentence: The movie was terrible."	→ "negative"
"sts sentence1: ... sentence2: ..."	→ "4.2"
"summarize: ..."	→ "..."

Objetivo de preentrenamiento elegido tras el barrido: **corrupción de tramos** — se enmascaran secuencias contiguas de tokens y el modelo debe emitirlas, con centinelas que marcan cada hueco. No es el enmascarado token a token de BERT, y el paper explica por qué gana.

6. Arquitectura o flujo

7. Qué observar en el paper original

- Es un artículo **largo** y su valor está en la sección de experimentos: cada subsección aísla una decisión de diseño. Leerlo como catálogo de resultados es más útil que leerlo en orden.
- La comparación de **objetivos de preentrenamiento**, que justifica la corrupción de tramos.
- La comparación de **arquitecturas** (encoder-decoder frente a solo decoder frente a solo encoder) a igualdad de parámetros y de cómputo.
- El apartado de **C4**: cómo se filtró Common Crawl y qué se descartó. Ese filtrado es una decisión editorial con consecuencias.
- La sección de **limitaciones y trabajo futuro**, inusualmente franca sobre lo que no midieron.

8. Evidencia y resultados

Resultados del estado del arte de la época en múltiples benchmarks (GLUE, SuperGLUE, SQuAD, resumen y traducción), obtenidos con el mismo modelo y el mismo procedimiento.

Las cifras por benchmark y por tamaño de modelo están en las tablas del artículo, junto con los resultados de cada ablación. Verificarlas allí: la mayoría del valor del paper está en esas tablas comparativas, no en la cifra final.

La miniatura de este eje muestra el marco: cinco tareas que exigían cinco tipos de cabeza se reducen a un único formato, con cero cabezas específicas y un solo objetivo.

9. Impacto

- Estableció **texto** → **texto** como interfaz por defecto, algo que hoy se da por evidente cada vez que se le pide algo a un modelo con una instrucción en lenguaje natural.
- **C4** se convirtió en corpus de referencia y en objeto de estudio por derecho propio.
- Su metodología —barrido sistemático con todo lo demás fijo— es el estándar de cómo se debe comparar en este campo, y sigue siendo raro verlo.
- La familia encoder-decoder que consolida sigue siendo la elección natural para traducir y resumir.

10. Limitaciones

1. **Precisión numérica:** emitir números como texto obliga a discretizar y pierde resolución.
2. **Salidas fuera del conjunto válido:** nada impide generar una etiqueta que no existe.
3. **Coste del estudio:** el barrido sistemático solo está al alcance de quien tiene mucho cómputo.
4. **C4 hereda los sesgos de Common Crawl**, y su filtrado introduce otros propios.
5. **Los prefijos son arbitrarios** y el rendimiento depende algo de cómo se redacten.
6. **No cubre** el aprendizaje en contexto de GPT-3, publicado meses después: T5 sigue asumiendo ajuste fino por tarea.

11. Errores comunes

Error	Corrección
«T5 es un modelo»	Es un marco más un estudio más un corpus. El modelo es el vehículo.
«Texto a texto es solo un formato»	Es lo que hace comparables las tareas, y por tanto lo que permite el estudio. Formato y método están acoplados.
«Usa el enmascarado de BERT»	Usa corrupción de tramos , y el paper justifica la diferencia con una ablación.
«Encoder-decoder es peor que solo decoder»	El paper mide ambos a igualdad de cómputo. La conclusión es matizada y depende de la tarea.
«La regresión como texto es un truco feo»	Es una decisión con coste explícito, medida y documentada en el propio artículo.

12. Relación con trabajos anteriores

- **P08 Transformer (2017)** — la arquitectura encoder-decoder.
- **P09 BERT (2018)** — el paradigma preentrenar-y-ajustar que sistematiza.
- **P24 ELMo (2018)** — la etapa anterior de transferencia en PLN.
- **Lewis et al. (2019), BART** — el otro encoder-decoder preentrenado, contemporáneo.
arXiv:1910.13461

13. Relación con trabajos posteriores

- **P10 GPT-3 (2020)** — cuestiona el ajuste fino por tarea que T5 asume.
- **P11 RAG (2020)** — usa un generador seq2seq de esta familia.
- **Modelos ajustados a instrucciones (2021+)** — llevan el prefijo a su conclusión: la tarea se describe en lenguaje natural.
- **Estudios sobre C4** — auditorías del corpus y de su filtrado.

14. Notebook asociado

P25_t5.ipynb

Qué implementa: cinco tareas reescritas al formato texto → texto, el recuento de cabezas específicas antes y después, y el coste de precisión al emitir números como texto.

Qué NO implementa: ningún modelo. Aquí se ve el **contrato de entrada y salida**, que es la idea; el estudio sistemático y C4 quedan por completo fuera de alcance local.

```
ai-evolution paper-lab P25 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe el objetivo único y di qué distingue una tarea de otra.
Explicar	Explica por qué unificar el formato es condición para poder comparar decisiones de diseño.
Aplicar	Reescribe una tarea propia al formato texto → texto con su prefijo.
Analizar	Calcula cuánta precisión se pierde al discretizar una escala continua en pasos de 0,2.
Evaluar	¿Qué tarea NO conviene reescribir como texto → texto? Justifica con el coste concreto.
Crear	Diseña una ablación propia sobre el objetivo de preentrenamiento y di qué mediría.

16. Autoevaluación

1. ¿Cuál es el objetivo de entrenamiento, exactamente, y para cuántas tareas vale?
2. ¿Qué distingue una tarea de otra en este marco?
3. ¿Por qué el marco es condición para el estudio, y no solo una comodidad?

4. ¿Qué objetivo de preentrenamiento elige el paper y en qué se diferencia del de BERT?
5. ¿Qué se pierde al emitir números como texto y cómo lo mitiga el paper?
6. ¿Qué es C4 y por qué su filtrado es una decisión editorial?
7. ¿Qué supuesto de T5 cuestiona GPT-3 pocos meses después?

17. Respuestas esperadas

1. Maximizar la verosimilitud del texto de salida dado el de entrada. Vale para todas: no hay objetivo específico de tarea.
2. Únicamente el prefijo del texto de entrada.
3. Porque si cada tarea tiene su propia cabeza, formato y métrica, cambiar el objetivo de preentrenamiento cambia varias cosas a la vez y no se puede atribuir la diferencia. Con formato único, se varía una sola cosa.
4. Corrupción de tramos: se enmascaran secuencias contiguas y se emiten con centinelas, frente al enmascarado token a token de BERT. El paper lo justifica con una ablación.
5. Resolución: 4.25 se convierte en 4.2. Se mitiga discretizando la escala en incrementos fijos, de modo que el modelo elige entre un conjunto pequeño de valores.
6. Un corpus derivado de Common Crawl con un filtrado explícito. Es editorial porque decidir qué se descarta —idioma, longitud, listas de palabras— determina qué aprende el modelo.
7. Que haga falta ajuste fino por tarea. GPT-3 muestra que se puede especificar la tarea en el prompt sin actualizar pesos.

18. Fuentes primarias

- Raffel, C. et al. (2020). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer*. **JMLR** 21(140). [arXiv:1910.10683](#) · JMLR · consultado 2026-08-16.
- Lewis, M. et al. (2019). *BART: Denoising Sequence-to-Sequence Pre-training*. [arXiv:1910.13461](#) · consultado 2026-08-16.

Evaluación — P25

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer* (2019, arXiv:1910.10683 · JMLR 21(140), 2020) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P25_t5.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P26 — DQN

Ruta de agentes · El primer agente que aprende a actuar directamente desde píxeles, con la misma arquitectura y los mismos hiperparámetros en decenas de juegos.

Nivel: L3 · Motor: `dqn` · Notebook: `P26_dqn.ipynb` · Anexo: cálculo y gradientes

1. Identificación

Campo	Valor
Título original	<i>Human-level control through deep reinforcement learning</i>
Autoría	Volodymyr Mnih, Koray Kavukcuoglu, David Silver y otros (DeepMind)
Año	2015
Venue	<i>Nature</i> 518, 529–533
Fuente primaria	doi.org/10.1038/nature14236
Acceso	Restringido (revista de suscripción)
Fecha de consulta	2026-08-16

2. Problema anterior

El aprendizaje por refuerzo tenía una teoría sólida —Q-learning es de 1989— pero funcionaba con tablas: un valor por cada par estado-acción. En cuanto el estado es una imagen, la tabla es imposible y hay que **aproximar** la función Q.

Y ahí estaba el problema conocido: combinar refuerzo con aproximación de función no lineal era notoriamente inestable, por dos motivos que se refuerzan. Las muestras consecutivas están muy **correlacionadas** (los fotogramas seguidos se parecen), y el **objetivo se mueve** mientras se aprende, porque se calcula con la misma red que se está actualizando.

3. Propuesta

Q-learning con una red convolucional que lee píxeles, estabilizado con dos mecanismos:

- Repetición de experiencia:** guardar las transiciones (s, a, r, s') en un buffer y entrenar con lotes muestreados al azar. Rompe la correlación temporal y permite reutilizar cada experiencia muchas veces.
- Red objetivo:** una copia congelada de la red que se usa para calcular el objetivo y solo se sincroniza cada cierto número de pasos. El blanco deja de moverse mientras se dispara.

Y una afirmación de generalidad: **la misma arquitectura y los mismos hiperparámetros** en decenas de juegos de Atari, sin ajuste específico por juego.

4. Intuición sin fórmulas

Aprender a jugar sin que nadie explique las reglas: pruebas, ves el marcador, ajustas. El problema es que si aprendes solo del último movimiento, te obsesionas con él. El buffer es una libreta de experiencias pasadas que relees en desorden.

Dónde deja de funcionar la analogía: una persona transfiere entre juegos parecidos. DQN aprende cada juego desde cero.

5. Matemática mínima

Actualización de Q-learning:

$$Q(s,a) \leftarrow Q(s,a) + \alpha \cdot \left[r + \gamma \cdot \max_{a'} Q(s',a') - Q(s,a) \right]$$

└──────── objetivo ─────────┘

Con red objetivo congelada θ^- :

$$L(\theta) = E_{\{(s,a,r,s') \sim \text{buffer}\}} \left[\left(r + \gamma \cdot \max_{a'} Q(s',a'; \theta^-) - Q(s,a; \theta) \right)^2 \right]$$

↑ NO se actualiza cada paso

$\gamma \in [0,1)$ descuenta el futuro. α es la tasa de aprendizaje. El **max** sobre a' es lo que hace a Q-learning **fuera de política**: aprende el valor de actuar óptimamente aunque esté explorando.

6. Arquitectura o flujo

7. Qué observar en el paper original

- La **figura de la arquitectura**: convoluciones sobre fotogramas apilados, con una salida por acción posible. Una sola pasada da el valor de todas las acciones.
- La **tabla de resultados por juego** y, sobre todo, en cuáles **falla** —los que requieren planificación a largo plazo, como Montezuma's Revenge—. Esa columna es la más informativa.
- La **visualización t-SNE** de las representaciones aprendidas: estados perceptualmente distintos con valor similar acaban juntos.
- Las **ablaciones** de repetición de experiencia y red objetivo, que es donde se demuestra que cada mecanismo aporta.

8. Evidencia y resultados

Evaluación en 49 juegos de Atari 2600 con la misma red y los mismos hiperparámetros, comparando con un jugador humano profesional y con los mejores métodos de refuerzo previos.

Las puntuaciones por juego, la normalización respecto al desempeño humano y las ablaciones están en el artículo y su material suplementario. Verificarlos allí antes de citar cualquier cifra: el resumen frecuente «supera al humano» esconde que en varios juegos queda muy por debajo.

La miniatura de este eje aísla las dos estabilizaciones con Q tabular en una rejilla: con ambas, la política converge cerca del óptimo; sin ellas, empeora y varía más entre semillas.

9. Impacto

- Convirtió el refuerzo profundo en un área central, tras años de escepticismo sobre la combinación de refuerzo y redes.
- La **repetición de experiencia** y la **red objetivo** pasaron a ser piezas estándar.
- Es el antecedente directo de AlphaGo y, más adelante, del uso de refuerzo para alinear modelos de lenguaje (P12) y para incentivar razonamiento (P22).
- Fijó Atari como banco de pruebas de la comunidad durante casi una década.

10. Limitaciones

1. **Ineficiencia extrema en muestras:** necesita decenas de millones de fotogramas por juego, órdenes de magnitud más que una persona.
2. **Falla donde hace falta exploración dirigida:** recompensas muy dispersas o planificación larga.
3. **Un modelo por juego:** no transfiere entre juegos.
4. **Solo acciones discretas:** el control continuo requiere otros métodos.
5. **Sobreestimación del valor** por el `max`, que trabajos posteriores corrigen (Double DQN).
6. **Sensible a hiperparámetros** pese a la afirmación de uniformidad, y con alta varianza entre semillas — algo que la comunidad tardó años en reportar de forma sistemática.

11. Errores comunes

Error	Corrección
«DQN inventó Q-learning»	Q-learning es de Watkins (1989). La contribución es estabilizarlo con aproximación no lineal.
«Supera a los humanos en Atari»	Los supera en muchos juegos y queda muy por debajo en otros. El agregado esconde la distribución.
«Aprende como una persona»	Necesita millones de partidas para lo que una persona aprende en minutos.
«La red objetivo es un detalle de implementación»	Sin ella el objetivo se mueve con cada actualización y el entrenamiento diverge.
«Es aprendizaje no supervisado»	Es aprendizaje por refuerzo : hay una señal de recompensa, aunque no haya etiquetas.

12. Relación con trabajos anteriores

- **Watkins (1989)** — Q-learning tabular.
- **P02 Backpropagation** y **P04 AlexNet** — la red que aproxima Q y su capacidad de leer píxeles.
- **Sutton y Barto** — el marco teórico del refuerzo. [libro abierto](#)

13. Relación con trabajos posteriores

- **P27 AlphaGo (2016)** — añade búsqueda a la ecuación.
- **Double DQN, Dueling DQN, Rainbow (2015–2017)** — corrigen la sobreestimación y combinan mejoras.
- **P12 InstructGPT (2022)** y **P22 DeepSeek-R1 (2025)** — el refuerzo aplicado a modelos de lenguaje.

14. Notebook asociado

P26_dqn.ipynb

Qué implementa: Q tabular en una rejilla 4×4, con y sin repetición de experiencia y red objetivo, más una demostración numérica de por qué un objetivo móvil desestabiliza.

Qué NO implementa: ninguna red neuronal, ningún píxel, ningún juego. Con 16 estados no hace falta aproximar nada: se ve el efecto de las estabilizaciones, no el problema que las motiva.

```
ai-evolution paper-lab P26 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la actualización de Q-learning y señala cuál es el objetivo.
Explicar	Explica por qué las muestras consecutivas correlacionadas perjudican el aprendizaje.
Aplicar	Ejecuta el notebook con tres semillas y compara ambas configuraciones.
Analizar	¿Por qué el <code>max</code> sobre <code>a</code> produce sobreestimación del valor?
Evaluar	«DQN supera a los humanos en Atari». Reescribe la afirmación de forma defendible.
Crear	Diseña un entorno donde la repetición de experiencia sea contraproducente y explica por qué.

16. Autoevaluación

1. ¿Qué dos problemas hacen inestable el refuerzo con aproximación no lineal?
2. ¿Cómo ataca cada uno de los dos mecanismos propuestos?
3. ¿Qué significa que Q-learning sea «fuera de política»?
4. ¿Qué papel juega `γ` y qué pasa si vale 0?
5. ¿En qué tipo de juegos falla DQN y por qué?

6. ¿Qué afirmación de generalidad hace el paper, y por qué es más fuerte que un buen resultado?
7. ¿Qué relación tiene esto con alinear un modelo de lenguaje?

17. Respuestas esperadas

1. La correlación entre muestras consecutivas y el hecho de que el objetivo se calcule con la misma red que se está actualizando, de modo que el blanco se mueve.
2. La repetición de experiencia muestrea al azar de un buffer, rompiendo la correlación. La red objetivo congela la red que calcula el objetivo durante N pasos, fijando el blanco.
3. Que aprende el valor de la política **óptima** (por el `max`) aunque los datos se hayan recogido con una política exploratoria distinta.
4. Descuenta el futuro. Con $\gamma = 0$ el agente solo optimiza la recompensa inmediata y no planifica.
5. En los de recompensa muy dispersa o que exigen planificación larga: explorar al azar casi nunca encuentra la primera recompensa.
6. Que la misma arquitectura y los mismos hiperparámetros valen para decenas de juegos. Es más fuerte porque descarta el ajuste específico como explicación del resultado.
7. Es el mismo marco: hay una señal de recompensa y una política que se optimiza. Cambia el entorno (texto en vez de juego) y el origen de la recompensa (humana o verificable).

18. Fuentes primarias

- Mnih, V. et al. (2015). *Human-level control through deep reinforcement learning*. **Nature** 518, 529–533. doi.org/10.1038/nature14236 · consultado 2026-08-16.
- Sutton, R. S. y Barto, A. G. *Reinforcement Learning: An Introduction*. versión abierta · consultado 2026-08-16.

Evaluación — P26

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Human-level control through deep reinforcement learning* (2015, Nature 518, 529–533 (2015)) ·

Nivel: L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P26_dqn.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P27 — AlphaGo

Ruta de agentes · Une las dos tradiciones que el programa enseña por separado: la búsqueda simbólica de la parte 01 y el aprendizaje profundo de la parte 04.

Nivel: L4 · **Motor:** alphago · **Notebook:** P27_alphago.ipynb · **Anexo:** complejidad y coste

1. Identificación

Campo	Valor
Título original	Mastering the game of Go with deep neural networks and tree search
Autoría	David Silver, Aja Huang, Chris J. Maddison y otros (DeepMind)
Año	2016
Venue	Nature 529, 484–489
Fuente primaria	doi.org/10.1038/nature16961
Acceso	Restringido (revista de suscripción)
Fecha de consulta	2026-08-16

2. Problema anterior

El ajedrez cayó en 1997 con búsqueda y una función de evaluación escrita por expertos. El go resistió veinte años más por dos razones concretas: su **factor de ramificación** es un orden de magnitud mayor, y nadie sabía escribir una función que dijera si una posición es buena.

La búsqueda de Monte Carlo en árbol había mejorado el nivel, pero seguía lejos del profesional. El cuello de botella era doble: demasiadas jugadas que considerar (anchura) y partidas demasiado largas para simular hasta el final (profundidad).

3. Propuesta

Atacar cada dimensión con una red:

- **Red de políticas** $p(a|s)$: propone las jugadas plausibles y **reduce la anchura** del árbol. Se entrena primero imitando partidas humanas y después se refina por autojuego.
- **Red de valor** $v(s)$: estima quién va ganando sin llegar al final y **reduce la profundidad**, sustituyendo simulaciones completas.
- **MCTS**: usa ambas para repartir un presupuesto de simulaciones y decidir la jugada.

Ninguna de las tres piezas basta sola, y el título del paper nombra las dos familias: redes profundas y búsqueda en árbol.

4. Intuición sin fórmulas

Un buen jugador no calcula todas las jugadas: su intuición descarta casi todo y solo analiza a fondo tres o cuatro. AlphaGo hace exactamente eso — la red da la intuición, la búsqueda hace el análisis.

Dónde deja de funcionar la analogía: la intuición humana viene de entender el juego; la de la red viene de correlaciones sobre millones de posiciones. Que el resultado se parezca no implica que el proceso lo haga.

5. Matemática mínima

Selección en el árbol (variante de UCT):

$$a^* = \operatorname{argmax}_a [Q(s,a) + u(s,a)] \quad \text{con} \quad u(s,a) \propto P(s,a) / (1 + N(s,a))$$

$Q(s,a)$ = valor medio observado en las simulaciones que pasaron por (s,a)

$P(s,a)$ = prior de la red de políticas $\leftarrow$ concentra el presupuesto

$N(s,a)$ = visitas $\leftarrow$ penaliza lo ya explorado

Evaluación de una hoja: mezcla de la red de valor y de un despliegue rápido

$$V(s) = (1-\lambda) \cdot v_{\theta}(s) + \lambda \cdot z_{\text{despliegue}}$$

El término u decae con las visitas: al principio manda el prior, y conforme se acumula evidencia manda Q . Es el compromiso explorar/exploitar de DQN, ahora dentro del árbol.

6. Arquitectura o flujo

7. Qué observar en el paper original

- La **figura del pipeline de entrenamiento**: red de políticas supervisada $\rightarrow$ refinada por refuerzo $\rightarrow$ red de valor entrenada con partidas de autojuego. Es una cadena, no un modelo suelto.
- Las **ablaciones**: qué pasa con solo política, solo valor, solo despliegues, y las combinaciones. Ahí está la demostración de que las tres piezas se necesitan.
- El **escalado con el número de simulaciones** y con el hardware: el nivel de juego es función del presupuesto de búsqueda, no solo de la red.
- La distinción entre la red de políticas **supervisada** y la refinada por **refuerzo**, y por qué usan una u otra en distintos puntos.

8. Evidencia y resultados

Victoria por 5-0 frente al campeón europeo Fan Hui, y una tasa de victoria muy alta frente a los mejores programas de go de la época.

Las tasas de victoria por configuración, las ablaciones y los detalles del hardware están en el artículo y su material suplementario. Verificarlos allí. El match posterior contra Lee Sedol (2016) es un hecho ampliamente documentado pero **no** forma parte de este artículo.

La miniatura de este eje aísla el mecanismo en tres en raya: el prior propone por preferencia fija, la búsqueda estima un valor por casilla. Ambas aciertan el tipo de jugada, pero solo la segunda produce números comparables y auditables, y solo la segunda mejora con más presupuesto.

9. Impacto

- Es el resultado que llevó la IA moderna a la conversación pública, más que cualquier paper técnico anterior.
- Demostró que **búsqueda y aprendizaje se potencian**: una lección que reaparece en Tree of Thoughts y en el cómputo en inferencia de P22.
- Su sucesor AlphaGo Zero (2017) eliminó las partidas humanas por completo, aprendiendo solo por autojuego.
- Consolidó el **autojuego** como forma de generar datos donde no los hay.

10. Limitaciones

1. **Requiere un simulador perfecto**: reglas conocidas, estado completamente observable y posibilidad de simular millones de partidas. Casi ningún problema real cumple eso.
2. **Coste computacional enorme**, tanto de entrenamiento como de juego.
3. **Dominio único**: no transfiere a otro juego sin rehacer el proceso.
4. **Depende de partidas humanas** en esta versión (AlphaGo Zero lo corregirá).
5. **Información perfecta**: no cubre juegos con azar u ocultación.
6. **No explica sus decisiones**: la jugada sale de un recuento de simulaciones, no de un argumento.

11. Errores comunes

Error	Corrección
«Una red neuronal venció al campeón»	La red sola no vence a nadie. El título nombra las dos piezas: redes y búsqueda.
«AlphaGo aprendió solo»	Esta versión parte de partidas humanas. La que aprende sola es AlphaGo Zero (2017).
«Demuestra que la IA razona»	Demuestra que búsqueda guiada por redes gana al go. Cualquier extrapolación es interpretación.
«MCTS es una novedad del paper»	MCTS es anterior. La novedad es guiarla y truncarla con redes aprendidas.
«Sirve para cualquier problema»	Necesita simulador perfecto e información completa. Es una restricción muy fuerte.

12. Relación con trabajos anteriores

- **P26 DQN (2015)** — refuerzo profundo del mismo grupo.
- **P04 AlexNet (2012)** — las convoluciones que leen el tablero.
- **MCTS y UCT (2006–2007)** — la búsqueda que se guía.
- **Deep Blue (1997)** — el precedente en ajedrez, con evaluación escrita a mano.

13. Relación con trabajos posteriores

- **AlphaGo Zero (2017)** y **AlphaZero (2018)** — sin partidas humanas, y generalizado a otros juegos.
DOI
- **P29 Tree of Thoughts (2023)** — la misma idea de buscar con evaluación, aplicada a pasos de razonamiento.
- **P22 DeepSeek-R1 (2025)** — gastar cómputo al decidir, no solo al entrenar.

14. Notebook asociado

P27_alphago.ipynb

Qué implementa: una posición de tres en raya resuelta con prior heurístico solo, y con búsqueda guiada por ese prior mediante despliegues aleatorios; más el reparto del presupuesto de simulaciones.

Qué NO implementa: nada del paper. No hay redes entrenadas, ni MCTS con UCT, ni autojuego. Tres en raya tiene 9 casillas; el go tiene más estados legales que átomos observables.

```
ai-evolution paper-lab P27 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Di qué reduce la red de políticas y qué reduce la red de valor.
Explicar	Explica el término $u(s,a)$ y por qué decae con las visitas.
Aplicar	Ejecuta el notebook y compara la jugada del prior con la de la búsqueda.
Analizar	¿Por qué el go resistió veinte años más que el ajedrez? Da las dos razones técnicas.
Evaluar	¿Qué problema real de tu entorno cumple los requisitos de este método? ¿Y cuál no?
Crear	Diseña una función de evaluación para conecta-4 y compárala con despliegues aleatorios.

16. Autoevaluación

1. ¿Qué dos dimensiones del árbol de búsqueda ataca cada red?
2. ¿Por qué no bastaba MCTS con despliegues aleatorios?
3. ¿Qué hace el prior cuando $N(s,a)$ es 0 y qué cuando es grande?
4. ¿Qué requisito del entorno hace inaplicable este método a la mayoría de problemas reales?

5. ¿Qué diferencia hay entre esta versión y AlphaGo Zero?
6. ¿Por qué el nivel de juego depende del presupuesto de simulaciones?
7. ¿Qué idea de este paper reaparece en el razonamiento de modelos de lenguaje?

17. Respuestas esperadas

1. La red de políticas reduce la **anchura** proponiendo pocas jugadas plausibles; la de valor reduce la **profundidad** evaluando sin llegar al final de la partida.
2. Porque el factor de ramificación del go hace que los despliegues aleatorios sean demasiado poco informativos: se desperdicia el presupuesto en jugadas absurdas.
3. Con $N = 0$ domina el prior, que es la única información disponible. Con N grande el término u decae y manda Q , la evidencia acumulada por las simulaciones.
4. Necesita un **simulador perfecto** con reglas conocidas e información completa, y la posibilidad de simular millones de partidas.
5. AlphaGo Zero prescinde de partidas humanas: aprende solo por autojuego desde cero.
6. Porque la decisión sale de un recuento de simulaciones: más simulaciones, mejor estimación de Q y por tanto mejor elección.
7. Que gastar cómputo **al decidir** —buscar, deliberar, verificar— puede rendir más que un modelo más grande que responde de una vez.

18. Fuentes primarias

- Silver, D. et al. (2016). *Mastering the game of Go with deep neural networks and tree search*. **Nature** 529, 484–489. doi.org/10.1038/nature16961 · consultado 2026-08-16.
- Silver, D. et al. (2017). *Mastering the game of Go without human knowledge*. **Nature** 550. doi.org/10.1038/nature24270 · consultado 2026-08-16.

Evaluación — P27

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Mastering the game of Go with deep neural networks and tree search* (2016, Nature 529, 484–489 (2016)) · **Nivel:** L4

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P27_alphago.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P28 — Chain-of-Thought

Ruta de agentes · Descomponer en pasos intermedios desbloquea tareas que el mismo modelo fallaba respondiendo de una sola vez.

Nivel: L2 · **Motor:** cot · **Notebook:** P28_chain_of_thought.ipynb · **Anexo:** probabilidad y verosimilitud

1. Identificación

Campo	Valor
Título original	Chain-of-Thought Prompting Elicits Reasoning in Large Language Models
Autoría	Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, Denny Zhou
Año	2022
Venue	arXiv:2201.11903 · NeurIPS 2022
Fuente primaria	arXiv:2201.11903
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

GPT-3 resolvía tareas sorprendentes con pocos ejemplos y fallaba en otras aparentemente más fáciles: aritmética de varios pasos, problemas de lógica, razonamiento de sentido común encadenado. Escalar el modelo mejoraba muchas cosas pero **no** estas: las curvas se quedaban planas.

La interpretación dominante era que esas capacidades simplemente no estaban. La hipótesis alternativa —que resultó ser la correcta— es que sí estaban, pero se le pedía el resultado sin dejarle espacio para llegar a él.

3. Propuesta

Cambiar los ejemplos del prompt: en vez de pares (pregunta, respuesta), usar tríos (pregunta, razonamiento paso a paso, respuesta).

Nada más. Sin ajuste fino, sin datos nuevos, sin cambiar el modelo. Ocho ejemplos escritos a mano bastan, y el efecto aparece solo en modelos suficientemente grandes: en los pequeños, las cadenas que generan son incoherentes y empeoran el resultado.

4. Intuición sin fórmulas

Pedir una multiplicación de tres cifras «de cabeza» falla; pedirla por pasos, no. No es que sepas más: es que te han dado sitio donde hacer la cuenta.

Dónde deja de funcionar la analogía: una persona sabe si su cuenta intermedia está bien. El modelo genera pasos que **parecen** cálculo y a veces no lo son, y aun así puede acertar.

5. Matemática mínima

No hay ecuación nueva. Lo que hay es aritmética de probabilidades, y explica el fenómeno entero:

$P(\text{acertar directo}) \approx \text{dificultad del problema completo}$
 $P(\text{acertar cadena}) \approx (\text{fiabilidad por paso})^n$

Descomponer gana **si y solo si** cada paso es mucho más fiable que el problema entero. De ahí salen las tres consecuencias observables:

- si la fiabilidad por paso es baja, descomponer **empeora** (se multiplican errores);
- por encima de un umbral, mejora;
- como la fiabilidad por paso crece con la escala del modelo, el efecto **emerge**: un producto de números cruza un umbral.

6. Arquitectura o flujo

7. Qué observar en el paper original

- Las **curvas de escala**: el efecto no existe en modelos pequeños y aparece de golpe. Esa forma es el argumento de la «emergencia».
- Los **ejemplos concretos** de cadenas, incluidos los erróneos. Los apéndices con fallos son lo más instructivo.
- El **análisis de ablación**: qué pasa si las cadenas son correctas pero irrelevantes, o si solo se añade la ecuación sin el razonamiento. Sirve para descartar explicaciones alternativas.
- Que se evalúa en **tres familias** de razonamiento —aritmético, de sentido común y simbólico—, no solo en matemáticas.

8. Evidencia y resultados

Mejoras sustanciales en benchmarks de razonamiento aritmético, de sentido común y simbólico, con modelos de varios tamaños para trazar las curvas de escala.

El resultado más citado: un modelo de 540 000 millones de parámetros con **ocho ejemplos** en el prompt alcanza el estado del arte en un benchmark de problemas matemáticos, superando a modelos ajustados específicamente para la tarea.

Las cifras por benchmark y por tamaño están en las tablas del artículo. Verificarlas allí.

La miniatura de este eje modela la aritmética del fenómeno: localiza el umbral de fiabilidad por paso por debajo del cual descomponer empeora, y muestra que la cadena se degrada con la longitud aun cuando siga ganando.

9. Impacto

- Convirtió el diseño del prompt en una variable de primer orden: la misma pregunta con otro formato da otro resultado.
- Es el punto de partida de casi todo el trabajo posterior sobre razonamiento: **autoconsistencia**, ReAct, Tree of Thoughts y el cómputo en inferencia de P22.
- Popularizó —para bien y para mal— la palabra «emergencia» en el vocabulario del campo.

10. Limitaciones

1. **Solo funciona a escala suficiente:** en modelos pequeños empeora.
2. **La cadena puede ser inválida y la respuesta correcta**, y al revés. No es una demostración.
3. **Cuesta tokens:** más latencia y más dinero por respuesta.
4. **Los ejemplos hay que escribirlos** y la calidad del resultado depende de ellos.
5. **Se degrada con la longitud:** cadenas muy largas multiplican la probabilidad de fallo.
6. **La palabra «emergencia» es discutida:** trabajo posterior señaló que parte del efecto puede ser un artefacto de métricas discontinuas (acierta/falla) sobre una mejora continua.

11. Errores comunes

Error	Corrección
«La cadena explica cómo razonó el modelo»	Es texto generado, optimizado para que la respuesta final sea correcta. Puede racionalizar en vez de describir.
«Siempre conviene pedir razonamiento»	En tareas de un paso añade coste sin mejorar, y en modelos pequeños empeora.
«Es una capacidad emergente inexplicable»	Se explica con aritmética elemental: un producto de fiabilidades cruzando un umbral.
«CoT enseña a razonar al modelo»	No cambia ningún peso. Cambia el formato del contexto.
«Si la cadena es correcta, la respuesta lo es»	No se sigue. El propio paper documenta ambos tipos de disociación.

12. Relación con trabajos anteriores

- **P10 GPT-3 (2020)** — el aprendizaje en contexto que hace posible el método.
- **P25 T5 (2019)** — la idea de describir la tarea en el propio texto.
- Trabajo previo sobre **explicaciones intermedias** en resolución de problemas matemáticos.

13. Relación con trabajos posteriores

- **Wang et al. (2022), autoconsistencia** — muestrear varias cadenas y votar la respuesta.
arXiv:2203.11171
- **P13 ReAct (2022)** — intercalar acciones reales entre pensamientos.
- **P29 Tree of Thoughts (2023)** — explorar varias ramas.
- **P22 DeepSeek-R1 (2025)** — hacer que el razonamiento emerja por refuerzo en vez de por ejemplos.

14. Notebook asociado

P28_chain_of_thought.ipynb

Qué implementa: el modelo probabilístico de por qué descomponer gana o pierde, el umbral de fiabilidad por paso, la degradación con la longitud y la emergencia con la escala.

Qué NO implementa: ningún modelo de lenguaje ni ninguna cadena generada. Las fiabilidades son didácticas: se modela el argumento, no se reproduce el experimento.

```
ai-evolution paper-lab P28 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe las dos probabilidades y di cuándo gana la cadena.
Explicar	Explica por qué el efecto no aparece en modelos pequeños.
Aplicar	Ejecuta el notebook y localiza el umbral de fiabilidad por paso.
Analizar	Calcula cuántos pasos aguanta una cadena con fiabilidad 0,95 antes de bajar del 50 %.
Evaluar	¿Es «capacidad emergente» una buena descripción? Argumenta a favor y en contra.
Crear	Diseña una tarea donde CoT empeore el resultado y explica por qué.

16. Autoevaluación

1. ¿Qué cambia exactamente en el modelo al usar CoT?
2. ¿De qué depende que descomponer compense?
3. ¿Por qué el efecto emerge con la escala?
4. ¿Puede una cadena inválida llevar a la respuesta correcta? ¿Y al revés?
5. ¿Qué cuesta CoT en producción?

6. ¿Por qué la palabra «emergencia» es discutida?
7. ¿Qué límite de CoT ataca directamente Tree of Thoughts?

17. Respuestas esperadas

1. Nada en el modelo: solo el formato de los ejemplos del prompt. Cero pesos actualizados.
2. De que la fiabilidad de cada paso sea suficientemente alta comparada con la dificultad del problema completo. Es una comparación entre q^n y la dificultad global.
3. Porque la fiabilidad por paso crece con el tamaño del modelo, y el producto q^n cruza el umbral de golpe al superarse cierta calidad.
4. Sí en ambos casos, y el paper lo documenta. Por eso la cadena no es una demostración.
5. Tokens: más latencia y más coste por respuesta, en cada llamada.
6. Porque parte del salto aparente puede deberse a medir con una métrica de todo o nada sobre una mejora subyacente continua.
7. Que la cadena es lineal e irreversible: un paso malo condena el resto sin posibilidad de retroceder.

18. Fuentes primarias

- Wei, J. et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models*. **NeurIPS 2022**. arXiv:2201.11903 · consultado 2026-08-16.
- Wang, X. et al. (2022). *Self-Consistency Improves Chain of Thought Reasoning*. arXiv:2203.11171 · consultado 2026-08-16.

Evaluación — P28

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models* (2022, arXiv:2201.11903 · NeurIPS 2022) · **Nivel:** L2

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P28_chain_of_thought.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P29 — Tree of Thoughts

Ruta de agentes · Devuelve la búsqueda clásica al razonamiento: explorar varias ramas, evaluarlas y poder retroceder.

Nivel: L3 · Motor: tot · Notebook: P29_tree_of_thoughts.ipynb · Anexo: complejidad y coste

1. Identificación

Campo	Valor
Título original	Tree of Thoughts: Deliberate Problem Solving with Large Language Models
Autoría	Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, Karthik Narasimhan
Año	2023
Venue	arXiv:2305.10601 · NeurIPS 2023
Fuente primaria	arXiv:2305.10601
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Chain-of-Thought razona de izquierda a derecha y **sin vuelta atrás**. Cada token se genera condicionado a los anteriores, y una decisión intermedia localmente razonable pero globalmente equivocada condena toda la solución sin que nada la corrija.

Hay una clase de problemas donde eso es fatal: los que requieren **explorar** —probar una combinación, ver que no lleva a nada y volver—. La generación autorregresiva no tiene ese mecanismo.

3. Propuesta

Tratar los pasos de razonamiento como **nodos de un árbol** y aplicar búsqueda clásica sobre ellos. Hacen falta tres piezas:

1. **Generar** varios candidatos de «pensamiento» por estado, no uno.
2. **Evaluar** estados parciales: el propio modelo juzga si una rama promete o es un callejón sin salida.
3. **Buscar**: anchura, profundidad, poda y retroceso, con un presupuesto de nodos.

El resultado más citado del artículo: en el juego de las 24, la tasa de éxito pasa del 4 % con cadena de pensamiento al 74 % con árbol.

4. Intuición sin fórmulas

Una cadena de pensamiento es escribir a bolígrafo: si el tercer paso está mal, sigues adelante con él. Un árbol es escribir a lápiz con varias hojas: exploras, comparas y borras.

Dónde deja de funcionar la analogía: quien escribe a lápiz sabe cuándo una hoja no va a ningún sitio. Aquí ese juicio lo hace el propio modelo, y si lo hace mal, el árbol solo multiplica el gasto.

5. Matemática mínima

Cadena :	$s_0 \rightarrow s_1 \rightarrow s_2 \rightarrow s_3$	una rama, decisión irreversible
Árbol :	$s_0 \rightarrow \{s_1^a, s_1^b, s_1^c\} \rightarrow \dots$	varias ramas, con evaluación y poda

Coste, con ramificación b , profundidad d y anchura de haz k :

cadena :	$b \cdot d$	nodos evaluados	
árbol :	$b \cdot k \cdot d$	nodos evaluados	$\rightarrow k$ veces más caro

Con $k = 1$, el árbol **es** la cadena. Cada unidad de anchura multiplica el coste y compra la posibilidad de recuperarse de un mal paso. El compromiso es explícito y presupuestable.

6. Arquitectura o flujo

7. Qué observar en el paper original

- Las **tres tareas** elegidas: juego de las 24, escritura creativa y crucigramas. No son arbitrarias: en las tres el progreso parcial es evaluable, que es el requisito del método.
- Cómo se implementa el **evaluador**: pedir al modelo que clasifique un estado parcial como seguro/quizá/imposible, o que puntúe. Ese diseño es la pieza frágil.
- La comparación con **cadena + autoconsistencia**, que es la línea base honesta: muestrear varias cadenas y votar ya mejora, y hay que superarla.
- El **coste en llamadas al modelo**, que el paper reporta. Sin ese número, la comparación de exactitud no significa nada.

8. Evidencia y resultados

Evaluación en tres tareas que requieren exploración, comparando con prompting directo, cadena de pensamiento y autoconsistencia.

En el juego de las 24, la tasa de éxito reportada pasa del **4 %** con cadena de pensamiento al **74 %** con árbol de pensamientos.

Las cifras de las otras dos tareas, la configuración de búsqueda y el número de llamadas al modelo están en el artículo. Verificarlos allí: el salto del 4 % al 74 % es real pero se paga en llamadas, y citar solo el primero es medio dato.

La miniatura de este eje compara nodos evaluados por ambos métodos y comprueba el caso límite: con anchura 1, el árbol se comporta exactamente como la cadena.

9. Impacto

- Reintrodujo la **búsqueda** —el tema de la parte 01 del programa— en el trabajo con modelos de lenguaje, que hasta entonces lo había ignorado.
- Es uno de los antecedentes conceptuales del **cómputo en inferencia**: gastar más al responder en vez de entrenar más grande, la línea que consolida P22.
- Popularizó la idea de que el modelo puede actuar como **evaluador de sí mismo** sobre estados parciales, no solo como generador.

10. Limitaciones

1. **Coste multiplicado**: cada nodo evaluado es una llamada al modelo. Es el método más caro de su familia.
2. **Depende por completo del evaluador**. Con un evaluador mediocre, la poda es aleatoria y solo se gasta más.
3. **Requiere que el progreso parcial sea evaluable**: en muchas tareas no lo es.
4. **Latencia alta**, difícil de justificar en un producto interactivo.
5. **Hay que diseñar el espacio de «pensamientos»** por tarea: qué es un paso no es evidente.
6. **La línea base correcta es autoconsistencia**, no cadena simple; compararse solo con la segunda infla la mejora aparente.

11. Errores comunes

Error	Corrección
«ToT siempre mejora sobre CoT»	Solo en tareas que requieren exploración y con un evaluador competente. En tareas de un paso, es gasto puro.
«Del 4 % al 74 %, luego es 18× mejor»	Ese salto se paga en llamadas al modelo. Sin el coste, la comparación está incompleta.
«El árbol razona mejor»	Busca mejor. Razonar y buscar no son lo mismo, y la distinción importa.
«Basta con ampliar la anchura»	Sin mejorar el evaluador, más anchura es más coste con la misma calidad de poda.
«Es una idea nueva»	La búsqueda con evaluación es de los años 60. Lo nuevo es aplicarla a pasos de razonamiento generados.

12. Relación con trabajos anteriores

- **P28 Chain-of-Thought (2022)** — la cadena lineal que generaliza.
- **P13 ReAct (2022)** — del mismo primer autor; actuar en vez de deliberar.
- **P27 AlphaGo (2016)** — búsqueda guiada por evaluación aprendida.
- **Búsqueda en espacios de estados** (parte 01 del programa) — el marco clásico que se reutiliza.

13. Relación con trabajos posteriores

- **P22 DeepSeek-R1 (2025)** — el razonamiento largo como política aprendida, en vez de como búsqueda explícita orquestada desde fuera.
- **Snell et al. (2024)** — el marco de escalar cómputo en inferencia. [arXiv:2408.03314](#)
- **Verificadores y modelos de recompensa por proceso (2023+)** — atacan justo la pieza frágil: la calidad del evaluador.

14. Notebook asociado

P29_tree_of_thoughts.ipynb

Qué implementa: la comparación de nodos evaluados entre cadena lineal y búsqueda con poda, el efecto de la anchura, el caso límite $k = 1$ y el experimento de qué pasa con un evaluador al azar.

Qué NO implementa: ningún modelo. El evaluador es una función hash determinista, no un modelo juzgando estados parciales — que es exactamente la parte difícil.

```
ai-evolution paper-lab P29 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe las tres piezas que necesita el método.
Explicar	Explica por qué con anchura 1 el árbol es la cadena.
Aplicar	Ejecuta el notebook y compara nodos evaluados para varias anchuras.
Analizar	¿Qué pasa con un evaluador al 50 % de acierto? Razónalo y compruébalo.
Evaluar	Te presentan «74 % frente a 4 %». ¿Qué dato pides antes de aceptarlo?
Crear	Define el espacio de «pensamientos» para un problema de tu dominio y di si el progreso parcial es evaluable.

16. Autoevaluación

1. ¿Qué limitación de la cadena de pensamiento ataca?
2. ¿Cuáles son las tres piezas del método?
3. ¿Cuál es el caso límite con anchura 1?
4. ¿Por qué el evaluador es la pieza frágil?
5. ¿En qué tipo de tarea NO conviene?
6. ¿Cuál es la línea base honesta con la que compararse?
7. ¿Qué relación tiene con AlphaGo?

17. Respuestas esperadas

1. Que decide de izquierda a derecha sin vuelta atrás: un paso malo no se puede deshacer.
2. Generar varios candidatos por estado, evaluar estados parciales y aplicar una estrategia de búsqueda con poda.
3. Con anchura 1 solo se conserva una rama en cada nivel: es exactamente una cadena lineal.
4. Porque la poda depende de él. Con un evaluador aleatorio, se descartan buenas ramas y se conservan malas: el coste se multiplica y la calidad no mejora.
5. En tareas de un solo paso, en las que el progreso parcial no se puede juzgar, y donde la latencia o el coste por respuesta son críticos.
6. Cadena de pensamiento con **autoconsistencia** (muestrear varias y votar), no cadena simple.
7. Es la misma estructura: un generador propone, un evaluador juzga y una búsqueda reparte el presupuesto. Cambia el dominio —pasos de texto en vez de jugadas— y que aquí no hay simulador.

18. Fuentes primarias

- Yao, S. et al. (2023). *Tree of Thoughts: Deliberate Problem Solving with Large Language Models*. **NeurIPS 2023**. arXiv:2305.10601 · consultado 2026-08-16.
- Yao, S. et al. (2023). *ReAct: Synergizing Reasoning and Acting in Language Models*. arXiv:2210.03629 · consultado 2026-08-16.

Evaluación — P29

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Tree of Thoughts: Deliberate Problem Solving with Large Language Models* (2023, arXiv:2305.10601 · NeurIPS 2023) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P29_tree_of_thoughts.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P30 — Reflexion

Ruta de agentes · El agente aprende entre intentos sin tocar un solo peso: el refuerzo ocurre en el contexto, en lenguaje natural.

Nivel: L2 · Motor: `reflexion` · Notebook: `P30_reflexion.ipynb` · Anexo: cálculo y gradientes

1. Identificación

Campo	Valor
Título original	<i>Reflexion: Language Agents with Verbal Reinforcement Learning</i>
Autoría	Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik Narasimhan, Shunyu Yao
Año	2023
Venue	arXiv:2303.11366 · NeurIPS 2023
Fuente primaria	arXiv:2303.11366
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Un bucle ReAct que falla vuelve a empezar de cero. No conserva nada del intento anterior, así que **repite el mismo error** con la misma confianza.

La alternativa clásica sería ajustar los pesos con refuerzo, pero eso es caro, lento y exige acceso al modelo. Para un agente construido sobre una API, ni siquiera es posible.

3. Propuesta

Cerrar el bucle **en el contexto**. Tras cada intento fallido:

- una señal de evaluación indica que falló (test que no pasa, error del entorno, heurística);
- el modelo genera una **reflexión verbal** sobre qué salió mal;
- esa reflexión se guarda en una memoria episódica;
- el siguiente intento se condiciona con ella.

Es refuerzo en el sentido de que la política mejora con la experiencia, pero la «actualización» es texto añadido al contexto. Cero gradientes.

4. Intuición sin fórmulas

Un agente sin memoria de sus fallos es alguien que repite el mismo error con entusiasmo. Reflexion añade lo mínimo para romper el bucle: escribir qué salió mal y leerlo antes de reintentar.

Dónde deja de funcionar la analogía: una persona reflexiona aunque nadie le diga que falló. Aquí la reflexión necesita una señal externa; sin ella, degenera en autoafirmación.

5. Matemática mínima

No hay ecuación. Lo que hay es un bucle con estado:

```
memoria = []
para intento en 1..N:
    trayectoria = política(tarea, memoria)      ← el contexto incluye las reflexiones
    resultado = evaluar(trayectoria)           ← señal EXTERNA, verificable
    si resultado == éxito: terminar
    memoria.append( reflexionar(trayectoria, resultado) )
```

La comparación con el refuerzo clásico es directa: donde RL actualiza θ , aquí se actualiza el **contexto**. Es más barato e inmediato, y también más frágil y efímero.

6. Arquitectura o flujo

7. Qué observar en el paper original

- Los **tres tipos de señal de evaluación** que consideran, según la tarea: exacta (tests), heurística y generada por el propio modelo. La calidad del método baja con la calidad de la señal.
- Las **curvas de éxito acumulado por número de intentos**: es la forma correcta de reportar un método iterativo, no una sola cifra.
- Los ejemplos de **reflexiones inútiles** («ser más cuidadoso»), que es el modo de fallo característico.
- La comparación con simplemente **reintentar sin reflexión**, que es la línea base honesta: parte de la mejora viene solo de tener más intentos.

8. Evidencia y resultados

Evaluación en tareas de toma de decisiones, razonamiento y generación de código, comparando con ReAct y con reintentos sin reflexión.

Las tasas de éxito por tarea y por número de intentos están en el artículo. Verificarlas allí, y comparar siempre contra «reintentar N veces sin reflexión»: sin esa columna, la mejora atribuida a la reflexión está inflada.

La miniatura de este eje muestra el mecanismo: sin memoria, el agente repite el primer fallo y no termina en cuatro intentos; con memoria verbal, elimina un error por intento y converge. Con cero pesos actualizados.

9. Impacto

- Instaló la idea de **memoria entre intentos** como componente estándar de un agente.
- Es uno de los antecedentes directos del bucle de autocrítica que hoy llevan casi todos los sistemas de generación de código.
- Marcó la distinción entre aprendizaje **en parámetros** y aprendizaje **en contexto** como una decisión de diseño, no como una limitación.

10. Limitaciones

1. **Depende de una señal de fallo fiable.** Sin verificador no hay sobre qué reflexionar.
2. **La reflexión puede ser incorrecta o genérica,** y entonces empeora el siguiente intento.
3. **La memoria ocupa contexto** y crece con los intentos: no escala a cientos.
4. **Efímero:** lo aprendido se pierde al terminar la sesión, salvo que se persista aparte.
5. **Coste multiplicado** por el número de intentos.
6. **Riesgo de bucle:** si la reflexión se repite, el agente puede quedar atrapado.

11. Errores comunes

Error	Corrección
«El modelo aprende»	Aprende en el contexto . Cierra la sesión y no queda nada.
«Basta con pedirle que revise su respuesta»	Sin señal externa, la autocrítica tiende a la autoafirmación o al cambio aleatorio.
«La mejora es de la reflexión»	Parte viene de tener más intentos. Hay que comparar contra reintentar sin reflexionar.
«Es aprendizaje por refuerzo»	Comparte la estructura (política, señal, mejora) pero no actualiza parámetros. La analogía es útil y también engañosa.
«Cuantos más intentos, mejor»	Con memoria creciente y sin criterio de parada, el coste crece y la calidad se estanca.

12. Relación con trabajos anteriores

- **P13 ReAct (2022)** — el bucle que se envuelve.
- **P28 Chain-of-Thought (2022)** — el razonamiento explícito.
- **P26 DQN (2015)** — el marco de refuerzo con el que se hace la analogía.

13. Relación con trabajos posteriores

- **Madaan et al. (2023), Self-Refine** — refinamiento iterativo sin señal externa; el contraste útil. [arXiv:2303.17651](#)
- **P31 Generative Agents (2023)** — memoria que persiste entre tareas, no solo entre intentos.
- **P16 Sistemas agentic** — la síntesis operativa.

14. Notebook asociado

P30_reflexion.ipynb

Qué implementa: el bucle de reintentos con y sin memoria verbal, el conteo de pesos actualizados (cero) y el análisis de cuánto contexto ocupa la memoria al crecer los intentos.

Qué NO implementa: las reflexiones son una lista escrita a mano. En el paper las genera el modelo, y pueden ser inútiles — que es justo el modo de fallo interesante.

```
ai-evolution paper-lab P30 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe los cuatro pasos del bucle.
Explicar	Explica por qué hace falta una señal externa.
Aplicar	Ejecuta el notebook y compara ambos bucles.
Analizar	Calcula cuántos intentos caben en un contexto de 8 000 tokens con reflexiones de 80.
Evaluar	Un equipo reporta +15 % con Reflexion. ¿Qué línea base exiges?
Crear	Diseña un detector de reflexión inútil («ser más cuidadoso») y di qué haría el agente al detectarla.

16. Autoevaluación

1. ¿Qué se actualiza exactamente en cada iteración?
2. ¿Por qué se le llama refuerzo si no hay gradientes?
3. ¿Qué pasa sin señal de evaluación externa?
4. ¿Cuál es la línea base correcta para medir la mejora?
5. ¿Qué límite impone el tamaño del contexto?
6. ¿Cómo se evita que el bucle no termine?
7. ¿Qué diferencia hay entre esta memoria y la de Generative Agents?

17. Respuestas esperadas

1. El **contexto**: se añade una reflexión a la memoria que condiciona el siguiente intento. Los parámetros no cambian.

2. Porque comparte la estructura del refuerzo —política, señal de resultado, mejora iterativa— aunque el mecanismo de actualización sea textual.
3. La reflexión no tiene información nueva que incorporar: el modelo tiende a declararse satisfecho o a cambiar cosas al azar.
4. Reintentar el mismo número de veces **sin** reflexión. Parte de la mejora viene solo de tener más oportunidades.
5. La memoria crece con los intentos y compite por el contexto con la tarea. A partir de cierto punto hay que resumir o priorizar.
6. Con límite de intentos, detección de reflexiones repetidas y escalamiento a un humano.
7. Reflexion recuerda entre **intentos de la misma tarea**; Generative Agents mantiene memoria **entre tareas y a lo largo del tiempo**, con recuperación puntuada.

18. Fuentes primarias

- Shinn, N. et al. (2023). *Reflexion: Language Agents with Verbal Reinforcement Learning*. **NeurIPS 2023**. arXiv:2303.11366 · consultado 2026-08-16.
- Madaan, A. et al. (2023). *Self-Refine: Iterative Refinement with Self-Feedback*. arXiv:2303.17651 · consultado 2026-08-16.

Evaluación — P30

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Reflexion: Language Agents with Verbal Reinforcement Learning* (2023, arXiv:2303.11366 · NeurIPS 2023) · **Nivel:** L2

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P30_reflexion.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P31 — Generative Agents

Ruta de agentes · Resuelve la memoria de un agente que vive mucho tiempo: qué recordar, cuándo y por qué, cuando el contexto no da para todo.

Nivel: L3 · **Motor:** generative_agents · **Notebook:** P31_generative_agents.ipynb · **Anexo:** álgebra y geometría

1. Identificación

Campo	Valor
Título original	Generative Agents: Interactive Simulacra of Human Behavior
Autoría	Joon Sung Park, Joseph C. O'Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, Michael S. Bernstein
Año	2023
Venue	arXiv:2304.03442 · UIST 2023
Fuente primaria	arXiv:2304.03442
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Reflexion recuerda entre intentos de una misma tarea. Pero un agente que **vive** —que actúa durante días, conversa, cambia de plan— acumula una historia que no cabe en ninguna ventana de contexto.

Y la solución obvia no funciona: guardar todo y recuperar lo más reciente devuelve lo trivial. Si acabas de comprar café, eso será lo último; que mañana hay una fiesta importante quedará enterrado.

3. Propuesta

Una arquitectura de memoria con tres piezas:

- Flujo de memoria:** todo se registra, con marca de tiempo, en lenguaje natural.
- Recuperación puntuada** por tres señales combinadas —relevancia respecto a la situación actual, recencia con decaimiento, e importancia intrínseca del recuerdo (que el propio modelo puntúa al guardarlo)—.
- Reflexión:** periódicamente, el agente sintetiza sus recuerdos en conclusiones de nivel superior («Klaus está muy metido en su investigación»), que se guardan como recuerdos nuevos y pueden a su vez sintetizarse.

Sobre esa memoria se construyen planificación y reacción, y todo ello se demuestra en una simulación con veinticinco agentes.

4. Intuición sin fórmulas

Una persona no recuerda su día en orden cronológico: recuerda lo que viene a cuento. Si un agente guarda todo y recupera lo último, recordará que compró café en vez de que mañana hay una fiesta.

Dónde deja de funcionar la analogía: la memoria humana olvida, distorsiona y consolida durmiendo. Aquí nada se borra: se puntúa. Es un archivo con buen buscador, no una memoria.

5. Matemática mínima

```
puntuación(m) = α_rel · relevancia(m, consulta)
                + α_rec · recencia(m)
                + α_imp · importancia(m)
```

relevancia	= similitud coseno entre embeddings	(ver A01)
recencia	= $\gamma^{(\text{pasos desde el último acceso})}$	decaimiento exponencial
importancia	= puntuación del modelo al guardar	de 1 a 10

Se recuperan los **k** de mayor puntuación y **solo esos** entran al contexto. Las tres señales son necesarias: sin relevancia se recupera ruido, sin recencia se ignora lo que acaba de pasar, y sin importancia lo trivial reciente gana siempre.

6. Arquitectura o flujo

7. Qué observar en el paper original

- La **arquitectura de memoria** en detalle: es la contribución técnica, por encima de la simulación que da titulares.
- El **árbol de reflexión**: las conclusiones se construyen sobre recuerdos y sobre otras conclusiones. Eso es lo que permite comportamiento coherente a lo largo de días simulados.
- La **evaluación con humanos**: se pide a personas que juzguen si el comportamiento es creíble, y se hacen **ablaciones** quitando memoria, reflexión o planificación. Esa tabla es la evidencia.
- El caso de la **fiesta de San Valentín**, que se propaga entre agentes sin que nadie lo programe: es el ejemplo más citado y conviene leer cómo se produce realmente.

8. Evidencia y resultados

Simulación de veinticinco agentes en un entorno tipo pueblo, con evaluación humana de la credibilidad del comportamiento y ablaciones de cada componente de la arquitectura.

El resultado que sostiene el paper es de **ablación**: quitar memoria, reflexión o planificación degrada la credibilidad juzgada por personas, y quitar la memoria es lo que más daña.

Las condiciones del experimento, el número de evaluadores y los resultados por ablación están en el artículo. Verificarlos allí, y tener presente que «credibilidad juzgada por humanos» es una métrica subjetiva por construcción.

La miniatura de este eje aísla la recuperación: con las tres señales sube lo relevante e importante; ordenando solo por recencia sube lo trivial.

9. Impacto

- Es la referencia obligada sobre **memoria de agentes de larga duración**, y su esquema de puntuación se copió por todas partes.
- Popularizó la idea de **reflexión como síntesis** —no solo como corrección de un fallo, que es lo de Reflexion—.
- Abrió una línea de simulación social con agentes que se usa en ciencias sociales computacionales, con las cautelas metodológicas del caso.

10. Limitaciones

1. **Coste alto**: cada acción requiere recuperar, y cada reflexión más llamadas al modelo.
2. **La importancia la puntúa el propio modelo**, con sus sesgos y sin calibración.
3. **Nada se olvida nunca**: el flujo crece indefinidamente y la recuperación se encarece.
4. **La credibilidad no es corrección**: que un comportamiento parezca humano no lo hace correcto ni útil.
5. **Los pesos de las tres señales** son hiperparámetros sin una teoría que los fije.
6. **Riesgo de sobreinterpretación**: son simulacros de comportamiento, no modelos de personas, y usarlos como sustituto de participantes humanos en investigación es metodológicamente delicado.

11. Errores comunes

Error	Corrección
«Los agentes tienen memoria»	Tienen un archivo con recuperación puntuada . No olvidan, no consolidan, no distorsionan.
«Es una demo tipo Los Sims»	La simulación es el vehículo; la contribución es la arquitectura de memoria y sus ablaciones.
«Simulan personas»	Simulan comportamiento creíble según jueces humanos. Los propios autores advierten contra el salto.
«Basta con RAG sobre el historial»	RAG recupera por relevancia. Aquí hacen falta las tres señales, y la reflexión no es recuperación sino síntesis.
«Se puede sustituir a participantes humanos»	Es precisamente el uso contra el que la comunidad ha advertido.

12. Relación con trabajos anteriores

- **P11 RAG (2020)** — la memoria consultable; aquí se le añaden recencia, importancia y síntesis.
- **P13 ReAct (2022)** — el bucle de acción sobre el que se monta.
- **P30 Reflexion (2023)** — reflexión como corrección; aquí es síntesis.

13. Relación con trabajos posteriores

- **P32 Voyager (2023)** — memoria **procedimental** (habilidades) en vez de episódica (recuerdos).
- **P16 Sistemas agentic** — la síntesis operativa del bloque.
- **Arquitecturas de memoria jerárquica (2023+)** — gestionar el contexto como un sistema operativo gestiona la memoria.

14. Notebook asociado

P31_generative_agents.ipynb

Qué implementa: la puntuación de cinco recuerdos con las tres señales, la comparación con ordenar solo por recencia, el efecto del factor de decaimiento y la cuenta de por qué concatenar el historial no escala.

Qué NO implementa: la relevancia se calcula por solapamiento de conjuntos, no por embeddings; la importancia está escrita a mano; y falta la reflexión, que es la mitad del aporte.

```
ai-evolution paper-lab P31 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la fórmula de puntuación y nombra las tres señales.
Explicar	Explica qué falla si se quita cada una de las tres.
Aplicar	Ejecuta el notebook y compara el ranking completo con el de solo recencia.
Analizar	Calcula cuántos tokens ocupa una semana de vida de un agente y decide si cabe.
Evaluar	«Credibilidad juzgada por humanos» como métrica: ¿qué mide y qué no?
Crear	Diseña una política de olvido y argumenta qué se pierde al no tenerla.

16. Autoevaluación

1. ¿Por qué recuperar por recencia no funciona?
2. ¿Qué aporta cada una de las tres señales?
3. ¿Qué es la reflexión aquí y en qué se diferencia de la de Reflexion?
4. ¿Por qué no cabe todo en el contexto? Da una cuenta aproximada.
5. ¿Qué demuestran las ablaciones del paper?
6. ¿Por qué «credibilidad» no equivale a «corrección»?
7. ¿Qué diferencia hay entre memoria episódica y procedimental?

17. Respuestas esperadas

1. Porque lo más reciente suele ser lo más trivial. La utilidad de un recuerdo no depende de cuándo ocurrió sino de si viene a cuento ahora.
2. Relevancia filtra por tema; recencia da peso a lo que acaba de pasar; importancia evita que lo trivial reciente desplace a lo significativo antiguo.
3. Aquí es **síntesis**: agregar recuerdos en conclusiones de nivel superior que se guardan como memoria. En Reflexion es **corrección**: analizar un fallo para no repetirlo.
4. Un agente con ~30 eventos por hora genera miles de eventos en una semana; a decenas de tokens cada uno, se van a cientos de miles. No cabe, y concatenar tampoco sería útil.
5. Que cada componente aporta: quitar memoria, reflexión o planificación degrada la credibilidad evaluada por personas, y la memoria es la más determinante.
6. Porque un comportamiento puede parecer humano y ser inútil, sesgado o falso. La credibilidad mide verosimilitud percibida, no acierto.
7. La episódica guarda **qué pasó** (recuerdos); la procedimental guarda **qué sé hacer** (habilidades ejecutables), que es lo de Voyager.

18. Fuentes primarias

- Park, J. S. et al. (2023). *Generative Agents: Interactive Simulacra of Human Behavior*. **UIST 2023**. arXiv:2304.03442 · consultado 2026-08-16.
- Lewis, P. et al. (2020). *Retrieval-Augmented Generation*. arXiv:2005.11401 · consultado 2026-08-16.

Evaluación — P31

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Generative Agents: Interactive Simulacra of Human Behavior* (2023, arXiv:2304.03442 · UIST 2023) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P31_generative_agents.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P32 — Voyager

Ruta de agentes · El agente acumula **habilidades reutilizables** en vez de contexto: memoria procedimental que no se borra al terminar la tarea.

Nivel: L3 · Motor: `voyager` · Notebook: `P32_voyager.ipynb` · Anexo: complejidad y coste

1. Identificación

Campo	Valor
Título original	<i>Voyager: An Open-Ended Embodied Agent with Large Language Models</i>
Autoría	Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, Anima Anandkumar
Año	2023
Venue	arXiv:2305.16291
Fuente primaria	arXiv:2305.16291
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Un agente que resuelve cada tarea desde cero **no mejora con la experiencia**. Y la solución inmediata — guardar lo aprendido como texto en el contexto— choca con dos muros: el contexto es finito, y el texto no es ejecutable.

Además faltaba la otra mitad: en un entorno de final abierto, **nadie dice qué hacer a continuación**. Sin objetivos, un agente competente no hace nada.

3. Propuesta

Tres componentes que se realimentan:

- Currículo automático:** el agente propone su siguiente tarea en función de lo que ya sabe y de lo que observa, buscando el punto en que sea alcanzable pero no trivial.
- Biblioteca de habilidades:** cada solución verificada se guarda como **código ejecutable** con nombre y descripción, e indexada para poder recuperarla. Las habilidades se componen unas con otras.
- Bucle iterativo de prompting:** se escribe código, se ejecuta en el entorno, y los errores del intérprete y la retroalimentación del juego alimentan el siguiente intento.

Se demuestra en Minecraft, un entorno de final abierto sin objetivo terminal.

4. Intuición sin fórmulas

Aprender a cocinar no es recordar cada vez la receta entera: es que «hacer un sofrito» pase a ser una sola cosa que sabes hacer. Voyager guarda habilidades, no anécdotas.

Dónde deja de funcionar la analogía: una persona generaliza una habilidad a situaciones parecidas. Aquí una habilidad es código concreto: si el entorno cambia lo suficiente, deja de funcionar y no hay adaptación automática.

5. Matemática mínima

No hay ecuación; hay una estructura de composición:

```
habilidad := [ primitiva | habilidad ]*

conseguir_madera = [talar, recoger]           → 2 primitivas
fabricar_mesa    = [conseguir_madera, fabricar] → 3 primitivas
fabricar_pico    = [conseguir_madera, fabricar_mesa, fabricar] → 6 primitivas
```

El **factor de compresión** —primitivas equivalentes dividido entre pasos declarados— crece con el currículo. Eso es lo que significa acumular capacidad, frente a acumular texto: cada nivel esconde más trabajo bajo el mismo número de pasos.

6. Arquitectura o flujo

La flecha que importa es la de **verificación**: solo entra en la biblioteca lo que se ejecutó y funcionó.

7. Qué observar en el paper original

- Cómo se genera el **currículo**: qué información recibe el modelo para proponer la siguiente tarea y cómo se evita que proponga cosas imposibles.
- El formato de las **habilidades** —código con descripción— y cómo se recuperan cuando hacen falta.
- Las **ablaciones**: sin biblioteca, sin currículo, sin bucle iterativo. Ahí se ve qué aporta cada pieza.
- Las **curvas de progreso**: número de ítems distintos obtenidos y distancia recorrida frente a las líneas base. Es un entorno sin puntuación única, así que las métricas son necesariamente indirectas.

8. Evidencia y resultados

Evaluación en Minecraft frente a líneas base de agentes con modelos de lenguaje, midiendo descubrimiento de ítems, progresión en el árbol tecnológico del juego y capacidad de generalizar a mundos nuevos reutilizando la biblioteca.

Las cifras concretas y las ablaciones están en el artículo. Verificarlas allí, y tener presente que es un preprint sin revisión por pares y que las métricas de un entorno de final abierto son discutibles por construcción.

La miniatura de este eje aísla la composición: la quinta tarea se declara en tres pasos pero equivale a once acciones primitivas, y una habilidad rota sin verificar contamina las cuatro que la componen.

9. Impacto

- Popularizó la **biblioteca de habilidades** como forma de memoria de agente, distinta de la episódica de Generative Agents.
- Puso el foco en el **aprendizaje de final abierto**: qué hace un agente cuando nadie le da objetivos.
- Reforzó la idea de que **el código es una buena representación** de una capacidad: es ejecutable, componible, verificable y legible.

10. Limitaciones

1. **Depende de un entorno con retroalimentación programática** rica: errores de intérprete, estado consultable. Casi ningún dominio real lo tiene igual de limpio.
2. **Una habilidad no verificada contamina** todas las que la compongan.
3. **Sin generalización**: el código es concreto; un cambio en el entorno lo rompe.
4. **Coste alto** en llamadas al modelo por el bucle iterativo.
5. **La biblioteca crece** y recuperar la habilidad correcta se vuelve un problema de búsqueda.
6. **Preprint sin revisión por pares**, y las métricas de un mundo abierto son difíciles de comparar entre trabajos.

11. Errores comunes

Error	Corrección
«El agente aprende como un humano»	Acumula funciones ejecutables verificadas. No generaliza ni transfiere por analogía.
«La biblioteca es memoria»	Es memoria procedimental . No recuerda qué pasó, sino qué sabe hacer.
«Basta con guardar lo que funcionó»	Sin verificación en el entorno, se guarda código que solo <i>parece</i> correcto y se propaga.
«Es un agente autónomo»	Opera dentro de un entorno concreto, con primitivas dadas y un objetivo implícito (explorar).
«Sirve para cualquier dominio»	Necesita ejecución verificable. Sin ella, el bucle no cierra.

12. Relación con trabajos anteriores

- **P13 ReAct (2022)** — el bucle de acción y observación.
- **P14 Toolformer (2023)** — aprender a usar herramientas; aquí se **crean** herramientas nuevas.
- **P30 Reflexion (2023)** — el bucle de reintento con retroalimentación.

13. Relación con trabajos posteriores

- **P33 AutoGen (2023)** — de un agente que acumula a varios que se coordinan.
- **P16 Sistemas agentic** — la síntesis operativa.
- **Agentes de programación (2023+)** — la biblioteca de habilidades como base de código que el agente mantiene.

14. Notebook asociado

P32_voyager.ipynb

Qué implementa: la composición de habilidades con su factor de compresión, el experimento de la habilidad rota que contamina a las que dependen de ella, y el contrato de entrada a la biblioteca.

Qué NO implementa: las habilidades son listas de nombres, no programas. No hay entorno, así que nada se ejecuta ni puede fallar — que es exactamente donde está la dificultad real.

```
ai-evolution paper-lab P32 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Enumera los tres componentes y di qué hace cada uno.
Explicar	Explica la diferencia entre memoria episódica y procedimental.
Aplicar	Ejecuta el notebook y calcula el factor de compresión de cada tarea.
Analizar	¿Cuántas tareas se rompen si falla una habilidad de nivel 1? Cuéntalo en el grafo.
Evaluar	¿Qué dominio de tu trabajo cumple el requisito de ejecución verificable?
Crear	Escribe el contrato de entrada a una biblioteca de habilidades para tu dominio.

16. Autoevaluación

1. ¿Qué problema resuelve la biblioteca que el contexto no puede resolver?
2. ¿Por qué el código es una buena representación de una habilidad?
3. ¿Para qué sirve el currículo automático?
4. ¿Qué pasa si se guarda una habilidad sin verificar?
5. ¿Qué requisito del entorno es imprescindible?
6. ¿Qué significa el factor de compresión?

7. ¿En qué se diferencia de Toolformer?

17. Respuestas esperadas

1. La persistencia y el coste: el contexto es finito y se pierde al terminar; la biblioteca crece sin ocupar contexto y se invoca por nombre.
2. Porque es ejecutable (se puede verificar), componible (se puede invocar desde otra), legible (se puede auditar) y persistente.
3. Para que el agente tenga objetivos en un entorno sin objetivo terminal, y que esos objetivos estén en el punto justo de dificultad dado lo que ya sabe.
4. Se propaga a todas las habilidades que la compongan: un fallo de nivel 1 rompe todo lo que dependa de él, directa o indirectamente.
5. Retroalimentación programática verificable: poder ejecutar y saber si funcionó.
6. Cuántas acciones primitivas esconde cada paso declarado. Crece con el currículo, y es la medida de que se está acumulando capacidad y no texto.
7. Toolformer aprende **cuándo llamar** a herramientas que ya existen; Voyager **crea** las herramientas y las guarda.

18. Fuentes primarias

- Wang, G. et al. (2023). *Voyager: An Open-Ended Embodied Agent with Large Language Models*. arXiv:2305.16291 · consultado 2026-08-16.
- Schick, T. et al. (2023). *Toolformer*. arXiv:2302.04761 · consultado 2026-08-16.

Evaluación — P32

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Voyager: An Open-Ended Embodied Agent with Large Language Models* (2023, arXiv:2305.16291) ·
Nivel: L3

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P32_voyager.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P33 — AutoGen

Ruta de agentes · El multiagente deja de ser una metáfora y pasa a ser un patrón de programación: agentes con rol que conversan hasta converger.

Nivel: L4 · Motor: `autogen` · Notebook: `P33_autogen.ipynb` · Anexo: complejidad y coste

1. Identificación

Campo	Valor
Título original	<i>AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation</i>
Autoría	Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu y otros
Año	2023
Venue	arXiv:2308.08155
Fuente primaria	arXiv:2308.08155
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Un solo agente escribe y juzga su propio trabajo, así que arrastra sus propios puntos ciegos: lee lo que quiso escribir. Reflexion mitiga eso con reintentos, pero sigue siendo el mismo agente evaluándose.

Y había un problema de ingeniería: componer varios agentes, decidir quién habla cuándo, meter a una persona en el bucle o permitir ejecución de código se hacía **ad hoc** en cada proyecto. No existía una abstracción común.

3. Propuesta

Modelar la aplicación como una **conversación entre agentes configurables**. Cada agente se define por tres ejes:

- qué modelo usa (o si no usa ninguno);
- si ejecuta código;
- si tiene una persona detrás (human-in-the-loop, en varios grados).

Y los patrones de conversación —quién habla, en qué orden, cuándo termina— se **programan**: hay conversaciones de dos, en grupo con moderador, jerárquicas y anidadas.

La tesis: muchas aplicaciones se expresan mejor como conversación multiagente que como un único prompt largo, y tener una abstracción común hace ese diseño reutilizable.

4. Intuición sin fórmulas

Quien escribe un texto es mal corrector de su propio texto. Poner a otro a revisarlo no es redundancia: es un punto de vista distinto sobre el mismo trabajo.

Dónde deja de funcionar la analogía: dos revisores humanos tienen experiencias distintas. Dos agentes con el mismo modelo base comparten sesgos, así que la «segunda opinión» puede ser la misma opinión con otro nombre. La diferencia hay que construirla con el rol y el objetivo.

5. Matemática mínima

No hay ecuación. Hay una cuenta de coste que conviene hacer siempre:

```
Un agente      : 1 llamada al modelo
Multiagente    : T llamadas, con T = turnos hasta converger

Coste relativo = T
Beneficio      = P(detectar error | multiagente) - P(detectar error | uno solo)
```

Multiagente compensa **si y solo si** el error que captura cuesta más que los turnos que añade. Esa desigualdad casi nunca se escribe, y es la que decide.

Fiabilidad compuesta, además: con n agentes y probabilidad p de que cada uno haga bien su parte, el sistema completo va como p^n salvo que haya verificación entre pasos.

6. Arquitectura o flujo

El crítico funciona porque su **objetivo es distinto**: encontrar fallos, no producir código. Si comparte prompt y objetivo con el programador, la crítica se vuelve ceremonial.

7. Qué observar en el paper original

- Los **patrones de conversación** que catalogan y en qué tipo de problema encaja cada uno.
- Los casos donde el multiagente **no** mejora: es la sección más útil y la que menos se cita.
- Cómo integran la **ejecución de código** y el **humano en el bucle** como grados de un mismo eje, no como casos especiales.
- Que es un **artículo de sistema**: describe un marco y sus aplicaciones, no un estudio controlado con ablaciones limpias. Hay que leerlo con esa expectativa.

8. Evidencia y resultados

Demostración del marco en seis aplicaciones —matemáticas, programación, respuesta a preguntas, juegos, análisis de datos— con comparaciones frente a implementaciones de un solo agente.

Los resultados por aplicación están en el artículo. Verificarlos allí, y con una cautela: en un paper de sistema, la línea base de «un solo agente» no siempre está tan optimizada como la propuesta. Es el sesgo estructural de este tipo de trabajos.

La miniatura de este eje muestra el mecanismo y su precio: la conversación detecta un fallo que el agente único entrega sin ver, a cambio de cinco veces más turnos.

9. Impacto

- Convirtió el multiagente en un patrón con vocabulario común: rol, turno, moderador, terminación.
- Popularizó el **crítico** como rol explícito, que hoy aparece en casi todos los sistemas de generación de código.
- Empujó la discusión hacia la **orquestración** —quién habla, cuándo se para, quién decide— que es donde están los fallos operativos reales.

10. Limitaciones

1. **Coste multiplicado** por el número de turnos, en dinero y en latencia.
2. **Sin criterio de parada, no converge**: dos agentes educados pueden felicitarse indefinidamente.
3. **Sesgos compartidos**: agentes con el mismo modelo base pueden coincidir en el error.
4. **Sicofancia entre agentes**: el crítico puede aprender a aprobar, sobre todo si no ejecuta nada.
5. **Fiabilidad compuesta**: más agentes, más puntos de fallo, salvo verificación entre pasos.
6. **Línea base débil**: la comparación honesta es contra un agente único **bien construido**, no contra uno ingenuo.
7. **Artículo de sistema**, no estudio controlado: difícil atribuir la mejora a la arquitectura.

11. Errores comunes

Error	Corrección
«Más agentes es mejor»	Cada agente añade coste, latencia y modos de fallo. Hay que demostrar qué error captura que uno solo no captura.
«El crítico garantiza calidad»	Solo si tiene objetivo distinto y acceso a evidencia (ejecución, tests). Si no, aprueba.
«Es como un equipo humano»	Comparten modelo base y por tanto sesgos. La diversidad hay que diseñarla, no viene dada.
«Multiagente resuelve la fiabilidad»	La empeora si no hay verificación: los fallos se componen.
«AutoGen inventó el multiagente»	Los sistemas multiagente son un área clásica de la IA. Lo nuevo es la abstracción conversacional sobre modelos de lenguaje.

12. Relación con trabajos anteriores

- **P13 ReAct (2022)** — el bucle de un agente.
- **P30 Reflexion (2023)** — autocrítica dentro del mismo agente.
- **Sistemas multiagente clásicos** — el área de la parte 10 del programa, anterior a los LLM.

13. Relación con trabajos posteriores

- **P16 Sistemas agentic** — la síntesis operativa: presupuesto, criterio de parada, permisos y trazas.
- **Model Context Protocol (2024)** — interoperabilidad de herramientas entre agentes y proveedores. modelcontextprotocol.io
- **Evaluación de sistemas multiagente (2024+)** — el problema abierto: cómo comparar arquitecturas de forma justa.

14. Notebook asociado

P33_autogen.ipynb

Qué implementa: la comparación entre una entrega de un solo agente y una conversación de tres roles que detecta y corrige el fallo, el coste relativo en turnos, el anti-patrón de conversación sin criterio de parada y un protocolo con terminación, detección de bucle y escalamiento.

Qué NO implementa: los mensajes están escritos a mano. En el paper los genera un modelo, y el crítico puede equivocarse o adular — que es el modo de fallo interesante.

```
ai-evolution paper-lab P33 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Nombra los tres ejes que definen un agente en este marco.
Explicar	Explica por qué el crítico necesita un objetivo distinto del programador.
Aplicar	Ejecuta el notebook y calcula el coste relativo en turnos.
Analizar	Con 4 agentes al 90 % de fiabilidad y sin verificación, ¿cuál es la fiabilidad del sistema?
Evaluar	Te presentan un sistema multiagente con +12 % de exactitud. ¿Qué línea base y qué costes pides?
Crear	Diseña un protocolo de conversación con criterio de parada, detección de bucle y escalamiento.

16. Autoevaluación

1. ¿Qué añade un crítico que un solo agente no puede aportar?
2. ¿Bajo qué condición deja de aportar?
3. ¿Cuál es el fallo operativo característico de una conversación multiagente?
4. ¿Por qué la fiabilidad puede empeorar al añadir agentes?

5. ¿Cuál es la línea base honesta?
6. ¿Qué significa que sea un «artículo de sistema»?
7. ¿Qué relación tiene con la sección de límites de P16?

17. Respuestas esperadas

1. Un punto de vista con **objetivo distinto**: buscar fallos en vez de producir. Eso cambia qué partes del trabajo recibe atención.
2. Cuando comparte prompt, modelo y objetivo con el autor, o cuando no tiene acceso a evidencia (ejecución, tests) y solo puede opinar.
3. No terminar: sin criterio de parada, los agentes pueden alternar indefinidamente, y cada turno cuesta una llamada al modelo.
4. Porque los fallos se componen: con n pasos al p de fiabilidad y sin verificación entre ellos, el sistema va como p^n .
5. Un agente único **bien construido** —con buen prompt, con acceso a las mismas herramientas y con reintentos—, no una versión ingenua.
6. Que describe un marco y sus aplicaciones sin aislar la contribución de la arquitectura frente a los prompts, el modelo o la ingeniería. Pide replicación independiente.
7. Es el paper que aporta la pieza de orquestación; P16 añade lo que falta para operarlo: presupuesto, criterio de parada, permisos, trazas y escalamiento.

18. Fuentes primarias

- Wu, Q. et al. (2023). *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation*. arXiv:2308.08155 · consultado 2026-08-16.
- *Model Context Protocol* — especificación abierta. modelcontextprotocol.io · consultado 2026-08-16.

Evaluación — P33

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation* (2023, arXiv:2308.08155) · **Nivel:** L4

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P33_autogen.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P34 — RoPE

Memoria y contexto · La posición se codifica **rotando**, y la atención pasa a depender solo de la distancia relativa. Es la base de casi todo modelo actual.

Nivel: L3 · Motor: rope · Notebook: P34_rope.ipynb · Anexo: álgebra y geometría

1. Identificación

Campo	Valor
Título original	RoFormer: Enhanced Transformer with Rotary Position Embedding
Autoría	Jianlin Su, Yu Lu, Shengfeng Pan, Ahmed Murtadha, Bo Wen, Yunfeng Liu
Año	2021
Venue	arXiv:2104.09864
Fuente primaria	arXiv:2104.09864
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

La codificación sinusoidal del Transformer se **suma** al embedding y codifica posición **absoluta**. Pero en lenguaje lo que importa casi siempre es la distancia: que dos palabras estén separadas por tres posiciones significa lo mismo al principio de un documento que al final.

Existían variantes con posición relativa (Shaw et al., 2018), pero añadían parámetros, tablas o términos extra en el cálculo de la atención.

3. Propuesta

Rotar los vectores de consulta y clave según su posición, por pares de coordenadas. La propiedad que se busca —y que se demuestra— es que el producto escalar entre una consulta en la posición m y una clave en la n depende **solo de $m - n$** .

La posición absoluta se usa para construir la rotación, pero desaparece del resultado. Sin parámetros nuevos, sin tablas y sin cambiar la forma de la ecuación de atención.

4. Intuición sin fórmulas

Dos relojes puestos en hora distinta. Si comparas sus manecillas, lo que obtienes no depende de qué hora marque cada uno, sino de la diferencia entre ambos.

Dónde deja de funcionar la analogía: las manecillas dan la vuelta y se repiten. Las frecuencias de RoPE están elegidas para que, en el rango de trabajo, posiciones distintas no colisionen — y ese rango es finito.

5. Matemática mínima

Para cada par de coordenadas $(2i, 2i+1)$ y posición m , con $\theta_i = 10000^{(-2i/d)}$:

$$R_m = \begin{bmatrix} \cos(m\theta_i) & -\sin(m\theta_i) \\ \sin(m\theta_i) & \cos(m\theta_i) \end{bmatrix}$$

$$q_m = R_m \cdot q \quad k_n = R_n \cdot k$$

$$\langle q_m, k_n \rangle = \langle R_m \cdot q, R_n \cdot k \rangle = q^T R_{\{n-m\}} k \quad \leftarrow \text{solo la DIFERENCIA}$$

La última igualdad sale de que las rotaciones componen: $R_m^T R_n = R_{\{n-m\}}$. Ahí está todo el paper.

6. Arquitectura o flujo

7. Qué observar en el paper original

- La **demostración** de que el producto solo depende de $m - n$. Es corta y merece seguirse.
- La elección de **frecuencias** por par de coordenadas, heredada de la codificación sinusoidal.
- El argumento sobre el **decaimiento** del producto con la distancia, que se presenta como propiedad deseable en lenguaje.
- Que se aplica a **q y k**, no al embedding de entrada: eso es lo que permite que la cancelación ocurra dentro de la atención.

8. Evidencia y resultados

Experimentos en tareas de comprensión y en modelado de lenguaje comparando RoPE con codificaciones absolutas y relativas previas.

Las cifras por tarea están en el artículo. Verificarlas allí. La adopción masiva de RoPE se debe tanto a sus resultados como a su simplicidad de implementación, y conviene no confundir ambas razones.

La miniatura de este eje comprueba la propiedad central: las parejas (5,3), (50,48) y (500,498) dan **exactamente** el mismo producto escalar.

9. Impacto

- Es la codificación posicional de la mayoría de modelos de lenguaje abiertos actuales.
- Al no añadir parámetros, se convirtió en el valor por defecto sin coste de adopción.
- Su formulación abrió la vía a las técnicas de **extensión de contexto** por interpolación de posiciones, que son posteriores y no están en este paper.

10. Limitaciones

1. **No garantiza extrapolación:** fuera del rango de posiciones visto en entrenamiento, el rendimiento cae. Las técnicas para arreglarlo son posteriores.
2. El **decaimiento con la distancia** es un sesgo, y hay tareas donde es indeseable.
3. Se aplica a la atención: **no** resuelve el coste cuadrático, que es otro problema (P35).
4. **Es un preprint** sin revisión por pares en su forma original.
5. La elección de la base (10000) es heredada y no está optimizada en el artículo.

11. Errores comunes

Error	Corrección
«RoPE permite contexto infinito»	Da posición relativa. Extender el contexto exige interpolación de posiciones, que es trabajo posterior.
«Es una codificación posicional más»	Es la única común que hace que la posición absoluta se cancele dentro del producto de atención.
«Se suma al embedding»	Se aplica rotando q y k , dentro de la atención. Ahí está la diferencia.
«Resuelve el coste de la atención»	No toca el coste. Ese es el problema de FlashAttention y de Mamba.

12. Relación con trabajos anteriores

- **Po8 Transformer (2017)** — la codificación sinusoidal que sustituye.
- **Shaw et al. (2018)** — posición relativa con parámetros extra. [arXiv:1803.02155](#)

13. Relación con trabajos posteriores

- **P35 FlashAttention (2022)** — el otro pilar del contexto largo.
- **Interpolación y extensión de posiciones (2023+)** — hacen extrapolable lo que RoPE no extrapola solo.
- **P20 Mamba (2023)** — la alternativa que prescinde de posiciones.

14. Notebook asociado

Qué implementa: la rotación por posición, la comprobación de invariancia relativa y la tabla de decaimiento con la distancia.

Qué NO implementa: ni modelo, ni entrenamiento, ni extensión de contexto.

```
ai-evolution paper-lab P34 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la matriz de rotación y di sobre qué se aplica.
Explicar	Explica por qué la posición absoluta se cancela en el producto.
Aplicar	Ejecuta el notebook y verifica la invariancia con tres parejas propias.
Analizar	¿Qué le pasa al producto con distancia 0? ¿Y por qué decae?
Evaluar	«RoPE da contexto infinito». Reescribe la afirmación de forma defendible.
Crear	Diseña un experimento que mida hasta qué posición extrapola un modelo con RoPE.

16. Autoevaluación

1. ¿Sobre qué vectores se aplica la rotación y por qué ahí?
2. ¿Por qué $R_m^T R_n = R_{\{n-m\}}$ es la clave de todo?
3. ¿Qué parámetros nuevos añade RoPE?
4. ¿Qué NO resuelve?
5. ¿Por qué el decaimiento con la distancia se considera deseable en lenguaje?
6. ¿Qué hace falta para extender el contexto más allá del entrenamiento?
7. ¿En qué se diferencia de la codificación sinusoidal de Po8?

17. Respuestas esperadas

1. Sobre q y k , dentro de la atención. Si se aplicara al embedding de entrada, la cancelación no ocurriría en el producto escalar.
2. Porque hace que el producto dependa solo de la diferencia de posiciones: es la propiedad que convierte una codificación absoluta en un efecto relativo.
3. Ninguno. La rotación se calcula, no se aprende.
4. El coste cuadrático de la atención, la extrapolación a longitudes no vistas y la calidad del uso del contexto largo.
5. Porque en lenguaje la dependencia entre palabras tiende a decaer con la distancia: es un sesgo inductivo alineado con el dominio.
6. Técnicas posteriores de interpolación o reescalado de posiciones. No está en este paper.
7. La sinusoidal se **suma** al embedding y codifica posición absoluta; RoPE **rota** q y k , y el resultado depende solo de la distancia.

18. Fuentes primarias

- Su, J. et al. (2021). *RoFormer: Enhanced Transformer with Rotary Position Embedding*. arXiv:2104.09864 · consultado 2026-08-16.
- Shaw, P., Uszkoreit, J. y Vaswani, A. (2018). *Self-Attention with Relative Position Representations*. arXiv:1803.02155 · consultado 2026-08-16.

Evaluación — P34

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *RoFormer: Enhanced Transformer with Rotary Position Embedding* (2021, arXiv:2104.09864) ·

Nivel: L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P34_rope.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P35 — FlashAttention

Memoria y contexto · El cuello de botella de la atención no eran los FLOPs sino las lecturas y escrituras a memoria. Y la solución es **exacta**, no una aproximación.

Nivel: L4 · **Motor:** flashattention · **Notebook:** P35_flashattention.ipynb · **Anexo:** complejidad y coste

1. Identificación

Campo	Valor
Título original	FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness
Autoría	Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, Christopher Ré
Año	2022
Venue	arXiv:2205.14135 · NeurIPS 2022
Fuente primaria	arXiv:2205.14135
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Desde 2019 la comunidad atacaba el coste $O(n^2)$ de la atención con **aproximaciones**: atención dispersa, lineal, de bajo rango. Reducían los FLOPs sobre el papel y, en la práctica, muchas no aceleraban nada — y sí perdían calidad.

El diagnóstico estaba equivocado. El proceso no estaba limitado por cómputo sino por **memoria**: materializar la matriz de atención $n \times n$ y moverla entre la memoria lenta de la GPU (HBM) y la rápida del chip (SRAM) costaba más que las multiplicaciones.

3. Propuesta

Reorganizar el cálculo para que la matriz de atención **nunca llegue a existir** en memoria lenta. Se recorre por bloques que caben en SRAM, se calcula el softmax de forma **incremental** con reescalado, y se acumula el resultado.

Dos consecuencias que conviene separar: el resultado es **numéricamente exacto** —no es una aproximación — y los FLOPs son los mismos. Lo único que cambia es el tráfico de memoria.

4. Intuición sin fórmulas

Cocinar con la despensa lejos. No es que cortar y freír sea lento: es que vas y vuelves a la despensa por cada ingrediente. La solución no es cocinar menos, es traer una bandeja con lo que necesitas y trabajar sin moverte.

Dónde deja de funcionar la analogía: el softmax necesita el máximo y la suma de **toda** la fila, que no cabe en la bandeja. Por eso hace falta el reescalado incremental, que es la parte técnicamente difícil.

5. Matemática mínima

Softmax incremental (estabilidad + acumulación por bloques):

```
para cada bloque j:  
    m_nuevo = max(m_viejo, max(S_j))  
    l_nuevo = e^{m_viejo - m_nuevo} · l_viejo +  $\sum e^{S_j - m_nuevo}$   
    O_nuevo = e^{m_viejo - m_nuevo} · O_viejo + e^{S_j - m_nuevo} · V_j
```

Accesos a memoria lenta:

estándar: $\Theta(n^2 + n \cdot d)$	← materializa la matriz
por bloques: $\Theta(n^2 \cdot d^2 / M)$	← M = tamaño de SRAM

Como $M \gg d$, el segundo es mucho menor. Y la salida es idéntica bit a bit salvo redondeo.

6. Arquitectura o flujo

7. Qué observar en el paper original

- El **análisis de accesos a memoria**: es la contribución conceptual, más que el algoritmo.
- La derivación del **softmax por bloques con reescalado**, que es lo que hace posible no materializar la matriz.
- Los **speedups medidos**, y en qué configuraciones: no son uniformes.
- El apartado de **Path-X y Path-256**: por primera vez un Transformer supera el azar en esas tareas de secuencia larguísima, no por ser más listo sino por poder ejecutarse.

8. Evidencia y resultados

Medidas de aceleración de extremo a extremo en entrenamiento, y resultados en tareas de secuencia larga que antes eran inabordables.

El artículo reporta **15 % de aceleración en BERT-large** (longitud 512), **3× en GPT-2** (longitud 1K) y **2,4× en Long Range Arena** (1K–4K), además de habilitar Path-X (16K) y Path-256 (64K), donde antes no se superaba el azar.

Verificar las condiciones exactas en el artículo: los speedups dependen del hardware, la longitud y la configuración.

La miniatura de este eje cuenta accesos a memoria frente a FLOPs, y muestra que con `n = 65 536` la reducción de tráfico es de más de un orden de magnitud **con los mismos FLOPs**.

9. Impacto

- Es una de las razones prácticas de que existan modelos con contexto largo: pasó de ser un problema de investigación a uno de ingeniería resuelta.
- Se integró como implementación por defecto en las bibliotecas principales, así que casi todo el mundo lo usa sin saberlo.
- Reorientó el trabajo sobre eficiencia: de reducir FLOPs a **considerar la jerarquía de memoria**.

10. Limitaciones

1. **No cambia la complejidad asintótica:** sigue siendo $O(n^2)$ en cómputo.
2. **Depende del hardware:** la ganancia se calcula sobre una jerarquía de memoria concreta.
3. **Implementación compleja:** es un kernel de bajo nivel, no unas líneas de código.
4. **No ayuda con lotes muy pequeños** ni secuencias cortas, donde el proceso no está limitado por memoria.
5. **No resuelve** que el modelo **use** bien el contexto largo, que es el problema de P36.

11. Errores comunes

Error	Corrección
«Es una atención aproximada»	Es exacta . Da el mismo resultado; cambia cómo se calcula.
«Reduce el coste a $O(n)$ »	El cómputo sigue siendo cuadrático. Lo que baja es el tráfico de memoria.
«Resuelve el contexto largo»	Lo hace viable . Que el modelo lo aproveche es otro problema.
«Optimizar FLOPs es optimizar velocidad»	Es la lección del paper: en hardware moderno, mover datos suele costar más que calcular.

12. Relación con trabajos anteriores

- **Po8 Transformer (2017)** — la atención cuyo coste se ataca.
- **Atención aproximada (2019–2021)** — la línea de trabajo que este paper desplaza.
- **Modelo roofline y jerarquía de memoria** — la clase O81 del programa.

13. Relación con trabajos posteriores

- **P36 Lost in the Middle (2023)** — el contexto largo ya es viable; ahora se mide si sirve.
- **P20 Mamba (2023)** — la alternativa por arquitectura al mismo problema.
- **Versiones posteriores de FlashAttention** — más optimizaciones sobre la misma idea.

14. Notebook asociado

P35_flashattention.ipynb

Qué implementa: el conteo comparado de FLOPs y accesos a memoria, y la memoria que ocuparía la matriz de atención a distintas longitudes.

Qué NO implementa: el kernel, el softmax por bloques con reescalado ni ninguna medida real de tiempo. Es un modelo de coste, no una implementación.

```
ai-evolution paper-lab P35 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Di qué se materializa en la versión estándar y qué no en la de bloques.
Explicar	Explica por qué el algoritmo es exacto pese a calcular por bloques.
Aplicar	Ejecuta el notebook y calcula la memoria de la matriz para $n = 128\,000$.
Analizar	¿Por qué no ayuda con secuencias cortas?
Evaluar	Un método reduce FLOPs a la mitad y no acelera. ¿Qué hipótesis formulas?
Crear	Diseña un experimento que distinga si tu proceso está limitado por cómputo o por memoria.

16. Autoevaluación

1. ¿Cuál era el diagnóstico equivocado que dominó tres años?
2. ¿Qué significa que el algoritmo sea «exacto»?
3. ¿Por qué hace falta reescalar el softmax al ir por bloques?
4. ¿Cambia la complejidad asintótica?
5. ¿Por qué la ganancia depende del hardware?
6. ¿Qué problema del contexto largo **no** resuelve?
7. ¿Qué lección general deja, más allá de la atención?

17. Respuestas esperadas

1. Que el cuello de botella eran los FLOPs. Por eso se buscaban aproximaciones que reducían cómputo y seguían moviendo la matriz por memoria lenta.

2. Que produce el mismo resultado numérico que la atención estándar, salvo redondeo. No sacrifica calidad por velocidad.
3. Porque el softmax necesita el máximo y la suma de toda la fila, y al procesar por bloques solo se conoce una parte: hay que corregir lo acumulado cuando aparece un máximo mayor.
4. No: sigue siendo cuadrática en cómputo. Cambia el término de memoria.
5. Porque se calcula sobre una jerarquía concreta (tamaño de SRAM, ancho de banda de HBM); en otra máquina el punto de equilibrio cambia.
6. Que el modelo **aproveche** el contexto largo, que es lo que mide P36.
7. Que en hardware moderno mover datos suele costar más que calcular, y que optimizar sin medir dónde está el límite lleva a soluciones que no aceleran nada.

18. Fuentes primarias

- Dao, T., Fu, D. Y., Ermon, S., Rudra, A. y Ré, C. (2022). *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness*. **NeurIPS 2022**. [arXiv:2205.14135](https://arxiv.org/abs/2205.14135) · consultado 2026-08-16.

Evaluación — P35

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness* (2022, arXiv:2205.14135 · NeurIPS 2022) · **Nivel:** L4

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P35_flashattention.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P36 — Lost in the Middle

Memoria y contexto · Tener contexto largo no es usarlo: el rendimiento cae en forma de U cuando el dato relevante está en el medio.

Nivel: L3 · **Motor:** `lost_in_middle` · **Notebook:** `P36_lost_in_middle.ipynb` · **Anexo:** complejidad y coste

1. Identificación

Campo	Valor
Título original	<i>Lost in the Middle: How Language Models Use Long Contexts</i>
Autoría	Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, Percy Liang
Año	2023
Venue	arXiv:2307.03172 · TACL
Fuente primaria	arXiv:2307.03172
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Con RoPE y FlashAttention, el contexto largo dejó de ser un problema técnico. La industria pasó a competir en tamaño de ventana: 4K, 32K, 128K, un millón de tokens.

Nadie estaba midiendo lo obvio: **si el modelo realmente usa todo ese espacio**. La ficha técnica dice cuántos tokens caben, no cuántos se aprovechan.

3. Propuesta

Un experimento controlado y sencillo. Se coloca el **mismo** documento relevante en distintas posiciones dentro del contexto, se hace la **misma** pregunta y se mide la exactitud en función de la posición.

Nada más cambia: ni el contenido, ni la pregunta, ni el modelo, ni la longitud total.

4. Intuición sin fórmulas

Un examen a libro abierto con veinte páginas de apuntes. Recuerdas bien lo primero que leíste y lo último; lo del medio se difumina. El modelo hace lo mismo.

Dónde deja de funcionar la analogía: una persona puede volver a mirar la página del medio. El modelo tiene todo delante a la vez y aun así lo usa peor.

5. Matemática mínima

No hay ecuación: hay un protocolo experimental, que es el aporte.

```
para posición p en 1..k:
  contexto = [doc1 ... doc_{p-1}, DOC_RELEVANTE, doc_{p+1} ... doc_k]
  exactitud[p] = evaluar(modelo, contexto, pregunta)

resultado: exactitud[p] tiene forma de U
  alta en p=1      (primacía)
  baja en p≈k/2    ← el hallazgo
  alta en p=k      (recencia)
```

6. Arquitectura o flujo

7. Qué observar en el paper original

- Las **curvas por modelo y por longitud**: la forma de U aparece de forma consistente, incluso en modelos anunciados como de contexto largo.
- El experimento con **modelos de contexto extendido**: tener más ventana no aplanla la curva.
- La conexión con **sistemas de recuperación**: cuántos pasajes conviene pasar y en qué orden.
- La discusión sobre posibles **causas**, que el paper plantea sin cerrar.

8. Evidencia y resultados

Evaluación en respuesta a preguntas con múltiples documentos y en recuperación de clave-valor, sobre varios modelos y longitudes de contexto.

El hallazgo central: el rendimiento es más alto cuando la información relevante está al principio o al final, y **cae de forma marcada** cuando está en el medio — incluso en modelos diseñados explícitamente para contexto largo.

Las magnitudes por modelo están en el artículo. Verificarlas allí: la caída varía y tomarla como constante

sería un error.

La miniatura de este eje reproduce la forma de la curva con un perfil didáctico de primacía y recencia, para poder razonar sobre sus consecuencias.

9. Impacto

- Cambió el discurso comercial: el tamaño de ventana dejó de ser un argumento suficiente.
- Introdujo la **prueba de aguja en el pajar por posición** como evaluación estándar de contexto largo.
- Tiene consecuencia directa en RAG: **el orden de los pasajes recuperados importa**, y colocar el mejor en medio es sabotearse.

10. Limitaciones

1. **Mide unos modelos concretos en un momento concreto**: los resultados envejecen.
2. **No explica la causa**, solo la documenta. Las hipótesis quedan abiertas.
3. Las tareas son **sintéticas y de recuperación**: no cubren razonamiento sobre todo el contexto.
4. La magnitud de la caída **depende del modelo**; generalizar un número sería incorrecto.
5. No dice **cuánto contexto conviene usar**, solo que más no es automáticamente mejor.

11. Errores comunes

Error	Corrección
«Este modelo tiene 128K de contexto, luego maneja 128K»	La ventana dice qué cabe, no qué se aprovecha. Hay que medirlo.
«Basta con meter todos los documentos»	Añadir pasajes irrelevantes empuja el bueno hacia el medio y empeora el resultado.
«El problema se arregla con más ventana»	El paper prueba modelos de contexto extendido y la curva sigue ahí.
«Es un fallo de un modelo concreto»	Se observa de forma consistente en varios modelos y familias.

12. Relación con trabajos anteriores

- **P34 RoPE y P35 FlashAttention** — hicieron viable el contexto largo cuya utilidad aquí se cuestiona.
- **P11 RAG (2020)** — el sistema al que más directamente afecta.
- **P10 GPT-3 (2020)** — la sensibilidad al prompt, de la que esto es un caso.

13. Relación con trabajos posteriores

- **P37 MemGPT (2023)** — si la ventana no basta ni usándola bien, hay que gestionarla como memoria.
- **Reordenamiento de pasajes en RAG (2023+)** — consecuencia práctica directa.
- **Evaluaciones de contexto largo (2024+)** — el género de benchmark que este paper inaugura.

14. Notebook asociado

P36_lost_in_middle.ipynb

Qué implementa: la curva en U con un perfil de primacía y recencia, la caída entre extremos y el protocolo de comprobación por posición.

Qué NO implementa: ningún modelo. El perfil está **modelado**, no medido: sirve para razonar sobre las consecuencias, no para afirmar magnitudes.

```
ai-evolution paper-lab P36 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Describe el protocolo experimental en tres líneas.
Explicar	Explica por qué esto afecta a un sistema RAG.
Aplicar	Ejecuta el notebook y localiza la peor posición.
Analizar	Si recuperas 10 pasajes y el mejor va tercero, ¿qué harías?
Evaluar	Un proveedor anuncia 1M de tokens. ¿Qué le pides antes de creerlo?
Crear	Diseña una prueba de aguja en el pajar para tu propio sistema.

16. Autoevaluación

1. ¿Qué se mantiene constante en el experimento y por qué es imprescindible?
2. ¿Qué forma tiene la curva y cómo se llaman sus dos extremos altos?
3. ¿Se arregla con una ventana mayor?
4. ¿Qué implicación tiene para ordenar pasajes en RAG?
5. ¿Explica el paper la causa?
6. ¿Por qué no conviene citar una magnitud concreta de caída?
7. ¿Qué evaluación estándar inaugura?

17. Respuestas esperadas

1. El documento, la pregunta, el modelo y la longitud total. Solo cambia la posición; si algo más cambiara, la diferencia no sería atribuible a la posición.
2. Forma de U: primacía al principio y recencia al final, con un valle en el medio.
3. No. El paper evalúa modelos de contexto extendido y el patrón persiste.
4. Que conviene colocar los pasajes más relevantes al principio o al final, y no pasar pasajes irrelevantes que empujen el bueno hacia el medio.
5. No. Documenta el fenómeno y plantea hipótesis, sin cerrarlas.
6. Porque varía según el modelo y la tarea; tomarla como constante convierte una observación en un dato falso.

7. La prueba de aguja en el pajar por posición, hoy habitual al evaluar contexto largo.

18. Fuentes primarias

- Liu, N. F. et al. (2023). *Lost in the Middle: How Language Models Use Long Contexts*. **TACL**. arXiv:2307.03172 · consultado 2026-08-16.

Evaluación — P36

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Lost in the Middle: How Language Models Use Long Contexts* (2023, arXiv:2307.03172 · TACL) ·

Nivel: L3

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P36_lost_in_middle.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P37 — MemGPT

Memoria y contexto · Aplica al contexto la idea de memoria virtual: una jerarquía que da la ilusión de memoria grande sobre una pequeña y rápida.

Nivel: L3 · Motor: `memgpt` · Notebook: `P37_memgpt.ipynb` · Anexo: complejidad y coste

1. Identificación

Campo	Valor
Título original	<i>MemGPT: Towards LLMs as Operating Systems</i>
Autoría	Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, Joseph E. Gonzalez
Año	2023
Venue	arXiv:2310.08560
Fuente primaria	arXiv:2310.08560
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

La ventana de contexto es un **límite duro**: lo que no cabe, no existe para el modelo. Ampliarla es caro y, como demuestra P36, tampoco garantiza que se aproveche.

Y hay casos donde ninguna ventana bastaría: un asistente que conversa durante meses, o el análisis de un corpus documental completo.

3. Propuesta

Tomar prestada la solución que la informática lleva décadas usando: **memoria virtual**.

- **Contexto principal**: pequeño, rápido, siempre visible. Es la RAM.
- **Almacén externo**: grande, lento, accesible bajo demanda. Es el disco.
- **El propio modelo gestiona el trasiego** mediante llamadas de función: decide qué desalojar y qué traer, igual que un sistema operativo pagina.

Se añaden interrupciones para ceder el control entre el modelo y el usuario.

4. Intuición sin fórmulas

Tu ordenador abre archivos mucho mayores que su RAM porque no los carga enteros: los pagina. Aquí igual, con una diferencia notable: **quien decide qué paginar es el propio modelo**.

Dónde deja de funcionar la analogía: un sistema operativo pagina con políticas deterministas y probadas. Aquí la política la decide un modelo que puede equivocarse, desalojar lo importante y no darse cuenta.

5. Matemática mínima

No hay ecuación; hay un contrato de gestión:

```
contexto_principal : capacidad fija C
almacén_externo   : capacidad ~ilimitada, acceso por función

si |contexto| > C:
    desalojar(según política) → almacén_externo

al necesitar algo ausente:
    buscar(almacén_externo) → traer al contexto ← cuesta UNA LLAMADA extra
```

El coste no desaparece: se convierte en **latencia por acceso**, exactamente como un fallo de página.

6. Arquitectura o flujo

7. Qué observar en el paper original

- La **analogía con el sistema operativo** desarrollada en serio: no es una metáfora suelta, es el marco de diseño.
- Cómo se le da al modelo el **control de la gestión** mediante funciones, y qué pasa cuando la usa mal.
- Las **dos aplicaciones** evaluadas: conversación de larga duración y análisis de documentos.
- El manejo de **interrupciones** y del flujo de control entre modelo y usuario.

8. Evidencia y resultados

Evaluación en conversación multisesión de larga duración y en preguntas sobre documentos que exceden ampliamente la ventana.

Las métricas y comparaciones están en el artículo. Verificarlas allí, y tener presente que es un preprint y que el sistema depende mucho del modelo base que lo ejecute.

La miniatura de este eje muestra la mecánica: con capacidad 5, los datos antiguos se desalojan sin perderse y se recuperan mediante una llamada de función.

9. Impacto

- Dio vocabulario prestado —paginación, jerarquía, interrupciones— a un problema que se estaba tratando sin marco.
- Es uno de los antecedentes de la **gestión explícita de contexto** en agentes, hoy una disciplina propia.
- Empujó la idea de que el modelo puede ser **gestor de su propio contexto**, no solo consumidor.

10. Limitaciones

1. **La política la decide el modelo**, y puede desalojar mal sin advertirlo.
2. **Cada page-in cuesta una llamada**: la ilusión de contexto infinito se paga en latencia.
3. **Depende de que el modelo base sepa usar funciones** de forma fiable.
4. **Preprint sin revisión por pares**.
5. **No resuelve P36**: lo que sí está en el contexto principal sigue sufriendo el problema de posición.
6. **Complejidad operativa**: más piezas, más modos de fallo, más difícil de depurar.

11. Errores comunes

Error	Corrección
«Da contexto infinito»	Da la ilusión de contexto grande, con coste por acceso. Como la memoria virtual.
«Sustituye a RAG»	Es complementario: RAG recupera de un corpus externo; esto gestiona la memoria del propio agente.
«El almacén externo es gratis»	Cada consulta es una llamada más al modelo: latencia y dinero.
«Es una arquitectura de modelo»	Es una arquitectura de sistema alrededor de un modelo que no se modifica.

12. Relación con trabajos anteriores

- **P11 RAG (2020)** — recuperación de conocimiento externo.
- **P31 Generative Agents (2023)** — la otra vía: memoria con recuperación puntuada en vez de paginación explícita.
- **P36 Lost in the Middle (2023)** — la evidencia de que agrandar la ventana no basta.

13. Relación con trabajos posteriores

- **P16 Sistemas agentic** — la memoria como componente operativo, con presupuesto y criterio de parada.
- **Gestión e ingeniería de contexto (2024+)** — la disciplina que se consolida a partir de aquí.

14. Notebook asociado

P37_memgpt.ipynb

Qué implementa: la jerarquía de dos niveles con desalojo y recuperación, la traza de page-out y page-in, y el coste en llamadas.

Qué NO implementa: el modelo no decide nada — aquí se desaloja lo más antiguo. En el paper, la política es del propio modelo, que es la parte interesante y la más frágil.

```
ai-evolution paper-lab P37 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Nombra los dos niveles de memoria y su equivalente en un sistema operativo.
Explicar	Explica por qué un page-in cuesta una llamada al modelo.
Aplicar	Baja la capacidad a 2 y observa cuántos datos acaban fuera.
Analizar	¿Qué política de desalojo usarías para un asistente personal? Justificala.
Evaluar	¿Cuándo conviene esto frente a simplemente usar un modelo de contexto mayor?
Crear	Diseña un presupuesto de accesos por respuesta y qué hacer al agotarlo.

16. Autoevaluación

1. ¿Cuál es la analogía central y hasta dónde se sostiene?
2. ¿Qué le pasa a un dato desalojado?
3. ¿Cuánto cuesta recuperarlo?
4. ¿Quién decide la política de paginación y por qué es frágil?
5. ¿En qué se diferencia de RAG?
6. ¿Resuelve el problema de la posición dentro del contexto?
7. ¿Qué hay que presupuestar al desplegarlo?

17. Respuestas esperadas

1. La memoria virtual de un sistema operativo. Se sostiene en la estructura (jerarquía, coste por acceso) y se rompe en la política: aquí la decide un modelo falible, no un algoritmo probado.
2. Baja al almacén externo. No se pierde.
3. Una llamada de función extra: latencia y coste, como un fallo de página.

4. El propio modelo, mediante llamadas de función. Es frágil porque puede desalojar lo importante o no buscar cuando debería, y nada lo detecta automáticamente.
5. RAG recupera de un corpus **externo** de conocimiento; esto gestiona la memoria **del propio agente** sobre su historia. Son complementarios.
6. No. Lo que sí está en el contexto principal sigue sujeto a la curva en U de P36.
7. Número máximo de page-ins por respuesta, qué vive siempre en el contexto principal, y qué hacer cuando se agota el presupuesto.

18. Fuentes primarias

- Packer, C. et al. (2023). *MemGPT: Towards LLMs as Operating Systems*. arXiv:2310.08560 · consultado 2026-08-16.

Evaluación — P37

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *MemGPT: Towards LLMs as Operating Systems* (2023, arXiv:2310.08560) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P37_memgpt.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P38 — VAE

Arquitectura y entrenamiento · Hace entrenable un modelo generativo latente: el truco de reparametrización deja pasar el gradiente a través del muestreo.

Nivel: L3 · Motor: vae · Notebook: P38_vae.ipynb · Anexo: probabilidad y verosimilitud

1. Identificación

Campo	Valor
Título original	Auto-Encoding Variational Bayes
Autoría	Diederik P. Kingma, Max Welling
Año	2013
Venue	arXiv:1312.6114 · ICLR 2014
Fuente primaria	arXiv:1312.6114
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Un modelo generativo con variables latentes dice: hay un z oculto que explica el dato x . Para entrenarlo hay que **muestrear** z , y muestrear es un nodo estocástico: no tiene derivada, así que la retropropagación no puede atravesarlo.

La inferencia variacional existía, pero requería aproximaciones costosas y no escalaba a conjuntos grandes ni encajaba con el entrenamiento por gradiente.

3. Propuesta

Dos piezas que se apoyan.

El truco de reparametrización: escribir $z = \mu + \sigma \cdot \epsilon$ con ϵ de una normal fija. La distribución de z es la misma, pero ahora el azar está **fuera** del camino del gradiente: μ y σ son funciones deterministas de la entrada, y se puede derivar respecto a ellas.

El objetivo: maximizar una cota inferior de la verosimilitud (ELBO), con dos términos —uno de reconstrucción y otro que mantiene el espacio latente cerca de una distribución simple—.

4. Intuición sin fórmulas

Quieres derivar respecto a la media de un dado trucado. Pero tirar el dado no tiene derivada. La solución: tira un dado normal aparte, y luego aplica la media y la escala con una fórmula que sí se puede derivar.

Dónde deja de funcionar la analogía: el truco solo vale para familias de distribuciones que se pueden escribir así. No es universal.

5. Matemática mínima

Reparametrización: $z = \mu_{\theta}(x) + \sigma_{\theta}(x) \cdot \epsilon, \quad \epsilon \sim N(0, I)$

ELBO: $\log p(x) \geq \underbrace{E_q[\log p(x|z)]}_{\text{reconstrucción}} - \underbrace{KL(q(z|x) \parallel p(z))}_{\text{regularización}}$

Con ambas gaussianas, la KL tiene forma cerrada:

$$KL = -\frac{1}{2} \sum (1 + \log \sigma^2 - \mu^2 - \sigma^2)$$

Sin el término KL, el codificador puede mapear cada x a una gaussiana estrechísima y aislada: el espacio latente deja de ser continuo y muestrear de la prior no genera nada coherente.

6. Arquitectura o flujo

7. Qué observar en el paper original

- La **derivación del ELBO** y por qué es una cota inferior de la verosimilitud.
- El **estimador reparametrizado** y su comparación de varianza con alternativas: ahí está el argumento cuantitativo.
- Los **experimentos con espacio latente 2D**, que muestran una variedad continua e interpolable.
- Que se publica casi a la vez que Rezende et al. (2014), con la misma idea: es un caso claro de descubrimiento simultáneo.

8. Evidencia y resultados

Experimentos de estimación de verosimilitud y generación en conjuntos de imágenes pequeños, comparando con métodos de inferencia variacional previos.

Las cifras están en el artículo. Verificarlas allí; lo transferible es el estimador, no los números de 2013.

La miniatura de este eje comprueba lo esencial: $z = \mu + \sigma \cdot \epsilon$ reproduce la distribución buscada y el gradiente respecto a μ existe y vale 1.

9. Impacto

- Hizo entrenable por gradiente toda una familia de modelos latentes profundos.
- El truco de reparametrización se usa hoy mucho más allá del VAE: políticas estocásticas, cuantización diferenciable, atención con puertas.
- Su espacio latente continuo e interpolable es un antecedente directo de los espacios latentes de los modelos de difusión modernos.

10. Limitaciones

1. **Muestras borrosas**: el término de reconstrucción penaliza el error medio, y la media de varias imágenes plausibles es borrosa.
2. **Colapso posterior**: si el decodificador es muy potente, puede ignorar z por completo.
3. **La KL en forma cerrada** solo existe para familias concretas.
4. **Es una cota inferior**: optimizarla no es optimizar la verosimilitud.
5. **Compromiso reconstrucción/regularización** difícil de ajustar.

11. Errores comunes

Error	Corrección
«Es un autoencoder con ruido»	Es inferencia variacional. El ruido tiene un papel probabilístico, no de regularización.
«Maximiza la verosimilitud»	Maximiza una cota inferior . La brecha depende de lo buena que sea q .
«El término KL es opcional»	Sin él, el espacio latente deja de ser muestreable y el modelo deja de ser generativo.
«Las muestras borrosas son un bug»	Son consecuencia directa del objetivo. Es un límite del método, no un error de implementación.

12. Relación con trabajos anteriores

- **P02 Backpropagation (1986)** — el método que el truco permite aplicar aquí.
- **Inferencia variacional clásica** — el marco que se hace escalable.
- **Rezende et al. (2014)** — derivación independiente y simultánea. [arXiv:1401.4082](https://arxiv.org/abs/1401.4082)

13. Relación con trabajos posteriores

- **P39 GAN (2014)** — la respuesta opuesta al mismo problema: un juego adversario en vez de una cota variacional.
- **P17 Difusión (2020)** — combina estabilidad de entrenamiento y calidad de muestra, que era justo lo que ninguno de los dos lograba a la vez.
- **Difusión latente (2022)** — difusión sobre un espacio latente tipo autoencoder.

14. Notebook asociado

P38_vae.ipynb

Qué implementa: el truco de reparametrización aislado, la comprobación de que la distribución se preserva, el gradiente respecto a μ y el cálculo de la KL en forma cerrada.

Qué NO implementa: codificador, decodificador ni datos. Solo el mecanismo.

```
ai-evolution paper-lab P38 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la reparametrización y los dos términos del ELBO.
Explicar	Explica por qué el azar tiene que quedar fuera del camino del gradiente.
Aplicar	Ejecuta el notebook y comprueba media y varianza empíricas.
Analizar	¿Qué pasa con el espacio latente si el peso de la KL tiende a 0?
Evaluar	¿Por qué las muestras son borrosas? ¿Es un fallo o una consecuencia?
Crear	Diseña un experimento que detecte colapso posterior.

16. Autoevaluación

1. ¿Por qué muestrear bloquea el gradiente?
2. ¿Qué hace exactamente la reparametrización?
3. ¿Qué mide cada término del ELBO?
4. ¿Por qué es una cota inferior y no la verosimilitud?
5. ¿De dónde vienen las muestras borrosas?
6. ¿Qué es el colapso posterior?
7. ¿Dónde más se usa el truco de reparametrización?

17. Respuestas esperadas

1. Porque es un nodo estocástico: la salida no es una función determinista de los parámetros, así que no hay derivada que propagar.
2. Reescribe la muestra como una función determinista de los parámetros y de una fuente de ruido independiente, de modo que el gradiente puede atravesarla.
3. Uno mide si el decodificador reconstruye x desde z ; el otro, cuánto se aleja la posterior aproximada de la prior, para que el espacio latente sea muestreable.
4. Porque se aproxima la posterior verdadera con una familia q más simple; la diferencia entre ambas es la brecha entre la cota y la verosimilitud real.
5. Del término de reconstrucción: penaliza el error medio, y la media de varias reconstrucciones plausibles es una imagen difusa.

6. Que el decodificador se vuelva tan potente que ignore z , y el término KL empuje q hacia la prior: el latente deja de codificar información.
7. En políticas estocásticas de refuerzo, en cuantización diferenciable y en cualquier sitio donde haya que derivar a través de una muestra.

18. Fuentes primarias

- Kingma, D. P. y Welling, M. (2014). *Auto-Encoding Variational Bayes*. **ICLR 2014**. [arXiv:1312.6114](#) · consultado 2026-08-16.
- Rezende, D. J., Mohamed, S. y Wierstra, D. (2014). *Stochastic Backpropagation and Approximate Inference in Deep Generative Models*. [arXiv:1401.4082](#) · consultado 2026-08-16.

Evaluación — P38

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Auto-Encoding Variational Bayes* (2013, arXiv:1312.6114 · ICLR 2014) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P38_vae.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P39 — GAN

Arquitectura y entrenamiento · Convierte la generación en un juego: dos redes compiten y ninguna necesita una verosimilitud explícita.

Nivel: L3 · Motor: gan · Notebook: P39_gan.ipynb · Anexo: probabilidad y verosimilitud

1. Identificación

Campo	Valor
Título original	Generative Adversarial Networks
Autoría	Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza y otros
Año	2014
Venue	arXiv:1406.2661 · NeurIPS (NIPS) 2014
Fuente primaria	arXiv:1406.2661
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Los modelos generativos de la época exigían definir y optimizar una **verosimilitud explícita**. Eso obligaba a aproximaciones costosas —cadenas de Markov, inferencia variacional— o producía muestras borrosas, como en el VAE publicado meses antes.

La pregunta abierta: ¿se puede entrenar un generador **sin** escribir nunca la probabilidad que asigna a cada muestra?

3. Propuesta

Sí, si otro modelo aporta la señal. Se entrenan dos redes en un juego de suma cero:

- el **discriminador** D aprende a distinguir muestras reales de sintéticas;
- el **generador** G aprende a producir muestras que D no distinga.

El generador nunca ve datos reales directamente: solo recibe gradiente **a través** del discriminador. No hay verosimilitud, no hay cadenas de Markov, no hay inferencia aproximada — solo retropropagación a través de dos redes.

En el óptimo teórico, D no puede distinguir y la distribución del generador iguala a la real.

4. Intuición sin fórmulas

Un falsificador y un policía que aprenden a la vez. El falsificador mejora porque el policía lo pilla; el policía mejora porque el falsificador se refina. Nadie le enseña al falsificador qué es un billete bueno: solo si coló o no.

Dónde deja de funcionar la analogía: al falsificador le basta con dominar **un** tipo de billete para colar siempre. No necesita saber hacer toda la serie — y eso es exactamente el modo de fallo del método.

5. Matemática mínima

$$\min_G \max_D V(D, G) = E_{\{x \sim p_{\text{datos}}\}}[\log D(x)] + E_{\{z \sim p_z\}}[\log(1 - D(G(z)))]$$

Discriminador óptimo para un G fijo:

$$D^*(x) = p_{\text{datos}}(x) / (p_{\text{datos}}(x) + p_G(x))$$

Sustituyendo, el objetivo de G equivale a minimizar la divergencia de Jensen-Shannon entre p_{datos} y p_G .

Detalle práctico del propio paper: al principio del entrenamiento D rechaza todo con facilidad, $\log(1 - D(G(z)))$ se satura y el gradiente para G casi desaparece. Por eso se entrena G maximizando $\log D(G(z))$ en lugar de minimizando el término original — mismo punto fijo, gradientes mucho mejores.

6. Arquitectura o flujo

7. Qué observar en el paper original

- La **demostración de convergencia** con capacidad infinita: es elegante y también explica por qué las garantías no se trasladan al caso real de redes con capacidad limitada.
- El **truco del gradiente no saturante**, en la sección de entrenamiento. Es un detalle de implementación con consecuencias enormes.
- Que el algoritmo alterna k pasos de D por cada paso de G , y por qué ese equilibrio importa.
- Que las muestras del paper son **modestas** para los estándares actuales: lo que se propone es un marco, y los resultados espectaculares llegan con trabajos posteriores.

8. Evidencia y resultados

Experimentos de generación en conjuntos de imágenes pequeños, con estimaciones de verosimilitud mediante ventanas de Parzen y comparación con modelos generativos de la época.

Las cifras están en el artículo. Verificarlas allí, y tener presente que la métrica usada (ventanas de Parzen) fue criticada después por poco fiable en alta dimensión: es un buen ejemplo de cómo una métrica dominante puede resultar inadecuada.

La miniatura de este eje muestra el modo de fallo característico: con tres modos en la distribución real, el generador converge a **uno solo** — y desde el punto de vista de su objetivo, hace bien.

9. Impacto

- Abrió una década de investigación en generación adversaria: rostros sintéticos, traducción de imagen a imagen, superresolución, datos sintéticos.
- Introdujo la idea de **aprender la función de pérdida** en vez de escribirla, que reaparece en el modelo de recompensa de RLHF.
- Popularizó el problema social de los medios sintéticos, con todo lo que trajo después.
- Y fue desplazado por la difusión hacia 2021-2022, precisamente por su inestabilidad y su colapso de modos.

10. Limitaciones

1. **Colapso de modos**: el generador cubre una región de la distribución y abandona el resto.
2. **Entrenamiento inestable**: dos objetivos en competencia, sin una pérdida que baje de forma monótona y que sirva para saber si va bien.
3. **Sin verosimilitud**: no se puede evaluar la probabilidad que el modelo asigna a un dato.
4. **Métricas difíciles**: no hay una medida sencilla y fiable de calidad y cobertura a la vez.
5. **Sensible a la arquitectura y a los hiperparámetros** de forma notoria.
6. **Las garantías teóricas suponen capacidad infinita** y optimización perfecta: ninguna de las dos se cumple.

11. Errores comunes

Error	Corrección
«Las muestras son buenas, luego el modelo es bueno»	El colapso de modos produce muestras excelentes y poco diversas. Hay que medir cobertura , no solo calidad.
«El generador aprende de los datos reales»	Nunca los ve. Solo recibe gradiente a través del discriminador.
«La pérdida baja, luego va bien»	En un juego adversario, la pérdida no es un indicador de progreso: puede oscilar mientras el modelo mejora, o bajar mientras colapsa.
«Las GAN quedaron obsoletas»	Fueron desplazadas en generación de imagen general, pero siguen siendo competitivas donde la latencia importa: generan en una pasada, no en cientos.
«El objetivo del paper es el que se usa»	Se usa la variante no saturante, que el propio artículo introduce por el problema de gradiente.

12. Relación con trabajos anteriores

- **P38 VAE (2013)** — el enfoque opuesto al mismo problema: cota variacional frente a juego adversario.
- **P02 Backpropagation (1986)** — el único mecanismo de entrenamiento que hace falta.
- **Modelos generativos con verosimilitud explícita** — la familia que se esquiva.

13. Relación con trabajos posteriores

- **DCGAN (2015), Wasserstein GAN (2017), StyleGAN (2018-2020)** — estabilización, mejores métricas y calidad fotorrealista.
- **P17 Difusión (2020)** — estabilidad de entrenamiento y calidad de muestra, que es justo lo que ni GAN ni VAE lograban a la vez.
- **P12 InstructGPT (2022)** — aprender la señal de entrenamiento con otro modelo, la misma idea en otro dominio.

14. Notebook asociado

P39_gan.ipynb

Qué implementa: la dinámica del generador persiguiendo el modo más cercano en una distribución de tres modos, el conteo de modos cubiertos y la lista de lo que hay que reportar en un modelo generativo.

Qué NO implementa: el discriminador está simplificado a una comparación de medias y no hay entrenamiento adversario alterno real. Se modela la dinámica del colapso, no se ejecuta una GAN.

```
ai-evolution paper-lab P39 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe el objetivo minimax e identifica qué maximiza y qué minimiza cada red.
Explicar	Explica por qué el generador nunca necesita ver datos reales.
Aplicar	Ejecuta el notebook y observa a qué modo colapsa según la posición inicial.
Analizar	Deriva $D^*(x)$ para un G fijo y explica qué significa que valga $1/2$ en todas partes.
Evaluar	Un equipo enseña 20 muestras excelentes. ¿Qué métrica falta y por qué?
Crear	Diseña una medida de cobertura de modos para una distribución 2D conocida.

16. Autoevaluación

1. ¿Qué señal de entrenamiento sustituye a la verosimilitud?
2. ¿Por qué se entrena G maximizando $\log D(G(z))$ y no con el término original?
3. ¿Qué es el colapso de modos y por qué es racional desde el objetivo del generador?
4. ¿Por qué la pérdida no sirve como indicador de progreso?

5. ¿Qué supone la demostración de convergencia que no se cumple en la práctica?
6. ¿En qué gana todavía una GAN frente a un modelo de difusión?
7. ¿Qué idea de este paper reaparece en RLHF?

17. Respuestas esperadas

1. El discriminador: una red que aprende a distinguir, y cuyo gradiente le dice al generador cómo mejorar. Se aprende la pérdida en vez de escribirla.
2. Porque al principio D rechaza casi todo, $\log(1 - D(G(z)))$ se satura y el gradiente para G se anula. La variante no saturante tiene el mismo punto fijo y gradientes útiles.
3. Que el generador cubre solo una parte de la distribución. Es racional porque su objetivo es engañar al discriminador, y para eso basta con ser convincente en una región.
4. Porque es un juego: si D mejora, la pérdida de G sube aunque G también haya mejorado. No hay una cantidad que descienda de forma monótona hacia el óptimo.
5. Capacidad infinita para ambas redes y optimización perfecta en cada paso. Con redes reales y descenso de gradiente alterno, ninguna se cumple.
6. En velocidad de muestreo: genera en **una** pasada, mientras la difusión necesita decenas o cientos. Donde la latencia manda, sigue siendo relevante.
7. Aprender la función objetivo con otro modelo en vez de escribirla: el modelo de recompensa de RLHF cumple el papel que aquí cumple el discriminador.

18. Fuentes primarias

- Goodfellow, I. J. et al. (2014). *Generative Adversarial Networks*. **NIPS 2014**. arXiv:1406.2661 · consultado 2026-08-16.
- Kingma, D. P. y Welling, M. (2014). *Auto-Encoding Variational Bayes*. arXiv:1312.6114 · consultado 2026-08-16.

Evaluación — P39

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Generative Adversarial Networks* (2014, arXiv:1406.2661 · NeurIPS 2014) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P39_gan.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P40 — Dropout

Arquitectura y entrenamiento · Apagar unidades al azar equivale a entrenar un ensamblado exponencial de subredes que comparten pesos.

Nivel: L2 · Motor: dropout · Notebook: P40_dropout.ipynb · Anexo: probabilidad y verosimilitud

1. Identificación

Campo	Valor
Título original	<i>Dropout: A Simple Way to Prevent Neural Networks from Overfitting</i>
Autoría	Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, Ruslan Salakhutdinov
Año	2014
Venue	<i>Journal of Machine Learning Research</i> 15(56), 1929–1958
Fuente primaria	jmlr.org/papers/v15/srivastava14a
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Las redes grandes memorizaban el conjunto de entrenamiento. La receta conocida contra eso — promediar muchos modelos entrenados por separado — funciona muy bien y es **inviable** con redes profundas: entrenar diez redes cuesta diez veces más, y servir las también.

Había además un problema más sutil que el sobreajuste clásico: la **co-adaptación**. Una unidad podía desarrollar una función que solo era útil en presencia de otra unidad concreta. El conjunto funcionaba en entrenamiento y era frágil ante cualquier cambio.

3. Propuesta

En cada paso de entrenamiento, poner a cero cada unidad con probabilidad p , de forma independiente. Ninguna función puede depender de que una unidad concreta esté presente, porque a menudo no lo estará.

La interpretación que da el paper: con n unidades hay 2^n subredes posibles, y entrenar con dropout es entrenar ese **ensamblado exponencial** con pesos compartidos. En inferencia se usan todas las unidades con los pesos escalados, lo que aproxima el promedio de todas esas subredes.

4. Intuición sin fórmulas

Si en un equipo cada tarea depende de dos personas concretas, el día que una falte no se hace nada. Si todos pueden cubrir varias tareas, el equipo aguanta. Dropout obliga a lo segundo haciendo faltar a gente al azar

cada día.

Dónde deja de funcionar la analogía: un equipo real reorganiza el trabajo conscientemente. La red no «decide» ser redundante: es que las representaciones redundantes son las únicas que reciben gradiente útil de forma consistente.

5. Matemática mínima

Entrenamiento:	$m \sim \text{Bernoulli}(1 - p)$ $\tilde{h} = m \odot h$	una máscara nueva por paso
Inferencia	: $h_{\text{test}} = (1 - p) \cdot h$	se usan todas, escaladas
Equivalente y más usado (inverted dropout):	$\tilde{h} = (m \odot h) / (1 - p)$ $h_{\text{test}} = h$	durante el entrenamiento sin cambio en inferencia

El escalado no es cosmético: sin él, la magnitud esperada de las activaciones en inferencia es $1/(1-p)$ veces la del entrenamiento, y el modelo trabaja en un régimen distinto del que aprendió.

Por qué premia la redundancia, con $p = 0,5$ y unidades independientes:

una función que necesita 2 unidades concretas	→ disponible el 25 % de los pasos
una repartida entre 3 unidades cualesquiera	→ disponible el 87,5 %

6. Arquitectura o flujo

7. Qué observar en el paper original

- La **interpretación como ensamblado** y por qué el escalado en inferencia aproxima el promedio geométrico de las subredes. Es una aproximación, y el paper lo dice.
- Los experimentos sobre **qué aprenden las unidades** con y sin dropout: las representaciones visualizadas son más localizadas e interpretables con dropout.
- La discusión sobre el **valor de p** : 0,5 en capas ocultas y valores menores en la entrada, y por qué.
- Que se evalúa en **muchos dominios** —visión, voz, texto, genómica—: la afirmación es de generalidad, no de un buen resultado puntual.

8. Evidencia y resultados

Mejoras consistentes en conjuntos de visión, reconocimiento de voz, clasificación de documentos y datos biológicos, comparando la misma arquitectura con y sin dropout.

Las cifras por conjunto están en el artículo. Verificarlas allí; lo transferible es el mecanismo y el patrón de mejora, no los números de 2014.

La miniatura de este eje cuantifica el argumento de la co-adaptación: una función que depende de dos unidades concretas está disponible una cuarta parte de las veces, y una repartida entre tres, casi siempre.

9. Impacto

- Fue la regularización por defecto en deep learning durante media década.
- Su interpretación como ensamblado influyó en toda una línea de trabajo sobre regularización estocástica.
- Y hoy se usa **menos** en visión y en Transformers grandes, donde la escala de datos y otras técnicas cumplen ese papel. Es un buen recordatorio de que las recetas del campo caducan.

10. Limitaciones

1. **Ralentiza la convergencia:** hacen falta más épocas para el mismo resultado.
2. **Interacción mal entendida con la normalización por lotes**, que produjo fallos sutiles durante años al combinarlas.
3. **La equivalencia con el ensamblado es aproximada**, no exacta, y el paper lo declara.
4. **Menos útil con muchos datos:** si el conjunto es grande, hay poco sobreajuste que evitar.
5. **Un hiperparámetro más** (p) que ajustar por capa.
6. **Poco efectivo en capas convolucionales** frente a las densas, por la compartición de pesos.

11. Errores comunes

Error	Corrección
«Dropout añade ruido para regularizar»	Cambia qué representaciones son rentables : penaliza la dependencia de unidades concretas.
Dejarlo activo en inferencia	La misma entrada daría salidas distintas y con magnitud equivocada. Es un fallo clásico en producción.
Aplicar el escalado dos veces	Existen dos convenciones (escalar en inferencia o dividir en entrenamiento). Usar ambas rompe el modelo.
«Siempre conviene ponerlo»	Con muchos datos o arquitecturas modernas puede no aportar y sí ralentizar.
«Es equivalente a entrenar 2^n redes»	Es una aproximación al promedio de esas subredes, no una equivalencia exacta.

12. Relación con trabajos anteriores

- **Po4 AlexNet (2012)** — lo usa en sus capas densas; este artículo es el desarrollo completo de la idea.
- **Hinton et al. (2012)** — la versión preliminar. [arXiv:1207.0580](#)
- **Bagging y ensamblados** — el marco clásico que se aproxima sin pagar su coste.

13. Relación con trabajos posteriores

- **P43 BatchNorm (2015)** — otra forma de estabilizar, con la que interactúa de forma no trivial.
- **DropConnect, DropPath, stochastic depth** — variantes sobre la misma idea.
- **Regularización moderna** — aumento de datos, decaimiento de pesos y escala, que en muchos casos lo han sustituido.

14. Notebook asociado

P40_dropout.ipynb

Qué implementa: el cálculo de disponibilidad de una función co-adaptada frente a una redundante, el conteo de subredes y la demostración del escalado en inferencia.

Qué NO implementa: no hay red, ni datos, ni entrenamiento. Se cuentan probabilidades de máscara para exhibir el argumento.

```
ai-evolution paper-lab P40 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe qué ocurre en entrenamiento y qué en inferencia.
Explicar	Explica por qué dropout premia las representaciones redundantes.
Aplicar	Calcula la disponibilidad de una función que necesita 4 unidades concretas con $p=0,5$.
Analizar	¿Por qué es menos efectivo en capas convolucionales?
Evaluar	Un modelo «funciona peor en producción» sin causa aparente. ¿Qué comprobarías primero?
Crear	Diseña un experimento que distinga el efecto de ensamblado del de anti-co-adaptación.

16. Autoevaluación

1. ¿Qué es la co-adaptación y por qué es un problema?
2. ¿Cuántas subredes hay con n unidades?
3. ¿Por qué hace falta escalar y qué pasa si no se hace?
4. ¿Cuáles son las dos convenciones de escalado?
5. ¿Por qué ralentiza la convergencia?
6. ¿Por qué se usa menos hoy en modelos grandes?

7. ¿En qué sentido la equivalencia con un ensamblado es aproximada?

17. Respuestas esperadas

1. Que una unidad desarrolle una función útil solo en presencia de otra concreta. Produce soluciones frágiles que dependen de una configuración exacta del resto de la red.
2. 2^n , una por cada subconjunto de unidades activas. Todas comparten los mismos pesos.
3. Porque en entrenamiento solo está activa la fracción $1-p$ de las unidades, así que la suma esperada es menor. Sin escalar, en inferencia las activaciones son $1/(1-p)$ veces mayores que las vistas al entrenar.
4. Escalar por $(1-p)$ en inferencia, o dividir por $(1-p)$ durante el entrenamiento (inverted dropout). Se elige una; aplicar las dos rompe el modelo.
5. Porque cada paso entrena una subred distinta y con gradiente más ruidoso: hacen falta más pasos para el mismo progreso efectivo.
6. Porque con conjuntos enormes hay poco sobreajuste, y otras técnicas —aumento de datos, decaimiento de pesos, la propia escala— cubren ese papel con menos coste.
7. El escalado en inferencia aproxima el promedio geométrico de las predicciones de las subredes, pero no lo calcula exactamente. El propio paper lo presenta como aproximación.

18. Fuentes primarias

- Srivastava, N. et al. (2014). *Dropout: A Simple Way to Prevent Neural Networks from Overfitting*. **JMLR** 15(56), 1929–1958. jmlr.org/papers/v15/srivastava14a · consultado 2026-08-16.
- Hinton, G. E. et al. (2012). *Improving neural networks by preventing co-adaptation of feature detectors*. arXiv:1207.0580 · consultado 2026-08-16.

Evaluación — P40

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Dropout: A Simple Way to Prevent Neural Networks from Overfitting* (2014, JMLR 15(56):1929–1958) · **Nivel:** L2

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P40_dropout.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P41 — Adam

Arquitectura y entrenamiento · Un paso de aprendizaje por dimensión, adaptado a la escala de su propio gradiente. Es el optimizador por defecto de casi todo lo que vino después.

Nivel: L2 · Motor: adam · Notebook: P41_adam.ipynb · Anexo: cálculo y gradientes

1. Identificación

Campo	Valor
Título original	Adam: A Method for Stochastic Optimization
Autoría	Diederik P. Kingma, Jimmy Ba
Año	2014
Venue	arXiv:1412.6980 · ICLR 2015
Fuente primaria	arXiv:1412.6980
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

El descenso de gradiente estocástico usa **la misma tasa de aprendizaje en todas las direcciones**. En un problema mal condicionado —donde una dirección es mucho más curva que otra— eso obliga a elegir entre dos males: una tasa grande hace oscilar la dirección empujada, y una pequeña deja la dirección plana avanzando a paso de tortuga.

Existían métodos adaptativos (AdaGrad, RMSProp) pero cada uno tenía su punto débil: AdaGrad acumula gradientes al cuadrado desde el inicio y su paso efectivo tiende a cero; RMSProp lo arregla con una media móvil pero no usa momento.

3. Propuesta

Combinar ambas ideas y añadir la pieza que faltaba:

- Primer momento:** una media móvil del gradiente, que suaviza la dirección (momento clásico).
- Segundo momento:** una media móvil del gradiente al cuadrado, que estima la escala típica de cada coordenada.
- Corrección de sesgo:** ambas medias empiezan en cero y por tanto subestiman al principio; se corrige dividiendo por $1 - \beta^t$.

El paso de cada coordenada acaba siendo del orden de η , sea cual sea la magnitud de su gradiente.

4. Intuición sin fórmulas

Bajar un valle largo y estrecho. En la dirección estrecha, un paso normal te hace rebotar de pared a pared; en la larga, ese mismo paso no avanza nada. Adam mide cuánto se mueve cada dirección y ajusta su paso por separado.

Dónde deja de funcionar la analogía: el valle real está fijo. En una red, el paisaje cambia con cada minilote, y las estimaciones de escala se hacen sobre un objetivo que se mueve.

5. Matemática mínima

$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot g_t$	primer momento
$v_t = \beta_2 \cdot v_{t-1} + (1 - \beta_2) \cdot g_t^2$	segundo momento
$\hat{m}_t = m_t / (1 - \beta_1^t)$	corrección de sesgo
$\hat{v}_t = v_t / (1 - \beta_2^t)$	
$\theta_t = \theta_{t-1} - \eta \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon)$	

Valores por defecto del paper: $\beta_1 = 0,9$, $\beta_2 = 0,999$, $\epsilon = 1e-8$.

Por qué hace falta la corrección de sesgo: con $m_0 = 0$ y $\beta_1 = 0,9$, tras el primer paso $m_1 = 0,1 \cdot g_1$, diez veces menor que el gradiente real. Sin corregir, los primeros pasos serían diminutos justo cuando más falta hace avanzar.

6. Arquitectura o flujo

7. Qué observar en el paper original

- La **derivación de la corrección de sesgo**: es la aportación técnica que distingue a Adam de RMSProp con momento.
- El **análisis de regret** en el caso convexo: da garantías, y conviene ver hasta dónde llegan (y hasta dónde no, porque las redes no son convexas).
- La discusión sobre el **paso efectivo acotado** por η , que es lo que da robustez frente a la escala del gradiente.
- **AdaMax**, la variante con norma infinito, que el mismo artículo introduce y casi nadie usa.

8. Evidencia y resultados

Experimentos de regresión logística, redes densas y convolucionales, comparando con SGD con momento, AdaGrad, RMSProp y otros.

Las curvas por experimento están en el artículo. Verificarlas allí, y con una cautela: la literatura posterior encontró que con SGD bien ajustado la comparación es más ajustada de lo que sugieren esas figuras, sobre todo en visión.

La miniatura de este eje aísla el argumento en un problema con número de condición 100: SGD queda oscilando con pérdida ~ 100 y Adam converge a $\sim 1e-8$.

9. Impacto

- Es el optimizador por defecto de prácticamente todos los modelos de este eje.
- Redujo drásticamente el esfuerzo de ajustar la tasa de aprendizaje, que era una de las mayores fricciones prácticas del deep learning.
- Su ubicuidad lo convirtió también en objeto de crítica: buena parte del trabajo posterior sobre optimización se define por comparación con él.

10. Limitaciones

1. **No siempre generaliza mejor:** hay evidencia de que SGD con momento bien ajustado generaliza mejor en algunas tareas de visión.
2. **El decaimiento de pesos se comporta mal** dentro de Adam: al dividirse también por $\sqrt{v}$, la regularización acaba siendo distinta por coordenada. Lo corrige AdamW (2017).
3. **Más memoria:** guarda dos estados por parámetro, lo que en modelos enormes es significativo.
4. **La prueba de convergencia original tenía un error**, señalado en 2018, que motivó variantes como AMSGrad.
5. **Sensible a ϵ** en regímenes de gradiente muy pequeño.
6. **Las garantías son para el caso convexo**, y las redes no lo son.

11. Errores comunes

Error	Corrección
«Adam es siempre mejor que SGD»	Converge más rápido en muchos casos; generalizar mejor es otra afirmación y no siempre se sostiene.
Usar el decaimiento de pesos de SGD tal cual	Dentro de Adam se divide también por $\sqrt{v}$ y deja de ser la regularización pretendida. Para eso está AdamW.
«La corrección de sesgo es un detalle»	Sin ella, los primeros pasos son diminutos, justo cuando más importa avanzar.
«Adapta la tasa de aprendizaje»	Adapta el paso por coordenada . La tasa global η sigue siendo un hiperparámetro que hay que elegir.
«Tiene garantías de convergencia»	Para el caso convexo, y la prueba original fue corregida después. En redes profundas no hay garantía.

12. Relación con trabajos anteriores

- **Po2 Backpropagation (1986)** — quien calcula el gradiente que Adam consume.
- **AdaGrad (2011) y RMSProp (2012)** — los métodos adaptativos que combina.
- **Momento de Polyak (1964)** — el primer momento.

13. Relación con trabajos posteriores

- **AdamW (2017)** — desacopla el decaimiento de pesos del paso adaptativo. [arXiv:1711.05101](#)
- **AMSGrad (2018)** — corrige el problema de la prueba de convergencia.
- **Optimizadores para modelos grandes (2023+)** — la línea que busca reducir el estado por parámetro que Adam necesita.

14. Notebook asociado

P41_adam.ipynb

Qué implementa: SGD y Adam sobre una cuadrática con número de condición 100, con los tres componentes explícitos, y la explicación de por qué el decaimiento de pesos necesita tratamiento aparte.

Qué NO implementa: ninguna red, ningún minilote, ningún paisaje no convexo. Es el mecanismo aislado.

```
ai-evolution paper-lab P41 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe las cuatro líneas del algoritmo y di qué hace cada una.
Explicar	Explica por qué el paso efectivo es del orden de η en toda coordenada.
Aplicar	Ejecuta el notebook y sube el número de condición a 10 000.
Analizar	Calcula $\hat{m}_1$ con y sin corrección de sesgo y compara.
Evaluar	¿Cuándo elegirías SGD con momento pese a la conveniencia de Adam?
Crear	Diseña un experimento que compare generalización, no solo velocidad de convergencia.

16. Autoevaluación

1. ¿Qué estima el segundo momento y para qué sirve?
2. ¿Por qué las medias móviles empiezan sesgadas?
3. ¿Qué problema de AdaGrad resuelve la media móvil?
4. ¿Por qué el decaimiento de pesos se comporta mal dentro de Adam?
5. ¿Cuánta memoria extra necesita por parámetro?
6. ¿Qué garantiza el análisis del paper y qué no?
7. ¿En qué caso puede convenir SGD?

17. Respuestas esperadas

1. La magnitud típica del gradiente de cada coordenada. Dividir por su raíz normaliza el paso, de modo que cada dirección avanza a un ritmo comparable.
2. Porque se inicializan a cero: las primeras iteraciones promedian con ese cero y subestiman el valor real. La corrección $1 - \beta^t$ lo compensa y desaparece al crecer t .
3. Que AdaGrad acumula **todos** los gradientes al cuadrado desde el inicio, así que el denominador solo crece y el paso efectivo tiende a cero. La media móvil olvida lo antiguo.
4. Porque el término de decaimiento se suma al gradiente y acaba dividido por $\sqrt{v}$: la fuerza de la regularización pasa a depender de la escala de gradiente de cada coordenada, que no es lo que se quería.
5. Dos estados por parámetro (m y v), es decir, aproximadamente el doble de memoria que los propios pesos.
6. Una cota de regret en el caso **convexo**. No garantiza nada en el no convexo, y la prueba original tuvo que corregirse.
7. En tareas donde la generalización importe más que la velocidad de convergencia y haya presupuesto para ajustar bien la tasa de aprendizaje y su planificación.

18. Fuentes primarias

- Kingma, D. P. y Ba, J. (2015). *Adam: A Method for Stochastic Optimization*. **ICLR 2015**. arXiv:1412.6980 · consultado 2026-08-16.
- Loshchilov, I. y Hutter, F. (2019). *Decoupled Weight Decay Regularization (AdamW)*. arXiv:1711.05101 · consultado 2026-08-16.

Evaluación — P41

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: Adam: *A Method for Stochastic Optimization* (2014, arXiv:1412.6980 · ICLR 2015) · **Nivel:** L2

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P41_adam.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P42 — Ejemplos adversarios

Arquitectura y entrenamiento · Una perturbación imperceptible cambia la predicción. Y la causa no es la profundidad: es la **linealidad** en dimensión alta.

Nivel: L3 · Motor: adversarial · Notebook: P42_adversarial.ipynb · Anexo: álgebra y geometría

1. Identificación

Campo	Valor
Título original	<i>Explaining and Harnessing Adversarial Examples</i>
Autoría	Ian J. Goodfellow, Jonathon Shlens, Christian Szegedy
Año	2014
Venue	arXiv:1412.6572 · ICLR 2015
Fuente primaria	arXiv:1412.6572
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Szegedy et al. (2013) habían descubierto algo desconcertante: perturbaciones minúsculas e invisibles al ojo humano hacían que una red clasificara mal con altísima confianza. La explicación que se manejaba era que las redes profundas son **extremadamente no lineales** y tienen bolsas de comportamiento errático repartidas por el espacio.

Esa hipótesis no explicaba dos observaciones incómodas: que los ejemplos adversarios **transfieran** entre modelos distintos, y que también afecten a modelos lineales sencillos.

3. Propuesta

La explicación contraria: el problema es que los modelos se comportan de forma **demasiado lineal** en dimensión alta.

Si la salida depende de $w^T x$ y perturbas cada componente en ϵ en la dirección del signo de w , el cambio total es $\epsilon \cdot \sum |w_i|$, que **crece con la dimensión**. Con miles de componentes, una perturbación invisible por píxel produce un cambio enorme en la salida.

De ahí sale un ataque de un solo paso —**FGSM**— y una defensa: entrenar con ejemplos adversarios generados sobre la marcha.

4. Intuición sin fórmulas

Mover una milésima cada uno de diez mil dígitos de una suma. Ninguna cifra cambia visiblemente, y el total se mueve diez unidades. La imperceptibilidad es **por componente**; el efecto es sobre la **suma**.

Dónde deja de funcionar la analogía: en una suma sabes exactamente cuánto contribuye cada sumando. Aquí lo que hace peligroso el ataque es que la dirección de máximo daño se puede calcular con un solo gradiente.

5. Matemática mínima

FGSM (Fast Gradient Sign Method):
$$x' = x + \epsilon \cdot \text{sign}(\nabla_x L(\theta, x, y))$$

Para un modelo lineal $w^T x$, el cambio en la salida es:

$$w^T(x' - x) = \epsilon \cdot \sum_i |w_i| = \epsilon \cdot \|w\|_1$$

→ crece linealmente con la DIMENSIÓN

dimensión	ϵ por componente	cambio en la salida
10	0,01	~0,1
784 (imagen 28×28)	0,01	~7,8
10 000	0,01	~100

La perturbación se mide en norma infinito —cuánto cambia **cada** componente— y el efecto se acumula en norma 1.

6. Arquitectura o flujo

7. Qué observar en el paper original

- El **argumento de la linealidad**, con el ejemplo del modelo lineal. Es corto y es toda la tesis.
- La famosa **figura del panda**: la imagen original, la perturbación amplificada y el resultado. Conviene mirar la escala real de la perturbación.
- La sección de **entrenamiento adversario** como regularizador, y cuánto cuesta en exactitud limpia.
- La discusión sobre **transferencia**: por qué el ataque generado contra un modelo funciona en otro. Es lo que convierte esto en un problema de seguridad y no en una curiosidad.

8. Evidencia y resultados

Experimentos sobre modelos lineales y redes en conjuntos de imágenes, midiendo la tasa de error bajo ataque y el efecto del entrenamiento adversario.

Las tasas por modelo y ϵ están en el artículo. Verificarlas allí: la exactitud robusta depende por completo del ataque usado y de su presupuesto, y una cifra sin esos dos datos no significa nada.

La miniatura de este eje aísla el argumento dimensional: con $\epsilon = 0,01$, el cambio en la salida pasa de $\sim 0,1$ en dimensión 10 a ~ 100 en dimensión 10 000.

9. Impacto

- Fundó el área de **aprendizaje automático adversario** como disciplina.
- El **entrenamiento adversario** sigue siendo la defensa de referencia, una década después.
- Cambió la forma de evaluar: la exactitud sobre la distribución natural dejó de considerarse suficiente en aplicaciones donde alguien puede atacar la entrada.
- Es un antecedente conceptual de los ataques a modelos de lenguaje: inyección de prompt y jailbreaks son la versión textual del mismo problema.

10. Limitaciones

1. **FGSM es un ataque débil**: un solo paso. Ataques iterativos como PGD son mucho más fuertes.
2. **El entrenamiento adversario cuesta exactitud limpia** y multiplica el coste de entrenamiento.
3. **Defenderse de un ataque concreto no es ser robusto**: muchas defensas publicadas cayeron ante ataques adaptativos.
4. **La explicación lineal es parcial**: describe bien el fenómeno pero no lo agota.
5. **La norma infinito es una elección**: hay amenazas que no encajan en ese modelo (rotaciones, parches físicos, cambios de iluminación).

11. Errores comunes

Error	Corrección
«Es un fallo de las redes profundas»	Afecta también a modelos lineales. La causa es la dimensión alta, no la profundidad.
«Exactitud alta = modelo robusto»	Son métricas distintas: una sobre la distribución natural, otra bajo entrada elegida por un atacante.
«Mi defensa funciona: baja la tasa de éxito de FGSM»	FGSM es débil. Hay que evaluar con ataques adaptativos que conozcan la defensa.
«La perturbación es invisible, luego es un truco de laboratorio»	Existen ataques físicos con parches y pegatinas. La invisibilidad no es un requisito del problema.
«Se resolvió»	Sigue abierto. La robustez certificada solo se consigue con garantías limitadas y a un coste alto.

12. Relación con trabajos anteriores

- **Szegedy et al. (2013)** — el descubrimiento original del fenómeno. [arXiv:1312.6199](#)
- **P04 AlexNet (2012)** — los modelos de visión sobre los que se demuestra.
- **P02 Backpropagation** — el gradiente respecto a la **entrada**, no a los pesos, es lo que hace posible el ataque.

13. Relación con trabajos posteriores

- **PGD y entrenamiento adversario robusto (2017)** — el ataque y la defensa de referencia actuales.
- **Robustez certificada (2019+)** — garantías demostrables, con un coste alto.
- **P52 Superposición** — otra vía para entender qué hay dentro del modelo y por qué se comporta así.
- **Inyección de prompt y jailbreaks** — la versión en lenguaje del mismo problema estructural.

14. Notebook asociado

P42_adversarial.ipynb

Qué implementa: el cálculo del cambio en la salida frente a la dimensión, para varios ϵ , y el protocolo de evaluación con exactitud limpia y robusta por separado.

Qué NO implementa: ninguna red ni ningún gradiente real. Se usa el signo de los pesos, que en el caso lineal coincide con el signo del gradiente.

```
ai-evolution paper-lab P42 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la fórmula de FGSM y di respecto a qué se deriva.
Explicar	Explica por qué el efecto crece con la dimensión.
Aplicar	Calcula el ϵ necesario para mover la salida 10 unidades en dimensión 784.
Analizar	¿Por qué la transferencia entre modelos es más preocupante que el ataque en sí?
Evaluar	Una defensa reporta 95 % de exactitud bajo FGSM. ¿Qué objetas?
Crear	Diseña un protocolo de evaluación de robustez que resista un ataque adaptativo.

16. Autoevaluación

1. ¿Cuál era la explicación anterior y por qué no encajaba?
2. ¿Respecto a qué se calcula el gradiente en FGSM?
3. ¿Por qué la perturbación es imperceptible y el efecto no?
4. ¿Qué es la transferencia y por qué importa?
5. ¿Qué cuesta el entrenamiento adversario?

6. ¿Por qué una defensa evaluada solo con FGSM no es creíble?
7. ¿Qué relación tiene esto con los jailbreaks de modelos de lenguaje?

17. Respuestas esperadas

1. Que las redes son extremadamente no lineales y tienen bolsas erráticas. No encajaba porque el fenómeno también afecta a modelos lineales y porque los ataques transfieren entre modelos.
2. Respecto a la **entrada**, no a los pesos. Es la misma retropropagación, usada para preguntar cómo cambiar la imagen en vez de cómo cambiar el modelo.
3. Porque la perturbación se acota **por componente** (norma infinito) y el efecto se acumula sobre **todas** las componentes: crece con la dimensión.
4. Que un ataque generado contra un modelo funciona contra otro entrenado por separado. Importa porque significa que no hace falta acceso al modelo objetivo para atacarlo.
5. Exactitud sobre datos limpios y coste de entrenamiento, porque hay que generar ejemplos adversarios en cada paso.
6. Porque FGSM es un ataque de un solo paso y muy débil. Una defensa puede vencerlo y caer ante un ataque iterativo o adaptativo que conozca la defensa.
7. Es el mismo problema estructural: una entrada cuidadosamente construida lleva al modelo fuera del comportamiento esperado, y la superficie de ataque crece con la expresividad de la entrada.

18. Fuentes primarias

- Goodfellow, I. J., Shlens, J. y Szegedy, C. (2015). *Explaining and Harnessing Adversarial Examples*. **ICLR 2015**. arXiv:1412.6572 · consultado 2026-08-16.
- Szegedy, C. et al. (2014). *Intriguing properties of neural networks*. arXiv:1312.6199 · consultado 2026-08-16.

Evaluación — P42

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Explaining and Harnessing Adversarial Examples* (2014, arXiv:1412.6572 · ICLR 2015) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P42_adversarial.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P43 — Batch Normalization

Arquitectura y entrenamiento · Normalizar las activaciones dentro de la red permite tasas de aprendizaje mucho mayores y hace el entrenamiento profundo mucho menos frágil.

Nivel: L2 · **Motor:** `batchnorm` · **Notebook:** `P43_batchnorm.ipynb` · **Anexo:** cálculo y gradientes

1. Identificación

Campo	Valor
Título original	<i>Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift</i>
Autoría	Sergey Ioffe, Christian Szegedy
Año	2015
Venue	arXiv:1502.03167 · ICML 2015
Fuente primaria	arXiv:1502.03167
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Entrenar redes profundas exigía tasas de aprendizaje pequeñas e inicializaciones cuidadosas. El motivo práctico era observable: la distribución de las activaciones de cada capa **se desplaza** durante el entrenamiento, porque los pesos de las capas anteriores cambian.

Con activaciones saturantes como sigmoide o tanh, eso es fatal: si las entradas de una capa se van a la zona plana, la derivada se acerca a cero y esa capa deja de aprender. El resultado era un entrenamiento lento y muy sensible a la configuración inicial.

3. Propuesta

Normalizar cada activación usando la media y la varianza **del minilote**, dentro de la red, como una capa más.

Y una pieza que evita perder capacidad expresiva: dos parámetros aprendidos, γ y β , que permiten reescalar y desplazar el resultado. Si a la red le conviene deshacer la normalización, puede hacerlo — pero ahora es una decisión aprendida, no un accidente.

En inferencia no hay minilote, así que se usan estadísticas acumuladas durante el entrenamiento.

4. Intuición sin fórmulas

Una tubería de doce filtros donde cada uno amplifica un poco. Al final, la señal está saturada o extinguida. Normalizar entre etapas es reajustar el caudal para que cada filtro reciba lo que sabe procesar.

Dónde deja de funcionar la analogía: el caudal se mide sobre el **lote**, no sobre cada muestra. Eso significa que la salida para una muestra depende de con quién le tocó viajar, algo que no tiene equivalente en la tubería y que es la raíz de sus problemas prácticos.

5. Matemática mínima

Sobre el minilote B, para cada activación:

$$\mu_B = (1/m) \sum x_i$$

$$\sigma^2_B = (1/m) \sum (x_i - \mu_B)^2$$

$$\hat{x}_i = (x_i - \mu_B) / \sqrt{(\sigma^2_B + \epsilon)}$$

normalizar

$$y_i = \gamma \cdot \hat{x}_i + \beta$$

reescalar y desplazar (APRENDIDOS)

Inferencia: μ y σ^2 son estadísticas acumuladas (media móvil), no del lote.

El ϵ no es decorativo: con un lote donde todas las activaciones sean iguales, $\sigma^2 = 0$ y la división explota. Ese epsilon evita un NaN que se propagaría a toda la red.

6. Arquitectura o flujo

7. Qué observar en el paper original

- La **definición de «internal covariate shift»** y cómo la miden. Esta parte es la que envejeció peor, y merece leerse sabiéndolo.
- Que la normalización se aplica **antes** de la no linealidad, y por qué.
- El experimento donde eliminan el **dropout** al añadir batch norm: sugiere que tiene un efecto regularizador propio, por el ruido que introduce la estadística del lote.
- Los resultados con **tasas de aprendizaje mucho mayores**: ese es el beneficio práctico principal y el más fácil de verificar.

8. Evidencia y resultados

Experimentos en clasificación de imágenes mostrando convergencia mucho más rápida con la misma arquitectura, tolerancia a tasas de aprendizaje mayores y menor dependencia de la inicialización.

Las curvas y el número de pasos hasta una exactitud dada están en el artículo. Verificarlos allí. La aceleración reportada es grande y es lo que explica su adopción inmediata.

La miniatura de este eje muestra el mecanismo: sin normalizar, la fracción de activaciones saturadas en la capa 11 es muy alta; con normalización, la desviación se mantiene cerca de 1 y las unidades siguen en su rango útil.

9. Impacto

- Adopción prácticamente universal en visión durante media década.
- Hizo entrenables arquitecturas mucho más profundas, y es una de las piezas que permitieron ResNet ese mismo año.
- Abrió la familia de técnicas de normalización: LayerNorm —la que usa el Transformer—, GroupNorm, InstanceNorm, RMSNorm.
- Y es un caso de estudio sobre la diferencia entre **que algo funcione** y **que la explicación ofrecida sea correcta**.

10. Limitaciones

1. **Depende del tamaño de lote:** con lotes muy pequeños las estadísticas son ruido y el método se degrada. Es su punto débil práctico.
2. **Comportamiento distinto en entrenamiento e inferencia**, lo que causa fallos sutiles si las estadísticas acumuladas no se gestionan bien.
3. **Complica el paralelismo de datos:** las estadísticas son por dispositivo salvo que se sincronicen, con su coste.
4. **La explicación original fue cuestionada:** Santurkar et al. (2018) argumentan que el beneficio viene de suavizar el paisaje de optimización, no de reducir el desplazamiento de covariables.
5. **Interacción no trivial con dropout**, que produjo errores frecuentes al combinarlos.
6. **No encaja bien en secuencias de longitud variable**, motivo por el que el Transformer usa LayerNorm.

11. Errores comunes

Error	Corrección
«Funciona porque reduce el internal covariate shift»	Es la explicación del paper y fue discutida después . Que la técnica funcione no valida la explicación.
Olvidar poner el modelo en modo evaluación	En inferencia se usan estadísticas acumuladas. Si se dejan las del lote, la salida depende de con quién viaje la muestra.
Usarlo con lotes de 1 o 2	La estadística es ruido puro. Para eso están GroupNorm o LayerNorm.
« γ y β son opcionales»	Sin ellos la red pierde capacidad expresiva: no podría representar una activación con media distinta de 0.
«Es lo mismo que normalizar la entrada»	Normalizar la entrada se hace una vez; esto ocurre en cada capa y en cada paso , con estadísticas que cambian.

12. Relación con trabajos anteriores

- **Po2 Backpropagation (1986)** — el gradiente que se intenta mantener vivo.
- **Po4 AlexNet (2012)** — su normalización de respuesta local es el antecedente que este método deja obsoleto.
- **Inicialización de Glorot (2010) y He (2015)** — la vía alternativa al mismo problema.

13. Relación con trabajos posteriores

- **P44 ResNet (2015)** — la usa en cada bloque.
- **LayerNorm (2016)** y **Po8 Transformer (2017)** — normalización por muestra, que no depende del lote.
- **Santurkar et al. (2018)** — la revisión de la explicación. [arXiv:1805.11604](https://arxiv.org/abs/1805.11604)

14. Notebook asociado

P43_batchnorm.ipynb

Qué implementa: la propagación de activaciones por doce capas con y sin normalización, midiendo media, desviación y fracción saturada; el error relativo de la estadística según el tamaño de lote; y la comparación con LayerNorm y GroupNorm.

Qué NO implementa: no hay entrenamiento, así que no se mide el efecto real sobre la convergencia — que es el beneficio principal del método.

```
ai-evolution paper-lab P43 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe las cuatro líneas del algoritmo y di qué se aprende.
Explicar	Explica por qué γ y β no anulan el propósito de normalizar.
Aplicar	Ejecuta el notebook y observa la fracción saturada por capa.
Analizar	¿Por qué el error de la estadística escala como $1/\sqrt{lote}$?
Evaluar	«Funciona porque reduce el covariate shift». Evalúa la afirmación.
Crear	Diseña un experimento que distinga aceleración de convergencia de efecto regularizador.

16. Autoevaluación

1. ¿Sobre qué se calculan la media y la varianza, y qué cambia en inferencia?
2. ¿Para qué sirven γ y β ?
3. ¿Por qué hace falta el ϵ y qué pasa sin él?
4. ¿Por qué falla con lotes pequeños?
5. ¿Cuál es el beneficio práctico más claro y verificable?
6. ¿Por qué el Transformer usa LayerNorm en vez de esto?
7. ¿Qué parte del paper envejeció peor?

17. Respuestas esperadas

1. Sobre el **minilote** durante el entrenamiento. En inferencia no hay lote, así que se usan estadísticas acumuladas por media móvil.
2. Permiten reescalar y desplazar el resultado normalizado. Sin ellos la red no podría representar activaciones con media o escala distintas de la normalizada, y perdería capacidad expresiva.
3. Para evitar dividir por cero cuando la varianza del lote es nula. Sin él, un lote de valores idénticos produce un NaN que se propaga a toda la red.
4. Porque la media y la varianza estimadas sobre pocas muestras son ruido: el error relativo va como $1/\sqrt{lote}$, y con lote 2 la normalización deja de tener sentido.
5. Que permite usar tasas de aprendizaje mucho mayores y reduce la dependencia de la inicialización. Es fácil de comprobar barriando tasas con y sin normalización.
6. Porque las secuencias tienen longitud variable y el tamaño de lote efectivo por posición cambia. LayerNorm normaliza por muestra y no depende del lote.
7. La explicación —el «internal covariate shift»—. Trabajo posterior argumenta que el beneficio viene de suavizar el paisaje de optimización.

18. Fuentes primarias

- Ioffe, S. y Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift*. **ICML 2015**. arXiv:1502.03167 · consultado 2026-08-16.
- Santurkar, S. et al. (2018). *How Does Batch Normalization Help Optimization?* arXiv:1805.11604 · consultado 2026-08-16.

Evaluación — P43

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift* (2015, arXiv:1502.03167 · ICML 2015) · **Nivel:** L2

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P43_batchnorm.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P44 — ResNet

Arquitectura y entrenamiento · Un atajo que suma la entrada a la salida convierte la profundidad de obstáculo en recurso.

Nivel: L2 · Motor: `resnet` · Notebook: `P44_resnet.ipynb` · Anexo: cálculo y gradientes

1. Identificación

Campo	Valor
Título original	<i>Deep Residual Learning for Image Recognition</i>
Autoría	Kaiming He, Xiangyu Zhang, Shaoqing Ren, Jian Sun
Año	2015
Venue	arXiv:1512.03385 · CVPR 2016
Fuente primaria	arXiv:1512.03385
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Una observación que no cuadraba con nada: al añadir capas, el error de **entrenamiento** empeoraba.

No era sobreajuste —eso empeora el error de test, no el de entrenamiento— y no era falta de capacidad —una red de 56 capas contiene a una de 20 más capas identidad, así que **como mínimo** debería igualarla—. El paper llama a esto **degradación**, y su conclusión es incómoda: el problema no estaba en el modelo, sino en la optimización. El descenso de gradiente no encontraba esa solución trivial.

3. Propuesta

Cambiar lo que cada bloque tiene que aprender. En vez de pedirle la transformación completa $H(x)$, se le pide el **residuo**:

$$y = F(x) + x$$

Si lo óptimo para ese bloque es no hacer nada, basta con que F se acerque a cero —y llevar pesos a cero es fácil. La identidad deja de ser una solución difícil de alcanzar y pasa a ser el punto de partida.

El atajo tiene además un efecto sobre el gradiente: la derivada de cada bloque es $1 + F'(x)$, y ese 1 sostiene el producto al retropropagar aunque los términos residuales sean pequeños.

4. Intuición sin fórmulas

Corregir un texto ajeno frente a reescribirlo entero. Si el original ya está bien, corregir es no tocar nada; reescribir obliga a reproducirlo de memoria y siempre sale peor.

Dónde deja de funcionar la analogía: el corrector ve el texto y decide. La red no decide: es que la parametrización hace que «no tocar nada» sea el estado más barato al que llegar.

5. Matemática mínima

Bloque plano : $y = H(x)$ → aprender H desde cero
Bloque residual: $y = F(x) + x$ → aprender solo la CORRECCIÓN

Al retropropagar:
 $\partial y / \partial x = I + \partial F / \partial x$

Producto sobre L bloques:
plano : $\prod f'_i$ → si $f'_i < 1$, colapsa exponencialmente
residual : $\prod (1 + F'_i)$ → el 1 sostiene el producto

El notebook lo hace explícito con factores fijados a mano: con 152 capas y un factor de 0,85 por capa, el gradiente sin atajo cae a **1,9e-11**; con $1 + (-0,02)$ por bloque, se queda en **4,6e-02**. Nueve órdenes de magnitud de diferencia entre desaparecer y sobrevivir.

6. Arquitectura o flujo

7. Qué observar en el paper original

- La **figura de la degradación**: 20 capas frente a 56, con el error de **entrenamiento** peor en la más profunda. Es el dato que motiva todo y conviene mirarlo antes que la solución.
- Que el atajo es **identidad y sin parámetros** en el caso general. Añade cero coste.
- El **bloque cuello de botella** (1×1, 3×3, 1×1) para las variantes de 50 capas en adelante, y por qué reduce el cómputo.
- Los resultados a **152 capas**: no solo entrena, sino que mejora — que es justo lo que la degradación impedía.

8. Evidencia y resultados

Resultados en clasificación de imágenes a gran escala y en detección, con redes de 18, 34, 50, 101 y 152 capas, comparadas contra sus equivalentes planas.

Las cifras de error por profundidad están en el artículo. Verificarlas allí. Lo relevante es el patrón: en las planas el error sube con la profundidad y en las residuales baja.

La miniatura de este eje aísla el mecanismo del gradiente, con factores puestos a mano. No mide exactitud ni entrena nada: muestra por qué el producto no colapsa.

9. Impacto

- Es, con el Transformer, una de las dos arquitecturas más citadas del campo.
- La conexión residual está **dentro** del Transformer: cada subcapa es $x + \text{Sublayer}(x)$. Sin ella, los modelos de lenguaje profundos no entrenarían.
- Cambió el techo práctico de profundidad de decenas a cientos de capas.
- Y reencuadró un problema: lo que parecía un límite de capacidad era un límite de optimización.

10. Limitaciones

1. **No elimina la necesidad de normalización:** sin BatchNorm y buena inicialización, la profundidad sigue siendo difícil.
2. **Más profundidad no es gratis:** más cómputo, más memoria de activaciones, más latencia.
3. **Rendimientos decrecientes:** de 152 a 1000 capas el paper reporta problemas, no mejoras proporcionales.
4. **La explicación de por qué funciona siguió discutiéndose:** hay trabajo que lo describe como un ensamblado implícito de rutas de distinta longitud (Veit et al., 2016).
5. **El atajo obliga a que las dimensiones cuadren;** cuando cambian hace falta una proyección, que ya no es gratuita.

11. Errores comunes

Error	Corrección
«Resuelve el gradiente que se desvanece»	Ayuda mucho, pero el problema que motiva el paper es la degradación del error de entrenamiento , que es otra cosa.
«El problema era que faltaba capacidad»	Al revés: la red profunda contiene a la superficial. Era la optimización la que no llegaba.
«El atajo tiene pesos»	En el caso general es la identidad, sin parámetros. Solo se proyecta cuando cambian las dimensiones.
«Es específico de visión»	La conexión residual es hoy universal: está en cada bloque de cada Transformer.
«Más capas siempre mejor»	El paper mismo muestra que el retorno decrece y que a profundidades extremas reaparecen problemas.

12. Relación con trabajos anteriores

- **P43 BatchNorm (2015)** — se usa dentro de cada bloque; las dos piezas juntas son lo que hace entrenable la profundidad.
- **P04 AlexNet (2012)** y **VGG (2014)** — la línea de «más profundo es mejor» que aquí choca contra un muro.
- **P03 LSTM (1997)** — la misma idea en el tiempo: un camino aditivo por el que el gradiente viaja sin atenuarse.

13. Relación con trabajos posteriores

- **P08 Transformer (2017)** — $x + \text{Sublayer}(x)$ en cada subcapa.
- **DenseNet (2016)** — lleva la idea al extremo conectando cada capa con todas las siguientes.
- **Veit et al. (2016)** — la lectura de las redes residuales como ensamblado de rutas. [arXiv:1605.06431](https://arxiv.org/abs/1605.06431)
- **P46 Vision Transformer (2020)** — el sucesor que discute su hegemonía en visión, y que también usa residuos.

14. Notebook asociado

P44_resnet.ipynb

Qué implementa: el producto de derivadas a 10, 20, 50 y 152 capas, con y sin atajo, para mostrar cuándo colapsa el gradiente.

Qué NO implementa: no hay convoluciones, ni datos, ni entrenamiento. Los factores están fijados a mano para aislar el efecto; el aporte experimental del paper —entrenar 152 capas de verdad— no está aquí.

```
ai-evolution paper-lab P44 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la ecuación del bloque residual y di qué aprende F .
Explicar	Explica por qué la degradación no es sobreajuste.
Aplicar	Ejecuta el notebook y cambia el factor por capa a 0,95.
Analizar	Deriva $\partial y / \partial x$ y explica el papel del término 1 .
Evaluar	«Una red de 56 capas debería ser al menos tan buena como una de 20». Evalúa el argumento.
Crear	Diseña una variante del atajo para cuando cambian las dimensiones y justifica el coste.

16. Autoevaluación

1. ¿Qué es la degradación y por qué no es sobreajuste?
2. ¿Qué aprende un bloque residual que no aprende uno plano?

3. ¿Por qué la identidad es fácil de alcanzar con el atajo y difícil sin él?
4. ¿Cuál es la derivada de un bloque residual y por qué importa?
5. ¿El atajo tiene parámetros?
6. ¿Dónde aparece esta idea fuera de visión?
7. ¿Qué no resuelve el atajo?

17. Respuestas esperadas

1. Que al añadir capas empeora el error de **entrenamiento**. El sobreajuste empeora el de test mientras el de entrenamiento baja: es un síntoma distinto y apunta a la optimización.
2. La **corrección** sobre su entrada, no la transformación completa. Si lo óptimo es no hacer nada, le basta con llevar F a cero.
3. Porque con atajo la identidad se obtiene con $F = 0$, que es el estado más barato. Sin atajo, la red tendría que aprender explícitamente la transformación identidad con sus pesos.
4. $I + \partial F / \partial x$. Al multiplicar sobre muchos bloques, ese 1 impide que el producto colapse exponencialmente hacia cero.
5. No, en el caso general es la identidad y no añade ningún coste. Solo hace falta una proyección cuando cambian las dimensiones entre entrada y salida.
6. En el Transformer: cada subcapa de atención y cada red feed-forward está envuelta en $x + \text{Sublayer}(x)$. Es una pieza universal en modelos profundos.
7. La necesidad de normalización y de una inicialización razonable. Y no hace que más profundidad sea siempre mejor: el retorno decrece.

18. Fuentes primarias

- He, K., Zhang, X., Ren, S. y Sun, J. (2016). *Deep Residual Learning for Image Recognition*. **CVPR 2016**. arXiv:1512.03385 · consultado 2026-08-16.
- Veit, A., Wilber, M. y Belongie, S. (2016). *Residual Networks Behave Like Ensembles of Relatively Shallow Networks*. arXiv:1605.06431 · consultado 2026-08-16.

Evaluación — P44

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Deep Residual Learning for Image Recognition* (2015, arXiv:1512.03385 · CVPR 2016) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P44_resnet.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P45 — Destilación de conocimiento

Arquitectura y entrenamiento · Lo que un modelo grande sabe no está en su respuesta, está en **cómo reparte** la probabilidad entre las respuestas que descartó.

Nivel: L2 · **Motor:** `distillation` · **Notebook:** `P45_distillation.ipynb` · **Anexo:** probabilidad y verosimilitud

1. Identificación

Campo	Valor
Título original	<i>Distilling the Knowledge in a Neural Network</i>
Autoría	Geoffrey Hinton, Oriol Vinyals, Jeff Dean
Año	2015
Venue	arXiv:1503.02531 · NIPS 2014 Deep Learning Workshop
Fuente primaria	arXiv:1503.02531
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Los mejores resultados venían de modelos enormes o de ensamblados de muchos modelos. Servir eso en producción —con latencia y coste acotados— era inviable.

La salida obvia, entrenar un modelo pequeño con las mismas etiquetas, daba resultados claramente peores. Y había una asimetría desaprovechada: el modelo grande produce, para cada entrada, una distribución completa sobre las clases, y de todo eso solo se usaba el `argmax`.

3. Propuesta

Entrenar el modelo pequeño contra la **distribución** del grande, no contra la etiqueta.

Una etiqueta dura dice «perro» y asigna 0 a lobo, a gato y a coche por igual. La distribución del maestro dice «perro, pero se parece bastante a un lobo, algo a un gato y nada a un coche». Esa estructura de similitud —el paper la llama **conocimiento oscuro**— es información que la etiqueta tira a la basura.

Para hacerla visible se aplica una **temperatura** `T` al softmax: subirla aplanar la distribución y saca a la luz las probabilidades pequeñas, que son justo las informativas.

4. Intuición sin fórmulas

Un examen corregido con solo «bien/mal» frente a uno donde el profesor anota qué alternativas estuviste a punto de marcar y cuáles ni te rozaron. Lo segundo enseña mucho más rápido.

Dónde deja de funcionar la analogía: el profesor sabe la verdad. El maestro aquí solo tiene sus creencias, aciertos y errores incluidos — y el alumno hereda ambos.

5. Matemática mínima

Softmax con temperatura:

$$p_i = \exp(z_i / T) / \sum_j \exp(z_j / T)$$

$T = 1$ → distribución habitual

$T > 1$ → más plana, más entropía, más estructura visible

Pérdida del alumno:

$$L = \alpha \cdot \text{CE}(\text{alumno}_T, \text{maestro}_T) \cdot T^2 + (1 - \alpha) \cdot \text{CE}(\text{alumno}_1, \text{etiqueta})$$

El factor T^2 no es opcional: los gradientes del término suave escalan como $1/T^2$, y sin compensarlo el peso relativo de los dos términos cambiaría al mover la temperatura.

Con logits $[4, 0 \cdot 2, 0 \cdot 1, 5 \cdot -1, 0]$ para perro, lobo, gato y coche:

T	perro	lobo	gato	coche	entropía
1	0,817	0,111	0,067	0,006	0,619
5	0,378	0,254	0,229	0,139	1,328
10	0,312	0,256	0,243	0,189	1,371

El orden `lobo > gato > coche` está siempre; la temperatura solo lo hace **utilizable** como señal de gradiente.

6. Arquitectura o flujo

7. Qué observar en el paper original

- El **ejemplo del 2 y el 7**: por qué la probabilidad que el maestro asigna a la clase equivocada contiene información sobre la geometría del problema.
- La **derivación del factor T^2** y la relación con entrenar contra logits directamente.

- El experimento donde el alumno aprende clases que **casi no vio** en su conjunto de transferencia: es el resultado más sorprendente del artículo.
- Los **modelos especialistas** para conjuntos con muchísimas clases, una parte del paper que casi nadie cita.

8. Evidencia y resultados

Experimentos en reconocimiento de dígitos, reconocimiento de voz a gran escala y un conjunto interno con miles de clases, comparando el alumno destilado contra el mismo alumno entrenado solo con etiquetas.

Las cifras están en el artículo. Verificarlas allí. Lo que hay que retener es el patrón: el alumno destilado se acerca al maestro mucho más que el alumno entrenado con etiquetas duras.

La miniatura de este eje no entrena nada: exhibe la información que contiene la distribución del maestro y cómo la temperatura la revela.

9. Impacto

- Es el fundamento de casi todos los modelos pequeños de uso práctico: DistilBERT, TinyBERT y la familia entera de modelos «mini» derivados de otros grandes.
- Se convirtió en pieza estándar del pipeline de despliegue, junto a la cuantización y la poda.
- En la era de los LLM, entrenar con salidas de un modelo mayor es una práctica generalizada — y una fuente de disputas sobre licencias y términos de servicio.
- Sostiene también el argumento de que **la capacidad no siempre necesita el tamaño** que se usó para descubrirla.

10. Limitaciones

1. **El alumno hereda los errores del maestro**, y también sus sesgos: la destilación transfiere comportamiento, no corrección.
2. **Sigue haciendo falta el maestro**: hay que entrenarlo y ejecutarlo sobre todo el conjunto de transferencia.
3. **Techo de capacidad**: si el alumno es demasiado pequeño, no hay temperatura que lo salve.
4. **T y α son hiperparámetros más** que ajustar.
5. **Explicación incompleta**: por qué funciona tan bien sigue siendo objeto de estudio; hay trabajo que lo atribuye tanto a la regularización como a la transferencia de similitud.
6. **Cuestiones de licencia**: destilar de un modelo ajeno puede violar sus términos de uso.

11. Errores comunes

Error	Corrección
«El alumno copia al maestro»	Aprende su distribución , que es una señal mucho más rica y suave que copiar predicciones.
«La temperatura crea información»	La información ya está en los logits. La temperatura solo la hace visible al gradiente.
Olvidar el factor T^2	Los gradientes del término suave escalan como $1/T^2$. Sin compensar, cambiar T cambia el equilibrio entre los dos términos.
«Un alumno destilado es tan bueno como el maestro»	Se acerca mucho más que sin destilar, pero hay un techo impuesto por su capacidad.
«Sirve para corregir al maestro»	Al contrario: le transfiere también sus errores y sesgos.

12. Relación con trabajos anteriores

- **Buciluă, Caruana y Niculescu-Mizil (2006)** — comprimir un ensamblado en un modelo único; el antecedente directo.
- **P04 AlexNet (2012)** — los modelos grandes cuyo despliegue motiva el problema.
- **Ensamblados** — la técnica cara que se busca comprimir.

13. Relación con trabajos posteriores

- **DistilBERT (2019)** — la aplicación más citada, sobre BERT.
- **P48 LoRA (2021)** y **P49 QLoRA (2023)** — la otra vía para reducir coste: no achicar el modelo, sino su ajuste y sus bits.
- **Destilación desde LLM (2023+)** — entrenar modelos pequeños con salidas de modelos grandes, con el debate de licencias que arrastra.

14. Notebook asociado

P45_distillation.ipynb

Qué implementa: la distribución del maestro a cuatro temperaturas, su entropía, y la comparación explícita contra la etiqueta dura.

Qué NO implementa: no hay maestro ni alumno entrenados, ni conjunto de transferencia, ni el factor T^2 en una pérdida real. Se exhibe la información, no se destila.

```
ai-evolution paper-lab P45 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe el softmax con temperatura y di qué hace $T > 1$.
Explicar	Explica qué es el conocimiento oscuro con el ejemplo perro/lobo/coche.
Aplicar	Ejecuta el notebook y añade $T = 20$. ¿Qué pasa con la entropía?
Analizar	¿Por qué la etiqueta dura y la distribución del maestro no son intercambiables?
Evaluar	Un maestro tiene un sesgo conocido. ¿Qué implica destilarlo?
Crear	Diseña un protocolo para destilar sin heredar un sesgo identificado.

16. Autoevaluación

1. ¿Qué información tira la etiqueta dura?
2. ¿Qué hace la temperatura y por qué hace falta?
3. ¿Por qué el término suave se multiplica por T^2 ?
4. ¿El alumno puede superar al maestro?
5. ¿Qué hereda el alumno además de la capacidad?
6. ¿Qué relación tiene esto con los modelos «mini» actuales?
7. ¿Qué problema legal puede plantear destilar?

17. Respuestas esperadas

1. La estructura de similitud entre clases: que un perro se parece a un lobo y nada a un coche. La etiqueta asigna o a todo lo que no es la clase correcta, sin distinguir entre esos ceros.
2. Aplana la distribución y aumenta su entropía, de modo que las probabilidades pequeñas —las que contienen la estructura de similitud— tengan magnitud suficiente para producir gradiente útil.
3. Porque los gradientes de ese término escalan como $1/T^2$. Sin el factor, subir la temperatura reduciría de facto el peso del término suave frente al de la etiqueta dura.
4. En general no lo supera; se acerca mucho más de lo que lograría entrenando solo con etiquetas. Su capacidad impone un techo.
5. Los errores y los sesgos del maestro. La destilación transfiere comportamiento, no corrección.
6. Es su mecanismo: DistilBERT y toda la familia de modelos pequeños de uso práctico se obtienen destilando un modelo mayor.
7. Que los términos de uso del modelo maestro pueden prohibir usar sus salidas para entrenar otro modelo. Es objeto de disputa activa.

18. Fuentes primarias

- Hinton, G., Vinyals, O. y Dean, J. (2015). *Distilling the Knowledge in a Neural Network*. arXiv:1503.02531 · consultado 2026-08-16.
- Buciluă, C., Caruana, R. y Niculescu-Mizil, A. (2006). *Model Compression*. **KDD 2006**. dl.acm.org/doi/10.1145/1150402.1150464 · consultado 2026-08-16.

Evaluación — P45

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Distilling the Knowledge in a Neural Network* (2015, arXiv:1503.02531) · **Nivel:** L2

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P45_distillation.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P46 — Vision Transformer

Arquitectura y entrenamiento · Trocea la imagen en parches y trátalos como palabras. La convolución deja de ser obligatoria en visión.

Nivel: L3 · Motor: vit · Notebook: P46_vit.ipynb · Anexo: complejidad y coste

1. Identificación

Campo	Valor
Título original	<i>An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale</i>
Autoría	Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov y otros
Año	2020
Venue	arXiv:2010.11929 · ICLR 2021
Fuente primaria	arXiv:2010.11929
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Desde AlexNet, la convolución era el supuesto de fondo de toda la visión por computador. Su fuerza está en tres **sesgos inductivos** que trae de fábrica: localidad —los píxeles cercanos importan juntos—, equivarianza a la traslación —un gato es un gato esté donde esté— y jerarquía espacial.

Mientras tanto, en lenguaje el Transformer había desplazado a las recurrentes por completo. La pregunta natural quedaba abierta: ¿esos sesgos son **necesarios**, o simplemente muy útiles cuando hay pocos datos?

3. Propuesta

Aplicar un Transformer casi sin modificar a la imagen. Se parte en parches fijos —16×16 píxeles—, cada parche se aplana y se proyecta linealmente a un vector, se le suma una codificación posicional y se le añade un token de clase. A partir de ahí, es literalmente el codificador del Transformer.

Ningún sesgo inductivo espacial. Ni localidad, ni equivarianza, ni jerarquía: solo el orden que inyecta la codificación posicional.

Y el resultado que da nombre al artículo — *at scale*: con datos suficientes, la ausencia de sesgos deja de ser un problema y se convierte en ventaja, porque el modelo aprende de los datos la estructura que la convolución imponía de antemano.

4. Intuición sin fórmulas

Un rompecabezas donde te dan las piezas numeradas por su posición. Nadie te dice que las piezas contiguas se relacionan más: lo descubres mirando muchísimos rompecabezas.

Dónde deja de funcionar la analogía: con pocos rompecabezas, quien ya sabe que las piezas vecinas encajan gana siempre. Ese «ya sabe» es exactamente el sesgo inductivo de la convolución, y por eso la CNN gana en régimen de datos medianos.

5. Matemática mínima

Imagen H×W×C, parche P×P
N = (H/P) · (W/P) parches
cada uno se aplanan a P²·C y se proyecta a dimensión d

 $z_o = [x_{clase} ; x^1E ; x^2E ; \dots ; x^NE] + E_{pos}$

Coste de la atención: O(N²)

imagen	parche	tokens (+clase)	coste de atención relativo
224×224	32×32	50	0,06
224×224	16×16	197	1,00
384×384	16×16	577	8,58
512×512	16×16	1025	27,07

Bajar el parche de 32 a 16 multiplica el coste por **17**: la resolución se paga al cuadrado. Ese es el compromiso central de la arquitectura.

6. Arquitectura o flujo

7. Qué observar en el paper original

- La **curva por tamaño de preentrenamiento**: es el resultado de verdad. Con conjuntos medianos la CNN gana; a partir de cierta escala, se invierte. Todo el artículo depende de esa figura.
- Que la arquitectura es **deliberadamente aburrida**: usan el Transformer estándar para que el resultado sea atribuible a la escala y no a un truco arquitectónico.
- Las **visualizaciones de atención**: en capas bajas atiende localmente aunque nadie se lo pidió. Redescubre la localidad.

- La discusión sobre la **codificación posicional 2D frente a 1D**, y por qué apenas importa.

8. Evidencia y resultados

Preentrenamiento en conjuntos de imágenes de tamaño creciente —del orden de un millón, diez millones y cientos de millones de imágenes— con evaluación por transferencia a varios conjuntos de clasificación.

Las cifras por conjunto y por tamaño de preentrenamiento están en el artículo. Verificarlas allí. El punto de cruce entre CNN y ViT es el dato que hay que leer, no el máximo absoluto.

La miniatura de este eje cuenta tokens y coste de atención: hace tangible por qué la resolución es cara y qué se gana bajando el tamaño de parche.

9. Impacto

- Unificó las arquitecturas de visión y lenguaje, lo que hizo posible que un mismo modelo procese ambas modalidades — la base de todo lo multimodal actual.
- Es el codificador visual de CLIP y de la mayoría de modelos de visión-lenguaje.
- Abrió la línea de preentrenamiento autosupervisado en imagen (MAE, DINO).
- Y validó una tesis más general del campo: los sesgos inductivos son un sustituto de los datos, y con datos suficientes, dejan de compensar.

10. Limitaciones

1. **Hambre de datos:** sin preentrenamiento a gran escala, una CNN comparable lo supera.
2. **Coste cuadrático en el número de parches**, que hace cara la alta resolución.
3. **Los parches son una rejilla rígida:** parten objetos por la mitad sin miramiento.
4. **Menos eficiente en datos por diseño:** lo que la convolución sabe gratis, aquí hay que aprenderlo.
5. **La comparación con CNN depende del régimen** de datos y de cómputo: no hay un ganador absoluto, y el propio paper es explícito en esto.
6. **Difícil en tareas densas** (segmentación, detección) sin modificaciones, que llegaron después con arquitecturas jerárquicas como Swin.

11. Errores comunes

Error	Corrección
«ViT es mejor que las CNN»	Solo por encima de cierta escala de preentrenamiento. Por debajo, la CNN gana — y el paper lo muestra.
«No tiene ningún sesgo inductivo»	Tiene uno: la partición en parches impone una rejilla. Lo que no tiene son localidad ni equivarianza.
«Es un Transformer adaptado a imagen»	Es lo contrario: la aportación es que no hace falta adaptarlo. La adaptación está en la entrada.
«El token de clase es imprescindible»	Es una elección heredada de BERT. El promediado de parches funciona de forma comparable.
«Parches más pequeños siempre mejor»	Mejora la resolución y multiplica el coste al cuadrado: de parche 32 a 16, $\times 17$.

12. Relación con trabajos anteriores

- **Po8 Transformer (2017)** — la arquitectura, usada casi tal cual.
- **Po9 BERT (2018)** — de donde viene el token de clase y el esquema de preentrenar y transferir.
- **Po4 AlexNet (2012)** y **P44 ResNet (2015)** — el paradigma convolucional que se cuestiona.
- **P19 Leyes de escalado (2020)** — el marco que explica por qué «a escala» es el argumento central.

13. Relación con trabajos posteriores

- **P18 CLIP (2021)** — usa un ViT como codificador de imagen.
- **Swin Transformer (2021)** — reintroduce jerarquía y ventanas locales para tareas densas.
- **MAE y DINO (2021)** — preentrenamiento autosupervisado sobre ViT.
- **Modelos multimodales (2022+)** — la unificación arquitectónica que este paper hace posible.

14. Notebook asociado

P46_vit.ipynb

Qué implementa: el conteo de tokens y el coste relativo de atención para cuatro configuraciones de imagen y parche, y la comparación explícita de sesgos inductivos entre CNN y ViT.

Qué NO implementa: no hay imagen, ni parches reales, ni modelo, ni entrenamiento. El resultado central del paper —qué pasa al preentrenar a escala— no es reproducible aquí.

```
ai-evolution paper-lab P46 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Enumera los tres sesgos inductivos de la convolución.
Explicar	Explica por qué ViT necesita más datos que una CNN.
Aplicar	Calcula los tokens de una imagen de 640×640 con parches de 16.
Analizar	¿Por qué el coste crece al cuadrado al bajar el tamaño de parche?
Evaluar	«Las CNN quedaron obsoletas». Evalúa con el régimen de datos en la mano.
Crear	Diseña un esquema de parcheado que reduzca el coste sin perder resolución efectiva.

16. Autoevaluación

1. ¿Cómo se convierte una imagen en una secuencia?
2. ¿Qué sesgos inductivos pierde ViT y qué gana a cambio?
3. ¿Cuántos tokens produce una imagen de 224×224 con parches de 16?
4. ¿Por qué la alta resolución es cara?
5. ¿En qué régimen gana la CNN?
6. ¿Qué redescubre el modelo en sus capas bajas?
7. ¿Por qué este paper importa para lo multimodal?

17. Respuestas esperadas

1. Se parte en parches de tamaño fijo, cada uno se aplanan y se proyecta linealmente a un vector, y se le suma una codificación posicional. Se añade un token de clase al principio.
2. Pierde localidad, equivarianza a la traslación y jerarquía espacial. Gana capacidad de modelar relaciones a larga distancia desde la primera capa y de escalar mejor con datos.
3. 196 parches más el token de clase: 197.
4. Porque el coste de la atención es cuadrático en el número de tokens, y el número de tokens crece con el cuadrado del lado de la imagen dividido por el del parche.
5. Con conjuntos de datos pequeños o medianos, donde sus sesgos inductivos sustituyen a datos que no existen.
6. La localidad: las visualizaciones muestran que en capas bajas la atención se concentra en parches cercanos, aunque nada en la arquitectura se lo imponga.
7. Porque unifica la arquitectura: si imagen y texto son secuencias de vectores procesadas por el mismo bloque, un solo modelo puede consumir ambas.

18. Fuentes primarias

- Dosovitskiy, A. et al. (2021). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. **ICLR 2021**. arXiv:2010.11929 · consultado 2026-08-16.
- Vaswani, A. et al. (2017). *Attention Is All You Need*. arXiv:1706.03762 · consultado 2026-08-16.

Evaluación — P46

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale* (2020, arXiv:2010.11929 · ICLR 2021) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P46_vit.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P47 — AlphaFold 2

Arquitectura y entrenamiento · El caso donde el aprendizaje profundo resolvió, en la práctica, un problema abierto de la biología durante cincuenta años.

Nivel: L4 · Motor: `alphafold` · Notebook: `P47_alphafold.ipynb` · Anexo: álgebra y geometría

1. Identificación

Campo	Valor
Título original	<i>Highly accurate protein structure prediction with AlphaFold</i>
Autoría	John Jumper, Richard Evans, Alexander Pritzel y otros (DeepMind)
Año	2021
Venue	<i>Nature</i> 596, 583–589
Fuente primaria	doi.org/10.1038/s41586-021-03819-2
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Una proteína es una cadena de aminoácidos que se pliega en una forma tridimensional concreta, y esa forma determina lo que hace. Determinarla experimentalmente —cristalografía, resonancia, criomicroscopía— cuesta meses o años y mucho dinero por estructura.

Predecirla desde la secuencia era un problema abierto desde los años setenta. La evaluación comunitaria **CASP**, celebrada cada dos años desde 1994, mostraba progreso lento: en 2018 el mejor resultado seguía lejos de la precisión experimental.

3. Propuesta

Un sistema entrenado de extremo a extremo con tres piezas que se refuerzan:

- Alineamientos múltiples de secuencias (MSA):** proteínas homólogas de otras especies. Si dos posiciones mutan juntas a lo largo de la evolución, probablemente estén en contacto. Es información evolutiva, no solo química.
- Evoformer:** atención que opera a la vez sobre el MSA y sobre una representación de **pares** de residuos, dejando que ambas se actualicen mutuamente.
- Módulo de estructura:** produce coordenadas 3D directamente, con una arquitectura equivariante a rotaciones y traslaciones, y **recicla** su salida como entrada varias veces.

El puente conceptual: la geometría 3D queda determinada —salvo rotación y reflexión— por las distancias entre pares. Predecir qué residuos están cerca es, en la práctica, predecir la forma.

4. Intuición sin fórmulas

Reconstruir el plano de una ciudad conociendo solo las distancias entre cada par de esquinas. Con suficientes distancias, la disposición queda fijada; solo puedes girar o reflejar el mapa entero.

Dónde deja de funcionar la analogía: las distancias de la ciudad te las dan. Aquí hay que **predecirlas** desde la secuencia, y ese es el problema entero.

5. Matemática mínima

De distancias a coordenadas:

dado d_{ij} para todo par (i,j) , encontrar $x_i \in \mathbb{R}^3$ tal que
 $\|x_i - x_j\| \approx d_{ij}$

minimizando $\sum_{i < j} (\|x_i - x_j\| - d_{ij})^2$

La solución es única salvo rotación, traslación y reflexión.

La miniatura del eje hace exactamente esto: parte de ocho puntos en posiciones **aleatorias** y, usando solo la matriz de distancias, converge por descenso de gradiente a la estructura correcta — el error cuadrático cae de 20,04 a 0,0 en menos de 150 pasos.

6. Arquitectura o flujo

7. Qué observar en el paper original

- El **resultado de CASP14**: mediana de GDT en torno a 92 sobre 100, con la precisión experimental como referencia. Es el dato que cambió el campo.
- El papel del **MSA**: cómo la señal coevolutiva sustituye a información física que no se modela.
- El **reciclado**: pasar la predicción de vuelta por la red varias veces, un truco simple con mucho efecto.
- La métrica **pLDDT** de confianza por residuo. Que el modelo diga *dónde* no está seguro es posiblemente tan importante como la predicción misma.

8. Evidencia y resultados

Evaluación en CASP14, la competición comunitaria a ciegas donde las estructuras reales no son públicas cuando se predice. AlphaFold 2 alcanzó una exactitud comparable a la determinación experimental en la mayoría de los objetivos.

Las cifras exactas de GDT por categoría están en el artículo de *Nature* y en los materiales de CASP14. Verificarlas allí. El diseño a ciegas de CASP es lo que hace creíble el resultado, y merece más atención que el número.

La miniatura de este eje **no predice** nada: recibe la matriz de distancias y reconstruye la geometría. Aísla la mitad geométrica del problema, no la difícil.

9. Impacto

- La base de datos derivada publicó predicciones para más de 200 millones de proteínas, de acceso abierto, transformando la práctica en biología estructural.
- Demis Hassabis y John Jumper compartieron el **Premio Nobel de Química de 2024** por este trabajo.
- Es el argumento más sólido a favor de que estos métodos producen conocimiento científico y no solo productos.
- Cambió las expectativas sobre qué problemas científicos son abordables así, con el riesgo de extrapolar de más.

10. Limitaciones

1. **Predice estructuras estáticas:** una proteína real se mueve, y su función suele depender de ese movimiento.
2. **Depende del MSA:** con proteínas huérfanas, sin homólogos conocidos, la calidad cae.
3. **No modela bien complejos ni ligandos** en la versión de 2021; eso llega con AlphaFold-Multimer y AlphaFold 3.
4. **Predecir la estructura no es entender el plegamiento:** el proceso físico por el que la proteína llega a esa forma sigue sin explicarse.
5. **No predice el efecto de mutaciones** de forma fiable, que es lo que muchas aplicaciones clínicas necesitan.
6. **Coste computacional alto** para el entrenamiento, y dependencia de bases de datos curadas durante décadas por la comunidad.

11. Errores comunes

Error	Corrección
«Resolvió el problema del plegamiento»	Resolvió la predicción de estructura . El plegamiento —cómo llega físicamente a esa forma— sigue abierto.
«Simula la física de la proteína»	No hay simulación física. Aprende de estructuras conocidas y de señal evolutiva.
«Sustituye al trabajo experimental»	Es una hipótesis de altísima calidad. La validación experimental sigue siendo necesaria, y el pLDDT indica dónde más.
«Funciona igual con cualquier proteína»	Depende críticamente de tener homólogos. Sin MSA útil, la calidad cae.
«Una proteína tiene una estructura»	Muchas tienen regiones desordenadas o varias conformaciones funcionales. Una predicción estática no captura eso.

12. Relación con trabajos anteriores

- **Po8 Transformer (2017)** — la atención, adaptada a operar sobre MSA y sobre pares.
- **P44 ResNet (2015)** — la profundidad con atajos que el sistema usa.
- **AlphaFold 1 (2020)** — la versión anterior, basada en predecir distancias y optimizar aparte.
doi.org/10.1038/s41586-019-1923-7
- **Anfinsen (1972)** — la hipótesis de que la secuencia determina la estructura, que es la premisa de todo el problema.

13. Relación con trabajos posteriores

- **AlphaFold-Multimer (2021)** y **AlphaFold 3 (2024)** — complejos, ácidos nucleicos y ligandos.
- **ESMFold (2022)** — predicción sin MSA usando un modelo de lenguaje de proteínas.
- **Modelos de fundación para la ciencia (2023+)** — la línea que este trabajo legitima.
- **P16 Sistemas agénticos** — la aplicación de estos métodos al ciclo de descubrimiento completo, todavía muy abierta.

14. Notebook asociado

P47_alphafold.ipynb

Qué implementa: la reconstrucción de coordenadas 3D a partir de una matriz de distancias por descenso de gradiente, con ocho puntos, partiendo de posiciones aleatorias.

Qué NO implementa: absolutamente nada del sistema real. No hay MSA, ni Evoformer, ni módulo de estructura, ni proteína. La matriz de distancias **se da**, y predecirla desde la secuencia es el problema entero. Es la parte geométrica, que es la fácil.

```
ai-evolution paper-lab P47 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Enumera las tres piezas del sistema y qué aporta cada una.
Explicar	Explica por qué la coevolución de dos posiciones sugiere contacto físico.
Aplicar	Ejecuta el notebook con 16 puntos y observa la convergencia.
Analizar	¿Por qué la solución es única solo salvo rotación y reflexión?
Evaluar	«AlphaFold resolvió el plegamiento de proteínas». Evalúa la afirmación.
Crear	Diseña un criterio para decidir cuándo una predicción necesita validación experimental.

16. Autoevaluación

1. ¿Qué predice exactamente el sistema y qué no?
2. ¿Qué información aporta el MSA que no está en la secuencia?
3. ¿Por qué las distancias entre pares determinan la geometría?
4. ¿Qué es el pLDDT y por qué importa tanto?
5. ¿Por qué CASP hace creíble el resultado?
6. ¿Qué pasa con una proteína sin homólogos conocidos?
7. ¿Qué diferencia hay entre predecir la estructura y entender el plegamiento?

17. Respuestas esperadas

1. Predice la **estructura tridimensional** de la proteína a partir de su secuencia. No predice el proceso físico de plegamiento, ni su dinámica, ni de forma fiable el efecto de mutaciones.
2. Señal **coevolutiva**: si dos posiciones mutan de forma correlacionada a lo largo de la evolución, es probable que estén en contacto en la estructura. Eso no se ve en una sola secuencia.
3. Porque fijar todas las distancias entre pares fija la configuración salvo movimientos rígidos: rotación, traslación y reflexión.
4. Es una estimación de confianza **por residuo**. Importa porque indica en qué partes de la predicción se puede confiar y cuáles requieren validación experimental.
5. Porque es una evaluación a ciegas: las estructuras reales no son públicas en el momento de predecir, así que no puede haber contaminación ni ajuste al conjunto de prueba.
6. La calidad de la predicción cae, porque falta la señal evolutiva de la que el sistema depende.
7. Predecir la estructura es dar el resultado final. Entender el plegamiento sería explicar el proceso físico por el que la cadena llega a esa forma, que sigue sin resolverse.

18. Fuentes primarias

- Jumper, J. et al. (2021). *Highly accurate protein structure prediction with AlphaFold*. **Nature** 596, 583–589. doi.org/10.1038/s41586-021-03819-2 · consultado 2026-08-16.
- Senior, A. W. et al. (2020). *Improved protein structure prediction using potentials from deep learning* (AlphaFold 1). **Nature** 577, 706–710. doi.org/10.1038/s41586-019-1923-7 · consultado 2026-08-16.

Evaluación — P47

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Highly accurate protein structure prediction with AlphaFold* (2021, Nature 596, 583–589 (2021)) ·

Nivel: L4

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P47_alphafold.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P48 — LoRA

Arquitectura y entrenamiento · Si la actualización útil de un modelo tiene rango bajo, se puede entrenar factorizada — y ajustar un modelo grande deja de exigir un centro de datos.

Nivel: L3 · Motor: `lora` · Notebook: `P48_lora.ipynb` · Anexo: álgebra y geometría

1. Identificación

Campo	Valor
Título original	<i>LoRA: Low-Rank Adaptation of Large Language Models</i>
Autoría	Edward J. Hu, Yelong Shen, Phillip Wallis y otros
Año	2021
Venue	arXiv:2106.09685 · ICLR 2022
Fuente primaria	arXiv:2106.09685
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Ajustar un modelo grande a una tarea significaba actualizar **todos** sus pesos. Con GPT-3 eso son 175 000 millones de parámetros: memoria de optimizador prohibitiva —Adam guarda dos estados por parámetro—, y una copia completa del modelo por cada tarea.

Existían alternativas de ajuste parcial. Los *adapters* insertaban capas nuevas, pero añadían latencia en inferencia. El *prefix tuning* consumía parte de la ventana de contexto. Ninguna era gratis.

3. Propuesta

Congelar `W` y aprender su **actualización** en forma factorizada:

$$W' = W + BA, \quad \text{con } B \in \mathbb{R}^{d \times r}, \quad A \in \mathbb{R}^{r \times d}, \quad r \ll d$$

La hipótesis, sugerida por trabajo previo sobre la dimensionalidad intrínseca del ajuste: la actualización que hace falta para adaptar el modelo a una tarea tiene **rango bajo**. Si es así, `BA` la representa con una fracción diminuta de los parámetros.

Y la propiedad que lo hace práctico: al desplegar, `BA` se **suma** a `W`. El modelo resultante es una matriz normal, sin capas extra. **Cero latencia añadida** en inferencia — a diferencia de los adapters.

4. Intuición sin fórmulas

Un manual de mil páginas y una fe de erratas de dos. No reimprimes el manual: publicas la fe de erratas, y cada especialidad tiene la suya sobre el mismo manual.

Dónde deja de funcionar la analogía: una fe de erratas corrige puntos sueltos. **BA** es una corrección que toca **toda** la matriz, pero con una estructura muy restringida —solo **r** direcciones independientes—. Es densa y de rango bajo, no dispersa.

5. Matemática mínima

Parámetros a entrenar:
ajuste completo : $d \times d$
LoRA : $2 \times d \times r$

Con $d = 128$:

rango r	ajuste completo	LoRA	fracción	reducción
1	16 384	256	1,6 %	64×
4	16 384	1 024	6,3 %	16×
16	16 384	4 096	25 %	4×
64	16 384	16 384	100 %	1×

Con $r = d/2$ ya no se ahorra nada: $2 \cdot d \cdot r = d^2$. El método solo tiene sentido con **r** pequeño, y eso es una **apuesta** sobre la estructura del problema, no un teorema.

A se inicializa aleatoria y **B** a cero, de modo que **BA** = **0** al empezar: el ajuste arranca exactamente desde el modelo base.

6. Arquitectura o flujo

7. Qué observar en el paper original

- El **estudio de rango**: $r = 1$ o 2 ya funciona sorprendentemente bien en muchas tareas. Es la evidencia principal de la hipótesis de rango bajo.

- **A qué matrices aplicarlo:** los autores encuentran que las proyecciones de consulta y valor de la atención dan mejor resultado que repartir el presupuesto entre todas.
- La comparación explícita de **latencia** frente a adapters. Es el argumento de ingeniería que explica su adopción.
- El análisis de la **relación entre el subespacio de B_A y las direcciones de W** , en el apéndice: sugiere que amplifica direcciones que ya existían pero estaban poco pesadas.

8. Evidencia y resultados

Experimentos sobre modelos de lenguaje de distintos tamaños en tareas de comprensión y generación, comparando ajuste completo, adapters, prefix tuning y LoRA a varios rangos.

Las cifras por tarea y rango están en el artículo. Verificarlas allí. Lo que hay que retener: con una fracción muy pequeña de parámetros entrenables, la calidad se mantiene comparable al ajuste completo en las tareas evaluadas.

La miniatura de este eje solo cuenta parámetros y enseña la forma de una actualización de rango 2. No entrena nada.

9. Impacto

- Hizo el ajuste de modelos grandes accesible fuera de los grandes laboratorios: un adaptador cabe en unos megabytes y se entrena en una GPU de consumo.
- Creó un **ecosistema**: miles de adaptadores compartidos sobre unos pocos modelos base, con servidores que cargan y descargan adaptadores por petición.
- Es la base de QLoRA, que lo combina con cuantización para bajar aún más el requisito de memoria.
- Y cambió la economía del ajuste fino: de decisión estratégica cara a experimento barato.

10. Limitaciones

1. **La hipótesis de rango bajo es empírica:** nada garantiza que la actualización útil lo sea para toda tarea, especialmente si exige conocimiento realmente nuevo.
2. **r y las matrices objetivo son hiperparámetros** cuya elección importa y no es automática.
3. **Menor capacidad de cambio que el ajuste completo:** con desplazamientos de dominio grandes, la diferencia aparece.
4. **Componer varios adaptadores no es trivial:** sumarlos no equivale a aprenderlos juntos.
5. **No reduce la memoria del modelo base** durante el entrenamiento —solo la del optimizador y los gradientes—; para eso hace falta cuantizar.
6. **La comparación con ajuste completo depende de la tarea y del presupuesto** del estudio.

11. Errores comunes

Error	Corrección
«Entrena menos parámetros, luego es más rápido»	Reduce memoria de optimizador y de gradientes, pero la pasada hacia delante sigue recorriendo el modelo entero.
«Añade latencia como los adapters»	No: BA se suma a W al desplegar y queda una matriz normal. Ese es su argumento central.
«Rango mayor siempre mejor»	El paper muestra que rangos muy pequeños bastan. Con $r = d/2$ se pierde todo el ahorro.
«Puede aprender cualquier cosa que aprenda el ajuste completo»	Está restringido a un subespacio de rango r . La hipótesis es empírica, no una garantía.
«Reduce la memoria necesaria para cargar el modelo»	La base sigue completa en memoria. Reducirla es lo que aporta QLoRA cuantizándola.

12. Relación con trabajos anteriores

- **P08 Transformer (2017)** — las matrices de proyección sobre las que se aplica.
- **P10 GPT-3 (2020)** — la escala que hace inviable el ajuste completo.
- **Adapters (Houlsby et al., 2019)** — la alternativa con coste de latencia. [arXiv:1902.00751](#)
- **Aghajanyan et al. (2020)** — la dimensionalidad intrínseca del ajuste, que motiva la hipótesis. [arXiv:2012.13255](#)

13. Relación con trabajos posteriores

- **P49 QLoRA (2023)** — LoRA sobre una base cuantizada a 4 bits.
- **DoRA, LoRA+, AdaLoRA (2023-2024)** — variantes que reparten el rango o descomponen magnitud y dirección.
- **Servidores multiadaptador (2023+)** — infraestructura que explota que la base es compartida.
- **P45 Destilación (2015)** — la vía alternativa para abaratar: achicar el modelo en vez de su ajuste.

14. Notebook asociado

P48_lora.ipynb

Qué implementa: el conteo de parámetros para cuatro rangos con $d = 128$, y la construcción explícita de una actualización de rango 2 como producto BA .

Qué NO implementa: no entrena nada. Ni modelo, ni tarea, ni comparación de calidad. La hipótesis central del paper —que la actualización útil es de rango bajo— aquí se enuncia, no se verifica.

```
ai-evolution paper-lab P48 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Escribe la ecuación de LoRA e identifica qué se congela.
Explicar	Explica por qué no añade latencia en inferencia.
Aplicar	Calcula el ahorro con $d = 4096$ y $r = 8$.
Analizar	¿A partir de qué r deja de haber ahorro y por qué?
Evaluar	Una tarea exige conocimiento de dominio muy nuevo. ¿Confiarías en LoRA?
Crear	Diseña un experimento que mida si la actualización útil de una tarea es de rango bajo.

16. Autoevaluación

1. ¿Qué se entrena y qué se congela?
2. ¿Por qué B se inicializa a cero?
3. ¿Por qué no hay latencia añadida en inferencia?
4. ¿Qué memoria ahorra exactamente y cuál no?
5. ¿A partir de qué rango desaparece el ahorro?
6. ¿Es la hipótesis de rango bajo un teorema?
7. ¿Qué añade QLoRA sobre esto?

17. Respuestas esperadas

1. Se entrenan las dos matrices pequeñas A y B ; la matriz original W queda congelada y no recibe gradiente.
2. Para que $BA = 0$ al inicio y el modelo arranque exactamente en el comportamiento del modelo base, sin una perturbación aleatoria de partida.
3. Porque al desplegar se calcula $W + BA$ una sola vez y queda una matriz de las mismas dimensiones. No hay capas adicionales que recorrer.
4. Ahorra la memoria del optimizador y de los gradientes, que solo se necesitan para los parámetros entrenables. **No** ahorra la memoria de cargar el modelo base, que sigue completo.
5. A partir de $r = d/2$, donde $2 \cdot d \cdot r = d^2$ y se entrenan tantos parámetros como en el ajuste completo.
6. No. Es una hipótesis empírica bien respaldada en las tareas evaluadas, no una garantía general.
7. Cuantizar la base congelada a 4 bits, que es justo la memoria que LoRA no reduce.

18. Fuentes primarias

- Hu, E. J. et al. (2022). *LoRA: Low-Rank Adaptation of Large Language Models*. **ICLR 2022**. arXiv:2106.09685 · consultado 2026-08-16.
- Aghajanyan, A., Zettlemoyer, L. y Gupta, S. (2020). *Intrinsic Dimensionality Explains the Effectiveness of Language Model Fine-Tuning*. arXiv:2012.13255 · consultado 2026-08-16.

Evaluación — P48

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *LoRA: Low-Rank Adaptation of Large Language Models* (2021, arXiv:2106.09685 · ICLR 2022) ·

Nivel: L3

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P48_lora.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P49 — QLoRA y cuantización

Arquitectura y entrenamiento · Menos bits por peso. Un modelo de 70 000 millones baja de 140 GB a 35 GB y pasa de necesitar un clúster a caber en una sola tarjeta.

Nivel: L3 · **Motor:** quantization · **Notebook:** P49_qlora.ipynb · **Anexo:** complejidad y coste

1. Identificación

Campo	Valor
Título original	QLoRA: Efficient Finetuning of Quantized LLMs
Autoría	Tim Dettmers, Artidoro Pagnoni, Ari Holtzman, Luke Zettlemoyer
Año	2023
Venue	arXiv:2305.14314 · NeurIPS 2023
Fuente primaria	arXiv:2305.14314
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

LoRA redujo drásticamente la memoria del **optimizador**, pero dejó intacto el problema mayor: el modelo base sigue cargado en memoria completo y en precisión de 16 bits. Un modelo de 70 000 millones de parámetros ocupa así **140 GB**, muy por encima de cualquier tarjeta única.

La cuantización existía y se usaba en inferencia, pero cuantizar y luego **entrenar** encima planteaba dudas: si los pesos base están degradados, ¿el ajuste todavía funciona?

3. Propuesta

Cuantizar la base congelada a 4 bits y entrenar los adaptadores LoRA en precisión alta. Como la base no recibe gradiente, su degradación no se propaga por la optimización — solo afecta a la función que el adaptador tiene que corregir.

Tres piezas técnicas concretas:

- NF4** (*NormalFloat 4-bit*): un formato de 4 bits cuyos niveles están colocados según la distribución normal que siguen los pesos, en vez de repartidos uniformemente.
- Doble cuantización**: cuantizar también las constantes de escala, que con bloques pequeños dejan de ser despreciables.
- Optimizadores paginados**: descargar estados del optimizador a memoria del sistema en los picos, para no agotar la tarjeta.

4. Intuición sin fórmulas

Guardar fotos con menos profundidad de color. Con suficientes niveles no se nota; a partir de cierto punto aparecen bandas. Y si los niveles se reparten donde hay más información —en vez de uniformemente— se aguanta mucho más antes de que se note.

Dónde deja de funcionar la analogía: en una foto el deterioro se ve. En un modelo hay que **medirlo en una tarea**, porque el error sobre los pesos no dice cuánto empeora el comportamiento.

5. Matemática mínima

Cuantización uniforme a b bits:

$$\text{niveles} = 2^b$$

$$\text{paso} = (\text{max} - \text{min}) / (\text{niveles} - 1)$$

$$\tilde{w} = \text{min} + \text{round}((w - \text{min})/\text{paso}) \cdot \text{paso}$$

Memoria de un modelo de N parámetros: $N \cdot b / 8$ bytes

Con 2000 pesos simulados de una gaussiana:

bits	niveles	error cuadrático medio	memoria de un modelo de 70 000 M
16	65 536	1,43e-06	140,0 GB
8	256	3,66e-04	70,0 GB
4	16	6,32e-03	35,0 GB
3	8	1,33e-02	26,2 GB
2	4	3,11e-02	17,5 GB

La memoria baja **linealmente** con los bits; el error crece mucho más deprisa. Ahí está el compromiso, y por qué 4 bits acabó siendo el punto habitual.

6. Arquitectura o flujo

7. Qué observar en el paper original

- La **justificación de NF4**: por qué colocar los niveles según la distribución normal es mejor que repartirlos uniformemente, dado que los pesos se distribuyen así.

- El experimento clave: **igualar la calidad de un ajuste en 16 bits** con la base cuantizada a 4. Es la afirmación central y hay que ver con qué modelos y tareas se sostiene.
- La familia de modelos **Guanaco** y su evaluación, incluyendo el uso de un modelo como juez —con todas las cautelas que eso merece.
- El análisis del **tamaño de bloque** y de la doble cuantización: cuánto ahorra realmente.

8. Evidencia y resultados

Ajuste de modelos de varios tamaños con la base cuantizada, comparado contra ajuste en 16 bits, con evaluación en conjuntos de instrucciones y comparativas por preferencia.

Las cifras están en el artículo. Verificarlas allí, con una cautela metodológica: parte de la evaluación usa modelos como jueces, y esa metodología tiene sesgos conocidos —hacia respuestas largas, hacia el estilo del propio juez—.

La miniatura de este eje mide el error de **reconstrucción de pesos**, no la calidad del modelo. Es deliberado: hace visible que son dos cosas distintas.

9. Impacto

- Puso el ajuste de modelos grandes al alcance de una sola tarjeta de consumo, con un efecto inmediato sobre quién puede participar en el campo.
- Aceleró el ecosistema de modelos abiertos ajustados, que a partir de 2023 se multiplicó.
- Consolidó los 4 bits como punto de operación estándar para inferencia local.
- Y desplazó la conversación sobre coste: de «cuántos parámetros» a «cuántos bits por parámetro».

10. Limitaciones

1. **La cuantización degrada:** puede ser imperceptible en promedio y notarse en casos concretos, sobre todo en razonamiento largo o en tareas de precisión.
2. **Descuantizar al vuelo cuesta cómputo:** se ahorra memoria, no necesariamente tiempo.
3. **Depende de núcleos especializados:** sin ellos, el rendimiento es malo.
4. **El error de reconstrucción de pesos no predice la degradación de la tarea;** hay que medir en la tarea.
5. **La evaluación con modelos como jueces es discutible,** y buena parte de los resultados comparativos la usan.
6. **Por debajo de 4 bits el deterioro se acelera** y deja de ser aceptable en general.

11. Errores comunes

Error	Corrección
«4 bits es la mitad de bueno que 8»	La relación entre bits y calidad no es lineal. Hay que medir en la tarea, no extrapolar.
«Cuantizar acelera la inferencia»	Ahorra memoria y ancho de banda. La velocidad depende de los núcleos y puede incluso empeorar sin ellos.
«El error de cuantización mide la pérdida de calidad»	No. Son magnitudes distintas y su relación no es directa.
«QLoRA es solo LoRA con menos bits»	Aporta NF4, doble cuantización y optimizadores paginados. Sin esas piezas, no funciona igual.
«Cuantizar los adaptadores también»	Los adaptadores se entrenan en precisión alta: son los que reciben gradiente y ahí la precisión sí importa.

12. Relación con trabajos anteriores

- **P48 LoRA (2021)** — la mitad del método.
- **LLM.int8() (2022)**, del mismo autor principal — cuantización a 8 bits para inferencia. [arXiv:2208.07339](https://arxiv.org/abs/2208.07339)
- **P45 Destilación (2015)** — la vía alternativa: achicar el modelo en vez de sus bits.
- **P10 GPT-3 (2020)** — la escala que crea el problema de memoria.

13. Relación con trabajos posteriores

- **GPTQ y AWQ (2023)** — cuantización guiada por datos para inferencia.
- **Cuantización a 1-2 bits (2024)** — el límite inferior, con degradación aún discutida.
- **Modelos locales (2023+)** — todo el ecosistema de ejecución en equipo propio depende de esto.
- **P21 Mezcla de expertos** — la otra vía para desacoplar capacidad de coste por token.

14. Notebook asociado

`P49_qlora.ipynb`

Qué implementa: cuantización uniforme de 2000 pesos gaussianos a 16, 8, 4, 3 y 2 bits, con el error de reconstrucción y la memoria resultante para un modelo de 70 000 millones.

Qué NO implementa: no hay NF4 —se usa cuantización uniforme, que es peor—, ni doble cuantización, ni adaptadores, ni ninguna medida de calidad en tarea, que es lo único que decide si una cuantización es aceptable.

```
ai-evolution paper-lab P49 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Enumera las tres aportaciones técnicas de QLoRA.
Explicar	Explica por qué la base puede cuantizarse y los adaptadores no.
Aplicar	Calcula la memoria de un modelo de 8000 M a 4 bits.
Analizar	¿Por qué NF4 es mejor que una rejilla uniforme para pesos gaussianos?
Evaluar	Un informe reporta «sin pérdida de calidad a 4 bits». ¿Qué exiges para creerlo?
Crear	Diseña un protocolo de evaluación de cuantización que no dependa de un modelo juez.

16. Autoevaluación

1. ¿Qué memoria reduce QLoRA que LoRA no reducía?
2. ¿Por qué la base cuantizada no arruina el entrenamiento?
3. ¿Qué es NF4 y por qué no es una rejilla uniforme?
4. ¿La cuantización acelera la inferencia?
5. ¿Por qué el error de cuantización no basta para evaluar?
6. ¿Qué precisión llevan los adaptadores y por qué?
7. ¿Qué cautela metodológica exige la evaluación del paper?

17. Respuestas esperadas

1. La del **modelo base cargado**. LoRA reducía la del optimizador y los gradientes, pero la base seguía ocupando 16 bits por parámetro.
2. Porque la base está congelada y no recibe gradiente: su degradación cambia la función de partida, pero no interfiere con la optimización de los adaptadores, que sí están en precisión alta.
3. Un formato de 4 bits cuyos niveles se colocan según los cuantiles de una distribución normal. Como los pesos siguen aproximadamente esa distribución, se asignan más niveles donde hay más masa, y el error baja frente a una rejilla uniforme.
4. No necesariamente. Ahorra memoria y ancho de banda; el tiempo depende de tener núcleos especializados y de descuantizar al vuelo, que cuesta cómputo.
5. Porque mide la distancia entre pesos originales y cuantizados, no el comportamiento del modelo. Dos cuantizaciones con el mismo error pueden degradar tareas de forma muy distinta.
6. Precisión alta, típicamente 16 bits, porque son los únicos parámetros que reciben gradiente y ahí la precisión numérica sí afecta a la optimización.
7. Que parte de los resultados comparativos usa modelos como jueces, una metodología con sesgos conocidos hacia respuestas largas y hacia el estilo del propio juez.

18. Fuentes primarias

- Dettmers, T., Pagnoni, A., Holtzman, A. y Zettlemoyer, L. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs*. **NeurIPS 2023**. arXiv:2305.14314 · consultado 2026-08-16.
- Dettmers, T. et al. (2022). *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale*. arXiv:2208.07339 · consultado 2026-08-16.

Evaluación — P49

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *QLoRA: Efficient Finetuning of Quantized LLMs* (2023, arXiv:2305.14314 · NeurIPS 2023) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P49_qlora.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P50 — IA constitucional

Evaluación y seguridad · Sustituir parte del juicio humano por un conjunto de principios **escritos**, y dejar que el modelo se critique a sí mismo contra ellos.

Nivel: L4 · **Motor:** constitutional_ai · **Notebook:** P50_constitutional_ai.ipynb · **Anexo:** probabilidad y verosimilitud

1. Identificación

Campo	Valor
Título original	Constitutional AI: Harmlessness from AI Feedback
Autoría	Yuntao Bai, Saurav Kadavath, Sandipan Kundu y otros (Anthropic)
Año	2022
Venue	arXiv:2212.08073
Fuente primaria	arXiv:2212.08073
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

RLHF funcionaba, y traía tres problemas de fondo.

El primero es de **escala**: hacen falta muchísimas comparaciones humanas, y son caras y lentas. El segundo es **humano**: revisar contenido dañino durante meses tiene un coste real sobre las personas que lo hacen. El tercero es de **auditoría**: lo que el modelo aprende son las preferencias implícitas de un grupo de anotadores, con sus criterios y sus sesgos, y no hay documento contra el cual verificar si el comportamiento resultante es el pretendido.

Además, los modelos alineados así tendían a la **evasión**: negarse sin explicar, que es seguro y poco útil.

3. Propuesta

Escribir los principios y ponerlos en el bucle. Dos fases:

- Supervisada**: el modelo genera una respuesta, se le pide que la **critique** contra un principio concreto de la lista y que la **revise**. Se entrena con las respuestas revisadas.
- Refuerzo (RLAIF)**: en vez de comparaciones humanas, el propio modelo compara pares de respuestas guiándose por los principios, y con esas preferencias se entrena el modelo de recompensa.

El cambio importante no es que se ahorren etiquetas. Es que **el criterio pasa a ser un documento legible**: se puede leer, discutir, versionar y criticar. Deja de estar disperso en el juicio implícito de anotadores.

4. Intuición sin fórmulas

Un código deontológico escrito frente a «pregúntale al jefe». El código no garantiza buenas decisiones, pero sí que se pueda discutir la regla, y no solo el caso.

Dónde deja de funcionar la analogía: un profesional entiende el código; el modelo solo lo recibe como texto en su contexto. Que lo cite no prueba que sea la causa de su comportamiento.

5. Matemática mínima

```
Fase 1 – supervisada:
  respuesta ← modelo(petición)
  crítica   ← modelo(respuesta, principio_i)    ¿lo viola?
  revisión  ← modelo(respuesta, crítica)
  entrenar el modelo sobre (petición → revisión)

Fase 2 – RLAIIF:
  preferencia ← modelo(respuesta_A, respuesta_B, principios)
  modelo de recompensa ← preferencias generadas por IA
  política ← RL contra ese modelo de recompensa

Etiquetas humanas de inocuidad usadas: 0
```

La miniatura del eje ejecuta la fase 1 con reglas explícitas: la respuesta inicial «*No pienso ayudarte con eso, deberías replantearte por qué lo preguntas*» cumple el principio de seguridad y **viola los otros dos** —juzga al usuario y se niega sin explicar—. La revisión corrige ambos sin una sola etiqueta humana nueva.

6. Arquitectura o flujo

7. Qué observar en el paper original

- **La constitución misma**, en el apéndice. Leerla completa es el ejercicio más instructivo del artículo: se ve qué es un principio operativo y qué es una declaración de intenciones.
- La **cadena crítica-revisión**, con ejemplos reales de respuestas antes y después.

- El resultado sobre **evasión**: modelos menos evasivos y a la vez menos dañinos, que es lo interesante — se suele asumir que ese intercambio es forzoso.
- Que la **utilidad** sigue entrenándose con retroalimentación humana. Solo la inocuidad pasa a retroalimentación de IA.

8. Evidencia y resultados

Comparaciones entre modelos entrenados con RLHF y con IA constitucional, evaluando inocuidad, utilidad y evasión mediante comparaciones por preferencia.

Las curvas están en el artículo. Verificarlas allí. El resultado que importa es la relación conjunta entre inocuidad y evasión, no cada métrica por separado.

La miniatura de este eje **no es el método**: la crítica está codificada con reglas fijas, mientras que en el paper la genera el propio modelo y puede fallar. Es la forma del bucle, no su contenido.

9. Impacto

- Estableció la retroalimentación de IA como alternativa práctica a la humana en parte del proceso de alineamiento.
- Popularizó publicar el criterio: hoy varias organizaciones publican documentos de especificación de comportamiento, versionados y criticables.
- Redujo la exposición de anotadores humanos a contenido dañino.
- Y trasladó el debate de «qué hace el modelo» a «quién escribe los principios y con qué autoridad» — que es la pregunta correcta, aunque el método no la responda.

10. Limitaciones

1. **No resuelve quién decide los principios.** Los hace explícitos, que es distinto y también valioso, pero la legitimidad sigue siendo una cuestión política abierta.
2. **La autocrítica hereda las limitaciones del modelo:** si no reconoce un daño, no lo criticará.
3. **Riesgo de retroalimentación circular:** el modelo evalúa según criterios que él mismo interpreta, y los sesgos pueden reforzarse.
4. **Los principios se contradicen entre sí** en casos reales, y su resolución no está especificada.
5. **Cumplir la letra no es cumplir la intención:** un modelo puede satisfacer el texto y fallar en el fondo.
6. **La utilidad sigue necesitando humanos**, así que el ahorro es parcial.

11. Errores comunes

Error	Corrección
«Elimina a los humanos del alineamiento»	Solo de la retroalimentación de inocuidad. Los principios los escriben personas y la utilidad sigue usando retroalimentación humana.
«Los principios garantizan el comportamiento»	Son entrada de entrenamiento, no una restricción verificable. El modelo puede violarlos.
«Es una constitución en sentido jurídico»	Es un documento de criterios de entrenamiento, sin mecanismo de aplicación ni de apelación.
«Resuelve el problema de los valores»	Lo hace explícito y discutible , que es un avance real, pero no decide de quién son esos valores.
«Si el modelo cita el principio, lo está siguiendo»	Citar es comportamiento superficial. Que el principio sea la causa del comportamiento es una afirmación causal que hay que probar aparte.

12. Relación con trabajos anteriores

- **P12 InstructGPT / RLHF (2022)** — el método que extiende y cuyos costes ataca.
- **P10 GPT-3 (2020)** — la capacidad de seguir instrucciones en contexto que hace viable la autocrítica.
- **P28 Cadena de pensamiento (2022)** — el razonamiento explícito en el que se apoya la crítica.

13. Relación con trabajos posteriores

- **P15 DPO (2023)** — simplifica la otra mitad, la optimización de preferencias.
- **RLAIF (2023)** — el estudio comparativo directo entre retroalimentación humana y de IA.
arXiv:2309.00267
- **Especificaciones de comportamiento publicadas (2024+)** — la práctica de publicar el criterio.
- **P51 SWE-bench (2023)** — el enfoque opuesto para evaluar: un verificador objetivo en vez de un juicio.

14. Notebook asociado

P50_constitutional_ai.ipynb

Qué implementa: el bucle crítica-revisión sobre una respuesta evasiva, con tres principios y la traza de qué principio viola y por qué.

Qué NO implementa: la crítica está codificada con reglas, no generada por un modelo. Falta entera la segunda fase de refuerzo. Es la **forma** del método, no el método.

```
ai-evolution paper-lab P50 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Enumera las dos fases y qué produce cada una.
Explicar	Explica por qué hacer explícito el criterio es un avance aunque no decida los valores.
Aplicar	Ejecuta el notebook y añade un cuarto principio.
Analizar	¿Qué pasa si dos principios se contradicen en un caso concreto?
Evaluar	«El modelo cita el principio, luego lo sigue». Evalúa el razonamiento.
Crear	Escribe tres principios operativos para un asistente de tu dominio, con su criterio de violación.

16. Autoevaluación

1. ¿Qué tres problemas de RLHF ataca?
2. ¿Qué hace la fase supervisada y qué la de refuerzo?
3. ¿Qué significa que el criterio sea auditable?
4. ¿Qué es la evasión y por qué es un problema?
5. ¿Qué parte del alineamiento sigue necesitando humanos?
6. ¿Cuál es el riesgo de que el modelo se evalúe a sí mismo?
7. ¿Qué pregunta deja abierta el método?

17. Respuestas esperadas

1. El coste de las etiquetas humanas, el impacto sobre los anotadores expuestos a contenido dañino, y la imposibilidad de auditar un criterio que solo existe implícito en sus juicios.
2. La supervisada genera crítica y revisión contra los principios y entrena sobre las respuestas revisadas. La de refuerzo genera **preferencias** con el propio modelo y entrena un modelo de recompensa con ellas.
3. Que existe un documento legible que se puede leer, discutir, versionar y criticar, en vez de un criterio disperso en el juicio implícito de un grupo de anotadores.
4. Negarse sin explicar. Es un problema porque es seguro pero inútil, y porque un modelo puede parecer alineado simplemente por no ayudar nunca.
5. Escribir los principios, y la retroalimentación de **utilidad**, que en el paper sigue siendo humana. Solo la inocuidad pasa a retroalimentación de IA.
6. Retroalimentación circular: si el modelo no reconoce un daño, no lo critica, y sus sesgos pueden reforzarse en vez de corregirse.
7. Quién escribe los principios y con qué legitimidad. El método hace la pregunta visible y explícita, pero no la responde.

18. Fuentes primarias

- Bai, Y. et al. (2022). *Constitutional AI: Harmlessness from AI Feedback*. arXiv:2212.08073 · consultado 2026-08-16.
- Lee, H. et al. (2023). *RLAIF: Scaling Reinforcement Learning from Human Feedback with AI Feedback*. arXiv:2309.00267 · consultado 2026-08-16.

Evaluación — P50

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Constitutional AI: Harmlessness from AI Feedback* (2022, arXiv:2212.08073) · **Nivel:** L4

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P50_constitutional_ai.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P51 — SWE-bench

Evaluación y seguridad · Deja de preguntar si el código *parece* correcto. Pregunta si pasan los tests del repositorio.

Nivel: L3 · **Motor:** swebench · **Notebook:** P51_swebench.ipynb · **Anexo:** complejidad y coste

1. Identificación

Campo	Valor
Título original	<i>SWE-bench: Can Language Models Resolve Real-World GitHub Issues?</i>
Autoría	Carlos E. Jimenez, John Yang, Alexander Wettig y otros
Año	2023
Venue	arXiv:2310.06770 · ICLR 2024
Fuente primaria	arXiv:2310.06770
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Las evaluaciones de programación de la época —HumanEval, MBPP— pedían funciones autocontenidas de pocas líneas, con el enunciado en el propio docstring. Los modelos las saturaron rápido, y la distancia entre ese resultado y ser útil en un repositorio real seguía siendo enorme.

Lo que faltaba en esas pruebas es justo lo que define el trabajo real: **localizar** dónde tocar en un código que no cabe en el contexto, entender una incidencia escrita por una persona con información incompleta, y no romper nada más.

3. Propuesta

Construir la evaluación a partir de **incidencias reales resueltas** en repositorios de Python populares. Cada instancia contiene:

- el texto de la incidencia tal como lo escribió alguien;
- el repositorio en el commit exacto anterior a la solución;
- los **tests** que el arreglo real hizo pasar.

El criterio de éxito es binario y no negociable: aplicar el parche generado y ejecutar la suite. O pasan los tests, o no.

Ese criterio es lo que aporta el trabajo. Un test es un verificador objetivo que **no se puede convencer con prosa**.

4. Intuición sin fórmulas

Un examen de mecánica con dos correcciones posibles: describir cómo arreglarías el motor, o arrancarlo. La segunda no admite interpretación.

Dónde deja de funcionar la analogía: un motor que arranca funciona. Un test que pasa solo prueba lo que ese test comprueba — y puede haberse conseguido de una forma terrible.

5. Matemática mínima

% resuelto = instancias con TODOS los tests en verde / instancias totales

Cinco intentos, tres criterios:

criterio	tasa
«parece correcto»	80 %
«compila»	80 %
tests pasan	40 %

La brecha entre 80 % y 40 % es exactamente el problema que ataca el benchmark. Los criterios blandos inflan, y siempre en la misma dirección.

6. Arquitectura o flujo

7. Qué observar en el paper original

- El **proceso de construcción**: cómo se filtran las incidencias para que sean verificables y reproducibles. Es la mitad del trabajo y casi nunca se comenta.
- Las **tasas iniciales**, muy bajas, y el contraste con los resultados saturados de HumanEval.
- La discusión sobre **contaminación**: las incidencias son públicas y anteriores al corte de datos de los modelos evaluados. El paper es honesto al respecto.

- El subconjunto **SWE-bench Lite** y por qué existe: el coste de ejecutar la evaluación completa no es trivial.

8. Evidencia y resultados

Evaluación de modelos de lenguaje sobre más de dos mil incidencias reales de doce repositorios de Python, con tasas de resolución iniciales de un solo dígito porcentual.

Las cifras están en el artículo, y **envejecen muy rápido**: la tasa del estado del arte ha subido mucho desde 2023. Verificar siempre contra la tabla pública vigente, no contra una cifra citada de memoria.

La miniatura de este eje no evalúa nada: con cinco casos escritos a mano, exhibe la diferencia entre medir apariencia y medir tests.

9. Impacto

- Se convirtió en la referencia de facto para agentes de programación, y en el número que se cita al presentar cada modelo nuevo.
- Impulsó la investigación en **localización de código**: encontrar dónde tocar resultó ser tan difícil como escribir el arreglo.
- Estableció un patrón para el resto del campo: evaluar con verificadores objetivos y ejecutables siempre que exista uno.
- Y dejó ver un límite del enfoque: cuando un benchmark se vuelve el objetivo, empieza a optimizarse para él.

10. Limitaciones

1. **Contaminación**: las incidencias y sus soluciones son públicas y pueden estar en los datos de entrenamiento.
2. **Pasar los tests no es una buena solución**: puede lograrse de forma frágil, rompiendo el diseño, o incluso modificando los tests si no se impide.
3. **Solo Python**, y solo repositorios con buena cobertura de tests: un sesgo importante.
4. **Muchas incidencias reales no tienen test asociado**, y quedan fuera por construcción.
5. **Coste de ejecución alto**, lo que empuja a evaluar en subconjuntos y complica comparar cifras.
6. **Optimizar para el benchmark** es un riesgo real desde que se convirtió en la métrica pública de referencia.

11. Errores comunes

Error	Corrección
«Resuelve el X % de incidencias reales»	El X % de estas incidencias, de repos con buenos tests, en Python. La generalización es una afirmación aparte.
«Pasar los tests es resolver bien»	Es un mínimo objetivo, no un juicio de calidad. Nada dice del diseño ni del mantenimiento.
«Comparar cifras entre informes»	Hay variantes (completo, Lite, Verified) y andamiajes distintos. Sin especificar cuál, las cifras no son comparables.
«La contaminación está descartada»	No lo está, y el propio paper lo discute. Es una limitación estructural del diseño.
«Es la mejor medida de un programador»	Mide reparación de incidencias con test. No mide diseño, revisión, comunicación ni trabajo en un código sin cobertura.

12. Relación con trabajos anteriores

- **HumanEval (2021)** — la evaluación que satura y que este trabajo reemplaza. [arXiv:2107.03374](#)
- **P13 ReAct (2022)** — el patrón de agente que se evalúa aquí.
- **P16 Sistemas agénticos** — los sistemas que este benchmark mide.

13. Relación con trabajos posteriores

- **SWE-agent (2024)** — el andamiaje de interfaz agente-computadora construido sobre este benchmark. [arXiv:2405.15793](#)
- **SWE-bench Verified (2024)** — subconjunto revisado por personas para eliminar instancias mal especificadas.
- **Benchmarks ejecutables en otros dominios (2024+)** — la generalización del criterio.
- **P50 IA constitucional (2022)** — el enfoque opuesto: criterios de juicio donde no hay verificador objetivo posible.

14. Notebook asociado

`P51_swebench.ipynb`

Qué implementa: cinco intentos con tres criterios de éxito distintos y el cálculo de la tasa según cada uno, para hacer visible cuánto inflan los criterios blandos.

Qué NO implementa: no hay repositorio, ni incidencia, ni parche, ni suite de tests. Cinco casos escritos a mano no son una evaluación: son la ilustración de un criterio.

```
ai-evolution paper-lab P51 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Enumera los tres componentes de una instancia del benchmark.
Explicar	Explica por qué un test es mejor verificador que un juicio.
Aplicar	Ejecuta el notebook y añade un caso que compile pero rompa otro test.
Analizar	¿Por qué la contaminación es un problema estructural aquí?
Evaluar	Dos informes reportan cifras distintas. ¿Qué preguntas antes de compararlas?
Crear	Diseña un benchmark ejecutable para una tarea de tu dominio.

16. Autoevaluación

1. ¿Qué contiene cada instancia?
2. ¿Cuál es el criterio de éxito y por qué es distinto de los anteriores?
3. ¿Qué muestra la brecha entre 80 % y 40 %?
4. ¿Por qué la contaminación es difícil de descartar?
5. ¿Qué no mide este benchmark?
6. ¿Por qué las cifras entre informes no son directamente comparables?
7. ¿Qué habilidad resultó ser más difícil de lo esperado?

17. Respuestas esperadas

1. El texto de una incidencia real, el repositorio en el commit anterior a su solución, y los tests que el arreglo real hizo pasar.
2. Aplicar el parche y que **todos** los tests pasen. Es objetivo y ejecutable, frente a criterios de similitud con una solución de referencia o de juicio sobre el código.
3. Cuánto inflan los criterios blandos: el doble de tasa aparente frente a la verificable, siempre en la dirección optimista.
4. Porque las incidencias y sus soluciones son públicas en GitHub y anteriores al corte de datos de los modelos evaluados. No hay forma limpia de garantizar que no estuvieran en el entrenamiento.
5. Diseño, mantenibilidad, revisión, comunicación, y el trabajo en repositorios sin cobertura de tests. Tampoco cubre lenguajes distintos de Python.
6. Porque existen variantes del conjunto (completo, Lite, Verified) y andamiajes de agente muy distintos. Sin especificar ambos, los números miden cosas diferentes.
7. Localizar dónde hay que tocar. En un repositorio que no cabe en el contexto, encontrar el archivo y la función resultó tan difícil como escribir el arreglo.

18. Fuentes primarias

- Jimenez, C. E. et al. (2024). *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* **ICLR 2024**. arXiv:2310.06770 · consultado 2026-08-16.
- Yang, J. et al. (2024). *SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering*. arXiv:2405.15793 · consultado 2026-08-16.

Evaluación — P51

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *SWE-bench: Can Language Models Resolve Real-World GitHub Issues?* (2023, arXiv:2310.06770 · ICLR 2024) · **Nivel:** L3

Parte A — Contexto histórico (20 pts)

- (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
- (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

- (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
- (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

- (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

- (10) Ejecuta `P51_swebench.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
- (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
- (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

- (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
- (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).

P52 — Superposición y autoencoders dispersos

Evaluación y seguridad · Un modelo guarda más conceptos que neuronas tiene. Por eso una neurona no significa una cosa — y por eso interpretarlo es difícil.

Nivel: L4 · Motor: `superposition` · Notebook: `P52_superposition.ipynb` · Anexo: álgebra y geometría

1. Identificación

Campo	Valor
Título original	<i>Toy Models of Superposition</i> (2022) y <i>Towards Monosemanticity: Decomposing Language Models With Dictionary Learning</i> (2023)
Autoría	Nelson Elhage, Tristan Hume, Trenton Bricken y otros (Anthropic)
Año	2022–2023
Venue	<i>Transformer Circuits Thread</i>
Fuente primaria	transformer-circuits.pub/2022/toy_model · transformer-circuits.pub/2023/monosemantic-features
Acceso	Abierto
Fecha de consulta	2026-08-16

2. Problema anterior

Al mirar dentro de una red buscando qué representa cada neurona, aparecía un patrón desconcertante: la mayoría responde a **cosas sin relación aparente**. Una misma unidad se activa con texto en japonés, con citas bíblicas y con nombres de compuestos químicos.

Ese fenómeno —la **polisemanticidad**— bloqueaba la interpretabilidad. Si la unidad de análisis natural no corresponde a un concepto, no hay por dónde empezar.

La explicación cómoda era que las redes son un desorden. Resultó ser lo contrario.

3. Propuesta

La polisemanticidad no es desorden: es **compresión**, y es óptima.

En un espacio de `d` dimensiones solo caben `d` direcciones perfectamente ortogonales. Pero si se acepta un poco de solape, caben **muchísimas más** direcciones casi ortogonales. Si el modelo necesita representar más conceptos que dimensiones tiene —y los necesita, porque el mundo tiene más conceptos que neuronas una capa—, guardarlos en superposición es la estrategia correcta.

El precio es la **interferencia**: los conceptos se pisan un poco. Y funciona porque los conceptos son **dispersos**: en un texto dado, casi ninguno está activo, así que la interferencia rara vez se manifiesta a la vez.

De ahí la propuesta práctica: entrenar un **autoencoder disperso** con muchas más unidades que la capa original, que descomponga las activaciones en características más interpretables.

4. Intuición sin fórmulas

Un armario donde caben diez cajas si no se tocan, y cuarenta si se aceptan solapes — sabiendo que casi nunca abres más de dos a la vez.

Dónde deja de funcionar la analogía: las cajas del armario existen físicamente separadas. Aquí los conceptos son **direcciones** en un espacio continuo, y su solape se suma en cada activación.

5. Matemática mínima

En $\mathbb{R}^d$ hay exactamente d direcciones ortogonales.
Pero hay exponencialmente muchas casi ortogonales (Johnson-Lindenstrauss).

Con vectores aleatorios normalizados en dimensión 8:

conceptos	ratio conceptos/dim	solape medio	solape máximo
8	1,0×	0,253	0,734
24	3,0×	0,310	0,900
80	10,0×	0,290	0,925

El solape **medio** apenas cambia —depende de la dimensión, no del número de conceptos—. Lo que crece es el **peor caso**: con 80 conceptos hay algún par que se solapa 0,925, casi paralelo. Esa es la interferencia que el modelo tiene que tolerar, y por eso la dispersión es la condición que hace viable el truco.

6. Arquitectura o flujo

7. Qué observar en el paper original

- Los **modelos de juguete** de 2022: el fenómeno se reproduce en una red minúscula y controlada, con la dispersión como parámetro. Es la evidencia más limpia.

- Las **estructuras geométricas** que emergen —dígonos, triángulos, pentágonos, tetraedros— según el nivel de dispersión. Es un resultado inesperado y bonito.
- En el trabajo de 2023, las **características encontradas** y sus ejemplos activadores. Vale la pena juzgar por uno mismo cuán interpretables son.
- Los **experimentos de ablación**: activar o suprimir una característica y ver qué cambia en la salida. Es lo único que respalda una afirmación causal.

8. Evidencia y resultados

Los modelos de juguete muestran superposición de forma controlada, con la dispersión como variable. El trabajo de 2023 aplica autoencoders dispersos a una capa de un modelo de lenguaje pequeño y obtiene miles de características con ejemplos activadores coherentes.

Los resultados están en los artículos originales, con visualizaciones interactivas que no se pueden resumir en una tabla. Consultarlos allí. Esta es también la parte del eje donde el trabajo está más **en curso**: es investigación abierta, no un método asentado.

La miniatura de este eje usa **direcciones aleatorias**, no características aprendidas. Muestra que el fenómeno es posible geoméricamente, no que sea lo que un modelo real hace.

9. Impacto

- Dio a la interpretabilidad mecanicista una unidad de análisis utilizable: la característica en lugar de la neurona.
- Convirtió el problema de «las redes son cajas negras» en algo más preciso y por tanto atacable: la base natural no es la correcta, hay que encontrar otra.
- Abrió una línea de trabajo activa sobre control de modelos mediante intervención en características.
- Y dio una explicación de por qué la interpretabilidad era tan difícil, que es en sí un avance.

10. Limitaciones

1. **Interpretable no es causal**: que una característica se active con textos legales no prueba que el modelo la **use** para nada. Hace falta intervenir y medir.
2. **Elegir el tamaño del diccionario es un arte**: demasiado pequeño y no separa, demasiado grande y las características se fragmentan.
3. **Se aplica a una capa a la vez**: componer una explicación del modelo completo sigue abierto.
4. **La reconstrucción no es perfecta**, y lo que se pierde puede importarse.
5. **Coste alto**: entrenar autoencoders sobre modelos grandes es caro.
6. **El fenómeno está bien demostrado en modelos de juguete**; su forma exacta en modelos de frontera es objeto de investigación activa.

11. Errores comunes

Error	Corrección
«Cada neurona representa un concepto»	La mayoría son polisemánticas. Ese es el punto de partida del trabajo.
«La superposición es un defecto»	Es una estrategia de compresión eficiente. El modelo hace bien en usarla.
«El autoencoder revela lo que el modelo piensa»	Da una descomposición que reconstruye las activaciones. Que corresponda a lo que el modelo usa causalmente hay que demostrarlo aparte.
«Más características, mejor interpretación»	Diccionarios muy grandes fragmentan conceptos en trozos sin sentido. Hay un compromiso.
«Esto ya resolvió la interpretabilidad»	Es investigación en curso, con limitaciones reconocidas por sus autores.

12. Relación con trabajos anteriores

- **Po8 Transformer (2017)** — la arquitectura que se intenta interpretar.
- **Po5 Word2Vec (2013)** — la idea de concepto-como-dirección, aquí llevada al interior del modelo.
- **Johnson-Lindenstrauss (1984)** — el resultado geométrico que hace posible la superposición.
- **P42 Ejemplos adversarios (2014)** — la otra cara de la dimensión alta: la interferencia también se puede explotar.

13. Relación con trabajos posteriores

- **Scaling Monosemanticity (2024)** — la aplicación a un modelo de producción, con millones de características. transformer-circuits.pub/2024/scaling-monosemanticity
- **Control mediante características (2024+)** — intervenir en el modelo actuando sobre ellas.
- **Interpretabilidad como herramienta de seguridad** — la promesa: auditar lo que el modelo hace antes de desplegarlo, todavía lejos de cumplirse.

14. Notebook asociado

P52_superposition.ipynb

Qué implementa: el almacenamiento de 8, 24 y 80 direcciones aleatorias en 8 dimensiones, con el solape medio y máximo entre pares.

Qué NO implementa: no hay autoencoder disperso, ni modelo, ni características aprendidas. Las direcciones aleatorias muestran que el fenómeno es geoméricamente posible; en un modelo real la estructura no es aleatoria y esa diferencia importa.

```
ai-evolution paper-lab P52 --seed 7
```

15. Actividades Bloom

Nivel	Actividad
Recordar	Define polisemanticidad y superposición con tus palabras.
Explicar	Explica por qué la dispersión es la condición que hace viable la superposición.
Aplicar	Ejecuta el notebook con dimensión 16 y compara los solapes.
Analizar	¿Por qué el solape medio no crece con el número de conceptos y el máximo sí?
Evaluar	«Encontramos la característica del engaño». ¿Qué evidencia exiges?
Crear	Diseña un experimento de ablación que distinga correlación de uso causal.

16. Autoevaluación

1. ¿Qué es la polisemanticidad y por qué bloquea la interpretabilidad?
2. ¿Cuántas direcciones ortogonales caben en d dimensiones, y cuántas casi ortogonales?
3. ¿Por qué la superposición es óptima y no un defecto?
4. ¿Qué papel juega la dispersión?
5. ¿Qué hace un autoencoder disperso?
6. ¿Por qué interpretable no implica causal?
7. ¿Qué distancia hay entre estos resultados y un modelo de frontera?

17. Respuestas esperadas

1. Que una misma neurona responde a conceptos sin relación entre sí. Bloquea la interpretabilidad porque la unidad de análisis natural —la neurona— no corresponde a nada interpretable.
2. Exactamente d ortogonales. Casi ortogonales, un número exponencial en d , que es lo que garantiza el resultado de Johnson-Lindenstrauss.
3. Porque el modelo necesita representar más conceptos de los que caben en direcciones ortogonales, y aceptar un poco de solape le permite guardar muchísimos más con una pérdida pequeña.
4. Es la condición que hace tolerable la interferencia: como casi ningún concepto está activo a la vez, los solapes rara vez se manifiestan simultáneamente.
5. Descompone las activaciones de una capa en muchas más unidades que la capa original, con una penalización que fuerza que casi todas estén en cero, buscando características que representen una sola cosa.
6. Porque el autoencoder solo garantiza que la descomposición **reconstruye** las activaciones. Que el modelo use esa característica para producir su salida es una afirmación causal que exige intervenir —activarla o suprimirla— y medir el efecto.
7. Los modelos de juguete demuestran el fenómeno de forma limpia y controlada; en modelos de frontera el trabajo está en curso, es caro, y sus propios autores reconocen las limitaciones.

18. Fuentes primarias

- Elhage, N. et al. (2022). *Toy Models of Superposition*. **Transformer Circuits Thread**. transformer-circuits.pub/2022/toy_model · consultado 2026-08-16.
- Bricken, T. et al. (2023). *Towards Monosemanticity: Decomposing Language Models With Dictionary Learning*. **Transformer Circuits Thread**. transformer-circuits.pub/2023/monosemantic-features ·

consultado 2026-08-16.

Evaluación — P52

Generado por `python scripts/generate_papers.py`. Se evalúa comprensión histórica, lectura crítica e interpretación — no memorización de definiciones.

Paper: *Towards Monosemanticity: Decomposing Language Models With Dictionary Learning* (2023, Transformer Circuits Thread) · **Nivel:** L5

Parte A — Contexto histórico (20 pts)

1. (10) Describe el estado del arte **inmediatamente anterior** a este paper y por qué era insuficiente. No menciones la solución del paper en tu respuesta.
2. (10) Nombra un trabajo anterior del que este paper depende y explica qué le tomó prestado.

Parte B — Lectura crítica (20 pts)

3. (10) Localiza en el paper original una afirmación **cuantitativa** y reescríbela indicando tarea, dataset, métrica, línea base y condiciones. Cita la tabla o sección.
4. (10) Identifica una idea que hoy se asocia a este paper pero que **apareció después**. Aporta la referencia posterior con año.

Parte C — Interpretación matemática (15 pts)

5. (15) Explica la ecuación central con tus palabras y señala qué ocurre en un caso límite (valor 0, dimensión muy grande, secuencia muy larga... según corresponda).

Parte D — Implementación e interpretación (25 pts)

6. (10) Ejecuta `P52_superposition.ipynb` con **tres semillas** y reporta qué varía y qué se mantiene.
7. (10) Reproduce el anti-patrón de la sección 11 y explica por qué produce una conclusión errónea.
8. (5) Aporta la corrección con su evidencia.

Parte E — Límites y transferencia (20 pts)

9. (10) Escribe una limitación de la **miniatura** y una del **paper original**. No pueden ser la misma idea.
10. (10) Conecta este hito con el siguiente de la ruta: ¿qué quedó sin resolver que motivó el paso siguiente?

Rúbrica

Nivel	Descripción
A — Excelente	Distingue hecho documentado, simplificación didáctica e inferencia propia. Cita fuentes primarias con sección o tabla. Sus límites son propios, no copiados.
B — Suficiente	Explica el mecanismo y ejecuta la miniatura correctamente, pero repite los límites de la ficha y cita de forma imprecisa.
C — Insuficiente	Describe el paper con narrativa retrospectiva, atribuye ideas posteriores, o presenta la salida de la miniatura como reproducción del experimento original.

Criterio automático de rechazo

Se devuelve sin nota cualquier entrega que:

- atribuya al paper resultados, métricas o autores que no aparecen en la fuente primaria;
- presente la ejecución del notebook como reproducción de los resultados del paper;
- cite un paper que no se abrió (se comprueba pidiendo el número de figura o tabla).